

МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ ФЕДЕРАЦИИ

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО
ПРОФЕССИОНАЛЬНОГО ОБРАЗОВАНИЯ**

**МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)**

«МАИ»

Факультет №3
Кафедра № 304

Г. А. ЗВОНАРЕВА

А.В. КОРНЕЕНКОВА

***Учебное пособие для лабораторных работ
по курсу:***

ОРГАНИЗАЦИЯ ЭВМ И СИСТЕМ

Под редакцией д.т.н. проф. О.М. Брехова

Москва, 2020



Оглавление

<i>Инструкция по запуску</i>	3
<i>Инструкция по использованию программного обеспечения</i>	4
<i>Описание языка MCL</i>	8
<i>Арифметико-логические устройства ЭВМ и принципы их функционирования</i>	21
<i>Лабораторная работа № 1. Моделирование функционирования АЛУ при выполнении операции сложения/вычитания над числами с фиксированной точкой</i>	22
<i>Лабораторная работа № 2. Моделирование функционирования АЛУ при выполнении операции умножения над числами с фиксированной точкой</i>	35
<i>Лабораторная работа № 3. Моделирование функционирования АЛУ при выполнении операции деления над числами с фиксированной точкой</i>	47
<i>Лабораторная работа № 4. Моделирование функционирования операционной части центрального процессора</i>	58
<i>Лабораторная работа № 5. Моделирование блока устройства управления</i>	73
<i>Лабораторная работа № 6. Моделирование функционирования селекторного канала</i>	86
<i>Лабораторная работа № 7. Моделирование функционирования мультиплексного канала</i>	100
<i>Лабораторная работа № 8. Моделирование секторной буферной памяти</i>	120
<i>Лабораторная работа № 9. Моделирование группно-ассоциативной буферной памяти</i>	147



Комплектация ПО

1. **MCL 1.7.exe** исполняемый файл графического интерпретатора языка MCL.
2. **Interpreter_1.0.dll** библиотека интерпретатора языка MCL.
3. **Helps** – папка содержащая следующие файлы справки в формате PDF:
 - 3.1 **Help1.pdf-Help9.pdf** – файлы справки по лабораторным работам.
 - 3.2 **ReadMe.pdf** – файл справки по программе.
4. **Distribute** – папка с дистрибутивами дополнительного ПО:
 - 4.1 **readerdc_ru_xa_cra_install.exe** дистрибутив Adobe Acrobat Reader DC.
 - 4.2 **dotNetFx40_Full_x86_x64.exe** дистрибутив библиотеки .NET 4.0.
5. **ReadMe.pdf** – данный файл.
6. **Учебное пособие по ОЭВМ.pdf** – методическое пособие по курсу лабораторных работ.

Инструкция по запуску

1. Проверить установлена ли версия .NET 2.5, если не установить ее из папки Distribute, которая вложена в папку с программой.
2. Если на диске, выделенном для студенческого использования, существует папка с номером группы, то создать папку с фамилией, если она отсутствует создать её, как и папку с номером группы, в случае её отсутствия. Скопировать папку с программой в вышеописанную папку с фамилией.
3. Если по каким-либо причинам файлы не могут быть скопированы или запущены, следует скопировать данные файлы на рабочий стол и запускать с него.
4. Все файлы, используемые студентом, следует хранить в папке с фамилией студента.

Примечание:

Последнюю версию программы следует уточнять у преподавателя.



Инструкция по использованию программного обеспечения

При загрузке приложения появляется выбор типа лабораторной работы (рис. 1).

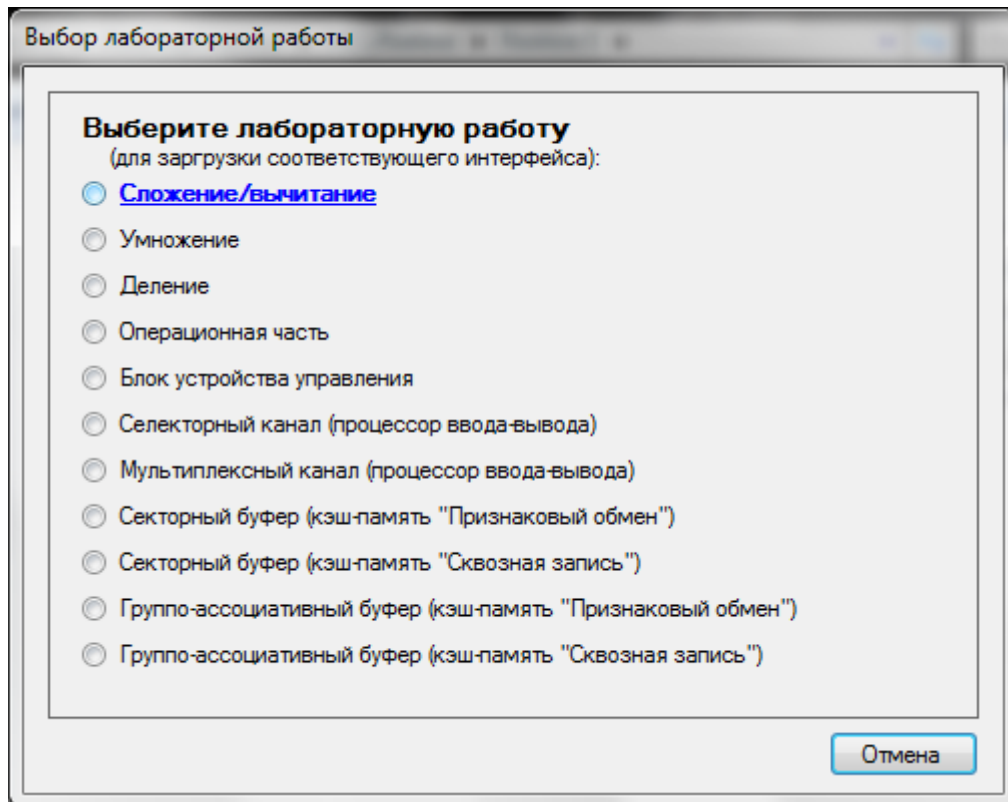


Рис. 1 Окно выбора лабораторной работы

После нажатия необходимый пункт загрузится графическая среда разработки. В зависимости от выбранной лабораторной работы слева будет отображен графический элемент управления, представляющий структуру изучаемого устройства, и поле для вывода текстовой информации (рис. 2).

Графические элементы управления содержат поля, иллюстрирующие различные элементы изучаемых устройств, а также поля для ввода их имен. Эти поля уже содержат предопределенные имена. Их можно изменять. Во время выполнения программы среди объявленных переменных будут искаться переменные с именами, указанными в данных полях, и их значения будут отображаться в соответствующих им «регистровых» полях.

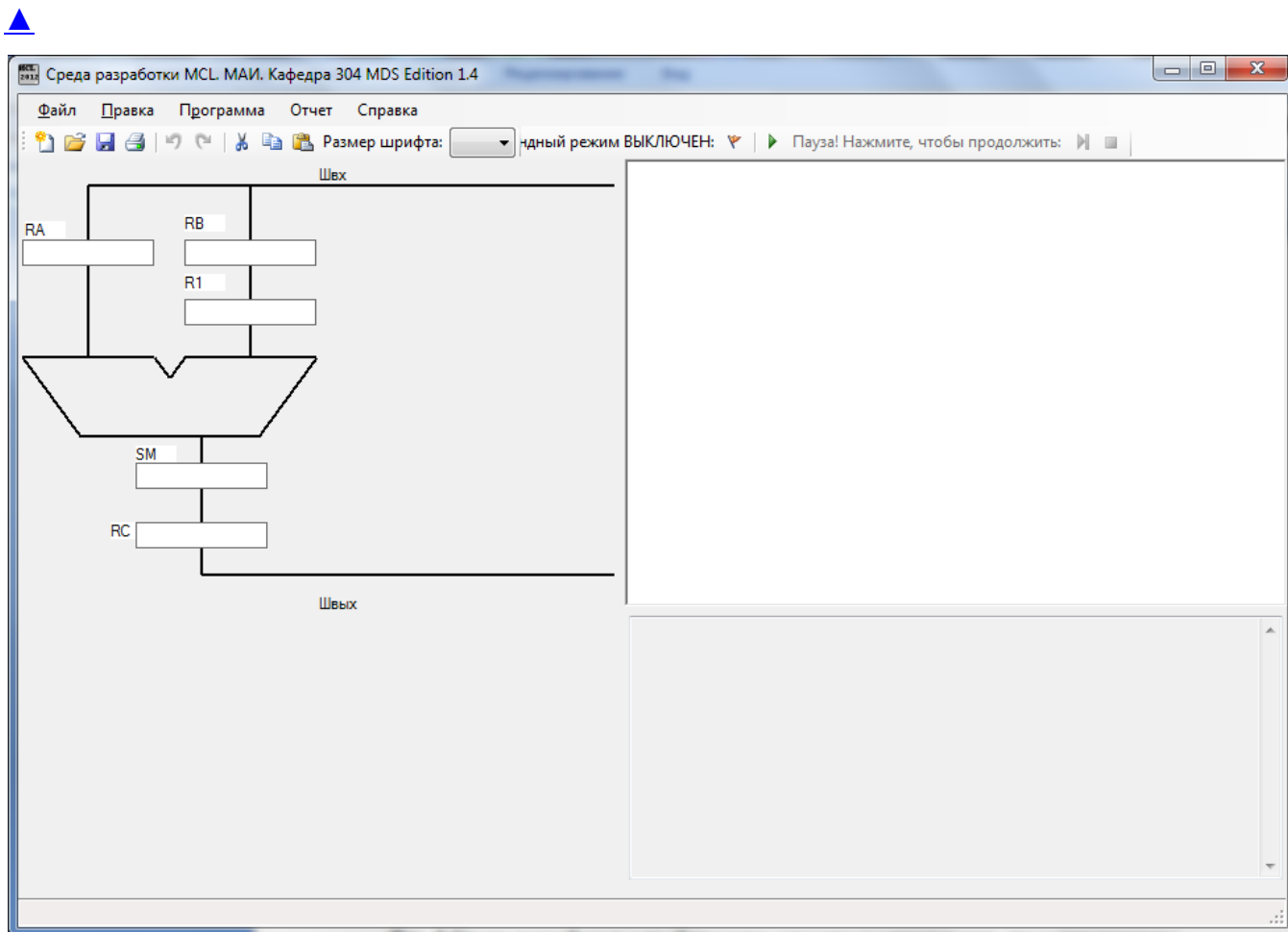


Рис. 2 Среда разработки с графическим элементом управления, представляющим структуру АЛУ

Справа находится редактор текстов. В него вводится текст разрабатываемой программы. С помощью стандартных команд меню (Файл - Открыть, Файл – Сохранить, Файл – Сохранить как) и кнопок можно открыть созданный ранее текстовый файл, сохранить созданный или редактируемый файл.

Выбрав пункт Файл – Новый можно выбрать другую лабораторную работу. При этом графическая среда будет переконфигурирована соответствующим образом.

Пункт меню Правка предоставляет стандартные средства редактирования текста: копирование, вырезание, вставка, выделение, отмена последних операции и повтор их выполнения.

Когда программа в текстовом редакторе готова к выполнению, следует обратиться к пункту меню Программа.

Программа может выполняться в двух режимах: покомандном и потоковом. При потоковом режиме команды выполняются одна за другой, интерпретация останавливается, когда считан оператор END. Если в программе встречается команда PAUSE, интерпретация приостанавливается и продолжается только после нажатия кнопки «Продолжить»  на панели под меню.

При покомандном режиме работы интерпретация приостанавливается после выполнения каждой команды и продолжается только после нажатия пользователем кнопки «Продолжить»  . При этом в редакторе тестов подсвечивается строка, которая будет выполнена следующей (рис. 3).

По умолчанию включен потоковый режим работы.

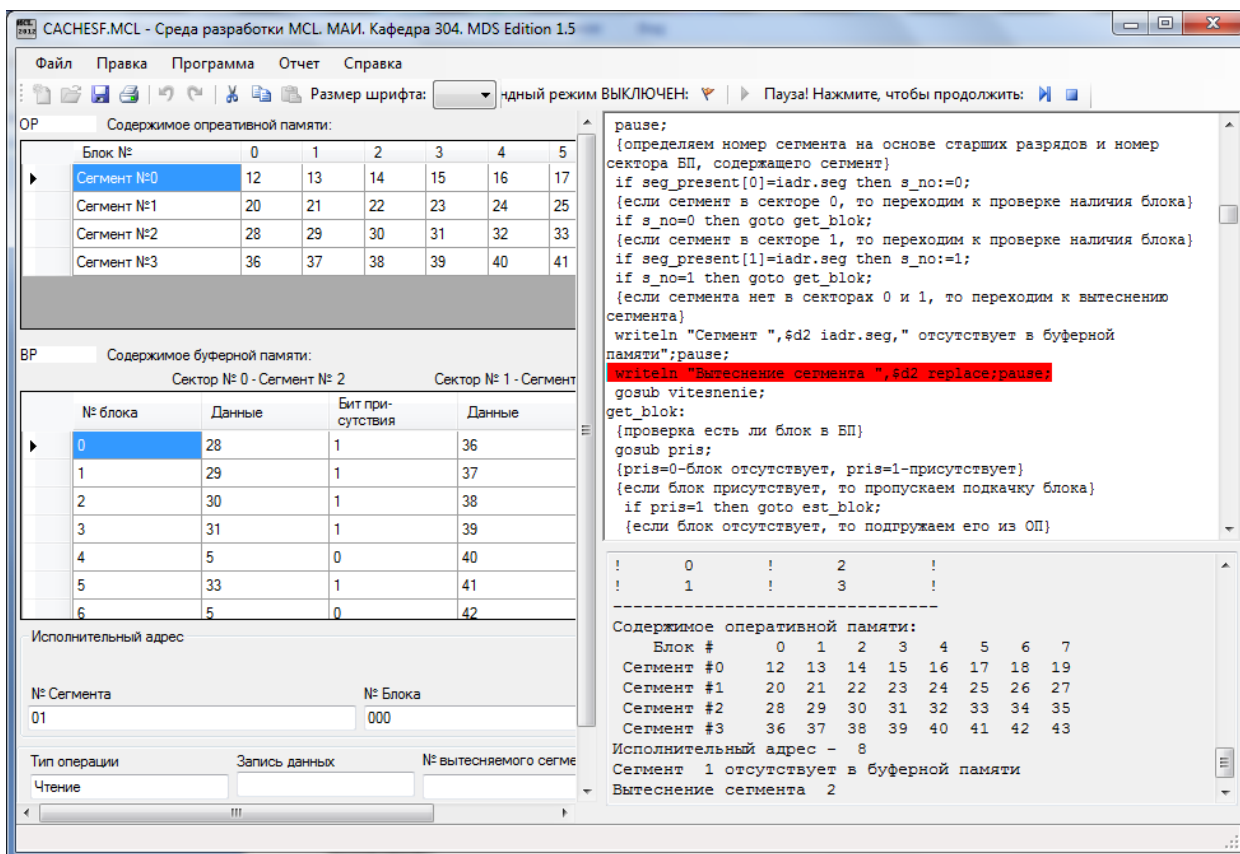


Рис. 3. Подсветка строки в редакторе текстов

Чтобы включить покомандный режим выполнения, необходимо выбрать пункт меню Программа – Покомандный режим, или нажать кнопку с флажком, или F4. Данная команда НЕ запускает интерпретацию программы, а только определяет её режим (покомандный или потоковый).

Чтобы запустить интерпретацию программы, необходимо выбрать пункт меню Программа – Выполнить, или нажать кнопку , или F5.

Во время выполнения программы текст, выводимый командами WRITE и WRITELN, отображается на форме тестового вывода слева внизу. В этой же форме отображаются сообщения об ошибках, если они найдены в программе. Иллюстрация перерисовывается в момент выполнения команды PAUSE в потоковом режиме и после выполнения каждой команды в покомандном режиме. Режим можно переключать во время выполнения программы: например, можно дойти до первой команды PAUSE в программе в потоковом режиме, переключить режим и продолжить выполнение программы в покомандном режиме.

Чтобы продолжить выполнение программы нужно выбрать пункт меню Программа – Продолжить, или нажать кнопку , или надпись «Пауза! Нажмите, чтобы продолжить», или F10.

Остановить интерпретацию (выполнение) программы можно в любой момент, нажав кнопку , выбрав пункт меню Программа – Остановить, или нажав F2.

Результат выполнения программы и её текст можно отправить на печать, выбрав пункт меню Файл – Печать или нажав кнопку . При этом содержимое редактора текстов будет совмещено с содержимым поля текстового вывода и отправлено на печать.

Кроме того, в программе имеется возможность делать скриншоты, копировать текст программы и лог в буфер обмена, для дальнейшей удобной вставки в текстовые файлы. Это существенно облегчает создание отчетов. Как итог, нет необходимости выделять текст



программы и содержимое поля для вывода текстовой информации вручную, а также использовать сторонние программы для создания скриншотов.

Чтобы сделать скриншот, нужно выбрать пункт меню Отчет - Скриншот в буфер обмена. После этого, готовый скриншот можно вставлять в отчет.

Для того, чтобы скопировать текст программы достаточно выбрать меню Отчет - Текст программы в буфер обмена. Затем в текстовых редакторах, с помощью комбинации клавиш Ctrl+V можно вставить исходный текст.

Чтобы копировать содержимое поля для вывода текстовой информации выберете меню Отчет - Копировать лог в буфер обмена. Затем в текстовых редакторах, с помощью комбинации клавиш Ctrl+V можно вставить исходный текст.

Для изменения размера шрифта редактора текста, необходимо выбрать его из выпадающего списка (рис 4).

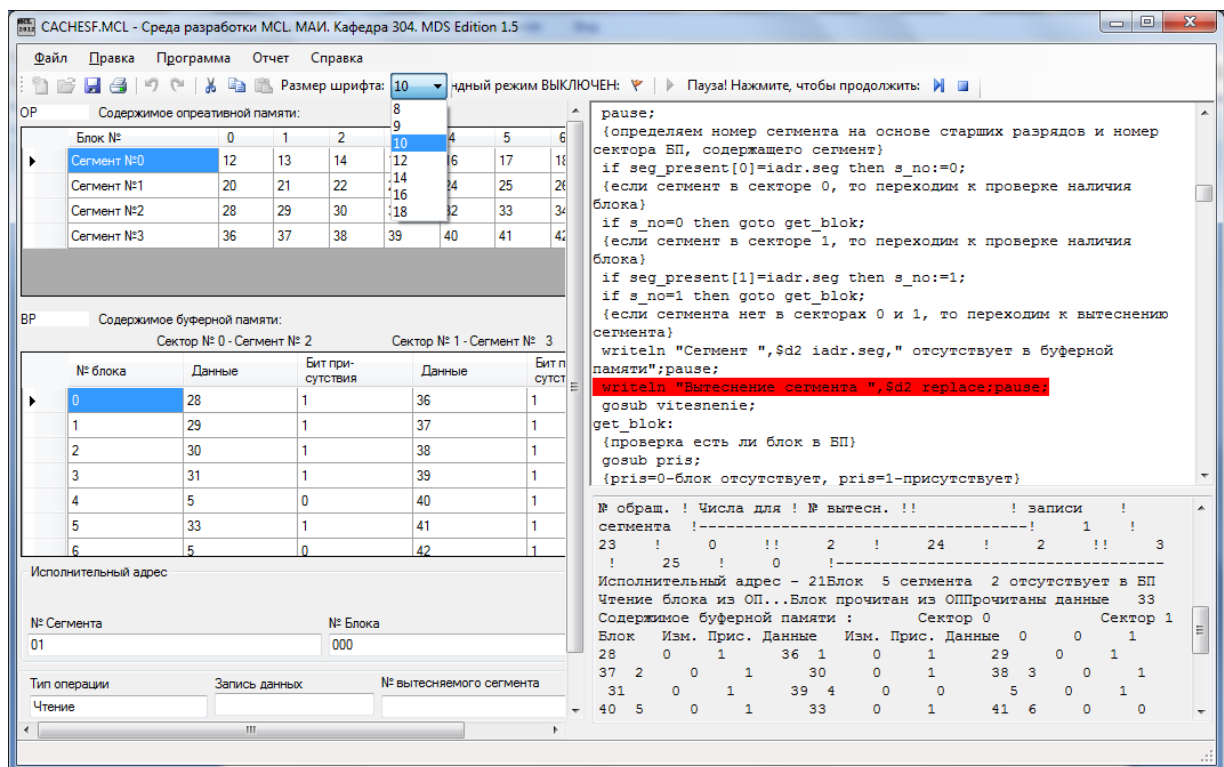


Рис. 4. Изменение размера шрифта редактора кода

Для вызова справки по программе выберите пункт меню Справка – По программе, после чего будет открыт файл справки по программе.

Для вызова справки по лабораторной работе, необходимо выбрать пункт меню Справка – По лабораторной работе.



Описание языка MCL

Содержание

1. Описание языка MCL	9
2. Алфавит	9
3. Использование разделителей	9
4. Использование комментариев	9
5. Идентификаторы	9
6. Константы	9
7. Операции и выражения	10
8. Приоритеты и порядок выполнения операций	11
9. Арифметические преобразования в выражениях	11
10. Типы языка MCL	11
11. Переменные языка MCL	13
12. Операторы языка MCL	13
13. Переменные-флаги	17
14. Общие замечания	17
15. Сообщения об ошибках интерпретатора языка MCL	18



1. Описание языка MCL

Язык микрокоманд MCL представляет собой "BASIC-подобный" язык программирования высокого уровня. Он включает в себя:

- операторы ввода-вывода,
- операторы перехода,
- операторы вызова подпрограмм,
- операторы цикла,
- условный оператор;

поддерживает достаточный набор типов данных:

- простые переменные,
- тип регистр,
- тип структура,
- тип память;

и операций над объектами этих типов:

- арифметические,
- побитовые,
- сдвига,
- сравнения.

Ниже приведено полное описание языка MCL.

2. Алфавит

Алфавит содержит:

- буквы: 26 латинских букв и \$,
- цифры от 0 до 9,
- знаки арифметических операций: +, -, *, / (целая часть от деления), % (остаток от деления),
- знаки операций отношения: >, >=, <, <=, =, <>.

Замечания: все операции отношения реализованы как целочисленные операции (истинность - 1, иначе - 0) и могут быть использованы в любом месте любого выражения. При сравнении операнды рассматриваются как целые числа без знака.

3. Использование разделителей

Пробелы, символы табуляции, перевода на новую строку используются как разделители. Любое количество этих символов может быть помещено в любое место, где допускается пробел.

4. Использование комментариев

Комментарий - любой текст, помещенный между символами '{' и '}'. Разрешены везде, где допускаются пробелы.

5. Идентификаторы

Идентификаторы - это последовательность букв и цифр, начинающихся с буквы. Их длина не должна превышать 16 символов. Они используются как имена переменных, полей структур и меток.

Правильные идентификаторы: A, R0, REG_RESULT, KOD22.

Неправильные идентификаторы:

- 0A - должен начинаться с буквы;
- A-REG - используется недопустимый символ '- '.

6. Константы

Все константы в языке MCL интерпретируются как 32-х разрядные целые числа без знака, отрицательные константы не поддерживаются. Для получения отрицательных значений необходимо использовать явное преобразование (операцию COM). Таким образом, могут быть использованы константы из диапазона: 0..4294967295. Константы могут быть записаны в



двоичной, десятичной или шестнадцатеричной системах счисления. Правило записи констант следующее:

<константа>:='0b'<b_посл>
'0d'<d_посл>
'0h'<h_посл>

Отсюда видно, что для записи константы в любой системе счисления необходимо перед числом указать спецификатор системы счисления вида '0b' или '0d' или '0h'. Без указания спецификатора формата константа интерпретируется как десятичная.

Примеры:

12345 или 0d12345 - десятичная константа 12345,
0b1000 - двоичная константа 8,
0hFF - шестнадцатеричная константа 255.

Замечание:

Для определения констант, больших по своему значению чем 2147483647 (0h7FFFFFFF), необходимо использовать двоичную или шестнадцатеричную систему счисления. Это связано с особенностями внутреннего представления целых чисел системами PASCAL VS и Turbo PASCAL.

7. Операции и выражения

Побитовые операции:

NOT (дополнение до единицы) NOT op

Дополнение до единицы (инвертирование) значения op. Значение выражения содержит 0 во всех разрядах, в которых op содержит 1, и 1 во всех разрядах, в которых op содержит 0.

COM (дополнение до двух) COM op

Дополняет до двух (переводит в дополнительный код) значение op. Фактически вычисляется выражение 1+NOT op.

AND (побитовое "И") op1 AND op2

Побитовая операция "И" двоичных представлений op1 и op2. Значение выражения содержит 1 во всех разрядах, в которых и op1 и op2 содержат 1, и 0 во всех остальных разрядах.

OR (побитовое "ИЛИ") op1 OR op2

Побитовая операция "ИЛИ" двоичных представлений op1 и op2. Значение выражения содержит 1 во всех разрядах, в которых op1 или op2 содержат 1, и 0 во всех остальных разрядах.

XOR (побитовое исключающее "ИЛИ") op1 XOR op2

Побитовая операция исключающее "ИЛИ" двоичных представлений op1 и op2. Значение выражения содержит 1 в тех разрядах, в которых и op1 и op2 имеют разные двоичные значения, и 0 во всех остальных разрядах.

Операции сдвига:

SHL (сдвиг влево) op1 SHL op2

Двоичное представление op1 сдвигается влево на op2 разрядов. Освобождающиеся разряды заполняются 0.

SHR (сдвиг вправо) op1 SHR op2

Двоичное представление op1 сдвигается вправо на op2 разрядов. Освобождающиеся разряды заполняются 0.

ASL (арифметический сдвиг влево) op1 ASL op2

Двоичное представление op1 сдвигается влево на op2 разрядов. Освобождающиеся разряды заполняются 0.

ASR (арифметический сдвиг вправо) op1 ASR op2



Двоичное представление `op1` сдвигается вправо на `op2` разрядов. Знаковый разряд (старший разряд) дублируется.

ROL (циклический сдвиг влево) `op1 ROL op2`

Двоичное представление `op1` циклически сдвигается влево на `op2` разрядов. Освобождающиеся разряды заполняются выдвигаемыми разрядами.

ROR (циклический сдвиг вправо) `op1 ROR op2`

Двоичное представление `op1` циклически сдвигается вправо на `op2` разрядов. Освобождающиеся разряды заполняются выдвигаемыми разрядами.

8. Приоритеты и порядок выполнения операций

Для каждой группы операций в таблице приоритетов операций приоритеты одинаковы. Операции с большим значением приоритета выполняются в первую очередь. Чем выше приоритет группы операций, тем выше она расположена в таблице.

Приоритеты операций:

9	NOT COM
8	* / %
7	+ -
6	SHL SHR ROR ROL ASR ASL
5	> >= < <=
4	= <>
3	AND
2	XOR
1	OR

Все операции с одинаковыми приоритетом выполняются справа налево. Можно группировать операции с помощью скобок. Операции в скобках будут выполняться в первую очередь.

9. Арифметические преобразования в выражениях

1. Если операнды в выражениях имеют различный размер (размер операнда зависит от типа, о типах см. ниже), то все результаты операций приводятся к большему размеру.

2. При выполнении присваивания значение выражения преобразуется к размеру переменной, которой оно присваивается.

3. Уменьшение размера значения производится путем отбрасывания старших битов и никаких диагностических сообщений при этом не выводится.

10. Типы языка MCL

Тип - простая переменная:

Простые переменные предназначены для использования в программе там, где не обязательно явное указание размера переменной - управляющие переменные циклов, счетчики и т.п. Описание простой переменной производится с помощью оператора `VAR` и синтаксис его определяется следующим правилом:

```
VAR <идентификатор>, ... ;
```

Например:

```
VAR i, j, simple_var; - описаны три простых переменных.
```



Простые переменные имеют predetermined размер 16 битов и интерпретируются как целые без знака. Кроме получения значения простой переменной, возможно получение значения определенного разряда или группы разрядов.

Например:

```
j - значение переменной j;  
j[3..3] - значение 3-го разряда переменной j;  
i[0..7] - значение восьми младших разрядов переменной j.
```

Тип - регистр:

Регистры используются там, где необходимо точно определить размер переменной. Регистры могут иметь произвольный размер в пределах от 1 до 32 разрядов. Описание регистра производится с помощью оператора REG. Синтаксис описания регистров определяется следующим правилом:

```
REG <идентификатор> [ <константа> ] , ... ;
```

Например:

```
REG ra[8],rb[8],f_zero[1],mul_res[32]; - описаны два  
восьмиразрядных регистра ra и rb, одноразрядный регистр f_zero и  
тридцатидвухразрядный регистр mul_res. Константа описывает  
размерность регистра.
```

Использование регистров полностью идентично использованию простых переменных. Так же возможно как получение значения всего регистра, так и получение значения его части.

Например:

```
ra - значение регистра ra;  
rb[7..7] - значение старшего бита регистра rb;  
mul_reg[16..31] - значение старшей половины регистра mul_reg.
```

Тип - структура:

Структуры могут быть использованы для упрощения доступа к отдельным полям переменных. Они позволяют присваивать группам битов имена. В остальном структуры сохраняют все свойства регистров. Описание структур производится с помощью оператора STRUCT. Синтаксис описания структур определяется следующим правилом:

```
STRUCT <имя структуры> [ <константа> ] <имя поля>  
[<константа>], ... ;
```

Например:

```
STRUCT rk[32] kop[16],r1[4],m1[4],r2[4],m2[4]; - в этом примере  
описана 32-х разрядная структура rk с полями kop[0..15],  
r1[16..19], m1[20..23], r2[24..27], m2[28..31].
```

Обратите внимание, что выделение полей в структуре производится начиная с младшего бита (т.е. от младших битов к старшим). Использование структур аналогично использованию регистров, но добавляется еще один метод доступа к значению группы бит - с помощью имени поля,



например:

rk - значение структуры rk;
rk[0..15] - значение младшей половины структуры rk;
rk.kor - то же самое, что и в предыдущем примере, но с использованием имени поля.

Тип - память:

Память представляет собой одномерный массив регистров. Ячейки памяти могут, как и регистры, иметь произвольный размер в пределах 1..32 разряда. Максимальный размер памяти 1024 (1K) ячеек. Индексация ячеек производится от 0. Описание памяти производится с помощью оператора MEM. Синтаксис описания памяти определяется следующим правилом:

<идентификатор> [<константа 1>] <константа 2>],...; - где первая константа определяет количество ячеек в памяти, а вторая - разрядность ячейки.

Например:

MEM mb[32][8] - описана память mb с числом ячеек 32 и с разрядностью ячейки - 8 бит;
MEM ml[1024][32] - описана память ml с числом ячеек 1024 и с разрядностью ячейки 32 бита;

Для доступа к ячейке памяти необходимо после имени памяти задать индекс этой ячейки. Также возможен доступ и к группам битов отдельной ячейки.

Например:

mb[5] - обращение к 6-й ячейке памяти mb (помните, что индексация ячеек начинается с 0);
ml[9,5..5] - обращение к 6-му разряду 10-й ячейки памяти ml;
ml[100,0..15] - обращение к младшей половине 101-й ячейки памяти ml.

11. Переменные языка MCL

Все переменные должны быть обязательно описаны перед ссылкой на них (неявное определение переменных не поддерживается). Максимальное число переменных, которые могут использоваться в программе одновременно составляет 100. Использование большего числа переменных приведет к ошибке. Как уже было отмечено выше, максимальная разрядность регистра или структуры - 32 разряда, а максимальное число ячеек памяти 1024.

12. Операторы языка MCL

Оператор - формат:

один оператор может занимать одну или более строк. Два или большее количество операторов могут находиться на одной строке. Каждый оператор должен завершаться точкой с запятой.

Оператор - метка:

метка может стоять перед любым оператором, чтобы на этот оператор можно было перейти с помощью оператора GOTO или GOSUB. Метка состоит из идентификатора за которым стоит двоеточие. Один оператор может иметь несколько меток.



Оператор - присваивание:

оператор присваивания имеет следующий формат:

<переменная> := <выражение>;

Значение выражения вычисляется и присваивается переменной. В качестве переменной может использоваться любая допустимая ссылка на переменную.

Например:

```
ra:=15+rb/2;  
ml[1,2..4]:=0b111;  
mb[3]:=0;  
rk.r1:=(ra[0..3]+rb[0..3]) ROL 2;
```

Оператор - безусловный переход GOTO:

оператор безусловного перехода имеет формат:

GOTO <идентификатор> ;

Управление безусловно передается на оператор перед которым стоит метка <идентификатор>.

Оператор - вызов подпрограммы GOSUB:

оператор вызова подпрограммы имеет формат:

GOSUB <идентификатор> ;

Управление передается на оператор перед которым стоит метка <идентификатор>. Когда будет выполнен оператор RETURN, управление будет возвращено оператору следующему за GOSUB.

Например:

```
GOSUB SUBR; _____  
Writeln "MAIN PROG"; ←  
...  
SUBR: ←  
Writeln "SUBROUTIN";  
RETURN; _____
```

-Вызов ПП
-Возврат из ПП

Допускается вложение вызовов подпрограмм. Максимальный уровень вложенности - 16.

Оператор - возврат из подпрограммы RETURN:

оператор возврата из подпрограммы имеет формат:

RETURN;

Управление передается на оператор следующий за оператором GOSUB (см. пример выше).



Оператор - условный оператор IF THEN:

условный оператор имеет формат:

```
IF <выражение> THEN <оператор> ;
```

Сначала вычисляется значение <выражения> и, если оно не равно 0, то выполняется оператор следующий за THEN. В противном случае выполняется следующий оператор. В качестве <выражения> можно использовать любое (не обязательно логическое) выражение.

Например:

```
I:=1;  
IF I THEN WRITELN "i<>0";  
J:=0; IF I<J THEN GOTO LBL2;  
PRINT "I>J";  
LBL2:  
END;
```

Результатом выполнения этой программы будет:

```
I<>0 I>J
```

Оператор - оператор цикла FOR TO NEXT:

оператор цикла имеет формат:

```
FOR <идентификатор>:=<выражение1>TO<выражение2>;  
{ <оператор> }  
NEXT;
```

Переменной с именем <идентификатор> присваивается значение <выражения1>, после чего выполняются все операторы до оператора NEXT. После этого к переменной <идентификатор> прибавляется 1 и ее значение сравнивается с значением выражения <выражение2>. Если значение переменной больше значения выражения, то выполнение цикла прекращается и управление передается следующему за NEXT оператору, в противном случае опять выполняются все операторы до оператора NEXT и т.д. Допускается использование вложенных циклов. Максимальный уровень вложенности не должен превышать 16.

Замечание:

после выхода из цикла управляющая переменная имеет значение <выражение2>+1. Цикл всегда выполняется хотя бы один раз (даже если <выражение1> больше <выражения2>) Не рекомендуется для выхода из цикла использовать оператор GOTO, безусловный переход из тела цикла возможен только на конец программы.

Например:

```
FOR I:=0 TO 1023; ML[I]:=0; NEXT;
```

Оператор - пауза PAUSE:

оператор пауза имеет следующий формат:

```
PAUSE;
```



Этот оператор приостанавливает выполнение программы и выводит на терминал сообщение: "Приостановлено. Нажмите клавишу ENTER..." После нажатия клавиши ENTER выполнение программы возобновляется с оператора, следующего за оператором PAUSE.

Оператор - вывод WRITE и WRITELN:

оператор предназначен для вывода информации на терминал, выводимую информацию можно разделить на две части: пояснительный текст (строки в кавычках) и значения выражений. Допускается три формата вывода значения выражений:

- двоичный (спецификатор формата \$B),
- десятичный (спецификатор формата \$D),
- шестнадцатеричный (спецификатор формата \$H).

Указание спецификатора формата и длины поля вывода обязательно!

Следующая за спецификатором формата константа определяет размер поля вывода для значения выражения. При выводе значений в двоичной и шестнадцатеричной СС если размер поля больше, чем размер реально выводимого числа, то число дополняется слева нолями. При выводе значений в десятичной СС число дополняется слева пробелами. Если выводимое число больше по размеру, чем выделенное поле, то оно усекается слева.

Например:

```
I:=0HFFFF; {I:=65535}
WRITE $B32 I {На выходе> 00000000000000000111111111111111}
WRITE $B16 I {На выходе> 1111111111111111}
WRITE $B8 I {На выходе> 11111111}
WRITE $D7 I {На выходе> 65535}
WRITE $D5 I {На выходе> 65535}
WRITE $D1 I {На выходе> 5}
WRITE $H6 I {На выходе> 00FFFF}
WRITE $H4 I {На выходе> FFFF}
WRITE $H2 I {На выходе> FF}
```

Оператор WRITELN отличается от оператора WRITE тем, что он добавляет к выводимой информации символ перевода на новую строку.

Оператор – ввод READ:

оператор предназначен для ввода значений любых переменных с терминала. Значения могут вводиться в виде констант в любой СС. Вводимые значения автоматически преобразуются к требуемому размеру.

Например:

```
READ "A=", A; READ "RK.KOP=", RK.KOP, "RA.LSB=", RA[0..7];
READ "M[I]=", M[I];
```

Замечание:

при вводе числа в десятичной СС допускается указание знака числа. После ввода, число будет автоматически преобразовано в дополнительный код.

Оператор - конец программы END:

Оператор служит для завершения выполнения программы. Имеет следующий формат:

```
END;
```




В общем случае этот оператор не обязателен (выполнение программы автоматически завершится после выполнения последнего оператора в тексте), но, тем не менее, желательно в любом случае использовать оператор END.

13. Переменные-флаги

Система представляет в распоряжение пользователя три предопределенные переменные-флага:

ZF - флаг нуля;

NF - флаг отрицательного значения;

CF - флаг переноса.

Можно сказать, что эти переменные описаны неявным образом как REG ZF[1],NF[1],CF[1]. Операции языка MCL могут изменить значение каждого из флагов. Рассмотрим назначение каждого флага более подробно.

Флаг нуля ZF:

Если результат операции равен нулю, то этот флаг устанавливается в 1, в противном случае - 0. Все без исключения операции изменяют значение этого флага.

Флаг отрицательного значения NF:

Если старший (знаковый) бит результата операции равен единице, то этот флаг устанавливается в 1, в противном случае - в 0. При определении значения этого флага учитывается длина операндов выполняемой операции. Все без исключения операции изменяют значение этого флага.

Флаг переноса CF:

Этот флаг устанавливается в 1, если при выполнении операции сложения или вычитания возникает перенос из старшего разряда результата. В противном случае этот флаг устанавливается в 0. При определении значения этого флага учитывается длина операндов выполняемой операции. Логические операции (<, >, =, <>, >=, <=, AND, OR, XOR, NOT) безусловно устанавливают этот флаг в 0. Операции сдвига помещают в этот флаг значение последнего "выдвигаемого" разряда.

Замечание:

следует четко понимать, что значение флагов изменяют именно операции, а оператор присваивания никакого влияния на флаги не оказывает. Т.е при выполнении оператора RA:=RB; значения флагов изменены не будут (поскольку не выполнялась ни одна операция), а при выполнении оператора M[I+1]:=0; значения флагов могут быть изменены, так как при вычислении индекса выполняется операция +.

14. Общие замечания

При написании программ на языке MCL рекомендуется следующая структура построения программы:

```
{Описание переменных} VAR ... REG ... MEM ... STRUCT ...  
{Тело главной программы} ...  
END {Конец главной программы}  
SUB1: {Подпрограмма 1} ...  
RETURN { Конец подпрограммы 1}  
... {Определение других подпрограмм}  
SUBN: {Подпрограмма N} ...  
RETURN {Конец подпрограммы N}
```



То есть, сначала описываются все переменные. После описания переменных располагаются операторы основной программы, которые завершаются оператором END. А затем идут операторы используемых подпрограмм. Безусловно, такая структура построения программ не является догмой, но она является наиболее простой для восприятия, и способствует большей ясности программы.

15. Сообщения об ошибках интерпретатора языка MCL

В случае обнаружения ошибки, интерпретация программы прекращается и выдается сообщение вида:

```
! ERROR ! в строке номер: <номер строки программы в которой
обнаружена ошибка>
<строка программы в которой обнаружена ошибка>
<сообщение об ошибке>
```

Ниже приведен полный список сообщений об ошибках, распознаваемых интерпретатором языка MCL. При необходимости даются дополнительные пояснения.

Синтаксическая ошибка - неизвестный символ.

При интерпретации программы встретился неизвестный интерпретатору символ.

Слишком длинное имя.

Превышена максимальная длина идентификатора (16 символов).

Слишком длинное число.

Превышена максимальная длина числа (32 символа).

Нет закрывающей кавычки в строке.

В строке вывода оператора WRITE(LN) не найдена закрывающая кавычка. Строка вывода не может содержать символ перевода каретки.

Не найден конец комментария.

Не найден символ завершения комментария - '}'.

Слишком много меток.

В программе используется слишком много меток. Максимальное число меток, которое может быть использовано в программе - 100.

Повторное определение меток.

В программе используются метки с одинаковыми именами. Все метки должны иметь уникальные имена.

Неопределенная метка.

В программе используется ссылка на неопределенную метку.

Слишком много вложенных циклов.

В программе используется слишком глубокое вложение циклов. Максимальный уровень вложенности циклов - 16.

Слишком много вложенных вызовов ПП.

В программе используется слишком глубокое вложение вызовов подпрограмм. Максимальный уровень вложенности вызовов подпрограмм - 16.



Команда NEXT без команды FOR.

В программе нет начала цикла FOR.

Команда RETURN без команды GOSUB.

В программе есть возвращение из функции, но нет её объявления.

Слишком много переменных.

В программе используется слишком много переменных. Максимальное число переменных в программе - 100.

Повторное определение переменной.

Попытка описать переменную, имя которой совпадает с именем уже описанной переменной. Все переменные должны иметь уникальные имена.

Неопределенная переменная.

В программе используются обращения к неописанным переменным.

Неверный размер памяти.

При описании переменной типа "память" используется слишком большое значение числа ячеек. Максимально допустимое значение числа ячеек - 1024.

Неверная разрядность регистра.

При описании переменной типа регистр, структура или память используется слишком большое значение разрядности регистра. Максимально допустимое значение разрядности регистра - 32.

Неверный размер полей структуры.

При описании переменной типа структура суммарный размер полей структуры больше размера самой переменной.

Слишком большой индекс.

При обращении к переменной типа "память" значение индекса ячейки памяти превышает количество ячеек указанное при описании переменной.

Неверное использование индекса или имени поля.

Попытка использовать индекс или имя поля при обращении к простой переменной или регистру.

Неопределенное имя поля.

При обращении к переменной типа структура используется неопределенное имя поля.

Диапазон превышает размеры переменной.

При обращении к диапазону переменной указанный размер диапазона превышает размер переменной.

Некорректное выражение.

Слишком сложное выражение.

Слишком много операндов в выражении.

Непарная скобка.



Неверный тип управляющей переменной цикла.

Управляющая переменная цикла не может быть переменной типа "память".

Неверный спецификатор формата.

Допустимы значения B,D,H.

Неверная длина поля в спецификаторе формата.

Допустимы значения 1..32.

Требуется "THEN".

Требуется имя переменной памяти.

Требуется имя простой переменной (типа VAR,REG или STRUCT).

Требуется ".." или "," или "]"

Требуется "..".

Требуется ":=".

Требуется "TO".

Требуется "," или ";"

Требуется "["

Требуется "]"

Требуется идентификатор или команда.

Требуется ":" или ":=".

Требуется ";"

Требуется идентификатор.

Требуется константа.

Требуется индекс.



Арифметико-логические устройства ЭВМ и принципы их функционирования

Аннотация.

Приведен цикл лабораторных работ по изучению принципов функционирования арифметико-логических устройств. Особое внимание уделено представлению чисел с фиксированной запятой в прямом, обратном и дополнительном коде, способам определения переполнения, моделированию функционирования арифметико-логического устройства при выполнении операций сложения/вычитания, умножения и деления чисел с фиксированной запятой, рассмотрены примеры выполнения указанных арифметических операций. Работы рассчитаны на самостоятельное выполнение студентами. В конце каждой работы приводится перечень вариантов заданий.

Учебное пособие предназначено для студентов, обучающихся по специальностям, связанным с разработкой и применением вычислительных систем.



Лабораторная работа № 1. Моделирование функционирования АЛУ при выполнении операции сложения/вычитания над числами с фиксированной точкой

Содержание

Цель работы	23
Функционирование АЛУ при выполнении операции сложения/вычитания	23
Моделирование функционирования АЛУ при выполнении операции сложения с использованием обратного кода	23
Моделирование функционирования АЛУ при выполнении операции сложения с использованием дополнительного кода	24
Алгоритм алгебраического сложения/вычитания двоичных чисел	25
Пример выполнения операции сложения	28
Контрольные вопросы	32
Содержание отчета	32
Задание к лабораторной работе	32



Цель работы

Изучение работы арифметико-логического устройства при выполнении операции сложения/вычитания двоичных чисел с фиксированной запятой при использовании модифицированного кода для определения переполнения разрядной сетки. Операция сложения/вычитания выполняется в арифметико-логическом устройстве (АЛУ).

Операция сложения/вычитания двоичных чисел с фиксированной запятой может быть выполнена с использованием прямого, обратного и дополнительного кода для представления слагаемых.

Операция вычитания в ЭВМ заменяется операцией сложения при использовании обратного и дополнительного кода. Отрицательные числа представляются в обратном и дополнительном кодах.

Функционирование АЛУ при выполнении операции сложения/вычитания

Моделирование функционирования АЛУ при выполнении операции сложения с использованием обратного кода

Чтобы получить обратный код отрицательного двоичного числа, необходимо значащие разряды проинвертировать, а в знаковый разряд поставить 1. При выполнении алгебраического суммирования с использованием обратного кода положительные числа представляются в прямом коде, отрицательные числа – в обратном и производится суммирование кодов чисел, включая знаковые разряды. При возникновении переноса из знакового разряда она подсуммируется к младшему разряду результата. Если знаковый разряд результата равен 0, то результат положительный и представлен в прямом коде. Если знаковый разряд результата равен 1, то результат отрицателен и представлен в обратном коде. Рассмотрим на примере выполнение алгебраического суммирования чисел с фиксированной точкой с использованием обратного кода. В качестве слагаемых возьмем 5-разрядные числа (4 информационных разряда и 1 разряд – знаковый): $A=5_{10}$, $B=4_{10}$. В двоичной системе счисления прямой код числа: $A_{пр}=00101$, $B_{пр}=00100$, и обратный код числа: $A_{обр}=11010$, $B_{обр}=11011$.

Пример 1.

Рассмотрим операцию алгебраического сложения: $C=A+(-B)$.

В двоичной системе счисления положительные числа представляется в прямом коде, а отрицательные – в обратном. Следовательно, $A_{пр}=00101$ и $B_{обр}=11011$. Производим операцию сложения:

Зн.	Инф.
	разр.
0	0101
1	1011
1 0	0000
	1
0	0001

единица переноса прибавляется к младшему разряду

В знаковом разряде результата 0, следовательно, результат положительный и представлен в прямом коде.

Пример 2.

Рассмотрим операцию алгебраического сложения: $C=(-A)+B$.



В двоичной системе счисления положительные числа представляются в прямом коде, а отрицательные – в обратном. Следовательно, $A_{обр}=11010$ и $B_{пр}=00100$. Производим операцию сложения:

Зн.	Инф. разр.
1	1010
0	0100
1	1110

В знаковом разряде результата 1, следовательно, результат отрицательный и представлен в обратном коде. Прямой код результата $C_{пр}=10001_{(2)}=-1_{(10)}$.

Моделирование функционирования АЛУ при выполнении операции сложения с использованием дополнительного кода

Чтобы получить дополнительный код отрицательного двоичного числа, необходимо значащие разряды проинвертировать, к младшему разряду прибавить 1 и в знаковый разряд поставить 1. При выполнении алгебраического суммирования с использованием дополнительного кода положительные числа представляются в прямом коде, отрицательные числа – в дополнительном и производится суммирование кодов чисел, включая знаковые разряды. При возникновении 1 переноса из знакового разряда она отбрасывается. Если знаковый разряд результата равен 0, то результат положительный и представлен в прямом коде. Если знаковый разряд результата равен 1, то результат отрицателен и представлен в дополнительном коде.

Рассмотрим на примере выполнение алгебраического суммирования чисел с фиксированной точкой с использованием дополнительного кода. В качестве слагаемых возьмем 5-разрядные числа (4 информационных разряда и 1 разряд – знаковый): $A=5_{10}$, $B=4_{10}$. В двоичной системе счисления прямой код числа: $A_{пр}=00101$, $B_{пр}=00100$, и дополнительный код числа: $A_{доп}=11011$, $B_{доп}=11100$.

Пример 3.

Рассмотрим операцию алгебраического сложения: $C=A+(-B)$.

В двоичной системе счисления положительные числа представляются в прямом коде, а отрицательные – в дополнительном. Следовательно, $A_{пр}=00101$ и $B_{доп}=11100$. Производим операцию сложения:

Зн.	Инф. разр.
0	0101
1	1100
0	0001

единица переноса отбрасывается

В знаковом разряде результата 0, следовательно, результат положительный и представлен в прямом коде.

Пример 4.

Рассмотрим операцию алгебраического сложения: $C=(-A)+B$.

В двоичной системе счисления положительные числа представляются в прямом коде, а отрицательные – в дополнительном. Следовательно, $A_{доп}=11011$ и $B_{пр}=00100$. Производим операцию сложения:



Зн.	Инф. разр.
1	1011
0	0100
1	1111

В знаковом разряде результата 1, следовательно, результат отрицательный и представлен в дополнительном коде. Прямой код результата $C_{пр}=10001_{(2)}=-1_{(10)}$.

В ЭВМ отрицательные числа с фиксированной точкой в памяти хранятся либо в обратном, либо в дополнительном коде. В настоящее время наиболее часто используется дополнительный код, поэтому в лабораторной работе рассматривается алгоритм алгебраического сложения/вычитания двоичных чисел с фиксированной запятой при использовании дополнительного кода для представления слагаемых.

Алгоритм алгебраического сложения/вычитания двоичных чисел с фиксированной запятой при использовании дополнительного кода для представления слагаемых.

Алгоритм алгебраического сложения/вычитания двоичных чисел с фиксированной запятой при использовании дополнительного кода для представления слагаемых.

Выполнение операции вычитания сводится к выполнению операции сложения с отрицательным вторым слагаемым, поэтому рассмотрим алгоритм выполнения операции сложения. Если слагаемое положительно, то прямой код слагаемого совпадает с дополнительным кодом. Поэтому, при выполнении операции сложения положительные слагаемые представляются в прямом коде, а отрицательные – в дополнительном. Производится сложение двоичных кодов, включая разряды знаков. Для отслеживания переполнения существует 2 основных метода:

- по наличию и отсутствию переноса;
- использование модифицированного кодирования.

В первом случае, если возникает перенос из знакового разряда суммы при отсутствии переноса в этот разряд или перенос в знаковый разряд при отсутствии переноса из разряда знака, то имеется переполнение разрядной сетки. Если нет переносов из знакового разряда и в знаковый разряд суммы или есть оба переноса, то переполнения нет. При этом, если знаковый разряд суммы равен 0, то сумма положительна, если 1, то сумма отрицательна и представлена в дополнительном коде.

Во втором случае для представления знака отводится два или более разрядов. В случае использования двух знаковых разрядов комбинация 00 позволяет сделать вывод о том, что результат получился положительным, 11 – отрицательным, 10 или 01 – переполнение.

Ниже на примерах демонстрируется анализ переполнения по наличию и отсутствию переносов. В лабораторной работе для отслеживания переполнения должно быть использовано модифицированное кодирование.

Пример 5.

Рассмотрим пример операции сложения: $C=A+B$. В качестве слагаемых возьмем 5-разрядные числа (4 информационных разряда и 1 разряд – знаковый): $A=11$, $B=-4$. В двоичной системе счисления положительные числа представляются в прямом коде, а отрицательные – в дополнительном. Следовательно, $A=01011$ и $B=11100$. Производим операцию сложения:

Зн.	Инф. разр.
0	1011



1	1100
0	0111

↺ ↻ ↷

Переполнения разрядной сетки не было, так как был перенос и в знаковый разряд, и из знакового разряда.

Так как знак результата равен 0, то получаем положительный результат в прямом коде $C=00111_{(2)}=7_{(10)}$.

Пример 6.

Рассмотрим пример операции вычитания: $C=A-B$. В качестве уменьшаемого и вычитаемого возьмем 5-разрядные числа (4 информационных разряда и 1 разряд – знаковый): $A=11_{(10)}=01011_{(2)}$, $B=-6_{(10)}=11010_{(2)}$.

Для выполнения операции вычитания берем дополнительный код вычитаемого. Получаем, что $B=00110$. Производим операцию сложения:

Зн.	Инф. разр.
0	1011
0	0110
1	0001

↺ ↻ ↷

Так как был перенос в знаковый разряд, и не было переноса из знакового разряда, то произошло переполнение разрядной сетки.

Моделирование функционирования АЛУ.

Структурная схема АЛУ показана на рис. 5.

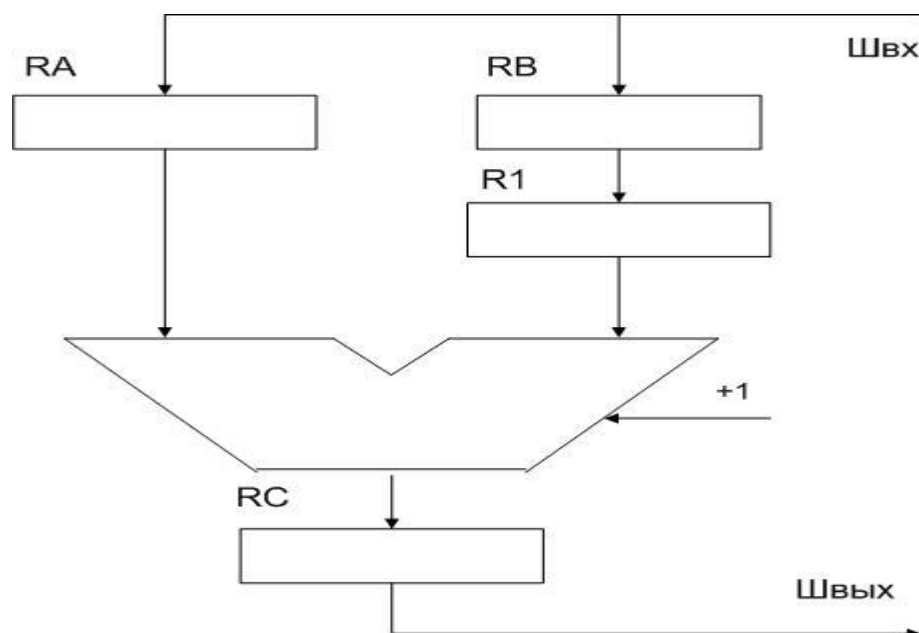


Рис. 5. Структурная схема АЛУ

Из оперативной памяти по входной информационной шине Швх в регистр RA поступает первое слагаемое/уменьшаемое A и в регистр RB – второе слагаемое/вычитаемое. В случае операции сложения второй операнд переписывается в регистр R1 без изменения. Если



выполняется операция вычитания, то операнд переписывается в регистр R1 в обратном коде. Далее, в случае операции сложения происходит суммирование операндов, а в случае операции вычитания – суммирование с добавлением 1. Результат операции записывается в регистр суммы RC и выдается из АЛУ в оперативную память по выходной информационной шине Швых.

Для того, чтобы отследить переполнение разрядной сетки, при выполнении операции сложения/вычитания используется модифицированный код. Под него отводится девять разрядов: семь информационных и два знаковых. После выполнения операции производится анализ знаковых разрядов, если их значения совпадают - переполнения нет, а если значения различны - было переполнение.

Таким образом, алгоритм выполнения операции сложения/вычитания заключается в следующем:

- 1) ввод операндов;
- 2) дешифрация кода операции;
- 3) в случае операции сложения: сложение операндов; в случае операции вычитания: представление вычитаемого в обратном коде и сложение операндов с добавлением 1;
- 4) проверка на переполнение разрядной сетки результата;
- 5) вывод результата.

Блок-схема выполнения операции сложения/вычитания показана на рис. 6.

В качестве Швх и Швых выступают входные восьмиразрядные регистры INPA и INPB, в которые вводятся операнды. Так как переполнение отслеживается с помощью модифицированных кодов, то регистры операндов RA и RB девятиразрядные.

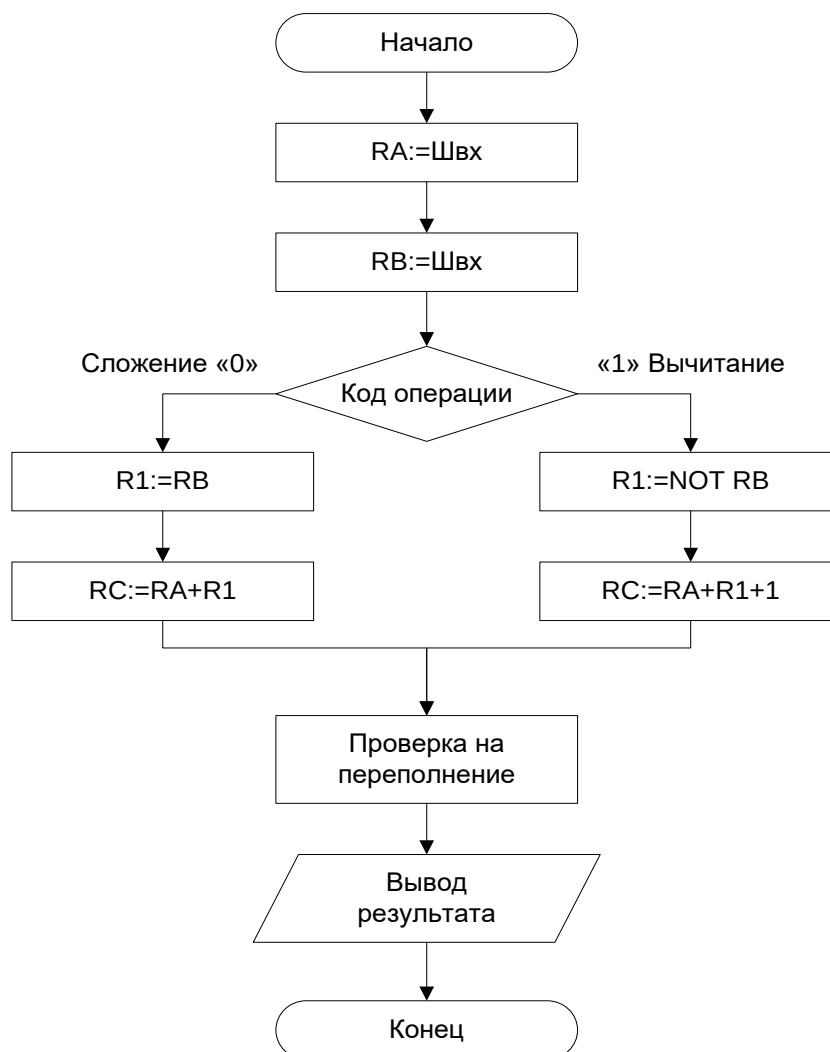


Рис. 6. Блок-схема выполнения операции сложения/вычитания



Пример микропрограммы выполнения операции сложения:

Текст программы:

```
writeln "*****";
writeln "*           Моделирование алгоритма сложения           *";
writeln "*****";

REG RA[9],RB[9],R1[9],RC[9],INPA[8],INPB[8],Z[9];

start:
    write "A=";read INPA;
    write "B=";read INPB;
    RA:=INPA; RA[8..8]:=INPA[7..7];
    RB:=INPB; RB[8..8]:=INPB[7..7];
    writeln "RA=", $B9 RA;
    writeln "RB=", $B9 RB;
    writeln "Сложение (A+B) :";
    R1:=RB;
    RC:=RA+R1;

    writeln "R1=", $B9 R1;
    writeln "RC=", $B9 RC;

    IF RC[8..8]<>RC[7..7] THEN
        writeln "!!! Переполнение разрядной сетки !!!";
    IF (RC[8..8]=RC[7..7]) AND (RC[8..8]=0) THEN
        writeln "Результат Z=", $B9 RC, "=", $D3 RC;
    IF (RC[8..8]=RC[7..7]) AND (RC[8..8]=1) THEN GOSUB WREZ;
writeln "***** Конеч программы *****";

END;

WREZ:
    Z:=COM(RC);
    Z[8..8]:=1;
    write "Результат Z= ", $B9 Z, "=-";
    Z[8..8]:=0;
    writeln $D3 Z;

RETURN;
```



Результат работы программы:

1. В случае получения положительного результата:

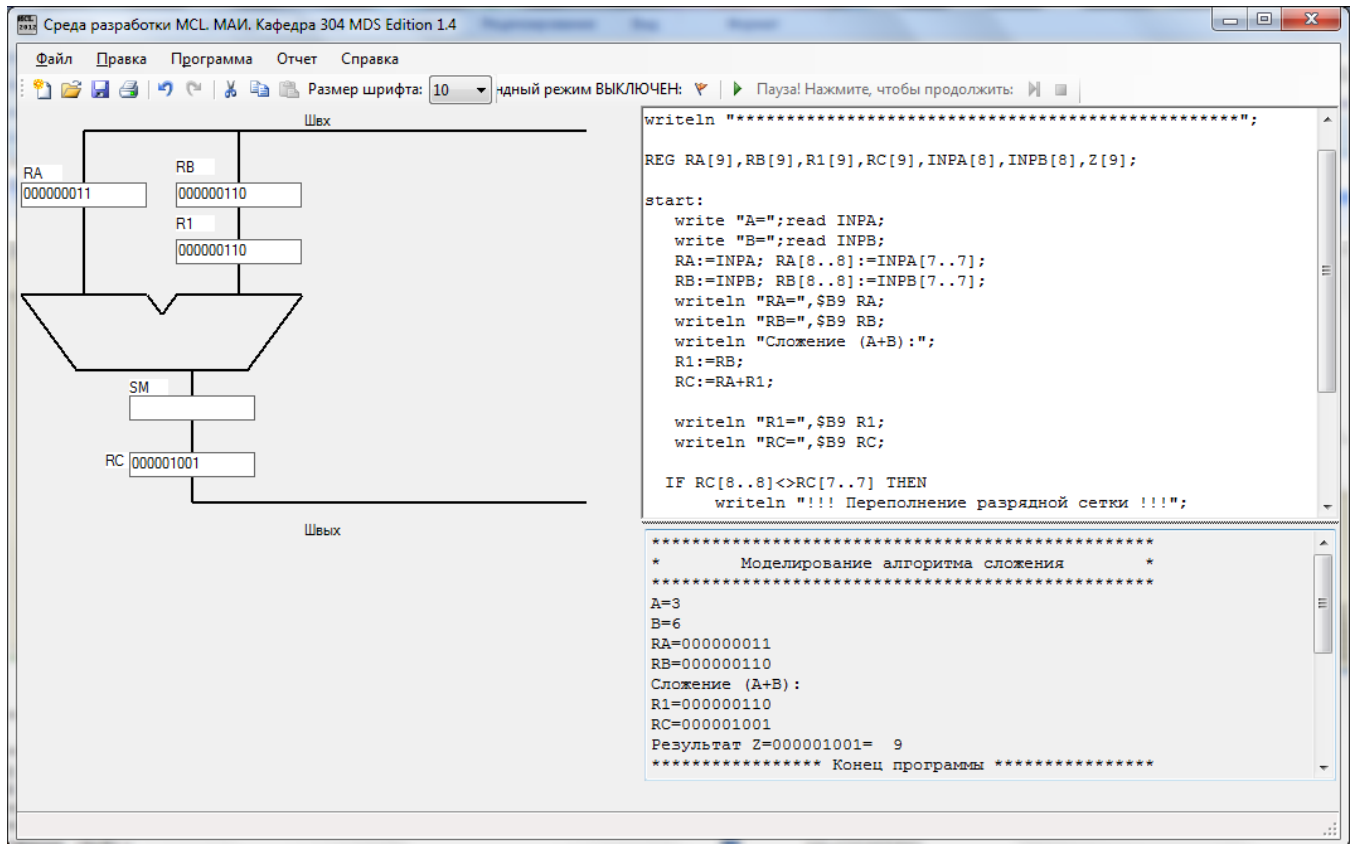


Рис. 7. Получение положительного результата

```
*****
*      Моделирование алгоритма сложения      *
*****
A=3
B=6
RA=000000011
RB=000000110
Сложение (A+B):
R1=000000110
RC=000001001
Результат Z=000001001= 9
***** Конеч программы *****

Содержимое таблицы переменных
00 Переменная: ZF типа регистр; разрядность 1 бит; значение = 1
01 Переменная: NF типа регистр; разрядность 1 бит; значение = 0
02 Переменная: CF типа регистр; разрядность 1 бит; значение = 0
03 Переменная: RA типа регистр; разрядность 9 бит; значение = 3
04 Переменная: RB типа регистр; разрядность 9 бит; значение = 6
05 Переменная: R1 типа регистр; разрядность 9 бит; значение = 6
06 Переменная: RC типа регистр; разрядность 9 бит; значение = 9
07 Переменная: INPA типа регистр; разрядность 8 бит; значение = 3
08 Переменная: INPB типа регистр; разрядность 8 бит; значение = 6
09 Переменная: Z типа регистр; разрядность 9 бит; значение = 0
```



2. В случае получения отрицательного результата:

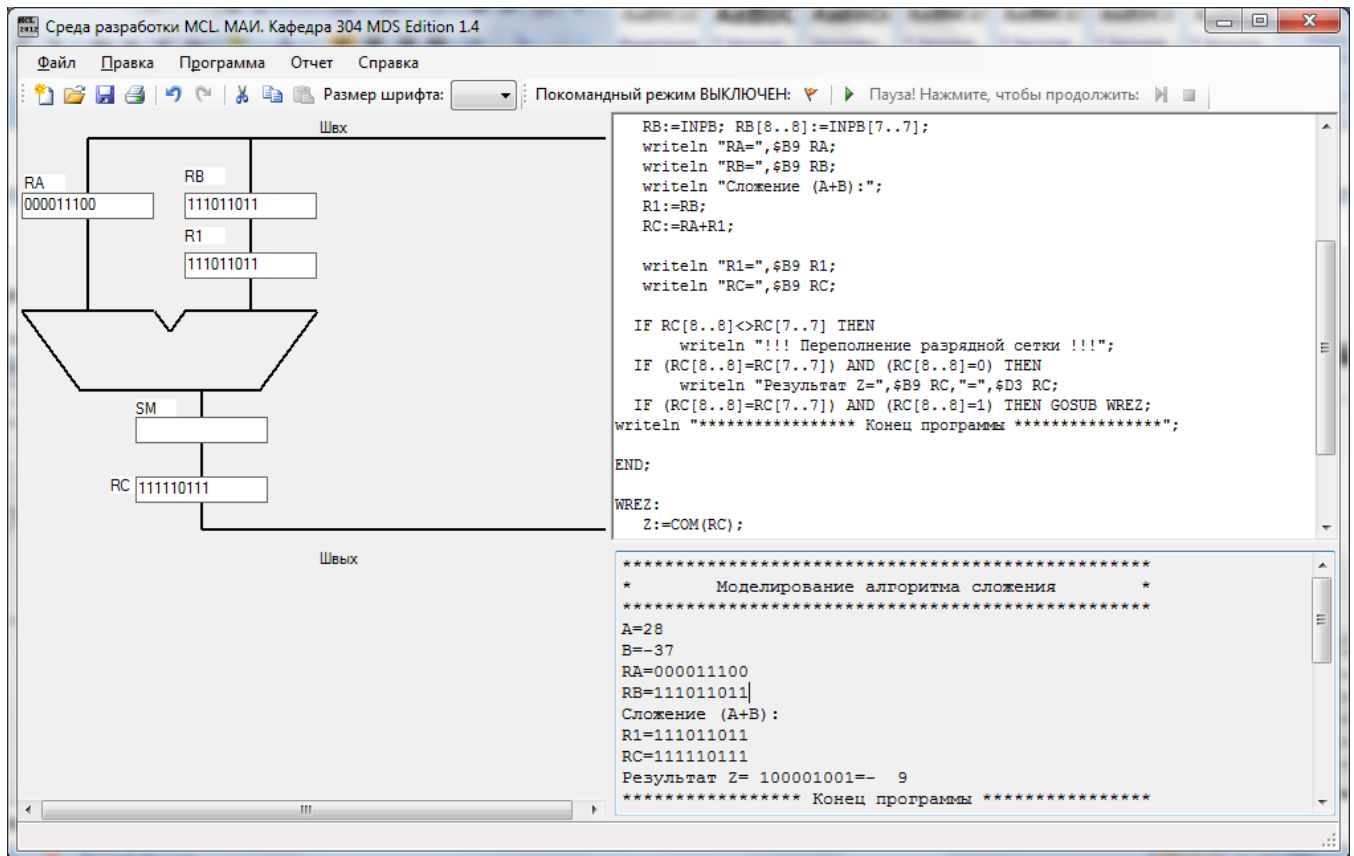


Рис. 8. Получение отрицательного результата

```

*****
*          Моделирование алгоритма сложения          *
*****
A=28
B=-37
RA=000011100
RB=111011011
Сложение (A+B):
R1=111011011
RC=111110111
Результат Z= 100001001=- 9
***** Конеч программы *****
  
```

Содержимое таблицы переменных

00	Переменная: ZF	типа регистр; разрядность 1 бит; значение = 0
01	Переменная: NF	типа регистр; разрядность 1 бит; значение = 0
02	Переменная: CF	типа регистр; разрядность 1 бит; значение = 0
03	Переменная: RA	типа регистр; разрядность 9 бит; значение = 28
04	Переменная: RB	типа регистр; разрядность 9 бит; значение = 475
05	Переменная: R1	типа регистр; разрядность 9 бит; значение = 475
06	Переменная: RC	типа регистр; разрядность 9 бит; значение = 503
07	Переменная: INPA	типа регистр; разрядность 8 бит; значение = 28
08	Переменная: INPB	типа регистр; разрядность 8 бит; значение = 4294967259
09	Переменная: Z	типа регистр; разрядность 9 бит; значение = 9



3. В случае переполнения:

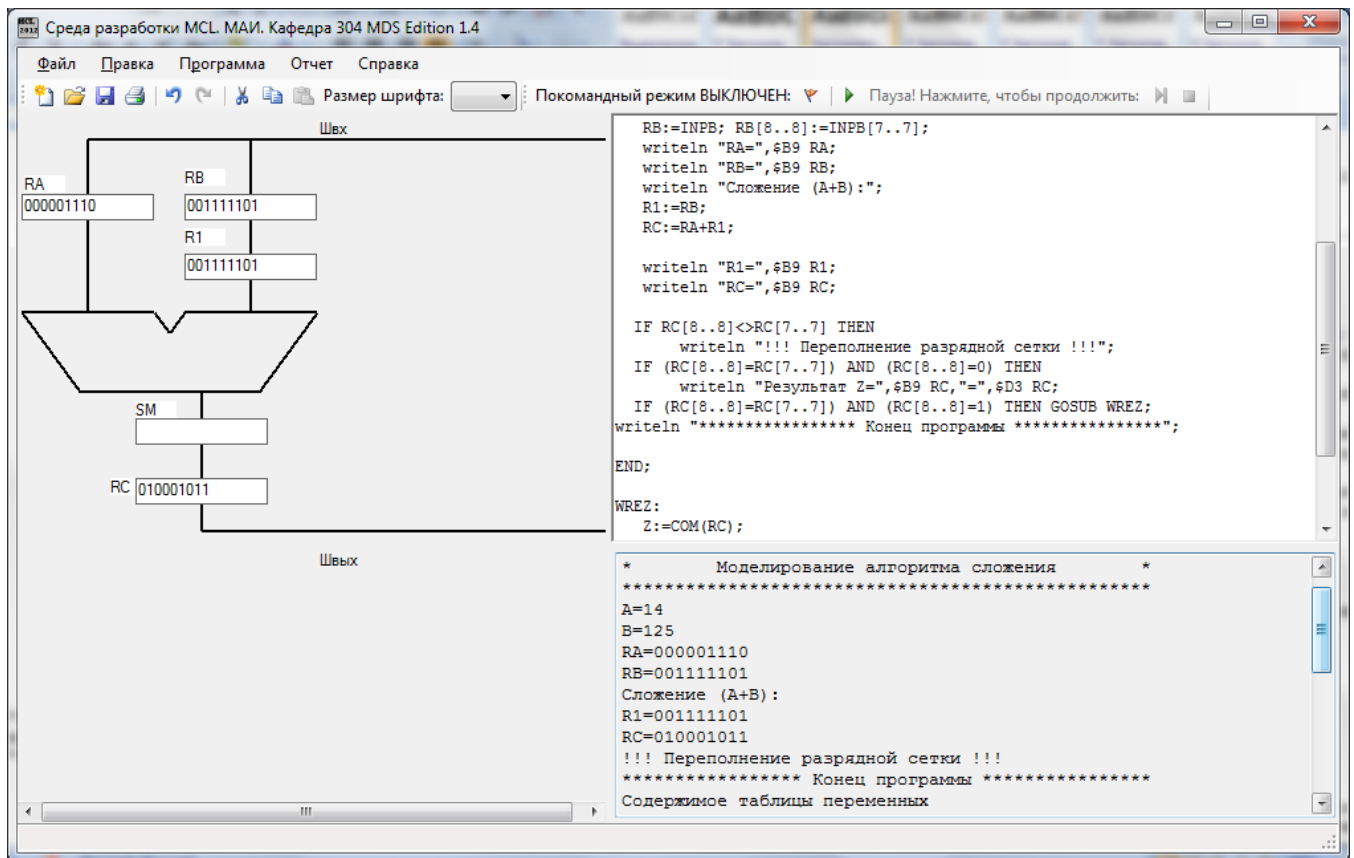


Рис. 9. Переполнение

```

*****
*          Моделирование алгоритма сложения          *
*****
A=14
B=125
RA=000001110
RB=001111101
Сложение (A+B):
R1=001111101
RC=010001011
!!! Переполнение разрядной сетки !!!
***** Конеч программы *****
Содержимое таблицы переменных
00 Переменная: ZF типа регистр; разрядность 1 бит; значение = 1
01 Переменная: NF типа регистр; разрядность 1 бит; значение = 0
02 Переменная: CF типа регистр; разрядность 1 бит; значение = 0
03 Переменная: RA типа регистр; разрядность 9 бит; значение = 14
04 Переменная: RB типа регистр; разрядность 9 бит; значение = 125
05 Переменная: R1 типа регистр; разрядность 9 бит; значение = 125
06 Переменная: RC типа регистр; разрядность 9 бит; значение = 139
07 Переменная: INPA типа регистр; разрядность 8 бит; значение = 14
08 Переменная: INPB типа регистр; разрядность 8 бит; значение = 125
09 Переменная: Z типа регистр; разрядность 9 бит; значение = 0
  
```



Контрольные вопросы

1. Особенности выполнения операции сложения/вычитания чисел с фиксированной запятой при использовании обратного и дополнительного кодирования.
2. Методы определения переполнения.
3. Состав и назначение основных регистров АЛУ для выполнения операции сложения/вычитания чисел с фиксированной запятой.

Содержание отчета

1. Формулировка задания.
2. Структурная схема АЛУ для выполнения операции сложения/вычитания чисел с фиксированной запятой при использовании дополнительного кода для представления слагаемых.
3. Блок-схема выполнения операции сложения/вычитания чисел с фиксированной запятой при использовании дополнительного кода для представления слагаемых.
4. Программа на языке MCL, моделирующая выполнение операции сложения/вычитания чисел с фиксированной запятой при использовании дополнительного кода для представления слагаемых.
5. Результаты выполнения программы в виде распечатки содержимого поля для вывода текстовой информации и скриншота (как показано в примере на рис. 7, 8, 9) в момент завершения выполнения программы.

Задание к лабораторной работе

Промоделировать работу АЛУ при выполнении команд сложения/вычитания в соответствии с вариантом задания. Варианты задания представлены в таблице 1. Рекомендуется использовать следующие обозначения:

I-код операции: 0 - сложение
1 - вычитание

INPA, INPB-регистры для ввода операндов;

RA-первый операнд АЛУ;

RB-второй операнд АЛУ;

R1-второй операнд в обратном коде;

RC-результат операции в АЛУ.

При моделировании использовать девятиразрядные регистры, т.к. используются модифицированные коды. Регистры INPA, INPB – восьмиразрядные. При моделировании должно быть отслежено переполнение (используются модифицированные коды). Вывести на печать содержимое всех регистров в двоичной системе счисления, исходные данные и результат – в двоичной и десятичной системе счисления.

Таблица 1. Варианты задания для моделирования операции сложения/вычитания

№ варианта	Первый операнд	Второй операнд	Операция
1	-25	40	+
	7	25	-
	75	59	+
2	-45	10	+
	5	45	-
	100	30	+
3	-30	7	+
	6	30	-
	89	60	+



4	22	-9	+
	-22	-8	-
	90	-38	+
5	73	-13	+
	-73	-10	-
	67	-93	+
6	46	-17	+
	-46	-23	-
	-57	-73	+
7	27	17	+
	10	27	-
	92	-43	-
8	70	19	+
	11	70	-
	94	-45	-
9	28	16	+
	14	28	-
	103	-34	-
10	68	17	+
	25	68	-
	98	-55	-
11	16	-70	+
	-16	17	-
	-105	70	-
12	26	-28	+
	-26	15	-
	-28	117	-
13	-24	40	+
	5	25	-
	65	63	+
14	-43	10	+
	6	45	-
	99	30	+
15	-32	7	+
	5	30	-
	88	60	+
16	20	-9	+
	-19	-8	-
	95	38	+
17	61	-13	+
	-63	-10	-
	-77	-93	+
18	46	-17	+
	-46	-23	-
	-57	-73	+
19	25	17	+
	11	27	-
	-87	-43	-
20	68	19	+
	10	70	-
	-92	-45	-



21	27 -13 -102	-16 28 -34	+ - -
22	58 23 -96	17 68 -55	+ - -
23	15 -15 -104	-70 17 70	+ - -
24	25 -25 -26	-28 15 -117	+ - -



Лабораторная работа № 2. Моделирование функционирования АЛУ при выполнении операции умножения над числами с фиксированной точкой

Содержание

Цель работы	36
Функционирование АЛУ при выполнении операции умножения	36
Моделирование функционирования АЛУ при выполнении операции умножения двоичных чисел, представленных в прямом коде	36
Моделирование функционирования АЛУ при выполнении операции умножения двоичных чисел, представленных в дополнительном коде	39
Алгоритм выполнения операции умножения двоичных чисел	40
Пример выполнения операции умножения	41
Контрольные вопросы	44
Содержание отчета	44
Задания к лабораторной работе	45



Цель работы

Изучение работы арифметико-логического устройства при выполнении операции умножения над числами с фиксированной запятой.

Функционирование АЛУ при выполнении операции умножения

Алгоритм умножения двоичных чисел с фиксированной запятой, представленных в прямом коде

В каждом цикле выполнения операции умножения анализируется очередная цифра множителя. Если очередная цифра множителя равна 1, то к сумме частичных произведений прибавляется множимое, в противном случае прибавляется нуль. Цикл завершается сдвигом множимого относительно суммы частичных произведений либо сдвигом суммы частичных произведений относительно неподвижного множимого. Таким образом, выполнение операции умножения в АЛУ сводится к последовательности операций сложения и сдвига.

В случае отрицательного операнда при умножении чисел, представленных в прямом коде, операция умножения сводится к выполнению следующих этапов:

- 1) определение знака произведения путем сложения по модулю 2 знаковых разрядов множимого и множителя;
- 2) обнуление знаковых разрядов отрицательных операндов;
- 3) выполнение операции умножения чисел, представленных в прямом коде.

Существует четыре способа умножения чисел с фиксированной точкой. При выполнении операции умножения можно сдвигать либо множимое, либо промежуточный результат и начинать анализ множителя либо с младших разрядов, либо со старших. Соответственно различают четыре структуры АЛУ для этой операции.

Рассмотрим указанные способы умножения чисел с фиксированной точкой.

1. Умножение чисел, представленных в прямом коде, начиная с младших разрядов множителя, со сдвигом суммы частичных произведений вправо и при неподвижном множимом.

Рассмотрим пример выполнения операции умножения: $C=A*B$, где $A=14_{(10)}=01110_{(2)}$, $B=4_{(10)}=00100_{(2)}$. При выполнении операции умножения будем сдвигать сумму частичных произведений вправо и анализировать разряды множителя, начиная с младшего разряда. Для простоты будем работать на 5-разрядной сетке (4 информационных разряда, 1 знаковый разряд).

Множитель				
Зн.	Информ. разряды			
4-й	3-й	2-й	1-й	0-й
0	0	1	0	0

Зн.	Информ. разряды	Описание действий
0 1110		Множимое (мн-ое)
0 0100		Множитель (мн-ль)
0 0000	->	Сумма частичных произведений (СЧП)
0 0000	0 ->	Сдвиг СЧП на 1 разряд вправо (0-й разряд множителя =0)
+0 0000	00	Сдвиг СЧП на 1 разряд вправо (1-й разряд множителя =0)
0 1110		Множимое
0 1110	00 ->	СЧП (2-й разряд множителя =1)
0 0111	000 ->	Сдвиг СЧП на 1 разряд вправо
0 0011	1000 ->	Сдвиг СЧП на 1 разряд вправо (3-й разряд множителя =0)
0 0001	11000	Сдвиг СЧП на 1 разряд вправо (4-й разряд множителя =0)



Знак результата формируется операцией сложения по модулю 2 знаков множимого и множителя: $Z=0\oplus 0=0$. Результат $C=0000111000_{(2)}=56_{(10)}$.

2. Умножение чисел, представленных в прямом коде, начиная с младших разрядов множителя, со сдвигом множимого влево и при неподвижной сумме частичных произведений.

Рассмотрим пример выполнения операции умножения: $C=A*B$, где $A=11_{(10)}=01011_{(2)}$, $B=-5_{(10)}=10101_{(2)}$. Числа представлены в прямом коде. При выполнении операции умножения будем сдвигать множимое влево, и анализировать разряды множителя, начиная с младшего разряда. Для простоты будем работать на 5-разрядной сетке (4 информационных разряда, 1 знаковый разряд).

Множитель				
Зн.	Информац. разряды			
4-й	3-й	2-й	1-й	0-й
0	0	1	0	1

Информ. разряды	Описание действий	
	01011	Множимое
	00101	Множитель
+	00000	Сумма частичных произведений, 0-ой разряд множителя =1
	01011	Множимое
+	01011	СЧП, Сдвиг мн-ого на 1 разряд влево, 1-й разряд множителя =0
0	0000	
+0	01011	СЧП, Сдвиг мн-ого на 1 разряд влево, 2-й разряд множителя =1
01	011	Сдвинутое множимое
+01	10111	СЧП, Сдвиг мн-ого на 1 разряд влево, 3-й разряд множителя =0
000	00	
+001	10111	СЧП, Сдвиг мн-ого на 1 разряд влево, 4-й разряд множителя =0
0000	0	
0001	10111	Результат операции умножения

Знак результата формируется операцией сложения по модулю 2 знаков множимого и множителя: $Z=0\oplus 1=1$. Результат $C=1000110111_{(2)}=-55_{(10)}$.

3. Умножение чисел, представленных в прямом коде, начиная со старших разрядов множителя, со сдвигом суммы частичных произведений влево и при неподвижном множимом.

Рассмотрим пример выполнения операции умножения: $C=A*B$, где $A=13_{(10)}=01101_{(2)}$, $B=9_{(10)}=01001_{(2)}$. При выполнении операции умножения будем сдвигать сумму частичных произведений влево, и анализировать разряды множителя, начиная со старшего разряда. Для простоты будем работать на 5-разрядной сетке (4 информационных разряда, 1 знаковый разряд).

Множитель				
Зн.	Информац. разряды			
4-й	3-й	2-й	1-й	0-й
0	1	1	0	1

Информ. разряды	Описание действий	
	01101	Множимое
	01001	Множитель
<	00000	СЧП, 4-ый разряд множителя =0
+0	00000	Сдвиг СЧП на 1 разряд влево, 3-й разряд множителя =1
	01101	Множимое
<0	01101	Сумма частичных произведений
+00	11010	Сдвиг СЧП на 1 разряд влево, 2-й разряд множителя =0
	00000	
<00	11010	Сумма частичных произведений
+001	10100	Сдвиг СЧП на 1 разряд влево, 1-й разряд множителя =0
	00000	



<.001	10100	Сумма частичных произведений
+0011	01000	Сдвиг СЧП на 1 разряд влево, 0-й разряд множителя =1
	01101	Множимое
0011	10101	Результат операции умножения

Знак результата формируется операцией сложения по модулю 2 знаков множимого и множителя: $Z=0\oplus 0=0$. Результат $C=0001110101_{(2)}=117_{(10)}$.

4. Умножение чисел, представленных в прямом коде, начиная со старших разрядов множителя, со сдвигом множимого вправо и при неподвижной сумме частичных произведений.

Рассмотрим пример выполнения операции умножения: $C=A*B$, где $A=11_{(10)}=01011_{(2)}$, $B=15_{(10)}=01111_{(2)}$. Числа представлены в прямом коде. При выполнении операции умножения будем сдвигать множимое вправо, и анализировать разряды множителя, начиная со старшего разряда. Для простоты будем работать на 5-разрядной сетке (4 информационных разряда, 1 знаковый разряд).

Множитель				
Зн.	Информац. разряды			
4-й	3-й	2-й	1-й	0-й
0	1	1	1	1

Зн.	Информ. разряды	Описание действий
0	1011	Множимое
0	1111	Множитель
0	0000	СЧП, 4-й разряд мн-ля =0, Сдвиг мн-го на 1р. вправо
+0	0000	СЧП, 3-й разряд мн-ля =1
0	0101 1	Мн-мое, сдвинутое на 1 разряд вправо
+0	0101 1	СЧП, 2-й разряд мн-ля =1
0	0010 11	Мн-мое, сдвинутое на 1 разряд вправо
+0	1000 01	СЧП, 1-й разряд мн-ля =1
0	0001 011	Мн-мое, сдвинутое на 1 разряд вправо
+0	1000 01	СЧП, 0-й разряд мн-ля =1
0	0000 1011	Мн-мое, сдвинутое на 1 разряд вправо
0	1010 0101	Результат операции умножения

Знак результата формируется операцией сложения по модулю 2 знаков множимого и множителя: $Z=0\oplus 0=0$. Результат $C=0010100101_{(2)}=165_{(10)}$.

Примем разрядность операндов n , тогда разрядность произведения будет $2n$.

Из сравнения структуры АЛУ для каждого рассмотренного метода умножения чисел, представленных в прямом коде, по таблице 2.1 видно, что:

1) если сдвигать множимое (методы 2 и 4), то аппаратные затраты составят:

- для множимого $2n$ разрядов;
- для множителя n разрядов;
- для произведения $2n$ разрядов;

2) если сдвигать сумму частичных произведений влево (метод 3), то для произведения аппаратные разряды составят $2n$ разрядов;

3) если сдвигать сумму частичных произведений вправо (метод 1), то вытесняемые разряды можно записывать в регистр множителя. При использовании этого способа размерность всех регистров может быть n разрядов. Это наиболее экономичный способ организации выполнения операции умножения. Он будет использоваться при моделировании.



Таблица 2.1. Сравнение структуры АЛУ для каждого метода умножения

Элементы структуры АЛУ	Методы умножения			
	1	2	3	4
Регистр множителя	Регистр со сдвигом вправо, n - разрядный, после выполнения операции умножения содержит мл. разряды произведения	Регистр со сдвигом вправо, n -разрядный	Регистр со сдвигом влево, n - разрядный	Регистр со сдвигом влево, n - разрядный
Регистр множимого	n -разрядный регистр	Регистр со сдвигом влево, $2n$ -разрядный	n -разрядный регистр	Регистр со сдвигом вправо, $2n$ -разрядный
Сумматор частичных произведений	Регистр со сдвигом вправо, n - разрядный, после выполнения операции умножения содержит ст. разряды произведения	$2n$ -разрядный регистр, после выполнения операции умножения содержит произведение	Регистр со сдвигом влево, $2n$ - разрядный, после выполнения операции умножения содержит произведение	$2n$ -разрядный регистр, после выполнения операции умножения содержит произведение

Алгоритм умножения двоичных чисел с фиксированной запятой, представленных в дополнительном коде

Рассмотрим алгоритм умножения двоичных чисел с фиксированной запятой, начиная с младших разрядов множителя, со сдвигом суммы частичных произведений и использованием дополнительного кода для отрицательных чисел.

Данный алгоритм умножения заключается в следующем:

1. Сомножители представляются в дополнительном коде. Так как дополнительный код положительного числа совпадает с прямым кодом, то положительные числа будут представлены в прямом коде.
2. Исходное значение суммы частичных произведений принимается равным 0.
3. Если анализируемая цифра множителя равна 1, то к сумме частичных произведений прибавляется множимое в том коде, в котором оно представлено. Прибавление производится с выравниванием складываемых чисел по старшим разрядам. Если анализируемая цифра множителя равна 0, прибавление не производится.
4. Сумма частичных произведений сдвигается на один разряд вправо, при этом, если сумма отрицательна, осуществляется модифицированный сдвиг.
5. Пункты 3 и 4 последовательно выполняются для всех цифровых разрядов множителя, начиная с младшего.
6. Если множитель – положительное число, полученный результат представляет собой произведение. Если множитель отрицателен, то для получения произведения к результату прибавляется множимое с обратным знаком. При этом прибавление производится с выравниванием складываемых чисел по старшим разрядам. Если результат размещается в двойном слове, то он предварительно сдвигается на один разряд вправо.
7. Произведение получается в прямом коде, если сомножители имеют одинаковые знаки, и в дополнительном, если сомножители имеют разные знаки.



Алгоритм выполнения операции умножения двоичных чисел

Моделирование функционирования АЛУ при выполнении операции умножения чисел, представленных в прямом коде

При выполнении операции умножения будем сдвигать сумму частичных произведений, и анализировать разряды множителя, начиная с младшего разряда.

Операнды в АЛУ записываются в прямом коде. Множимое хранится в регистре R1, множитель в регистре R2. Результат операции умножения будет формироваться в регистрах RC и R2, в RC - старшая часть, в R2 - младшая.

Анализ множителя начинается с младшего разряда, т.е. на каждом шаге анализируется нулевой разряд регистра R2 и после анализа содержимое этого регистра сдвигается вправо на один разряд. Если нулевой разряд регистра R2 равен 1, то к сумме частичных произведений прибавляется множимое, а если в нулевом разряде множителя стоит 0, то к сумме частичных произведений прибавляется 0. Структурная схема АЛУ представлена на рис. 10.

Знак произведения формируется в результате анализа знаков сомножителей. Если сомножители имеют одинаковые знаки, то произведение получается положительным, если знаки различны - произведение будет отрицательным. После определения знака результат знаковые разряды сомножителей обнуляются и умножение производится над положительными числами.

При выполнении операции умножения используется не модифицированный сдвиг, т.е. при сдвиге промежуточного произведения старший разряд заполняется нулем.

Число циклов выполнения операции умножения равно n . Это объясняется тем, что до этапа коррекции (после выполнения $(n-1)$ циклов) требуется сдвиг на 1 разряд вправо в связи с тем, что знаковый разряд обнуляется в начале выполнения операции. Этого можно достичь заноса в счетчик циклов n , а не $(n-1)$. Представление результата до и после этапа коррекции показано на рис. 11.

Блок-схема алгоритма операции умножения представлена на рис. 12.

Разрядность регистров АЛУ принимается n бит. Число циклов умножения равно разрядности операндов, т.е. n .

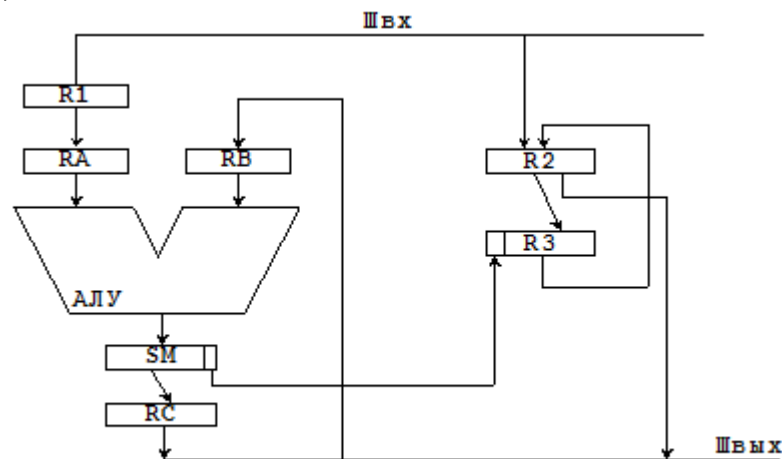
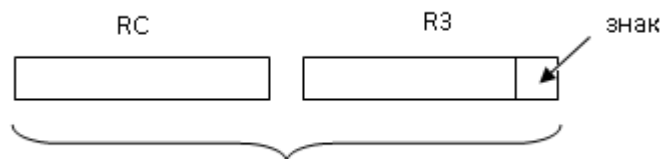


Рис. 10. Структурная схема АЛУ для выполнения операции умножения



Результат до этапа коррекции

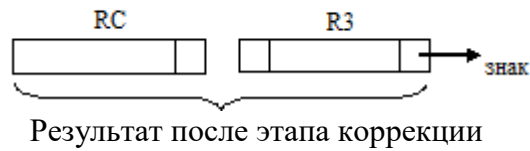


Рис. 11. Представление результата до и после этапа коррекции

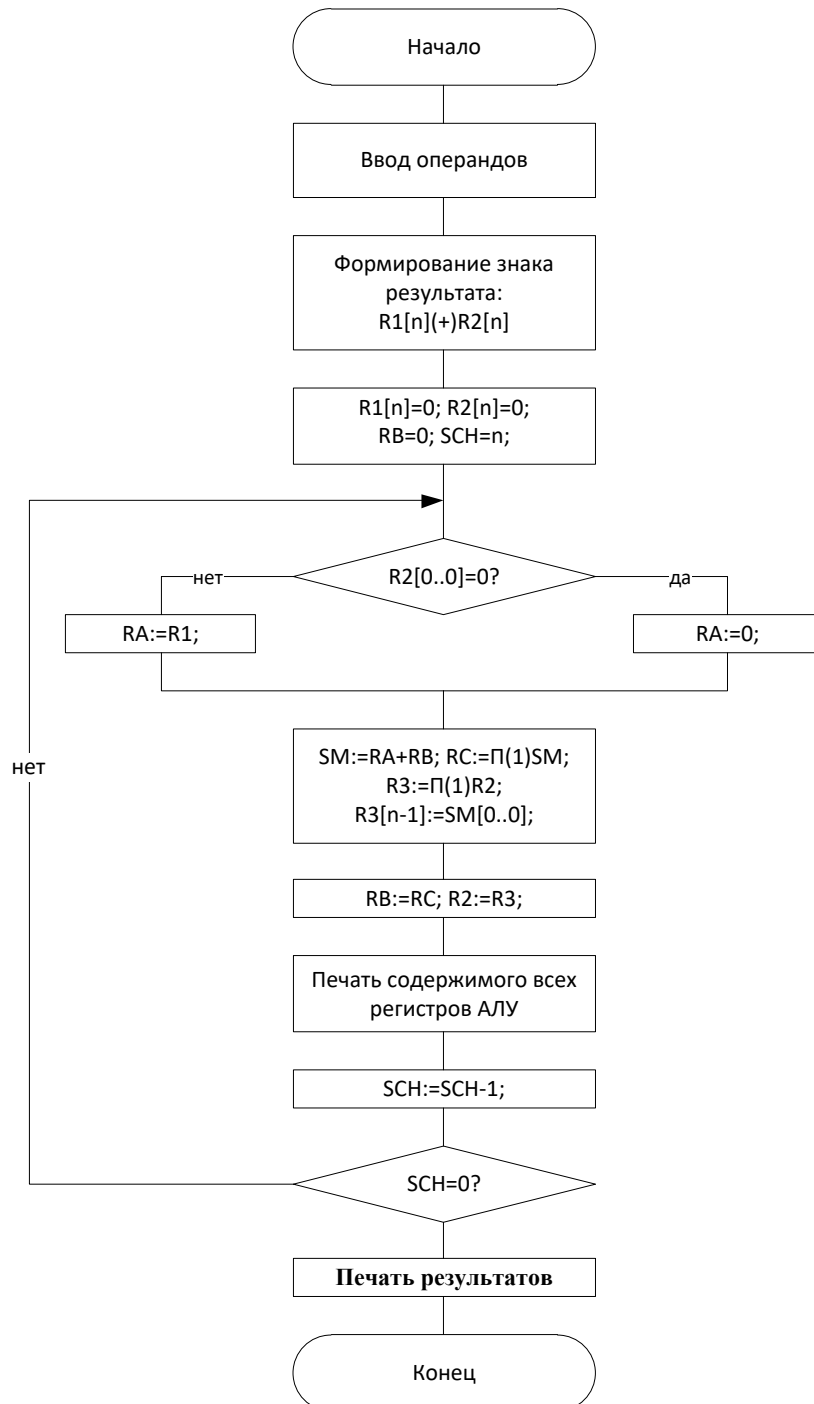


Рис. 12. Блок-схема алгоритма выполнения операции умножения

Пример выполнения операции умножения

Для наглядности проиллюстрируем выполнение операции умножения на микропрограммном уровне на примере.



Умножим 117 на 45. Введем операнды:

множимое $X=01110101=117$

множитель $Y=00101101=45$

Формируем знак результата. Т.к. знаковый разряд множимого равен знаковому разряду множителя, следовательно, результат положительный. После ввода множимое записывается в регистр R1, множитель в регистр R2. Число циклов равно 8.

Цикл 1.

Младший разряд множителя - 1.

В регистр RA переписывается содержимое R1.

$R1=01110101$, $RA=01110101$, $RB:=0$

Содержимое регистров RA и RB суммируется и записывается в регистр SM= 01110101

Регистр R2= 00101101 сдвигается вправо на один разряд и записывается в R3.

Регистр SM сдвигается вправо на один разряд и записывается в RC, вытесненный разряд записывается на место старшего разряда R3.

R3 переписывается в R2, RC в RB.

После сдвигов содержимое регистров равно:

$SM=01110101$

$RC=00111010$

$R2=00101101$ (в конце цикла $R2=10010110$)

$R3=10010110$

$RB=00111010$

Содержимое счетчика циклов уменьшаем на единицу SCH:=7

Проверяем счетчик циклов на равенство нулю. $SCH \neq 0$, следовательно, выполняем следующий цикл.

Цикл 2.

Младший разряд множителя - 0.

В RA записываем 0.

Суммируем.

$RA=00000000$

$RB=00111010$ (в конце цикла $RB=00011101$)

$SM=00111010$

После сдвигов содержимое регистров равно:

$SM=00111010$

$RC=00011101$

$R2=10010110$ (в конце цикла $R2=01001011$)

$R3=01001011$

Содержимое счетчика циклов уменьшаем на единицу SCH:=6

Проверяем счетчик циклов на равенство нулю. $SCH \neq 0$, следовательно, выполняем следующий цикл.

Цикл 3.

Младший разряд множителя - 1.

Суммируем.

$RA=01110101$

$RB=00011101$ (в конце цикла $RB=01001001$)

$SM=10010010$

После сдвигов содержимое регистров равно:

$SM=10010010$

$RC=01001001$

$R2=01001011$ (в конце цикла $R2=00100101$)



R3=00100101

Содержимое счетчика циклов уменьшаем на единицу SCH:=5

Проверяем счетчик циклов на равенство нулю. SCH ≠ 0, следовательно, выполняем следующий цикл.

Цикл 4.

Младший разряд множителя - 1.

Суммируем.

RA=01110101

RB=01001001 (в конце цикла RB=01011111)

SM=10111110

После сдвигов содержимое регистров равно:

SM=10111110

RC=01011111

R2=00100101 (в конце цикла R2=00010010)

R3=00010010

Содержимое счетчика циклов уменьшаем на единицу SCH:=4

Проверяем счетчик циклов на равенство нулю. SCH ≠ 0, следовательно, выполняем следующий цикл.

Цикл 5.

Младший разряд множителя - 0.

В RA записываем 0.

Суммируем.

RA=00000000

RB=01011111 (в конце цикла RB=00101111)

SM=01011111

После сдвигов содержимое регистров равно: SM=01011111

RC=00101111

R2=00010010 (в конце цикла R2=10001001)

R3=10001001

Содержимое счетчика циклов уменьшаем на единицу SCH:=3

Проверяем счетчик циклов на равенство нулю. SCH ≠ 0, следовательно, выполняем следующий цикл.

Цикл 6.

Младший разряд множителя - 1.

Суммируем.

RA=01110101

RB=00101111 (в конце цикла RB=01010010)

SM=10100100

После сдвигов содержимое регистров равно:

SM=10100100

RC=01010010

R2=10001001 (в конце цикла R2=01000100)

R3=01000100

Содержимое счетчика циклов уменьшаем на единицу SCH:=2

Проверяем счетчик циклов на равенство нулю. SCH ≠ 0, следовательно, выполняем следующий цикл.

Цикл 7.

Младший разряд множителя - 0.

В RA записываем 0.



Суммируем.

RA=00000000

RB=01010010 (в конце цикла RB=00101001)

SM=01010010

После сдвигов содержимое регистров равно:

SM=01010010

RC=00101001

R2=01000100 (в конце цикла R2=00100010)

R3=00100010

Содержимое счетчика циклов уменьшаем на единицу SCH:=1

Проверяем счетчик циклов на равенство нулю. SCH ≠ 0, следовательно, выполняем следующий цикл.

Цикл 8.

Младший разряд множителя - 0.

В RA записываем 0.

Суммируем.

RA=00000000

RB=00101001 (в конце цикла RB=00010100)

SM=00101001

После сдвигов содержимое регистров равно:

SM=00101001

RC=00010100

R2=00100010 (в конце цикла R2=00010001)

R3=10010001

Содержимое счетчика циклов уменьшаем на единицу SCH:=0

Счетчик циклов равен нулю.

В 16-ти разрядный регистр REZ записываем результат: из регистра R2 младшую часть, из RC старшую. Производим печать результата. Для этого обращаемся к подпрограмме печати чисел со знаком. Параметры подпрограммы:

P - печатаемое число; P=REZ;

PL - его разрядность; PL=16;

REZ=0001010010010001=5265.

Моделирование функционирования АЛУ при выполнении операции умножения чисел, представленных в дополнительном коде

При выполнении операции умножения чисел, представленных в дополнительном коде, будем сдвигать сумму частичных произведений и анализировать разряды множителя, начиная с младшего разряда. Для выполнения такого умножения можно использовать структурную схему, представленную на рис. 10.

Контрольные вопросы

1. Особенности выполнения операции умножения чисел с фиксированной запятой, представленных в прямом и дополнительном коде.
2. Способы умножения чисел с фиксированной запятой.
3. Состав и назначение основных регистров АЛУ для выполнения операции умножения чисел с фиксированной запятой, представленных в прямом коде.

Содержание отчета

1. Формулировка задания.
2. Структурная схема АЛУ для выполнения операции умножения чисел с фиксированной запятой, представленных в прямом коде.



3. Блок-схема выполнения операции умножения чисел с фиксированной запятой, представленных в прямом коде.
4. Программа на языке MCL, моделирующая выполнение операции умножения чисел с фиксированной запятой, представленных в прямом код.
5. Результаты выполнения программы в **виде распечатки** содержимого поля для вывода текстовой информации и **скриншота** в момент завершения выполнения программы.

Задания к лабораторной работе

Промоделировать работу АЛУ при выполнении операции умножения чисел, представленных в прямом коде в соответствии с номером варианта. Номера вариантов даны в таблице 2.2. Рекомендуется использовать следующие обозначения:

INPA, INPB – регистры для ввода операндов;

R1 – регистр для хранения множимого;

R2 – регистр для хранения множителя и формирования младшей части произведения;

RA – регистр для хранения первого операнда АЛУ (множимое, либо 0);

RB – регистр для хранения второго операнда АЛУ (сумма частичных произведений);

R3 – регистр сдвига множителя;

SM – регистр АЛУ суммы множимого и суммы частичных произведений;

RC – регистр для формирования старшей части произведения.

Разрядность регистров АЛУ принимается 8 бит. Число циклов умножения равно разрядности операндов, т.е. восьми. При выполнении лабораторной работы для наглядности, необходимо печатать содержимое всех регистров АЛУ на каждом такте умножения. Содержимое регистров должно быть представлено в двоичной форме, исходные значения операндов и результаты – в десятичной системе счисления.

Таблица 2.2. Варианты заданий для моделирования операции умножения

Номер варианта	Множимое	Множитель
1	-2	3
	125	13
	-15	-87
2	-2	4
	115	18
	-20	-92
3	-2	5
	105	23
	-26	-98
4	-2	6
	119	31
	-32	-64
5	-3	2
	95	110
	-18	-37
6	-3	3
	75	39
	-120	-13
7	-3	4
	118	30
	-27	-81
8	-4	2
	36	99
	-84	-32
9	-4	3



	24 -46	70 -28
10	-5 28 -32	2 35 -21
11	-6 79 -73	2 96 -36
12	-3 111 -7	5 85 -77
13	-2 125 -15	3 13 -87
14	-2 115 -20	4 18 -92
15	-2 105 -26	5 23 -98
16	-2 119 -32	6 31 -64
17	-3 95 -18	2 110 -37
18	-3 75 -120	3 39 -13
19	-3 118 -27	4 30 -81
20	-4 36 -84	2 99 -32
21	-4 24 -46	3 70 -28
22	-5 28 -32	2 35 -21
23	-6 79 -73	2 96 -36
24	-3 111 -7	5 85 -77



Лабораторная работа № 3. Моделирование функционирования АЛУ при выполнении операции деления над числами с фиксированной точкой

Содержание

Цель работы	48
Функционирование АЛУ при выполнении операции деления	48
Моделирование функционирования АЛУ при выполнении операции деления методом с восстановлением остатка над числами, представленными в прямом коде	48
Моделирование функционирования АЛУ при выполнении операции деления методом без восстановления остатка над числами, представленными в прямом коде	52
Контрольные вопросы	54
Содержание отчета	55
Задания к лабораторной работе	55



Цель работы

Изучение работы арифметико-логического устройства при выполнении операции деления над числами с фиксированной запятой, представленными в прямом коде.

Функционирование АЛУ при выполнении операции деления

Выполнение операции деления в АЛУ сводится к последовательности операций сложения и сдвига. Существующие способы деления можно разделить на две группы:

1. Выполнение операции деления методом с восстановлением остатка.
2. Выполнение операции деления методом без восстановлением остатка.

Алгоритмы деления аналогичны алгоритму деления при ручном счете. Рассмотрим особенности деления на примере деления целых чисел. Пусть $Z=X/Y$, где X – делимое, представляемое обычно двойным словом ($2n-1$ информационных разрядов и 1 знаковый разряд), Y – делитель и Z – частное, представляемое словом, содержащим $n-1$ информационных и 1 знаковый разряд. Пусть для простоты X и Y – целые числа, представленные в прямом коде.

Так как Z – слово, то должно выполняться равенство $|Z| < 2^{n-1}$, т.е. $(|X| - |Y| * 2^{n-1}) < 0$. Если неравенство не выполняется, то возникает переполнение разрядной сетки частного и результат получается неверным. Это условие корректного выполнения деления.

Таким образом, для проверки корректности деления надо сдвинуть делитель на $n-1$ разряд влево и вычесть его из делимого. Если результат окажется отрицательным, то деление возможно. Иначе возникнет переполнение разрядной сетки частного.



Таким образом выполнение операции деления начинается с пробного вычитания делителя, сдвинутого на $n-1$ разряд влево, из делимого. Если результат меньше нуля, то деление возможно. Если результат больше нуля, то деление невозможно. Знак частного формируется в результате анализа знаков операндов. Если операнды имеют одинаковые знаки, то частное получается положительным, если знаки различны - частное будет отрицательным. После определения знака результата знаковые разряды операндов обнуляются и деление производится над положительными числами.

Моделирование функционирования АЛУ при выполнении операции деления методом с восстановлением остатка над числами, представленными в прямом коде

Метод деления с восстановлением остатка состоит в следующем:

1. Из сдвинутого частичного остатка вычитается делитель. Если результат больше либо равен нулю, очередной разряд частного получает значение 1. Если результат меньше нуля, то разряд частного равен нулю и происходит восстановление промежуточного остатка путем прибавления к текущему остатку делителя.
2. Промежуточный остаток сдвигается влево на один разряд и в него заносится очередной разряд делимого. Цикл деления повторяется.

Рассмотрим пример деления целых чисел, представленных в прямом коде, методом с восстановлением остатка.

$$X=15, Y=-3; Z=X/Y;$$

	Знак	Информационные разряды							
X	0	0	0	0	0	0	1	1	1
Y	1	0	0	1	1				



Формируем знак результата путем сложения по модулю 2 знаков делимого и делителя:
 $ZN=0\oplus 1=1$. Обнуляем знаковый разряд делителя, получаем $Y=00011$.

Дополнительный код делителя: $[Y]_{\text{доп.код}}=11101$

Описание операции	Знак	Информационные разряды	
Делимое	+00	0000	1111
Сдвиг делителя влево на 4 разряда, вычитание делителя	1	1101	
Остаток от деления $< 0 \Rightarrow$ деление возможно	+1	1101	1111
Восстановление остатка: прибавление делителя	0	0011	
Восстановленный остаток	0	0000	1111
Сдвиг влево остатка на 1 разряд	+0	0001	1110
Вычитание делителя	1	1101	
1-ый остаток от деления < 0	+1	1110	1110
Восстановление остатка: прибавление делителя	0	0011	
Восстановленный остаток	0	0001	1110
Сдвиг влево остатка на 1 разряд	+0	0011	1100
Вычитание делителя	1	1101	
2-ой остаток от деления > 0	0	0000	1100
Сдвиг влево остатка на 1 разряд	+0	0001	1000
Вычитание делителя	1	1101	
3-ий остаток от деления < 0	+1	1110	1000
Восстановление остатка: прибавление делителя	0	0011	
Восстановленный остаток	0	0001	1000
Сдвиг влево остатка на 1 разряд	+0	0011	0000
Вычитание делителя	1	1101	
4-ый остаток от деления $= 0$	0	0000	0000

Z	1	0	1	0	1
	Знак	1 ост	2 ост	3 ост	4 ост

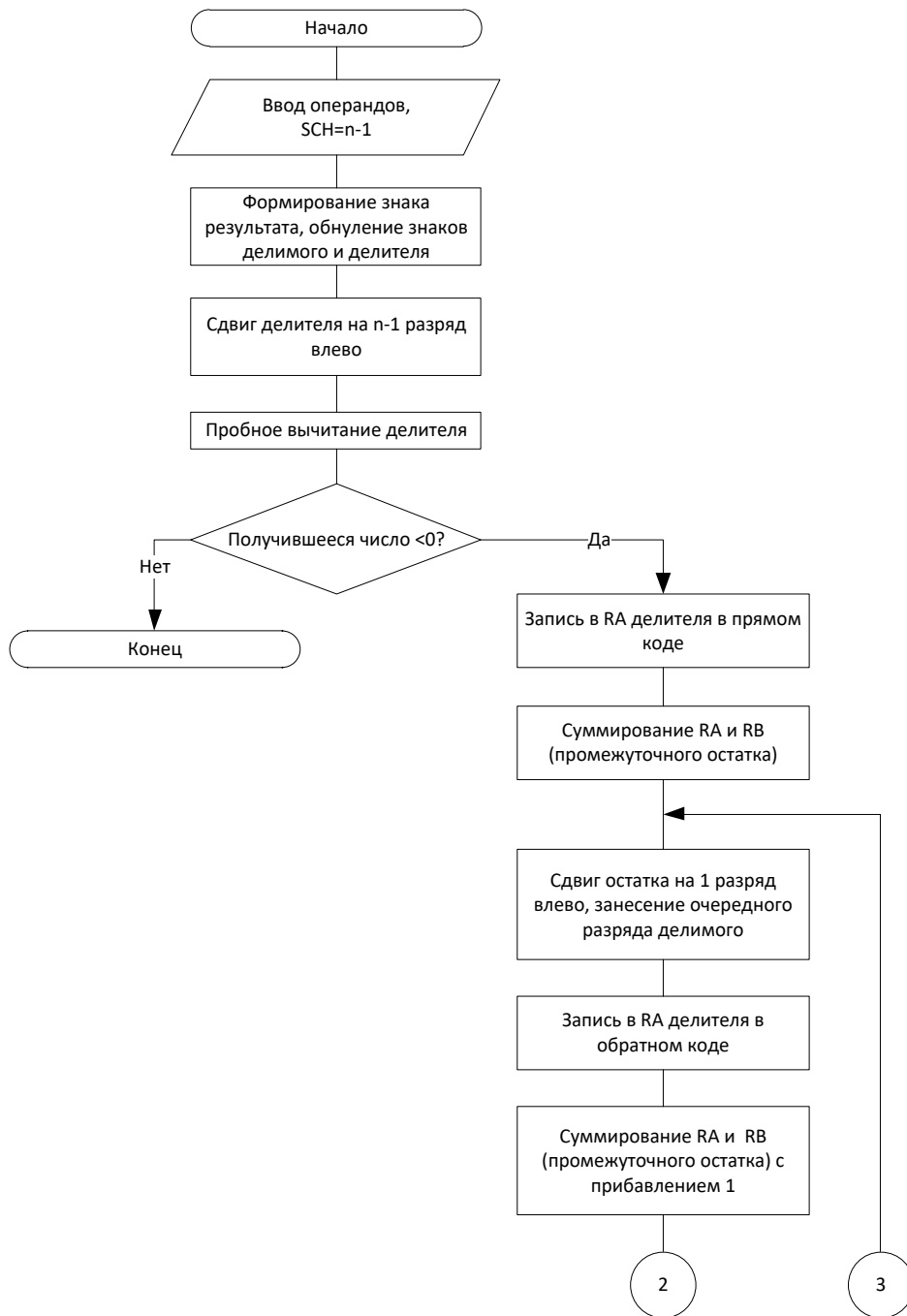
Выполнение операции деления с восстановлением остатка можно организовать по-другому, сдвигая делитель вправо на один разряд. Промежуточный остаток при этом остается на месте.

Структурная схема АЛУ представлена на рис. 14.

Делимое записывается в регистры RB и R2: в RB - старшая часть, в R2 - младшая.

Делитель записывается в регистр R1. Частное формируется в регистре R3.

Появляется связь между регистрами R2 и RC для сдвига промежуточного остатка и организован доступ к нулевому разряду регистра R3 для записи очередного разряда частного. Старшая и младшая части промежуточного остатка хранятся в регистрах RB и R2 соответственно. Если промежуточный остаток отрицательный, то делитель переписывается в регистр RA в прямом коде и в очередной разряд частного в регистр R3 записывается 0, если промежуточный остаток неотрицательный, то делитель переписывается в регистр RA в обратном коде и в очередной разряд частного в регистр R3 записывается 1. Регистры SM и R2 используются для организации сдвигов. При сдвиге промежуточного остатка в освободившийся разряд регистра RC заносится очередной разряд делимого. В каждом цикле результат с регистра RC переписывается на регистр RB. Триггер TS хранит знак результата, который формируется сложением по модулю 2 знаковых разрядов делимого и делителя. Так как деление производится над положительными числами в прямом коде, в случае отрицательных операндов, сначала формируется знак результата и заносится в триггер TS, операнды переводятся в прямой код и только тогда выполняется деление по уже известному алгоритму. Число циклов равно $n-1$, т.е. семи (без учета пробного вычитания).



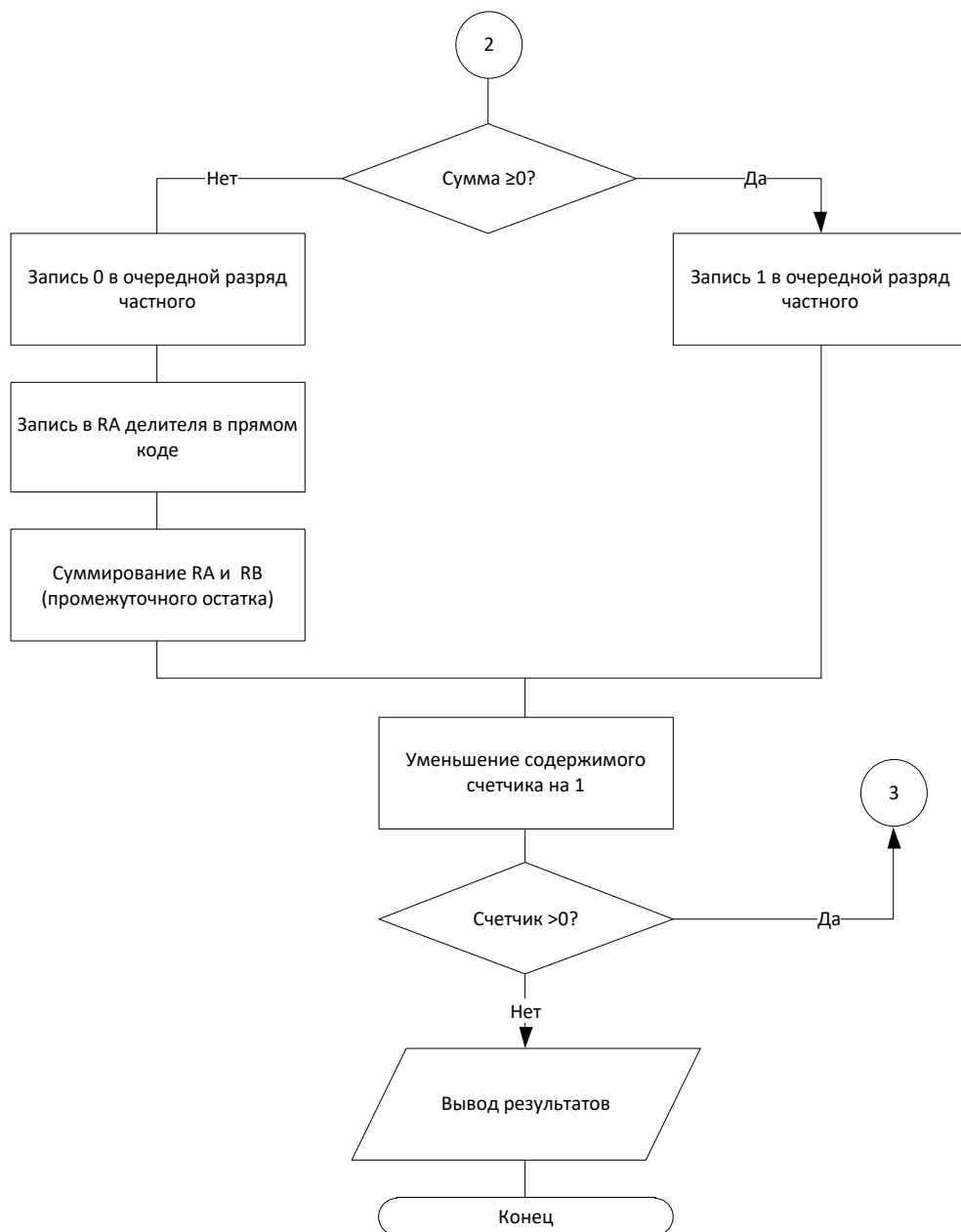


Рис. 13. Блок-схема выполнения операции деления с восстановлением остатка



Моделирование функционирования АЛУ при выполнении операции деления методом без восстановления остатка над числами, представленными в прямом коде.

Используя этот метод деления удастся избежать дополнительного сложения при восстановлении остатка. Суть метода в том, что после сдвига промежуточного остатка на один разряд влево, в зависимости от его знака вычитают или прибавляют делитель.

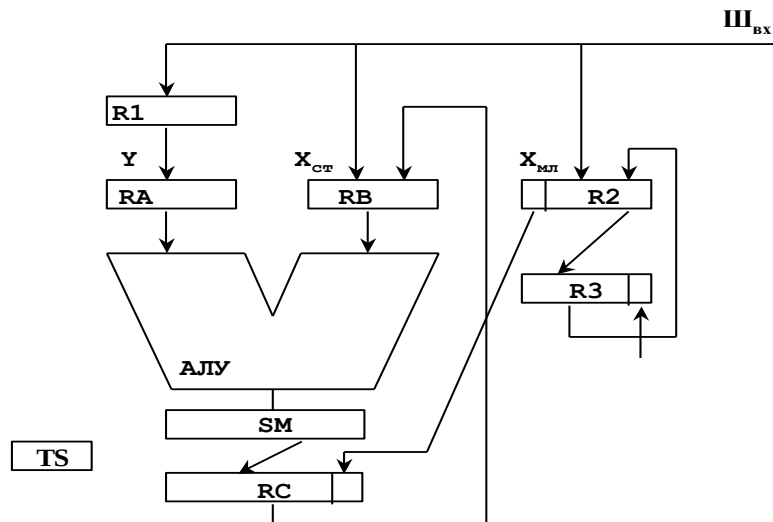


Рис. 14. Структурная схема АЛУ для операции деления

Предположим, что текущий остаток A больше или равен нулю, сдвигаем его влево на 1 разряд ($2A$) и вычитаем из него делитель d . Получаем $2A-d$. Выполненные действия аналогичны методу с восстановлением остатка. Если текущий остаток меньше нуля, в предыдущем методе восстанавливали остаток $A+d$, сдвигали его $2(A+d)$ и на следующем шаге вычитали делитель $2(A+d)-d=2A+d$. Но можно поступить и по-другому: при $A<0$ сдвинуть промежуточный остаток на 1 разряд влево ($2A$) и прибавить к нему делитель $2A+d$. Получим тот же результат, что и при восстановлении остатка.

Таким образом для выполнения операции деления методом без восстановления остатка необходимо выполнить следующие действия:

1. Анализ знака промежуточного остатка.
2. Если $A>0$, то в очередной разряд частного заносят значение 1 и вычитают делитель, если $A<0$, то в очередной разряд частного заносят значение 0 и прибавляют делитель.
3. Остаток сдвигают влево на один разряд.

Рассмотрим пример деления целых чисел, представленных в прямом коде, методом без восстановления остатка.

$$X=15, Y=-3; Z=X/Y;$$

	Знак	Информационные разряды							
X	0	0	0	0	0	0	1	1	1
Y	1	0	0	1	1				

Формируем знак результата путем сложения по модулю 2 знаков делимого и делителя: $ZN=0\oplus 1=1$. Обнуляем знаковый разряд делителя, получаем $Y=00011$.

Дополнительный код делителя: $[Y]_{\text{доп.код}}=11101$.

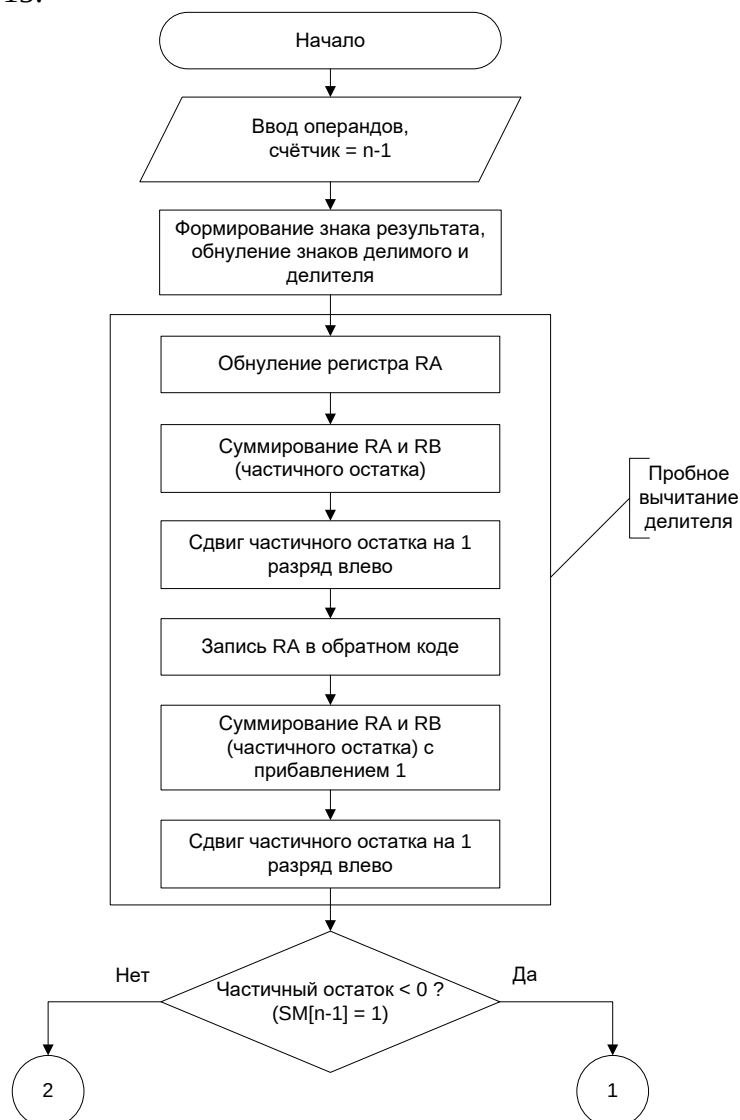


Описание операции	Знак	Информационные разряды	
Делимое	+00	0000	1111
Сдвиг делителя влево на 4 разряда, вычитание делителя	1	1101	
Остаток от деления $< 0 \Rightarrow$ деление возможно	1	1101	1111
Сдвиг влево остатка на 1 разряд	+1	1011	1110
Сложение с делителем	0	0011	
1-ый остаток от деления < 0	1	1110	1110
Сдвиг влево остатка на 1 разряд	+1	1101	1100
Сложение с делителем	0	0011	
2-ой остаток от деления > 0	0	0000	1100
Сдвиг влево остатка на 1 разряд	+0	0001	1000
Вычитание делителя	1	1101	
3-ий остаток от деления < 0	1	1110	1000
Сдвиг влево остатка на 1 разряд	+1	1101	0000
Сложение с делителем	0	0011	
4-ый остаток от деления $= 0$	0	0000	0000

Z	1	0	1	0	1
	Знак	1 ост	2 ост	3 ост	4 ост

Структурная схема АЛУ для операции деления без восстановления остатка аналогична структурной схеме АЛУ при выполнении операции деления с восстановлением остатка.

Блок-схема выполнения операции деления методом без восстановления остатка представлена на рис. 15.



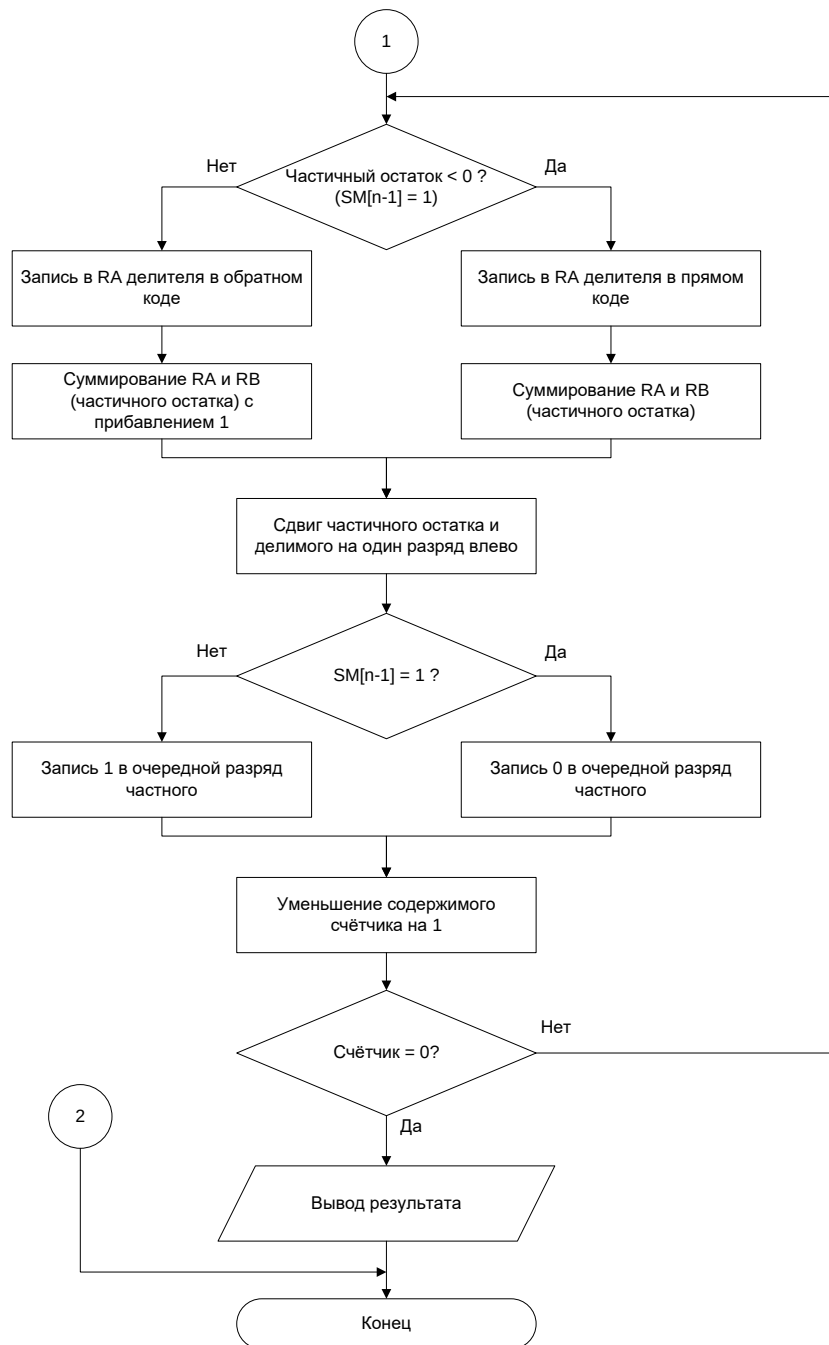


Рис. 15. Блок-схема выполнения операции деления без восстановления остатка

Примечание:

При выполнении лабораторной работы рекомендуется использовать функции сдвига SHL и SHR.

Контрольные вопросы

1. Особенности выполнения операции деления чисел с фиксированной запятой, представленных в прямом коде.
2. Способы деления чисел с фиксированной запятой.
3. Состав и назначение основных регистров АЛУ для выполнения операции деления чисел с фиксированной запятой, представленных в прямом коде.



Содержание отчета

1. Формулировка задания.
2. Структурная схема АЛУ для выполнения операции деления чисел с фиксированной запятой, представленных в прямом коде.
3. Блок-схема выполнения операции деления чисел с фиксированной запятой, представленных в прямом коде.
4. Программа на языке MCL, моделирующая выполнение операции деления чисел с фиксированной запятой, представленных в прямом код.
5. Результаты выполнения программы в **виде распечатки** содержимого поля для вывода текстовой информации **и скриншота** в момент завершения выполнения программы.

Задания к лабораторной работе

Промоделировать работу АЛУ в системе MCL при выполнении операции деления в соответствии с номером варианта. Номера вариантов даны в таблице 3. При написании программы рекомендуется использовать следующие обозначения:

TS – знак операции деления;

SCH – счетчик циклов;

R1 – регистр делителя;

R2, RB – регистр делимого;

R3, SM – регистры АЛУ для организации сдвига;

RC – регистр суммы АЛУ;

RA – регистр для формирования дополнительного и прямого кода делителя;

INP1, INP2 – входные регистры.

Разрядность регистров АЛУ принимается 8 бит. Число циклов деления равно разрядности операндов, т.е. восьми. При выполнении лабораторной работы для наглядности, необходимо печатать содержимое всех регистров АЛУ на каждом такте деления. Содержимое регистров должно быть представлено в двоичной форме, исходные значения операндов и результаты – в десятичной системе счисления.

Таблица 3. Варианты заданий для моделирования операции деления

Номер варианта	Делимое	Делитель
1	20	4
	-2322	9
	1980	-36
	-1152	-96
2	18	3
	-1701	6
	345	-15
	-1342	-23
3	12	4
	-1890	7
	1872	-36
	-833	-17
4	14	2
	-999	3
	720	-45
	-455	-13
5	24	4
	-1176	4
	935	-11



	-2625	-75
6	24 -2210 1368 -1012	8 8 -19 -23
7	32 -1768 2120 -1212	4 6 -40 -101
8	20 -2814 1701 -1363	5 10 -21 -29
9	18 -1089 3000 -720	6 4 -40 -24
10	20 -1008 3920 -1632	2 3 -49 -17
11	10 -1582 1290 -2997	5 6 -15 -81
12	12 -728 1288 -1593	2 2 -23 -27
13	15 -1432 980 -152	5 5 -36 -96
14	14 -1801 345 -1342	7 6 -15 -23
15	18 -1890 1872 -833	9 7 -36 -17
16	18 -1999 720 -455	2 7 -45 -13
17	36 -1200 935 -2625	9 4 -11 -75
18	21	7



	-2565 1368 -1012	10 -19 -23
19	18 -1878 2120 -1212	9 7 -40 -101
20	9 -2345 1701 -1363	3 9 -21 -29
21	21 -1190 3000 -720	3 45 -40 -24
22	22 -1248 3920 -1632	11 4 -49 -17
23	22 -1792 1290 -2997	2 7 -15 -81
24	24 -538 1288 -1593	3 2 -23 -27



Лабораторная работа № 4. Моделирование функционирования операционной части центрального процессора

Содержание

Цель работы	59
Функционирование операционной части центрального процессора	59
Центральный процессор	59
Команды ЭВМ	59
4-х адресные команды	60
3-х адресные команды	61
2-х адресные команды	62
Одноадресные команды	62
Безадресные команды	62
Способы адресации	62
1)Прямая адресация	63
2)Непосредственная адресация	63
3)Косвенная адресация	63
4)Регистровая адресация	63
5)Базовая адресация	65
6)Индексная адресация	66
7)Базово – индексная адресация	66
8)Косвенно – регистровая адресация	68
Пример моделирования операционной части ЦП	68
Содержание отчета	71
Задание к лабораторной работе	71



Цель работы

Изучение функционирования операционной части центрального процессора при выполнении команд с различными способами адресации.

Функционирование операционной части центрального процессора

Центральный процессор

Центральный процессор (ЦП) – это устройство, которое предназначается для непосредственной обработки данных и работает под управлением программ.

Основные узлы центрального процессора:

- 1) Арифметико-логическое устройство (АЛУ). Предназначается для выполнения арифметических и логических операций над данными. В АЛУ выполняются операции с фиксированной точкой, с плавающей точкой, операции двоично – десятичной арифметики, логические операции, обработка алфавитно-цифровой информации.
- 2) Устройство управления (УУ) – предназначается для выработки управляющих сигналов, под воздействием которых выполняются машинные команды.

Основные этапы выполнения машинной команды.

1 этап: выбор машинной команды из памяти.

2 этап: дешифрация кода операции.

3 этап: формирование исполнительного адреса и выбор операндов.

4 этап: непосредственное выполнение операции в АЛУ.

5 этап: запись результата.

- 3) Регистры общего назначения (РОНы). РОНы - это программно-адресуемые регистры, предназначаются для хранения операндов, результатов, а также для хранения индекса, базы, используются при вычислении адреса.
- 4) Управляющие регистры.
Счетчик команд (СК) – хранит адрес следующей выполняемой команды.
Регистр команд (РК) – хранит текущую выполняемую команду.
- 5) Вспомогательные блоки – к ним относится блок прерывания, блок связи центрального процессора (ЦП) и оперативной памяти (ОП), блок контроля и диагностики и т.д.

Центральный процессор (ЦП) состоит из: операционной части и устройства управления (УУ), где операционная часть центрального процессора включает в себя регистры, счетчики, сумматоры, дешифраторы и связи между ними и предназначена для выполнения микропрограмм.

Устройство управления служит для выработки управляющих сигналов, с помощью которых выполняется пересылка информации в операционной части. Другими словами УУ координирует работу всех устройств.

Организация операционной части центрального процессора

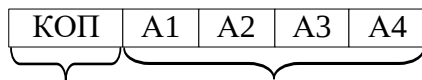
Команды ЭВМ

Машинная команда – это двоичный код, который включает в себя операционную часть и адресную часть. В адресной части содержится информация об адресах операндов и результатов. Различаются следующие команды:

- 4-х адресные,
- 3-х адресные,
- 2-х адресные,
- одноадресные,
- безадресные.



4-х адресные команды



Операционная часть Адресная часть

Поле A1 – информация об адресе первого операнда.

Поле A2 – информация об адресе второго операнда.

Поле A3 – информация об адресе результата.

Поле A4 – информация об адресе следующей выполняемой команды.

Микропрограмма и структурная схема операционной части ЦП при выполнении 4-х адресной команды.

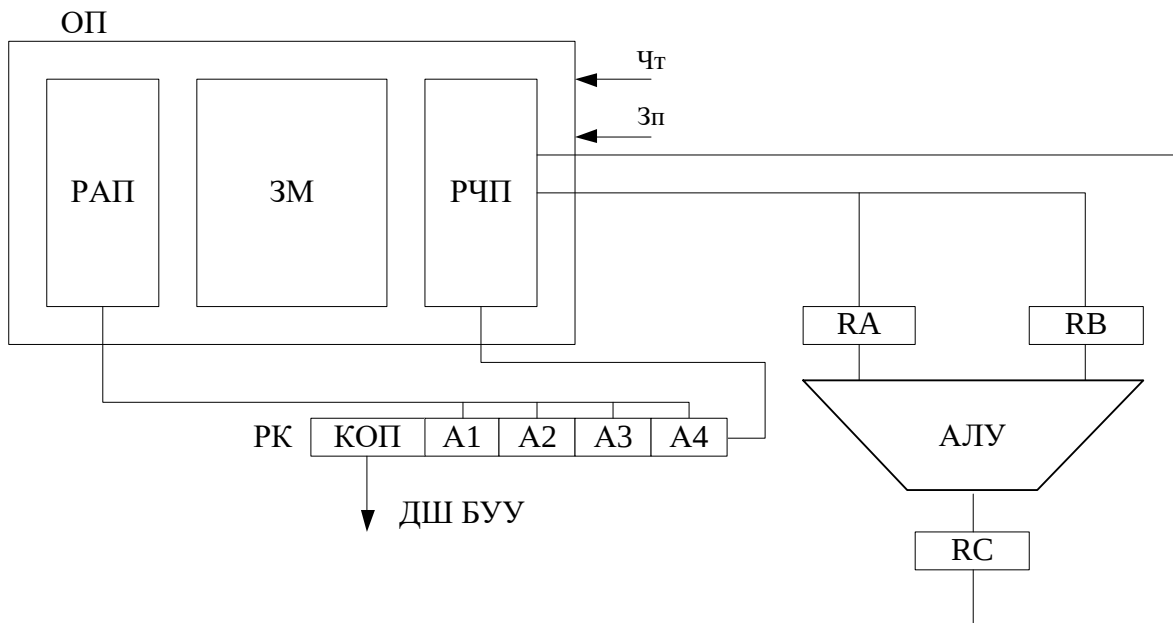


Рис. 16. Структурная схема операционной части ЦП при выполнении 4-х адресной команды

ЗМ – запоминающий массив.

РАП – регистр адреса памяти.

РЧП – регистр числа памяти.

ПК – регистр команд, предназначается для хранения текущей выполняемой команды.

АЛУ – арифметико-логическое устройство.

РА и RB - входные регистры АЛУ, предназначаются для хранения операндов.

RC - выходной регистр, предназначается для хранения результата выполненной операции в АЛУ.

1 этап. Выбор команды из памяти:

РАП:=ПК(A4) - адрес следующей выполняемой команды подается в память на регистр адреса памяти.

РЧП:=Чт(РАП) - непосредственный выбор самой команды из ЗМ

ПК:=РЧП - пересылается команда из памяти на ПК.

2 этап. Дешифрация кода операции.

3 этап. Формирование исполнительного адреса и выбор операндов:

Выбор 1-го операнда { РАП:=ПК(A1) - адрес первого операнда подается в ОП на РАП
РЧП:=Чт(РАП) - чтение из ЗМ на РЧП первого операнда
РА:=РЧП - первый операнд подается на входной регистр АЛУ



Выбор 2-го операнда $\left\{ \begin{array}{l} \text{РАП} := \text{РК}(\text{А2}) - \text{адрес второго операнда подается в ОП на РАП} \\ \text{РЧП} := \text{Чт}(\text{РАП}) - \text{чтение из ЗМ на РЧП второго операнда} \\ \text{RB} := \text{РЧП} - \text{второй операнд подается на входной регистр АЛУ} \end{array} \right.$

4 этап. Выполнение операции в АЛУ:

$\text{RC} := \text{РА}$ операция RB

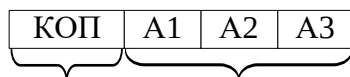
5 этап. Запись результата:

$\text{РАП} := \text{РК}(\text{А3})$ - адрес, по которому необходимо записать результат, подается в ОП на РАП

$\text{РЧП} := \text{RC}$ - результат подается в ОП на РЧП

$\text{Зп}(\text{РАП}) := \text{РЧП}$ - результат записывается в ЗМ

3-х адресные команды



Операционная часть Адресная часть

В 3-х адресной команде отсутствует А4. При использовании 3-х адресной команды добавляется дополнительное устройство – счетчик команд и дополнительно появляются команды перехода.

Микропрограмма и структурная схема операционной части ЦП при выполнении 3-х адресной команды.

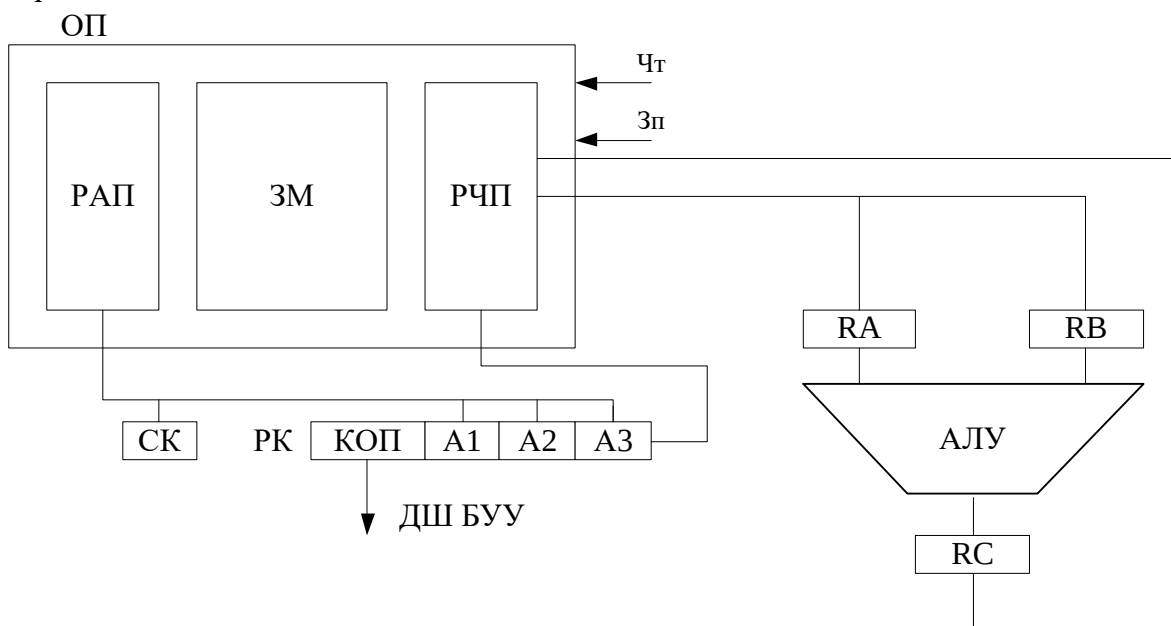


Рис. 17. Структурная схема операционной части ЦП при выполнении 3-х адресной команды

СК – счетчик команд, хранит адрес следующей команды.

1 этап. Выбор команды из памяти:

$\text{РАП} := \text{СК}$

$\text{РЧП} := \text{Чт}(\text{РАП})$

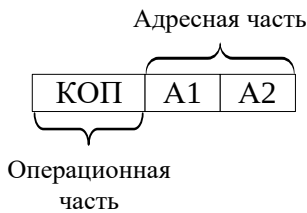
$\text{РК} := \text{РЧП}$

$\text{СК} := \text{СК} + n$ {n-длина команды}

Этапы 2,3,4,5 аналогично 4-х адресные команды.

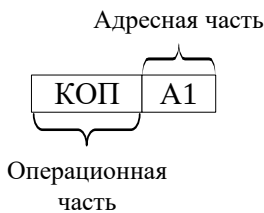


2-х адресные команды



Поле A3 – отсутствует. Результат записывается на место 1-го или 2-го операнда. При этом операнд затирается и если его необходимо использовать в дальнейшем, он предварительно должен быть сохранен.

Одноадресные команды



Есть только поле A1. В структуру ЦП вводится дополнительный регистр аккумулятора (РА), на который предварительно дополнительной командой засылается второй операнд. Результат записывается на место либо первого операнда, либо в аккумулятор.

Безадресные команды

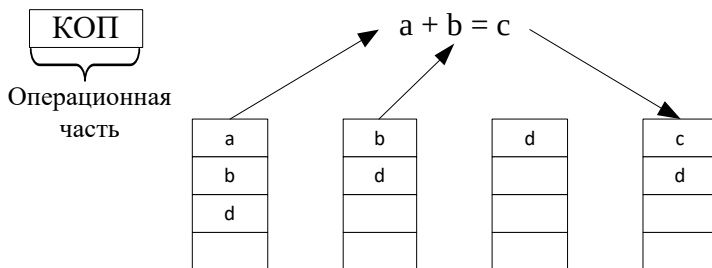


Рис. 18. Структура безадресной команды

Имеется только поле кода операции. Стек используется для хранения операндов и записи результата.

Способы адресации

Различают понятия: адресный код в команде и исполнительный адрес операнда. Адресный код в команде - это информация об адресе.

Исполнительный адрес операнда – это физический адрес ячейки памяти, в которой хранится операнд или в которую необходимо записать результат.

Рассмотрим варианты адресаций на примере 2-х адресной команды





Прямая адресация.

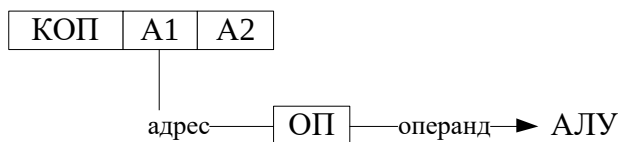


Рис. 19. Структура прямой адресации

В адресном поле команды A1 задаётся исполнительный адрес операнда.

Выбор операнда $\left\{ \begin{array}{l} \text{РАП:}=\text{РК}(\text{A1}) \\ \text{РЧП:}=\text{Ч}_\text{Т}(\text{РАП}) \\ \text{РА:}=\text{РЧП} \end{array} \right.$

РА - входной регистр АЛУ.

Непосредственная адресация.

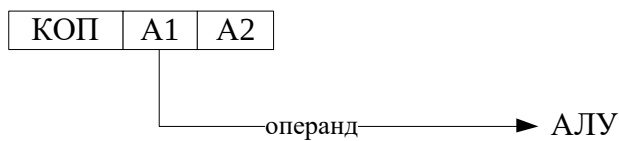


Рис. 20. Структура непосредственной адресации

В адресном поле команды задается непосредственно сам операнд.

$\text{РА:}=\text{РК}(\text{A1})$

Косвенная адресация.

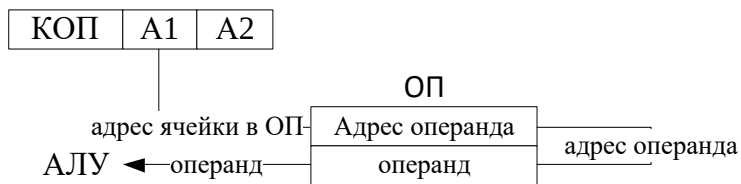


Рис. 21. Структура косвенной адресации

В адресном поле команды задается адрес ячейки оперативной памяти (ОП), в которой хранится адрес операнда. Иначе, эта адресация называется адрес адреса.

$\text{РАП:}=\text{РК}(\text{A1})$

$\text{РЧП:}=\text{Ч}_\text{Т}(\text{РАП})$ – выбор адреса операнда из ЗМ

$\text{РАП:}=\text{РЧП}$

$\text{РЧП:}=\text{Ч}_\text{Т}(\text{РАП})$ – выбор операнда из ЗМ

$\text{РА:}=\text{РЧП}$

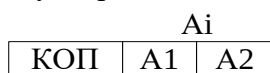
Регистровая адресация.

В структуру ЦП дополнительно вводятся регистры общего назначения (РОНы).

Они вводятся для того, чтобы:

1. Сократить время выбора операндов по сравнению с ОП.
2. Сократить размер поля A_i для кодирования номера РОНа.

Микропрограмма и структурная схема операционной части ЦП при выполнении двухадресной команды формата регистр-регистр.



При использовании регистровой адресации в адресном поле команд A_i задается номер регистра, где хранится операция.



При написании микропрограммы принимается, что запись результата производится на место первого операнда:

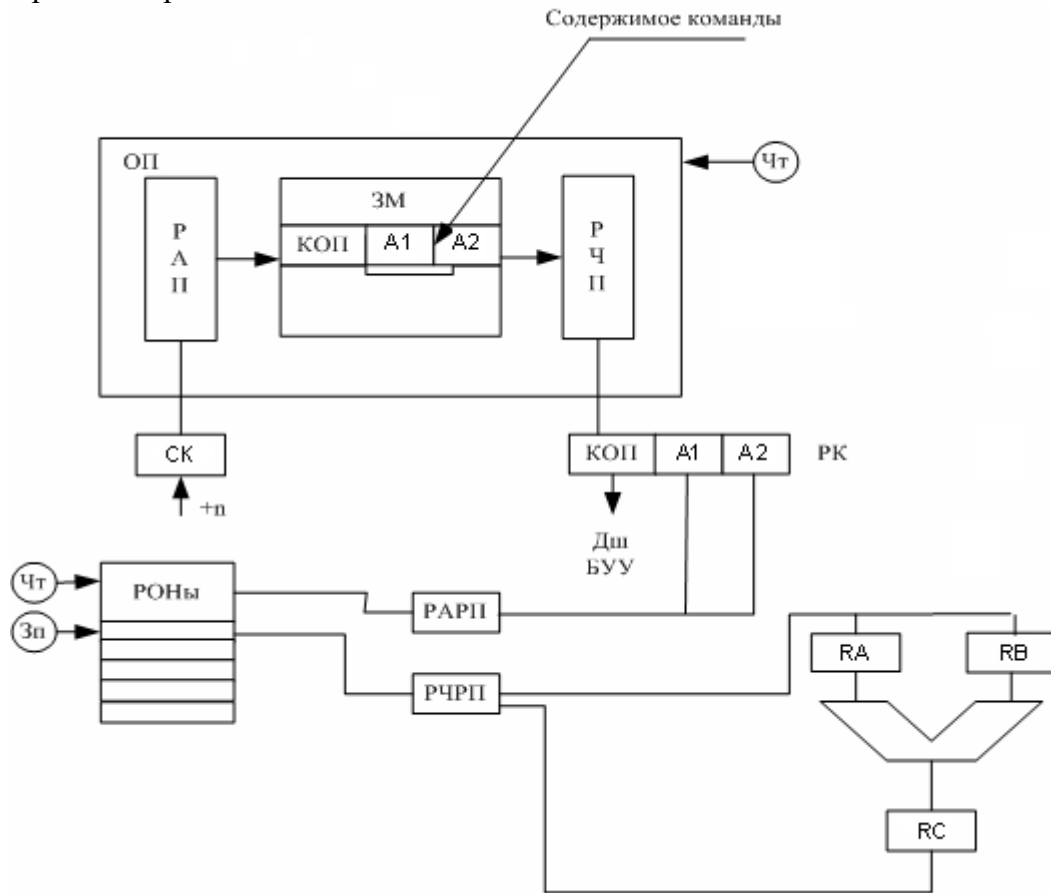


Рис. 22. Структурная схема операционной части ЦП при выполнении двухадресной команды формата регистр-регистр

1 этап. Выбор команды из памяти:

РАП:=СК - адрес следующей команды подается на РАП

РЧП:=ЧТ(РАП) - выбор команды из ЗМ

ПК:=РЧП - машинная команда из памяти передается на ПК в ЦП

СК:=СК+n - n-длина выполняемой машинной команды в байтах

2 этап. Дешифрация кода операции.

3 этап. Формирование исполнительного адреса и выбор операндов:

Выбор 1-го операнда	{	РАРП:=ПК(А1) - на РАРП подается номер регистра, где хранится 1-ый операнд
		РЧРП:=ЧТ(РАРП) - выбор первого операнда из РОНов
		РА:=РЧРП - пересылка первого операнда на входной регистр АЛУ
Выбор 2-го операнда	{	РАРП:=ПК(А2) - на РАРП подается номер регистра, где хранится 2-ой операнд
		РЧРП:=ЧТ(РАРП) - выбор второго операнда из РОНов
		РВ:=РЧРП - пересылка второго операнда на входной регистр АЛУ

4 этап. Выполнение операции в АЛУ:

RC:=RA операция RB

5 этап. Запись результата:

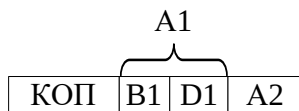
РАРП:=ПК(А1)

РЧРП:=RC

Зп(РАРП):=РЧРП



Базовая адресация.



B1 – номер базового регистра, (B1) – база.

D1 – смещение относительно базы.

Исполнительный адрес не задан явно в адресном поле.

Исполнительный адрес $A1 = (B1) + D1$.

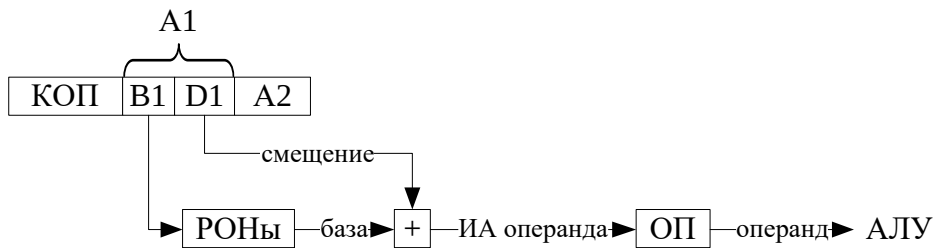
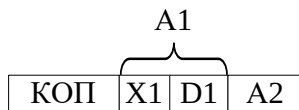


Рис. 23. Структура базовой адресации

Базовая адресация – обеспечивает перемещаемость программ в памяти.

При базовой адресации операнды хранятся в ОП. Для вычисления исполнительного адреса операнда необходимо к содержимому базового регистра прибавить смещение, которое выбирается из адресного поля команды.

Индексная адресация.



X1 – номер индексного регистра, (X1) – индекс.

Исполнительный адрес $A1 = (X1) + D1$.

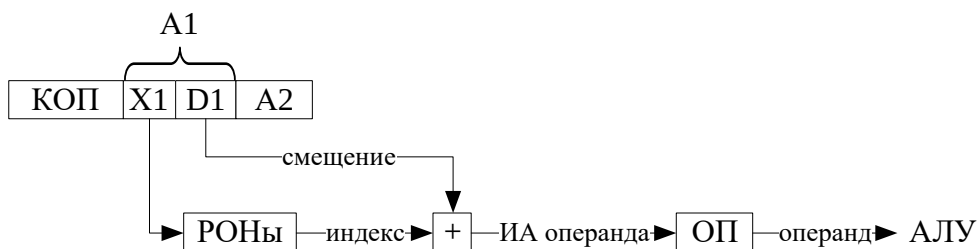
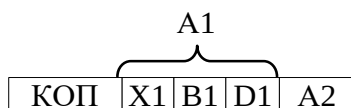


Рис. 24. Структура индексной адресации

Индексная адресация - используется при работе с массивами, таблицами.

Исполнительный адрес операнда вычисляется аналогично базовой адресации.

Базово – индексная адресация.



(X1) – индекс (содержимое индексного регистра).

(B1) – база (содержимое базового регистра).

D1 – смещение (выбирается из команды и подается на АЛУ).

Исполнительный адрес $A1 = (X1) + (B1) + D1$.

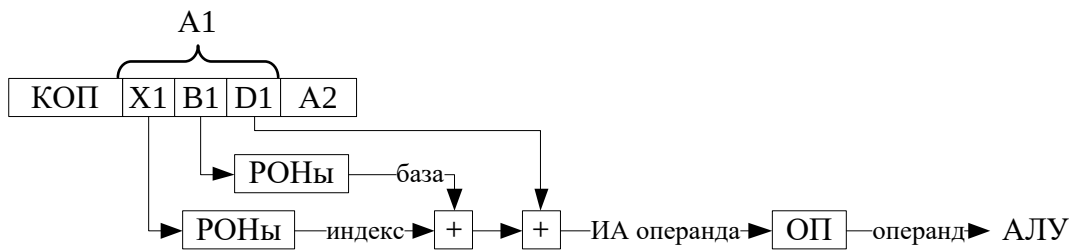


Рис. 25. Структура базово-индексной адресации

Микропрограмма и структурная схема операционной части ЦП при выполнении двухадресной команды с записью результата на место первого операнда:

1-ый операнд: регистровая адресация.

2-ой операнд: базово - индексная адресация.

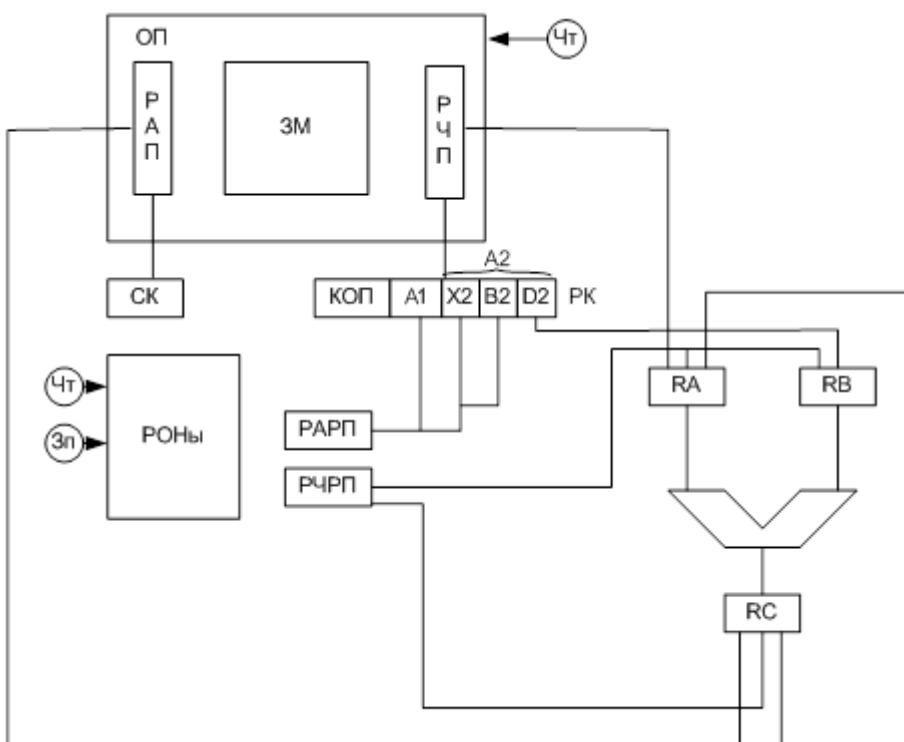


Рис. 26. Структурная схема операционной части ЦП при выполнении двухадресной команды с записью результата на место первого операнда

1 этап. Выбор команды из памяти:

РАП:=СК

РЧП:=Чт(РАП)

РК:=РЧП

СК:=СК + n

2 этап. Дешифрация кода операции.

3 этап. Формирование ИА и выбор операндов:

Выбор индекса X2	{	РАРП:=РК(X2) – номер индексного регистра подается в РОНЫ
		РЧРП:=Чт(РАРП) – выбор индекса
		РА:=РЧРП - пересылка индекса на входной регистр АЛУ
Выбор индекса B2	{	РАРП:=РК(B2) – номер базового регистра подается в РОНЫ
		РЧРП:=Чт(РАРП) – выбор базы
		RB:=РЧРП - пересылка базы на входной регистр АЛУ



$RC := RA + RB$ – вычисление $X2 + B2$

$RA := RC$ - на RA пересылается часть ИА

$RB := PK(D2)$ - выбор смещения

$RC := RA + RB$ - вычисление ИА

$РАП := RC$ - ИА подается на РАП

$РЧП := Чт(РАП)$ - выбор 2-го операнда

$RA := РЧП$ - пересылка второго операнда на входной регистр АЛУ

Выбор 1-го операнда $\left\{ \begin{array}{l} \text{РАРП} := PK(A1) \text{ - на РАРП подается номер регистра, где хранится 1-ый операнд} \\ \text{РЧРП} := Чт(\text{РАРП}) \text{ - выбор первого операнда из РОНов} \\ \text{RB} := \text{РЧРП} \text{ - пересылка первого операнда на входной регистр АЛУ} \end{array} \right.$

4 этап. Выполнение операции в АЛУ:

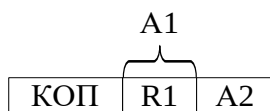
$RC := RA$ операция RB

5 этап. Запись результатов:

$РЧРП := RC$

$Зп(\text{РАРП}) := \text{РЧРП}$

Косвенно – регистровая адресация.



В адресном поле команды указывается номер РОНа, который содержит адрес операнда, хранящийся в ОП (оперативной памяти).

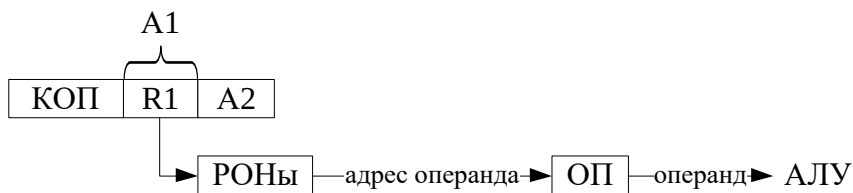


Рис. 27. Структура косвенно-регистровой адресации

$РАРП := PK(R1)$

$РЧРП := Чт(РАРП)$ - выбор адреса операнда из РОНов

$РАП := РЧРП$

$РЧП := Чт(РАП)$ - выбор операнда из ОП

$RA := РЧП$

Если нужно записать результат на место операнда с косвенно – регистровой адресацией, то результат записывается в ОП, а не в РОНЫ.

Рассмотрим моделирование операционной части ЦП на примере выполнения двухадресной команды формата регистр-регистр.

Структурная схема операционной части ЦП при выполнении двухадресной команды формата регистр-регистр (R-R) изображена на рис. 22.

При моделировании примем, что формат двухадресной команды формата R-R имеет следующий вид:

КОП	A1	A2
15...14	13...7	6...0

A1 - поле, содержащее номер РОНа, где хранится 1-й операнд,

A2 - поле, содержащее номер РОНа, где хранится 2-й операнд.



Определим регистры:

RCH[16] – регистр числа памяти 16 бит, в него происходит выборка из памяти и из него происходит запись в память.

RAP[4] – регистр адреса, по которому производится выборка из памяти или запись в память. Команды:

- RCH:=ZS[RAP]; - чтение по адресу RAP;
- ZS[RAP]:=RCH; - запись по адресу RAP.

RCHR[16] – регистр числа регистровой памяти 16 бит, в него происходит выборка из РОНов и из него происходит запись в них.

RARP[4] – регистр адреса регистровой памяти, по которому производится выборка из РОНов или запись в них. Команды:

- RCHR:=RON[RARP]; - чтение по адресу RARP;
- RON[RARP]:=RCHR; - запись по адресу RARP.

SK[4] – счётчик команд, по его адресу из памяти извлекается первая команда. При запуске программы лабораторной работы происходит запрос его начального значения.

MEM RON[16][16] – массив для хранения РОНов. 16 РОНов по 16 бит.

MEM ZS[16][16] – запоминающая среда памяти, массив 16 ячеек по 16 бит.

Структура регистра команд. Возможные варианты в зависимости от выбранной адресации:

STRUCT RK[16] OP2[7], OP1[7], KOP[2];

STRUCT RK[16] OP2[7], D1[3], R1[4], KOP[2];

STRUCT RK[16] D2[3], R2[4], OP1[7], KOP[2];

STRUCT RK[16] D2[3], R2[4], D1[3], R1[4], KOP[2];

OP1[7] или OP2[7] – поля, предназначенные для задания первого или второго операнда или информации об их адресах, если для них выбраны соответственно адресации: непосредственная, прямая, регистровая, косвенно-регистровая, косвенная.

R1[4] или R2[4] – поля, предназначенные для задания номеров базового и индексного регистров для первого или второго операнда.

D1[3] или D2[3] – смещение первого или второго операнда для базовой или индексной адресации.

Пример моделирования операционной части ЦП

Ниже приводится программа моделирования операционной части ЦП на примере выполнения двухадресной команды формата R-R. Этот код будет работать в режиме «без графического интерфейса». При моделировании с графическим интерфейсом вставлять этот код не нужно, т.к. разработчик интерпретатора «вшил» его в данный вариант лабораторной работы.

```
VAR I;  
REG RA[16], RB[16], RC[16], RCH[16], RAP[4], SK[4], RARP[4], RCHR[16],  
RAMK[5], RMK[32];  
MEM RON[16][16];  
MEM ZS[16][16];  
MEM PMK[32][32];  
MEM PAMK[32][5];  
STRUCT RK[16] OP2[7], OP1[7], KOP[2];  
RON[0]:=0;  
RON[1]:=0;  
RON[2]:=0;  
RON[3]:=32;  
RON[4]:=0;  
RON[5]:=0;
```



```
RON[6]:=0;
RON[7]:=15;
RON[8]:=0;
RON[9]:=0;
RON[10]:=0;
RON[11]:=0;
RON[12]:=0;
RON[13]:=0;
RON[14]:=0;
RON[15]:=0;
ZS[0]:=0;
ZS[1]:=16775;
ZS[2]:=0;
ZS[3]:=0;
ZS[4]:=0;
ZS[5]:=0;
ZS[6]:=0;
ZS[7]:=0;
ZS[8]:=0;
ZS[9]:=0;
ZS[10]:=0;
ZS[11]:=0;
ZS[12]:=0;
ZS[13]:=0;
ZS[14]:=0;
ZS[15]:=0;
START:
READ "Введите ячейку, в которую записана команда" SK;
RAP:=SK;
PAUSE;
RCH:=ZS[RAP];
PAUSE;
RK:=RCH;
PAUSE;
SK:=SK+1;
PAUSE;
RARP:=RK.OP1;
PAUSE;
RCHR:=RON[RARP];
PAUSE;
RA:=RCHR;
PAUSE;
RARP:=RK.OP2;
PAUSE;
RCHR:=RON[RARP];
PAUSE;
RB:=RCHR;
PAUSE;
RC:=RA+RB;
PAUSE;
RARP:=RK.OP1;
```

```

PAUSE;
RCHR:=RC;
PAUSE;
RON[RARP]:=RCHR;
PAUSE;    PAUSE;
END;

```

На рис. 28 приведено содержимое экрана для рассматриваемого примера. В примере команда вводится по адресу 1, КОП – 1, 1-ый операнд хранится в 3 РОНе, 2-ой операнд – в 7 РОНе. Данные вводятся и отображаются в 10-ой системе счисления.

Операционная часть MCL - Среда разработки MCL. МАИ. Кафедра 304

Файл Правка Программа

Покомандный режим ВЫКЛЮЧЕН: Пауза! Нажмите, чтобы продолжить:

Адресация: регистровая - регистровая

Адрес	Данные	Адрес	Данные	Адрес	Данные
0	0	1	16775	2	0
4	0	5	0	6	0
8	0	9	0	10	0
12	0	13	0	14	0
				15	0

Операционная часть

```

RAP:=SK;
PAUSE;
RCH:=ZS[RAP];
PAUSE;
RK:=RCH;
PAUSE;
SK:=SK+1;
PAUSE;
RARP:=RK.OP1;
PAUSE;
RCHR:=RON[RARP];
PAUSE;
RA:=RCHR;
PAUSE;
RARP:=RK.OP2;
PAUSE;
RCHR:=RON[RARP];
PAUSE;
RB:=RCHR;
PAUSE;
RC:=RA+RB;
PAUSE;
RARP:=RK.OP1;
PAUSE;
RCHR:=RC;
PAUSE;
RON[RARP]:=RCHR;
PAUSE;

```

Ввод содержимого РОНов:

РОН №	Значение
РОН 0	
РОН 1	
РОН 2	
РОН 3	32
РОН 4	
РОН 5	
РОН 6	
РОН 7	15
РОН 8	
РОН 9	

Ввод содержимого ОП:

Нач. КОП: 1, ОП1: 3, ОП2: 7

Ввод команды: 1, 3, 7

Нач. Данные: 0, 00000

Введите ячейку в которую записана команда

1

Содержимое таблицы переменных

№	Переменная	Тип	Разрядность	Значение
00	Переменная: ZF	типа регистр	разрядность 1 бит	значение = 0
01	Переменная: NF	типа регистр	разрядность 1 бит	значение = 0
02	Переменная: CF	типа регистр	разрядность 1 бит	значение = 0
03	Переменная: I	типа простая переменная		Значение = 0
04	Переменная: RA	типа регистр	разрядность 16 бит	значение = 32
05	Переменная: RB	типа регистр	разрядность 16 бит	значение = 15
06	Переменная: RC	типа регистр	разрядность 16 бит	значение = 47
07	Переменная: RCH	типа регистр	разрядность 16 бит	значение = 16775
08	Переменная: RAP	типа регистр	разрядность 4 бит	значение = 1
09	Переменная: SK	типа регистр	разрядность 4 бит	значение = 2
10	Переменная: RARP	типа регистр	разрядность 4 бит	значение = 3
11	Переменная: RCHR	типа регистр	разрядность 16 бит	значение = 47
12	Переменная: RAMK	типа регистр	разрядность 5 бит	значение = 0
13	Переменная: RK	типа регистр	разрядность 32 бит	значение = 0
14	Переменная: RON	типа память	из 16 ячеек по 16 бит	
15	Переменная: ZS	типа память	из 16 ячеек по 16 бит	
16	Переменная: PK	типа память	из 32 ячеек по 32 бит	

Рис. 28. Моделирование операционной части ЦП на примере выполнения двухадресной команды формата R-R

Примечания

- на рисунке 28 слева на графике расставлены вентили, которые в данной лабораторной работе не используются (они нужны для моделирования блока устройства управления);
- значения РОНов и ОП следует задавать при помощи интерфейса, а не в коде программы.
- при написании программы моделирования необходимо вставлять паузу (PAUSE) после каждой команды для контроля правильного выполнения программы (правильной передачи информации с регистра на регистр);
- в командах записи и чтения необходимо использовать квадратные скобки;
- в данной лабораторной работе пошаговый режим не использовать;
- в программе задается содержимое всех РОНов равное 0 (выделяется ячейка в таблице и печатается значение);



- чтобы задать команду, необходимо выбрать номер ячейки, в которую записывается команда, выделить поле КОП и задать код операции, выделить следующее поле и ввести следующее значение;
- чтобы задать данные, необходимо выбрать номер ячейки, в которую записываются данные, выделить поле данных и ввести число, которое необходимо задать;
- в ячейке 1 имеется число 16775 (по адресу 1-команда), которое является десятичной интерпретацией команды.

Содержание отчета

1. Формулировка задания.
2. Структурная схема ЦП при выполнении команд в соответствии с заданными способами адресации.
3. Блок-схема функционирования операционной части центрального процессора.
4. Программа на языке MCL, моделирующая выполнение команд в соответствии с заданными способами адресации.
5. Результаты выполнения программы в **виде распечатки** содержимого поля для вывода текстовой информации и **скриншота** в момент завершения выполнения программы.

Задание к лабораторной работе

Промоделировать функционирование операционной части на микропроцессорном уровне при выполнении 2-х адресной арифметико-логической команды, хранящейся в памяти по адресу, значение которого указано в счётчике команд (СК).

КОП для всех вариантов представлен двоичным набором 1, что соответствует операции сложения.

Задать содержимое РОН и ОП. Принять, что:

- а) длина команды 16-ть бит и под код операции отводится 2 разряда. Разрядность полей, отводимых под операнды, 7 бит;
- б) объём памяти 16 ячеек по 16 бит;
- в) существует 16-ть регистров общего назначения разрядностью 16 бит;
- г) РАП - 4 разрядов,
РЧП - 16 разрядов,
РАРП - 4 разряда,
РЧРП - 16 разрядов.

Таблица 4. Варианты заданий для моделирования операционной части

Номер варианта	1-ый операнд	2-ой операнд	Результат
1	Регистровая	Прямая	1ый операнд
2	Регистровая	Косвенная	1ый операнд
3	Регистровая	Косвенно-регистровая	1ый операнд
4	Регистровая	Базовая	1ый операнд
5	Прямая	Регистровая	1ый операнд
6	Прямая	Косвенная	1ый операнд
7	Прямая	Косвенно-регистровая	1ый операнд
8	Прямая	Базовая	1ый операнд
9	Косвенная	Регистровая	1ый операнд
10	Косвенная	Косвенно-регистровая	1ый операнд



11	Косвенная	Базовая	1ый операнд
12	Косвенная	Непосредственная	1ый операнд
13	Косвенная	Прямая	1ый операнд
14	Косвенно- регистравая	Регистровая	1ый операнд
15	Косвенно- регистравая	Прямая	1ый операнд
16	Косвенно- регистравая	Косвенная	1ый операнд
17	Косвенно- регистравая	Индексная	1ый операнд
18	Базовая	Регистровая	1ый операнд
19	Базовая	Непосредственная	1ый операнд
20	Прямая	Регистровая	2ой операнд
21	Косвенная	Регистровая	2ой операнд
22	Косвенно- регистравая	Регистровая	2ой операнд
23	Индексная	Регистровая	2ой операнд
24	Регистровая	Прямая	2ой операнд
25	Косвенная	Прямая	2ой операнд
26	Косвенно- регистравая	Прямая	2ой операнд
27	Базовая	Прямая	2ой операнд
28	Регистровая	Косвенная	2ой операнд
29	Косвенно- регистравая	Косвенная	2ой операнд
30	Базовая	Косвенная	2ой операнд
31	Прямая	Косвенная	2ой операнд
32	Непосредственная	Косвенно-регистравая	2ой операнд
33	Регистровая	Косвенно-регистравая	2ой операнд
34	Прямая	Косвенно-регистравая	2ой операнд
35	Косвенная	Косвенно-регистравая	2ой операнд
36	Индексная	Косвенно-регистравая	2ой операнд
37	Регистровая	Базовая	2ой операнд
38	Непосредственная	Базовая	2ой операнд
39	Прямая	Регистровая	1ый операнд
40	Непосредственная	Косвенно-регистравая	2ой операнд

Контрольные вопросы

1. Типы адресации. Основные различия между типами адресации.
2. Типизация команд по адресной части.
3. Эффективность применения конкретного типа адресации в конкретной ситуации.



Лабораторная работа № 5. Моделирование блока устройства управления

Содержание

Цель работы	74
Функционирование блока устройства управления	74
Описание функционирования блока устройства управления	74
Структурная схема горизонтального микропрограммного устройства	75
Пример работы БУУ при выполнении двухадресной команды формата R-R	75
Структурная схема	77
Пример выполнения программы	79
Содержание отчета	84
Задание к лабораторной работе	84



Цель работы

Изучение функционирования блока УУ при выполнении команд центрального процессора с различными способами адресации.

Функционирование блока устройства управления

Описание функционирования блока устройства управления

Устройство управления центрального процессора вырабатывает последовательность функциональных сигналов, необходимую для выполнения команды. Команда в процессоре реализуется, обычно, за несколько тактов, в каждом из которых выполняется одна или несколько микроопераций.

Цикл выполнения команды начинается с выборки ее из памяти. Затем происходит дешифрация кода операции, формирование исполнительного адреса и выбор операндов, выполнение операции в АЛУ и, последнее, запись результата в память. Процесс выполнения команды состоит из выполнения микрокоманд, каждая микрокоманда включает в себя одну или несколько микроопераций. Каждой микрооперации соответствует определенный управляющий (функциональный) сигнал.

Известны два основных метода построения логики формирования функциональных сигналов. Один из них состоит в том, что для каждой команды процессора имеется набор логических схем, которые в нужных тактах возбуждают соответствующие управляющие сигналы, такой принцип управления получил название "жесткой" или "запамятной" логики.

В современных процессорах при построении устройств управления часто используется другой подход, основанный на использовании принципа микропрограммного управления. В таком устройстве управления схемы включения функциональных сигналов управляются регистром, в который считывается управляющая информация из постоянного запоминающего устройства (ПЗУ). Считывая последовательно слова из ПЗУ в этот регистр, можно организовать требуемую последовательность функциональных сигналов.

Микропрограммный принцип организации управления заключается в том, что каждой микрооперации ставится в соответствие слово (или часть слова), называемая микрокомандой и хранимое в памяти подобно тому, как хранятся в памяти команды обычной пользовательской программы. При таком подходе команде соответствует микропрограмма, то есть набор микрокоманд, указывающих, какие функциональные импульсы и в какой последовательности необходимо возбуждать для выполнения данной операции. Этот подход получил название микропрограммный или "хранимой логики".

Микропрограммное устройство управления может быть реализовано следующими способами:

- горизонтальный способ;
- вертикальный способ;
- горизонтально - вертикальный;
- вертикально - горизонтальный.

Горизонтальное микропрограммное устройство

При горизонтальном микропрограммировании каждому разряду микрокоманды соответствует определенная микрооперация, выполняемая независимо от содержания других разрядов.



Структурная схема горизонтального микропрограммного устройства

Структурная схема горизонтального микропрограммного устройства представлена на рисунке 29. В состав устройства управления входят:

1. Регистр адреса микрокоманд (РАМК);
2. Дешифратор (ДШ);
3. Постоянное запоминающее устройство (ПЗУ);
4. Регистр микрокоманд (РМК).



Рис. 29. Структурная схема горизонтального микропрограммного устройства

Достоинствами горизонтального микропрограммирования являются возможность одновременного выполнения нескольких микроопераций и простота формирования функциональных импульсов, так как последние могут возбуждаться непосредственно от сигналов, получаемых из регистра хранения микрокоманды без каких либо промежуточных схем дешифрации. Однако, оно имеет и существенный недостаток, заключающийся в том, что требуется большая длина микрокоманды, поскольку число функциональных импульсов в современном процессоре может достигать нескольких сотен. Соответственно, для хранения микрокоманд требуются запоминающие устройства большой емкости.

Чтобы обеспечить выполнение требуемых микроприказов не постоянно, а в строго определенное время, необходимо расчлнить связи, соединяющие соответствующие регистры, управляющими вентилями (схемами И). В этом случае управляющий вентиль имеет два входа (от передающего триггера и управляющий) и один выход, подсоединенный к приемному триггеру.

Каждому микроприказу взаимно-однозначно соответствует номер управляющего вентиля. Поэтому устройство управления, выдавая управляющие единичные сигналы на определенные управляющие вентили, в требуемое время обеспечивает выполнение операционным устройством требуемого действия.

Пример работы БУУ при выполнении двухадресной команды формата R-R

Для примера рассмотрим работу блока устройства управления при выполнении двухадресной команды формата R-R.

Разобьем выполнение данной команды на такты на основе уже существующей микропрограммы (смотри предыдущую лабораторную работу):

- | | |
|-----------------------------|--|
| 1 такт: $РАП := СК$ | $\left\{ \begin{array}{l} \text{выборка} \\ \text{команды} \\ \text{и} \\ \text{ее дешифрация} \\ \text{выборка 1-го операнда} \\ \text{и его размещение на} \\ \text{входном регистре АЛУ} \end{array} \right.$ |
| 2 такт: $РЧП := Ч_Т(РАП)$ | |
| 3 такт: $РК := РЧП$ | |
| 4 такт: $СК := СК + 1$ | |
| 5 такт: $РАП := РК(A1)$ | |
| 6 такт: $РЧРП := Ч_Т(РАРП)$ | |
| 7 такт: $РА := РЧРП$ | |



8 такт: $РАПП:=PK(A2)$	{	выборка 2-го операнда
9 такт: $РЧРП:=ЧТ(РАПП)$		
10 такт: $RB:=РЧРП$	{	и его размещение на входном регистре АЛУ
11 такт: $RC:=RA+RB$		
12 такт: $РЧРП:=RC$	{	выполнение операции в АЛУ запись результата
13 такт: $РАПП:=PK(A1 \text{ или } A2)$		
14 такт: $Зп(РАПП):=РЧРП$		

Каждый такт работы УУ полностью определяется управляющими вентилями, номера которых не обязательно совпадают с номерами тактов и на которые подаются единичные управляющие сигналы.

$V1$ $РАП:=СК$
 $V5$ $РЧП:=ЧТ(РАП)$
 $V2, V3$ $PK:=РЧП; СК:=СК+1$
 $V6$ $РАПП:=PK(A1)$
 $V8$ $РЧРП:=ЧТ(РАПП)$
 $V9, V10$ $RA:=РЧРП; РАПП:=PK(A2)$
 $V8$ $РЧРП:=ЧТ(РАПП)$
 $V11, V6$ $RB:=РЧРП; РАПП:=PK(A1 \text{ или } A2)$
 $V12$ $RC:=RA+RB$
 $V13$ $РЧРП:=RC$
 $V7$ $Зп(РАПП):=РЧРП$

Учитывая, что горизонтальное микропрограммирование позволяет совмещать микрооперации в одном такте времени, получим следующую последовательность включения управляющих вентилях: $V1-V5-V2, V3-V6-V8-V10, V9-V8-V11, V6-V12-V13-V7$.

Введем в блоке УУ регистр микрокоманд (РМК) длиной N бит. Установим взаимно-однозначное соответствие между номерами управляющих вентилях по правилу: i -разряд РМК соответствует i -вентилу V_i . Тогда работа УУ сводится к систематическому формированию двоичных последовательностей. Определим ряд компонент структурной схемы операционной части ЦП при выполнении 2-х адресной команды формата R-R(рис.30), где:

РК - регистр команд;
 СК - счетчик команд;
 АЛУ - арифметико-логическое устройство;
 ДШ - дешифратор;
 РОН - регистры общего назначения;
 РАП - регистр адреса регистровой памяти;
 РЧРП - регистр числа регистровой памяти;

в оперативном запоминающем устройстве:

ЗМ - запоминающий массив;
 РАП - регистр адреса памяти;
 РЧП - регистр числа памяти;
 V_i - i -тый управляющий вентиль.



Структурная схема

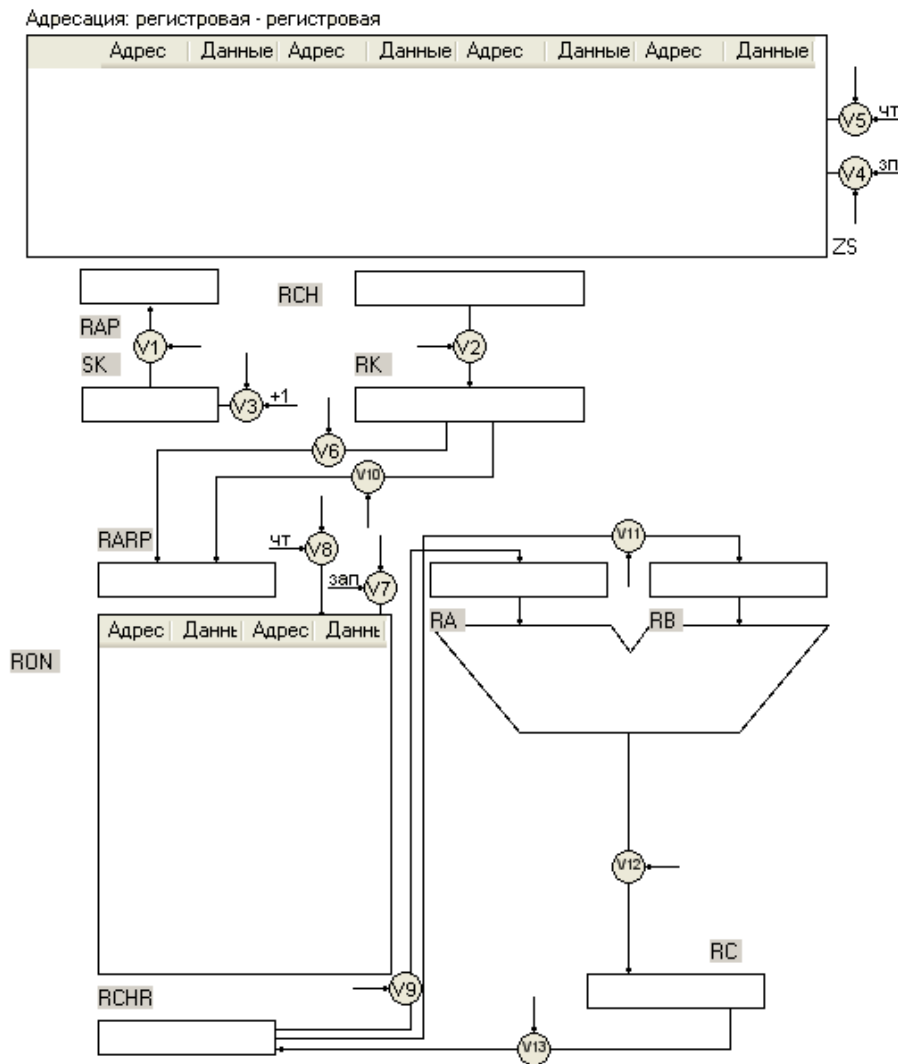


Рис. 30. Структурная схема операционной части ЦП при выполнении 2-х адресной команды формата R-R

В блок УУ входят следующие компоненты:

1. Регистр микрокоманд (РМК);
2. Регистр адреса микрокоманд (РАМК);
3. Дешифратор (ДШ);
4. Микропрограммная память (МПП), состоящая из двух матриц: управляющей матрицы, обеспечивающей формирование требуемых микрокоманд, и матрицы, определяющей номер очередной микрокоманды.

Структурная схема горизонтального микропрограммного УУ изображена на рисунке 31, где число вертикальных шин управляющей матрицы равно числу управляющих вентилей (13), число горизонтальных шин равно числу микрокоманд (11), а число вертикальных шин в адресной части равно 4-м.

Определим следующие переменные для моделирования в системе MCL:

RA[16] - первый операнд АЛУ.

RB[16] - второй операнд АЛУ.

RC[16] - регистр результата.

RCH[16] – регистр числа памяти 16 бит, в него происходит выборка из памяти и из него происходит запись в память.



RAP[4] – регистр адреса, по которому производится выборка из памяти или запись в память.

Команды:

- RCH:=ZS[RAP]; - чтение по адресу RAP;
- ZS[RAP]:=RCH; - запись по адресу RAP.

RCHR[16] – регистр числа регистровой памяти 16 бит, в него происходит выборка из РОНов и из него происходит запись в них.

RARP[4] – регистр адреса регистровой памяти, по которому производится выборка из РОНов или запись в них. Команды:

- RCHR:=RON[RARP]; - чтение по адресу RARP;
- RON[RARP]:=RCHR; - запись по адресу RARP.

SK[4] – счётчик команд, по его адресу из памяти извлекается первая команда. При запуске программы лабораторной работы происходит запрос его начального значения.

RAMK[5], RMK[32] – регистры для работы с прошивкой БУУ.

MEM PMK[32][32] – массив для хранения прошивки БУУ.

MEM RAMK[32][5] – массив для хранения адресной части БУУ.

MEM RON[16][16] – массив для хранения РОНов. 16 РОНов по 16 бит.

MEM ZS[16][16] – запоминающая среда памяти, массив 16 ячеек по 16 бит.

Структура регистра команд (в зависимости от адресации):

STRUCT RK[16] OP2[7], OP1[7], KOP[2];

STRUCT RK[16] OP2[7], D1[3], R1[4], KOP[2];

STRUCT RK[16] D2[3], R2[4], OP1[7], KOP[2];

STRUCT RK[16] D2[3], R2[4], D1[3], R1[4], KOP[2];

OP1[7] или OP2[7] – поля, предназначенные для задания первого или второго операнда или информации об их адресах, если для них выбраны соответственно адресации: непосредственная, прямая, регистровая, косвенно-регистровая, косвенная.

R1[4] или R2[4] – поля, предназначенные для задания номеров базового и индексного регистров для первого или второго операнда.

D1[3] или D2[3] – смещение первого или второго операнда для базовой или индексной адресации.

Исходными данными является содержимое микропрограммной памяти, которое необходимо задать в двоичной системе счисления.

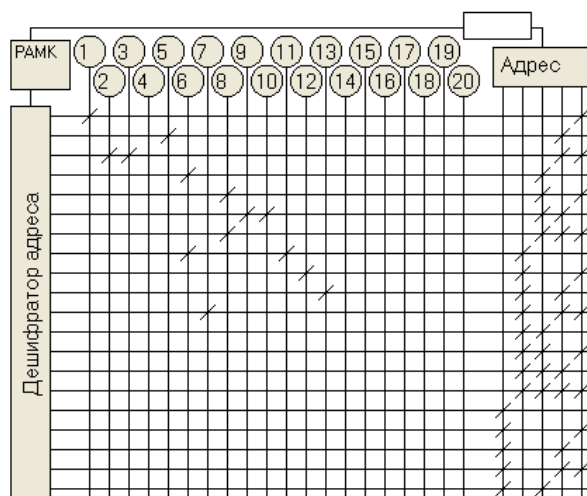


Рис.31 Структурная схема горизонтального микропрограммного УУ

Пример выполнения программы:

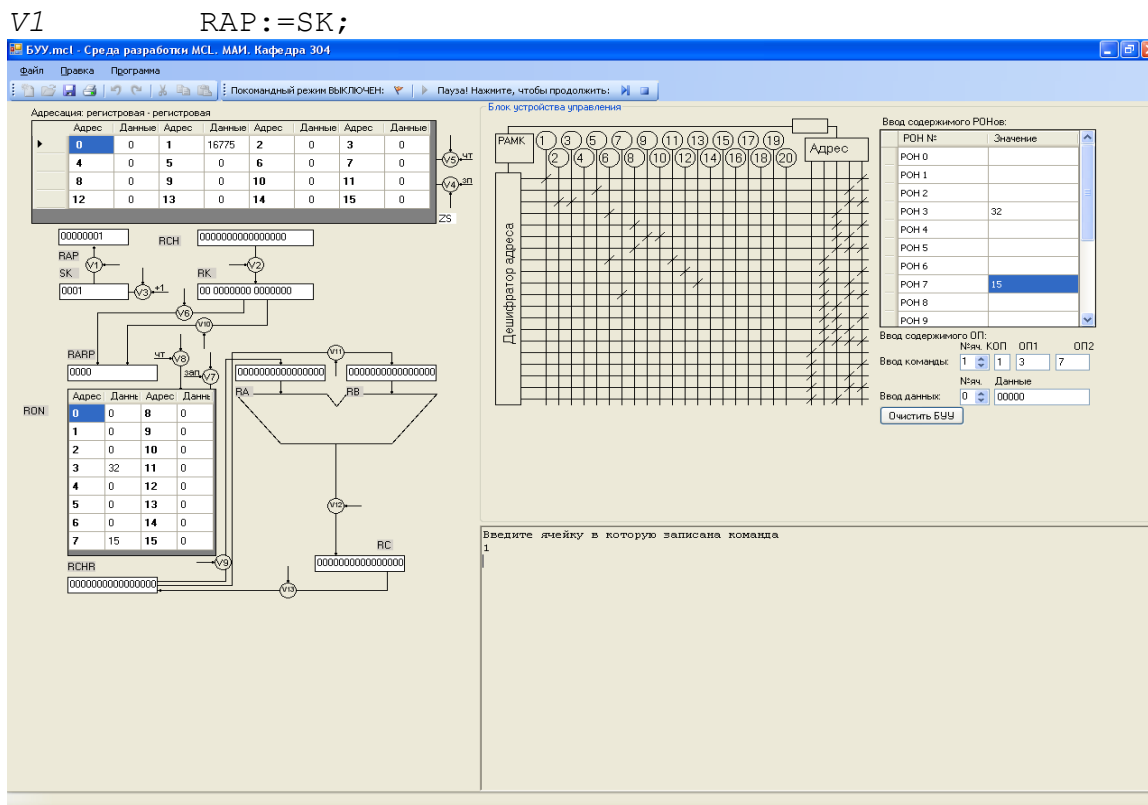


Рис. 32. Первый этап выполнения программы

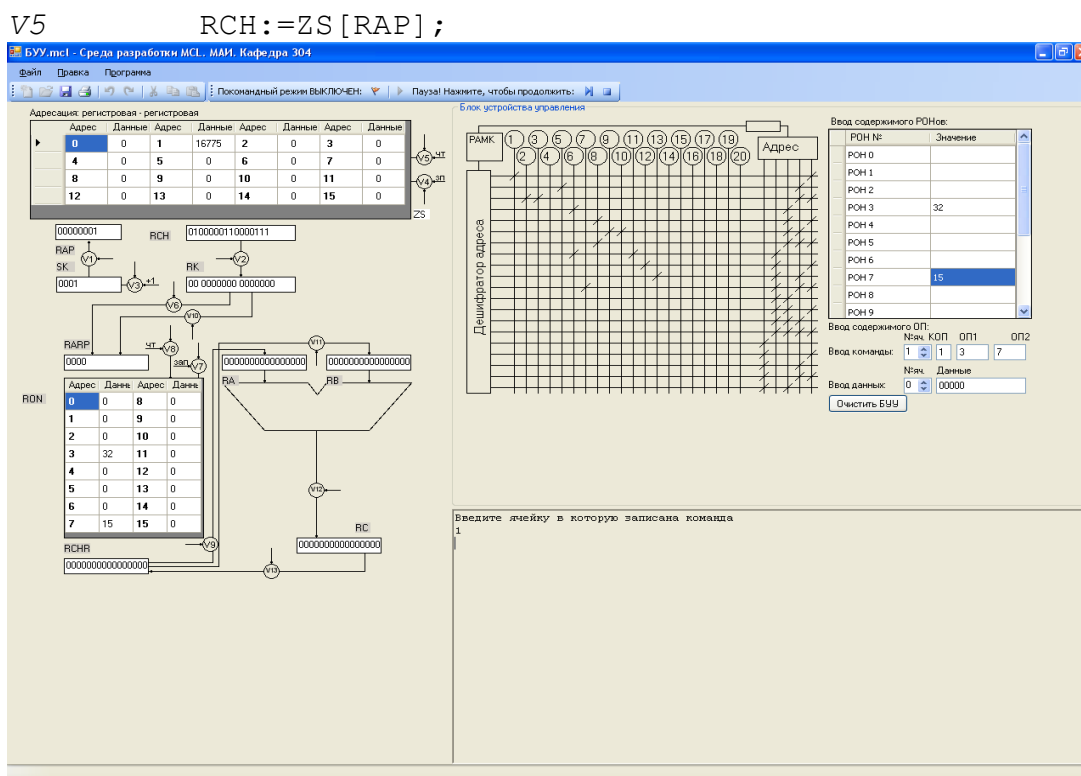


Рис. 33. Второй этап выполнения программы



V2, V3

RK:=RCH; SK:=SK+1;

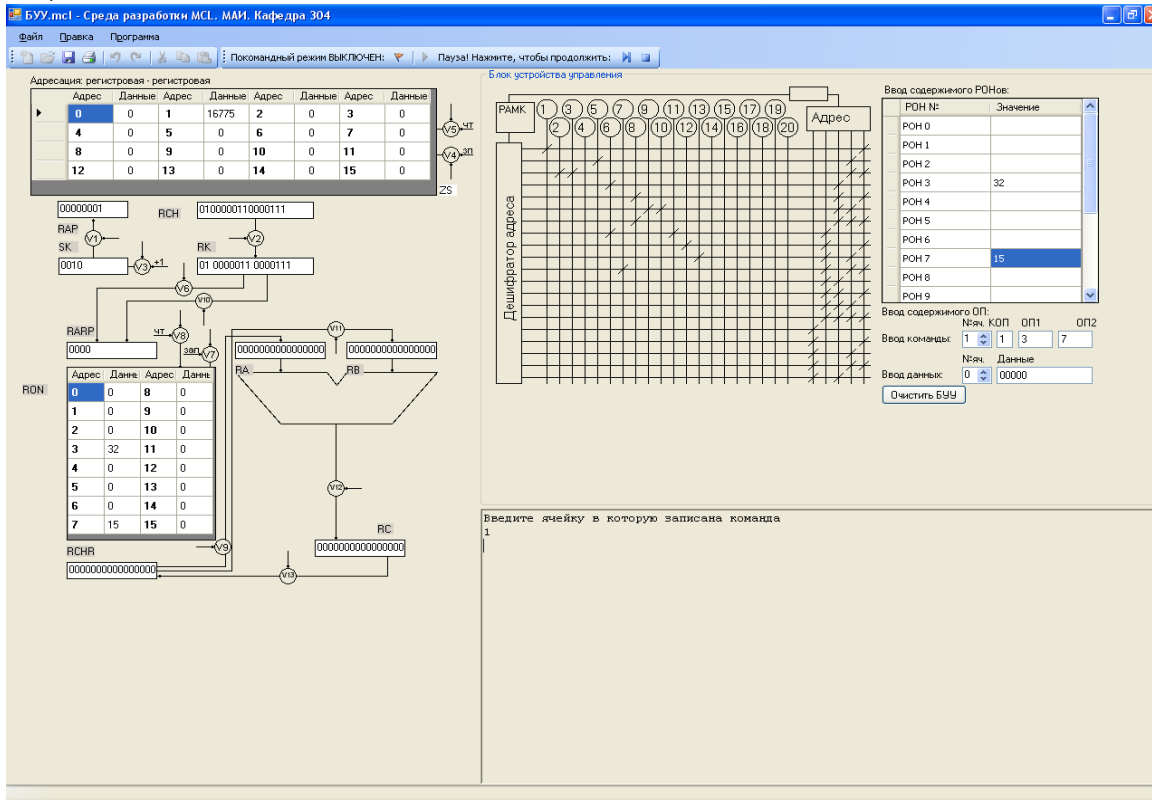


Рис. 34. Третий этап выполнения программы

V6

RARP:=RK.OP1;

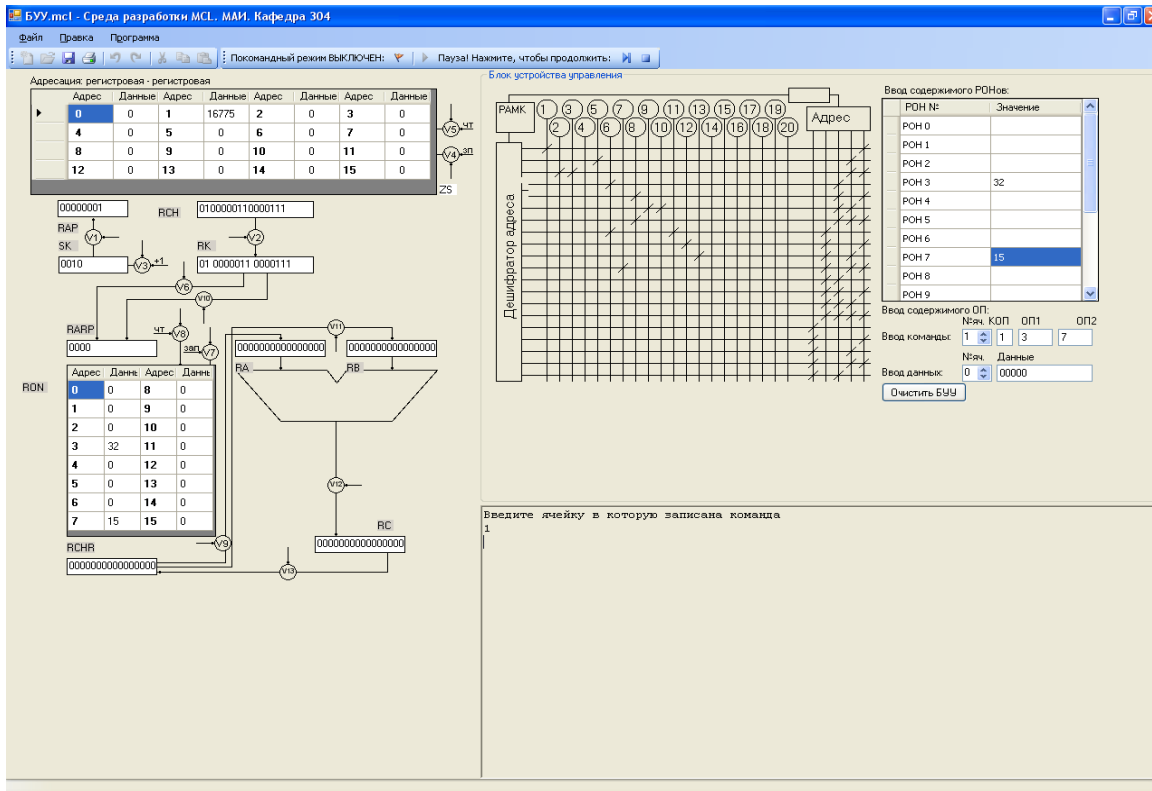


Рис. 35. Четвертый этап выполнения программы

V8

RCHR:=RON[RARP];

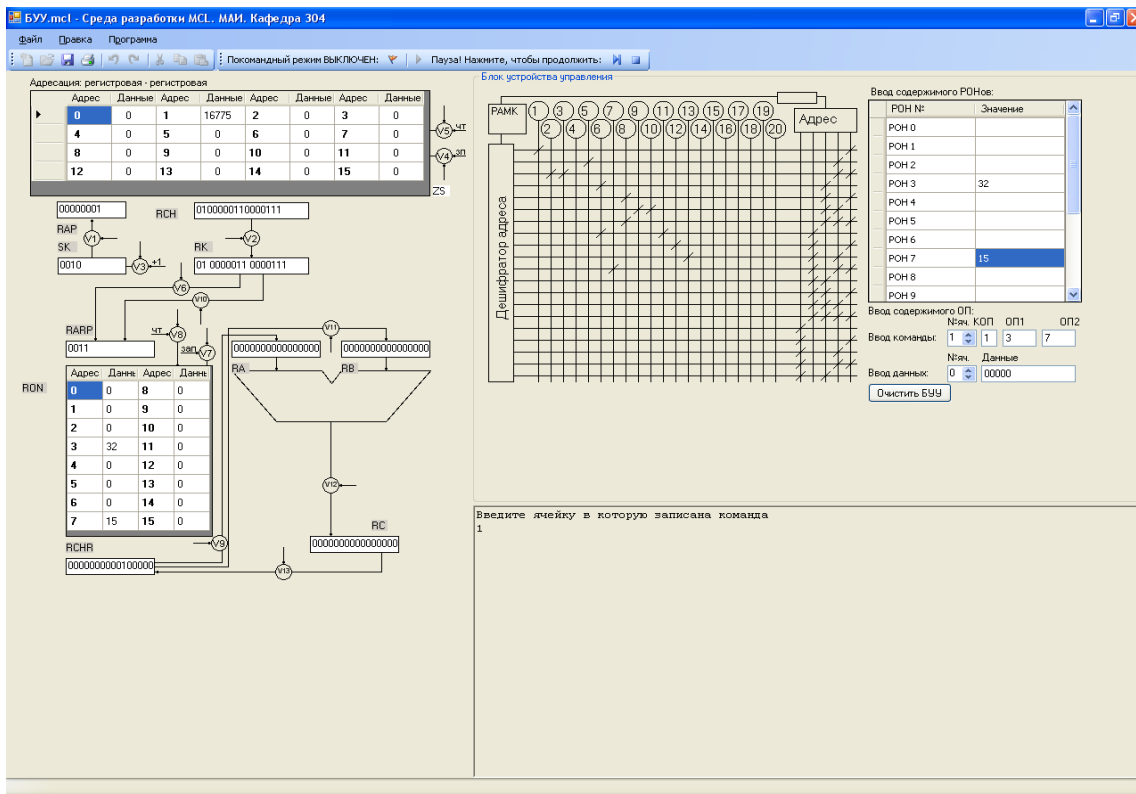


Рис. 36. Пятый этап выполнения программы

V9, V10 RA:=RCHN; RARP:=RK.OP2;

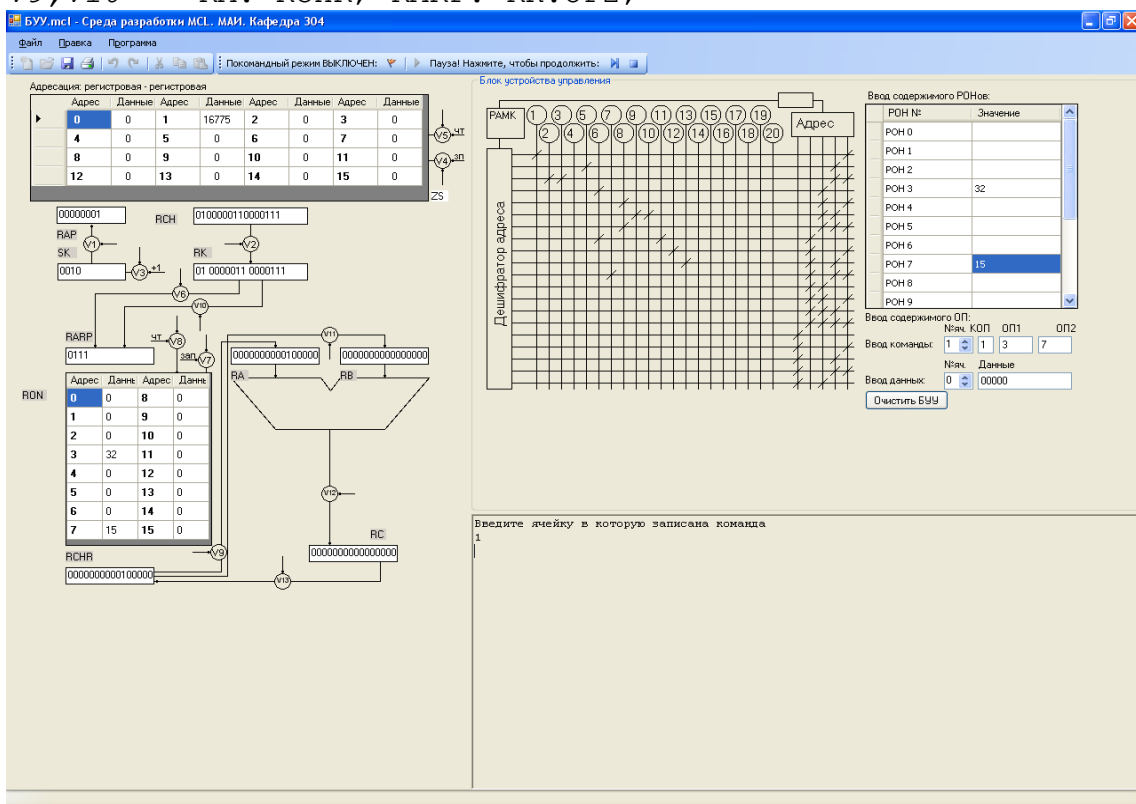


Рис. 37. Шестой этап выполнения программы



V8

RCHR:=RON[RARP];

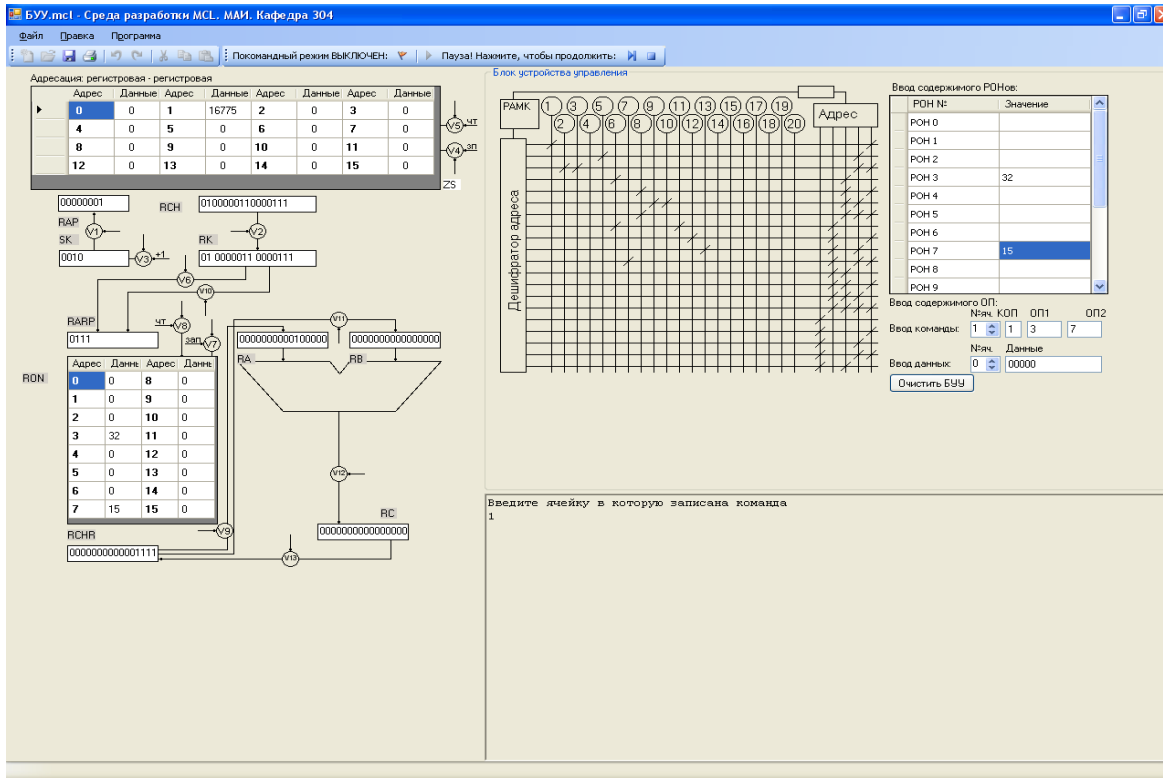


Рис. 38. Седьмой этап выполнения программы

V11, V6

RB:=RCHR; RARP:=RK.OP1;

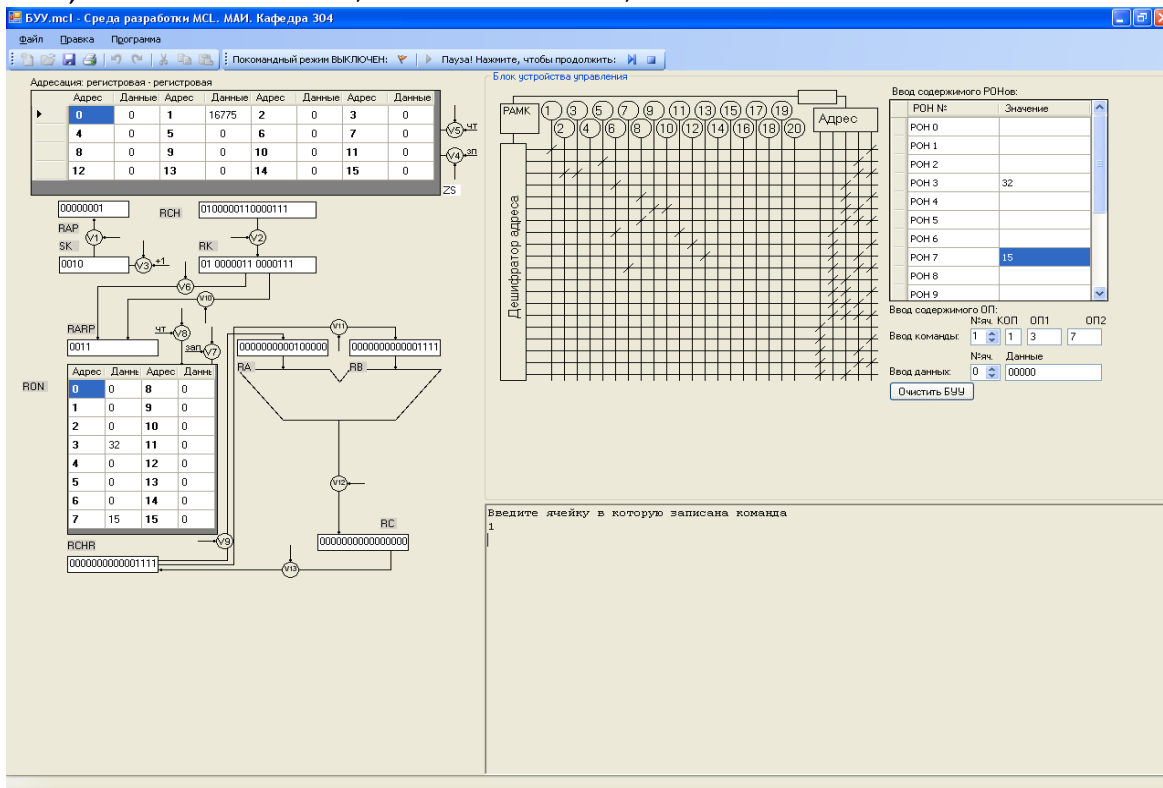


Рис. 39. Восьмой этап выполнения программы



V12

RC := RA + RB;

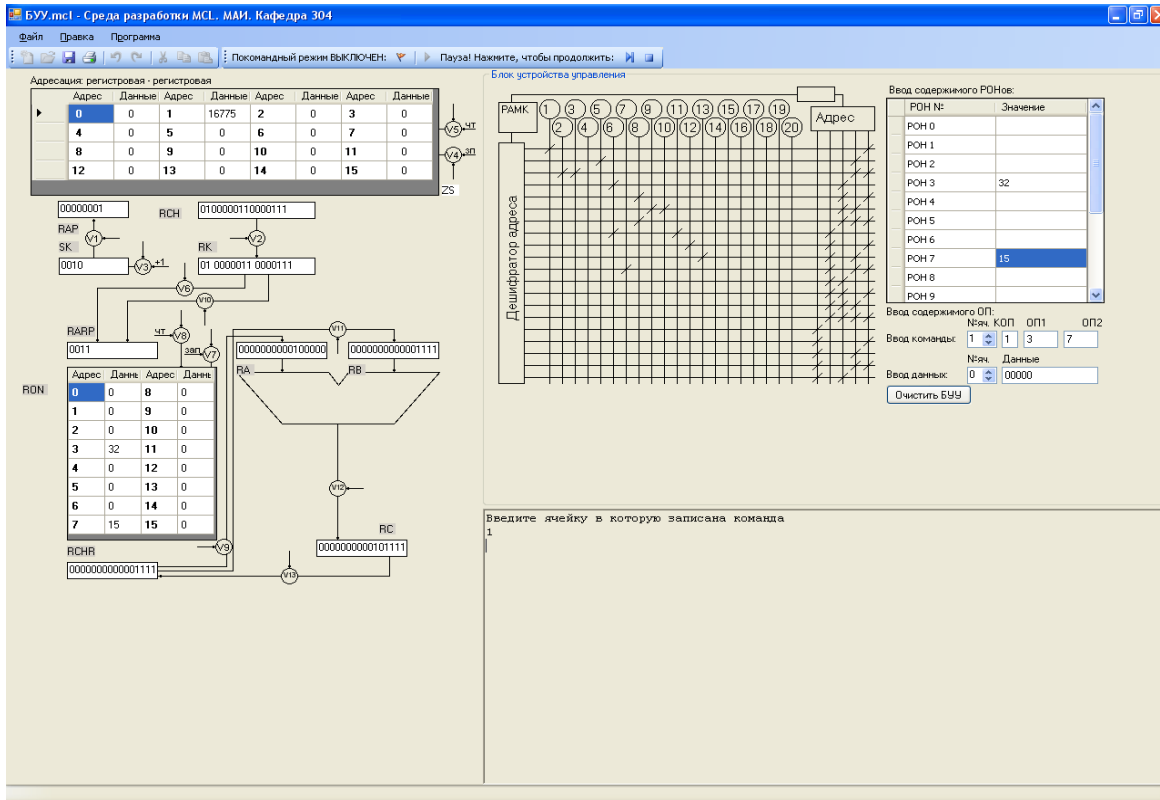


Рис. 40. Девятый этап выполнения программы

V13

RCHR := RC;

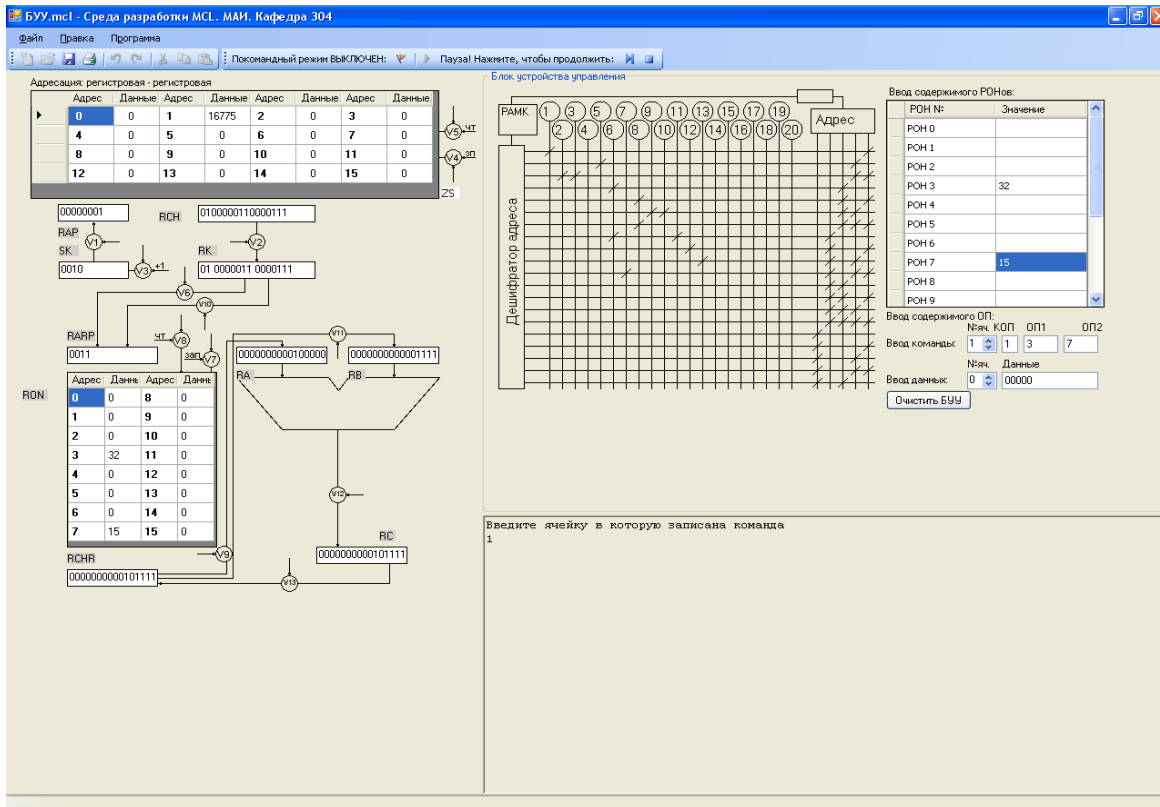


Рис. 41. Десятый этап выполнения программы



V7

RON[RARP]:=RCHR;

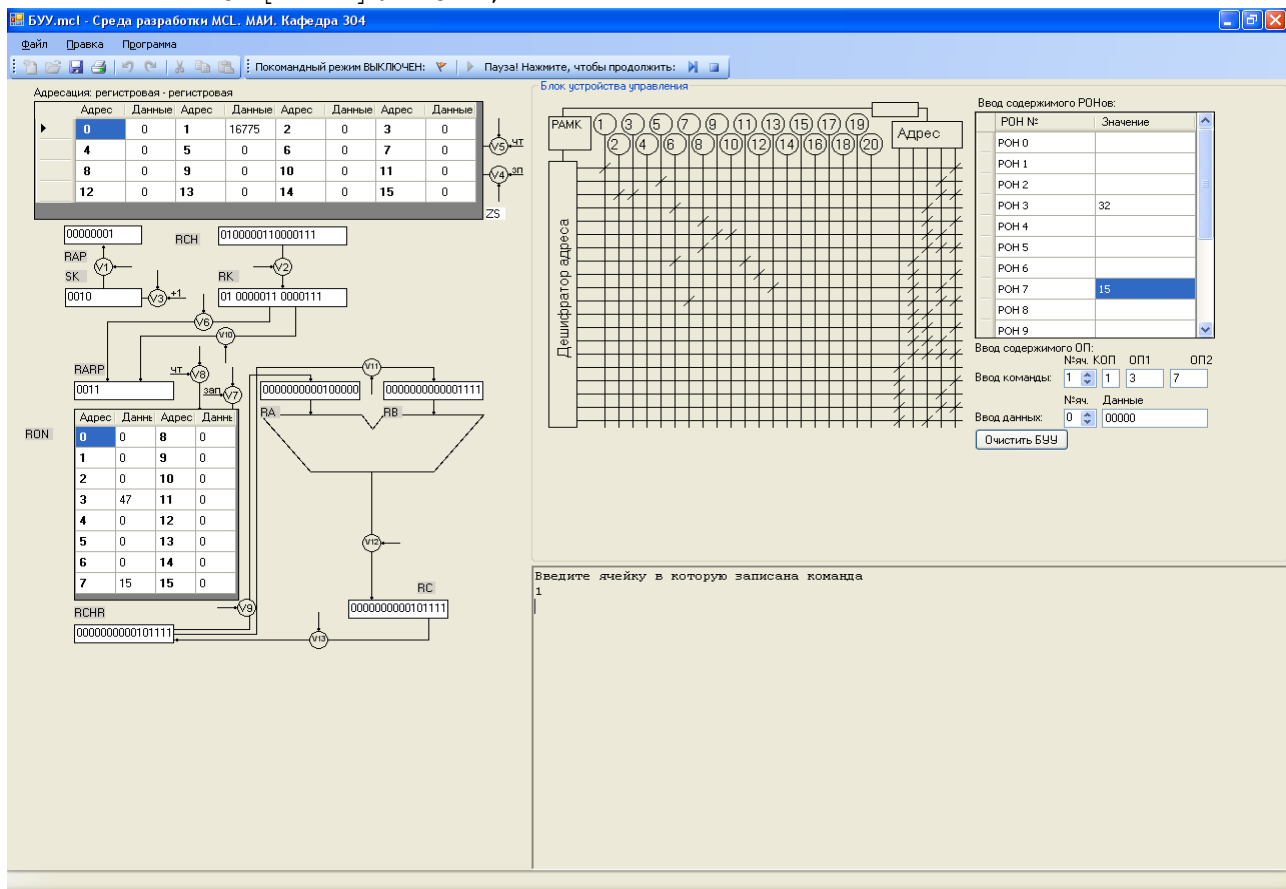


Рис. 42. Одиннадцатый этап выполнения программы

Примечание

Вентиль V4 при выполнении команды формата R-R не используется и поэтому в память ничего нельзя записывать; вентиль V4 используется при других способах адресации.

Содержание отчета

1. Структурная схема операционной части ЦП при выполнении команд в соответствии с заданными способами адресации.
2. Структурная схема горизонтального микропрограммного УУ в соответствии с вариантами задания.
3. Текст программы моделирования.
4. Распечатка результатов моделирования.

Задание к лабораторной работе

Промоделировать функционирование горизонтального микропрограммного УУ при выполнении 2-х адресной арифметико-логической команды, хранящейся на регистре команд. При моделировании использовать совмещение микроопераций во времени.

Принять, что:

- а) длина команды 16-ть бит и под код операции отводится 2 разряда. Разрядность полей, отводимых под операнды, 7 бит;
- б) объем памяти 16 ячеек по 16 бит;



- в) существует 16-ть регистров общего назначения разрядностью 16 бит;
- г) РАП - 4 разрядов,
РЧП - 16 разрядов,
РАРП - 4 разряда,
РЧРП - 16 разрядов.

КОП для всех вариантов представлять 1, что соответствует операции сложения. Задать содержимое РОН и ОП. Вывести на печать:

1. скриншот с заданным содержимым РОНов и ОП до выполнения программы;
2. скриншоты на каждом этапе выполнения микропрограммы.

Варианты заданий смотрите в таблице 4 из предыдущей лабораторной работы.

Контрольные вопросы

1. Микрооперация и микрокоманда.
2. Горизонтальный способ реализации УУ. Вертикальный способ реализации УУ. Связь с управляющими вентилями.
3. Способ определения адреса следующей микрокоманды в микропрограммном УУ.



Лабораторная работа № 6. Моделирование функционирования селекторного канала

Содержание

Цель работы	87
Функционирование селекторного канала	87
Структурная схема селекторного канала	87
Канальная команда	88
Структурная схема селекторного канала	90
Описание подпрограмм	91
Пример результатов моделирования работы селекторного канала	95
Содержание отчета	96
Задание к лабораторной работе	96



Цель работы

Изучение функционирования селекторного канала при обслуживании одного или нескольких высокоскоростных внешних устройств.

Функционирование селекторного канала

Структурная схема селекторного канала

Селекторный канал предназначен для обслуживания одного или нескольких подсоединенных к нему высокоскоростных ВУ с целью выполнения канальной программы. Структурная схема селекторного канала приведена на рис. 43. Графическое изображение окна программы при выборе лабораторной работы «Селекторный канал (процессор ввода-вывода)» приведено на рис. 44.

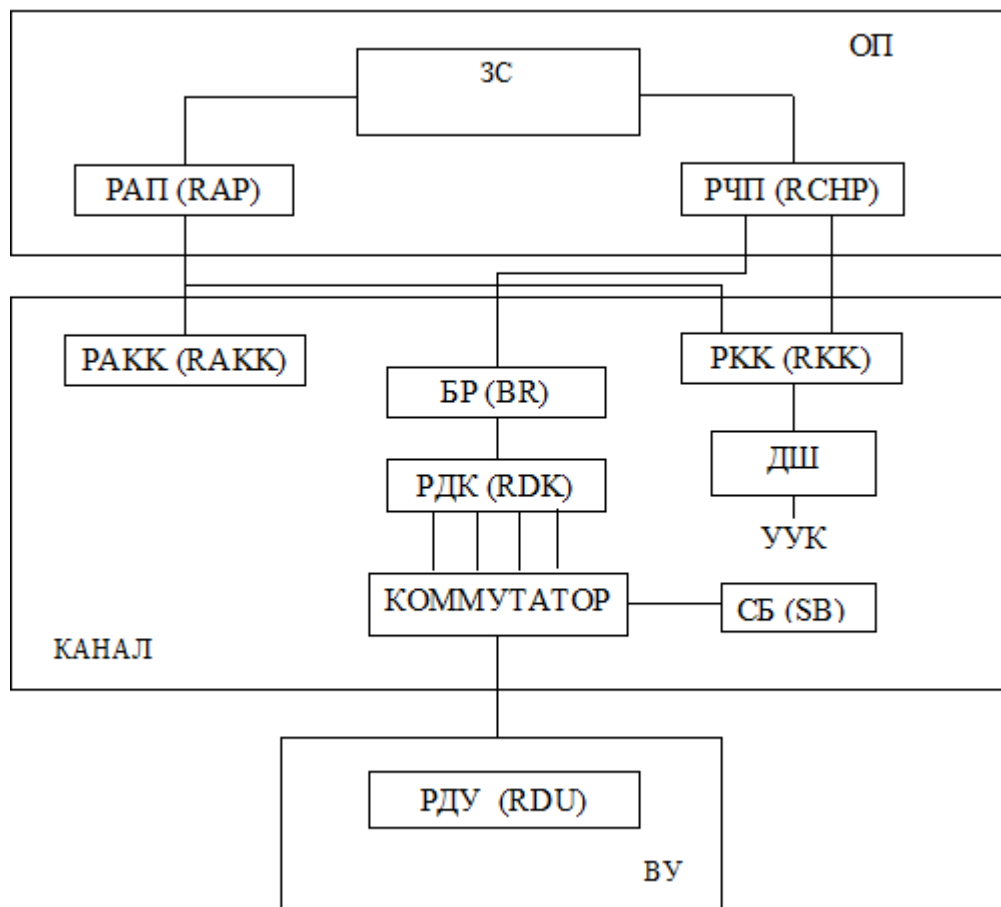


Рис. 43. Структурная схема селекторного канала

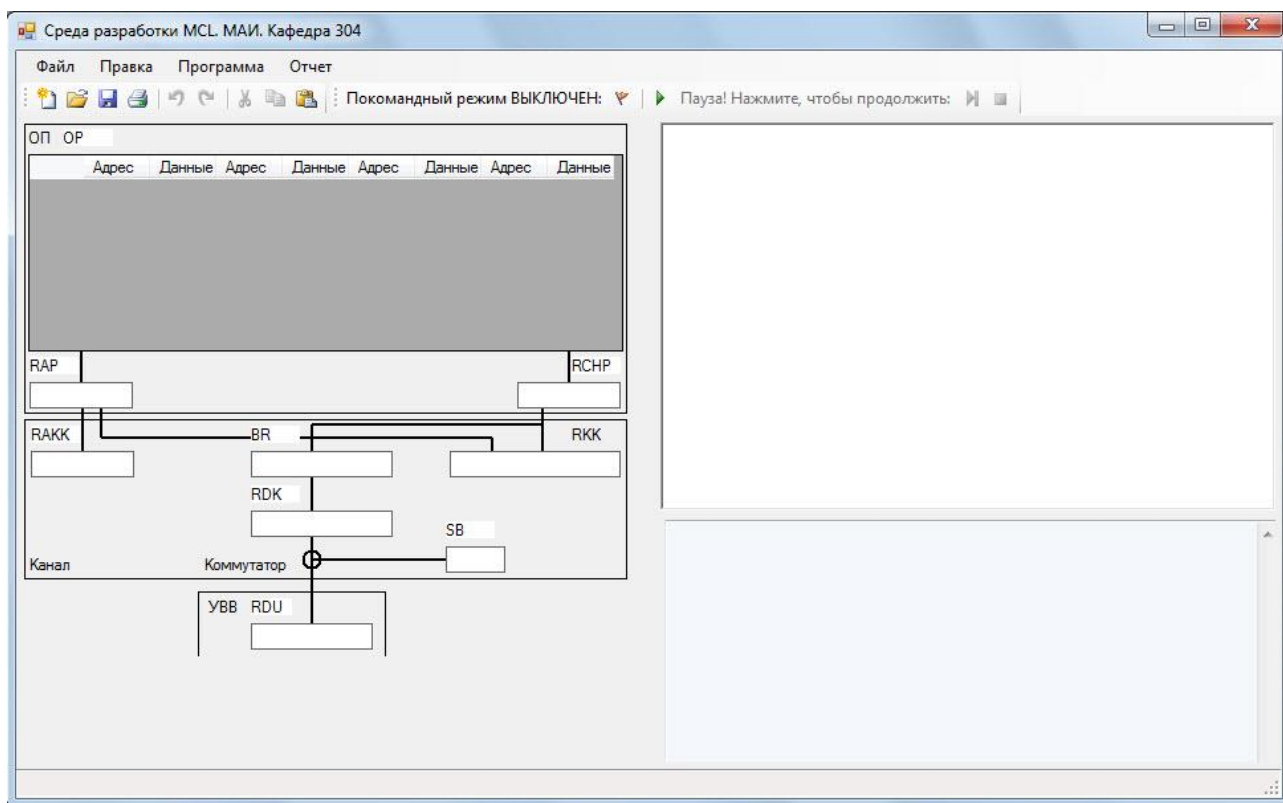


Рис. 44. Окно программы при выборе лабораторной работы «Селекторный канал (процессор ввода-вывода)»

Канальная команда

Адрес следующей выполняемой канальной команды (управляющего слова канала) хранится на регистре адреса канальной команды (РАКК). По этому адресу из ОП выбирается канальная команда, которая помещается на регистр канальной команды (РКК). Пусть одна канальная команда занимает 8 байт, поэтому при ширине выборки из памяти равной 4 байтам потребуется 2 обращения к ОП.

Формат канальной команды:

1 байт (1)	3 байта (2-4)	2 байта (5-6)	2 байта (7-8)
КОП	Поле адреса	Поле признаков	Поле длины массива

Будем обозначать соответствующие поля на РКК как РКК.КОП, РКК.АДР, РКК.ПРЗ и РКК.ДЛМ.

Поле кода операций (КОП) - занимает 1 байт. Пусть для лабораторной работы значение:

01 – означает операцию "чтение".

02 – означает операцию "запись".

Поле адреса (счетчик адреса) - занимает 3 байта. Оно указывает адрес ячейки ОП, в которой находится (или куда нужно записать) очередной элемент передаваемого массива данных.

Поле длины массива - занимает 2 байта. Оно указывает, сколько байт информации осталось передать. Равенство значения этого поля "0" указывает на то, что массив информации в соответствии с данной командой полностью передан, т.е. канальная команда выполнена. Далее анализируются признаки, находящиеся в поле признаков выполняемой команды.

Поле признаков - занимает 2 байта, из которых используется только старший (младший всегда равен 0 и рассматривать не будет). Из возможного набора признаков будем рассматривать "цепочку данных", "цепочку команд" и "конец канальной команды". Эти признаки и определяются значением 2-х старших разрядов поля признаков.



Признак	Значение старших разрядов поля	Значение поля признаков
цепочка данных	10	A000
цепочка команд	01	6000
конец канальной ком.	00	2000

Цепочка данных означает, что следующая канальная команда имеет тот же код операции, что и выполняемая, и при ее выборке из ОП поле РКК.КОП остается неизменным.

Цепочка команд означает, что следующая канальная команда имеет иной код операции, нежели выполняемая.

Конец канальной команды означает, что эта команда является последней. В реальных системах канал при обработке этого признака формирует запрос на прерывание ЦП.

Счетчик байт (СБ) представляет собой кольцевой (двухразрядный) счетчик. Младшие 2 бита поля адреса управляющего слова загружаются в счетчик байт.

Если выбранная команда оказалась командой записи, то из ОП через БП на регистр данных канала (РДК) считывается блок информации, равный по величине ширине выборки из памяти (в дальнейшем будем рассматривать память с шириной выборки 4 байта). Буферный регистр (БР) используется для увеличения скорости обработки: одновременно с записью информации из РДК на внешнее устройство (ВУ) происходит считывание данных из ОП на БР. После выборки информации из ОП на РДК происходит разгрузка РДК на регистр данных внешнего устройства (РДУ) блоками, равными по величине ширине информационного тракта ВУ (в дальнейшем - 1 байт). При считывании 1 байта с РДК на ВУ значения счетчика байт (СБ) и РКК.АДР увеличиваются на 1, а значение РКК.ДЛМ уменьшается на 1. По достижении значения, равного числу байтов в РДК (ширине выборки из ОП), СБ обнуляется, т.к. счетчик является кольцевым.

Если СБ или РКК.ДЛМ не равны 0, то происходит считывание с РДК следующего байта информации.

Если СБ=0, то значит массив данных из РДК полностью записан на ВУ, и, если РКК.ДЛМ не равно 0, нужно считать из ОП на РДК следующую порцию информации.

Если РКК.ДЛМ=0, то это означает, что массив полностью считан из ОП и записан на ВУ. Далее следует анализировать признак в поле признаков. В соответствии с признаками выбирается следующая канальная команда из ОП, а в случае, если выполняемая команда оказалась последней - формируется запрос на прерывание ЦП по вводу-выводу (в л/р означает конец моделирования).

Если выбранная команда оказалась командой чтения, то с ВУ считывается блок информации, величина которого равна ширине информационного тракта ВУ (1 байт), и помещается на РДК.

Номер байта в РДК, в который помещается блок информации, определяется по счетчику байт (СБ). Если СБ и РКК.ДЛМ не равны 0, с ВУ на РДК передается следующий байт информации.

После пересылки очередного байта информации значение СБ и РКК.АДР увеличивается на 1, а РКК.ДЛМ уменьшается на 1. Если СБ=0, то это означает, что РДК полностью заполнен, и происходит обращение к ОП для записи содержимого РДК. При этом для увеличения скорости обработки в канале используется буферный регистр (БР), в который пересылается содержимое РДК. После пересылки с РДК на БР канал начинает считывать новые порции информации с ВУ и помещать их на РДК. Если РКК.ДЛМ=0, то, независимо от значения СБ, также происходит запись содержимого РДК в ОП, т.к. массив в соответствии с текущей канальной командой полностью считан с ВУ. При этом СБ указывает, какое количество байт нужно переписать в ОП. После записи в ОП содержимого РДК при РКК.ДЛМ=0 анализируется поле признаков, как и при команде записи. Структурная схема алгоритма моделирования селекторного канала приведена на рис. 45.

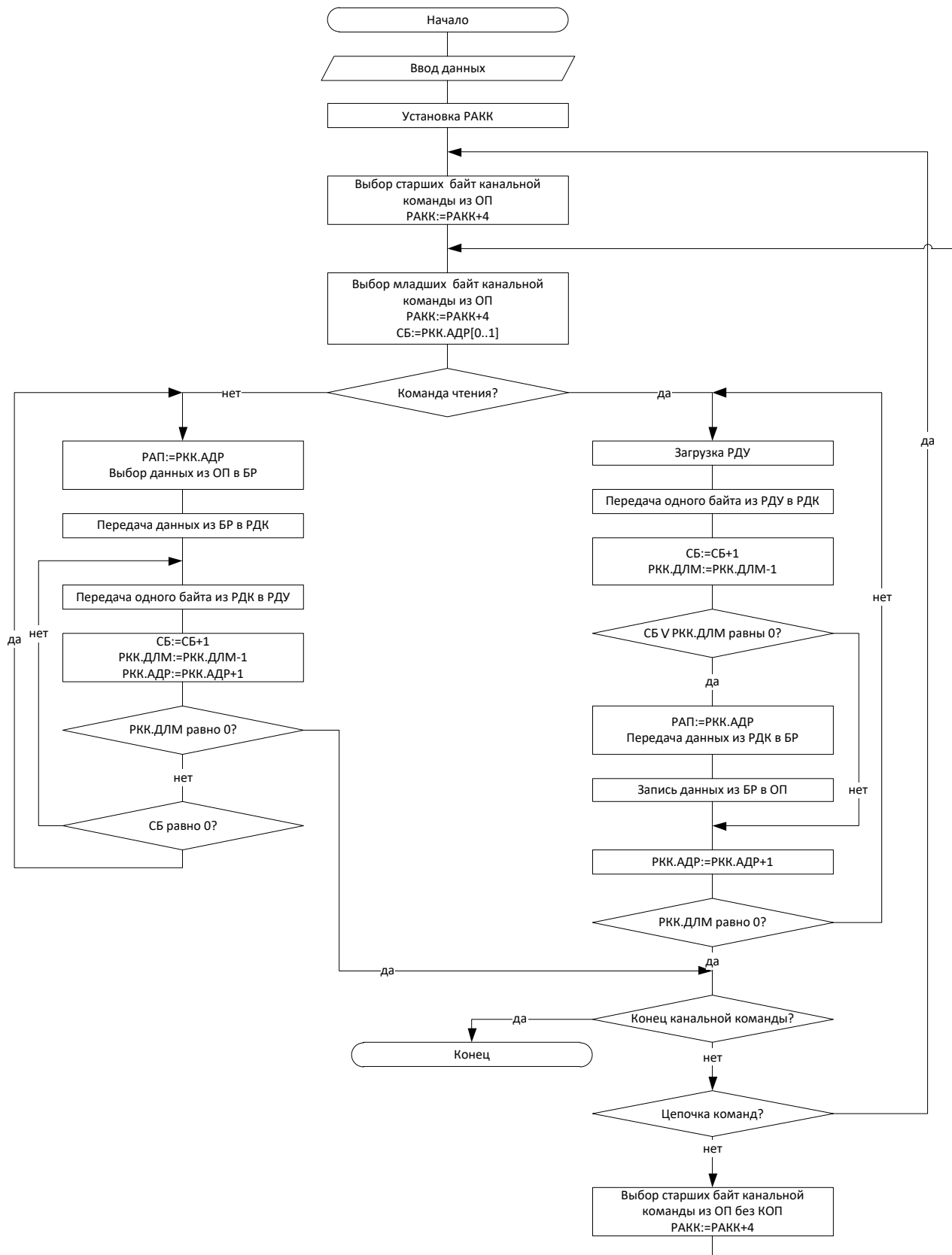


Рис. 45. Структурная схема алгоритма моделирования селекторного канала



Файл SK.MCL

В файле SK.MCL содержатся подпрограммы работы с памятью, передачи информации из РДК на ВУ (и наоборот), распечатки регистров канала и рабочих ячеек ОП. Эти подпрограммы рекомендуется использовать для моделирования работы селекторного канала. В них приняты следующие обозначения переменных и регистров (в квадратных скобках - размерность регистров):

WN - номер варианта

ADDR[24] - после выполнения подпрограммы ZAGR содержит адрес первой кан. команды

RAKK[24] - регистр адреса канальной команды (РАКК)

RAP[24] - регистр адреса памяти (РАП)

RCHP[32] - регистр числа памяти (РЧП)

BR[32] - буферный регистр данных (БР)

RDK[32] - регистр данных канала (РДК)

RDU[8] - регистр данных внешнего устройства (РДУ)

SB[2] - счетчик байт (СБ)

Регистр канальной команды (РКК) 64-х разрядный. Т.к. в языке MCL максимальная размерность регистра - 32 разряда, то этот регистр представлен в виде двух структур:

RKK1[32] с полями KOP[8] и ADR[24] - старшая часть РКК

RKK2[32] с полями PRZ[16] и DLM[16] - младшая часть РКК

RKK1.KOP	RKK1.ADR	RKK2.PRZ	RKK2.DLM
----------	----------	----------	----------

Оперативная память (ОП) описана как MEM OP[1024][8] - линейная память из 1024 ячеек по 1 байту каждая. Подпрограммы работы с памятью представляют, что её ширина выборки равна 4 байта.

MEM PU[16][8] - память для хранения массива чисел, считываемых с ВУ (заполняется, когда есть операция "чтение"). Все числа выводятся в 16-ричной системе счисления. Остальные переменные, начинающиеся с буквы W, используются как рабочие переменные внутри подпрограмм и физического смысла не имеют, поэтому использовать их в основной программе не рекомендуется.

Описание подпрограмм

ПОДПРОГРАММА K_U_K:

K_U_K:

WU:=24-SB*8;

IF RKK1.KOP=1 THEN RDK[WU..(WU+7)]:=RDU;

IF RKK1.KOP=2 THEN RDU:=RDK[WU..(WU+7)];

RETURN;

Записывает в РДК из РДУ (или наоборот) 1 байт информации. Направление передачи определяется по коду операции выполняемой команды. Номер байта в РДК, в который (из которого) будет записана (считана) информация, определяется значением счетчика байт. Байты в РДК нумеруются слева направо.

ПОДПРОГРАММА W_REG:

W_REG:

WRITELN " RKK СЧ.БАЙТ РДК РДУ РАКК ";

WRITE \$H2 RKK1.KOP, " ", \$H6 RKK1.ADR, " ";

WRITE \$H4 RKK2.PRZ, " ", \$H4 RKK2.DLM, " ";



```

WRITE $H1 SB, "      ", $H2 RDK[24..31], " ";
WRITE $H2 RDK[16..23], " ", $H2 RDK[8..15], " ";
WRITELN $H2 RDK[0..7], " ", $H2 RDU, " ", $H6 RAKK;
RETURN;

```

Распечатывает регистры канала РДК, СБ, РДК и РДУ. Вывод всех чисел производится в 16-ричной системе счисления и в виде:

РКК	СЧ.БАЙТ	РДК	РДУ
01 000045 6000 0003	01 03 19 C4 2A	03	

ПОДПРОГРАММА OUT_OP:

```

OUT_OP:
RAP[0..1]:=0;
FOR WI:=0 TO 3;
WU:=24-WI*8;
RCHP[WU..(WU+7)]:=OP[RAP+WI];
NEXT;
RETURN;

```

Производит выборку из ОП содержимого 4-х ячеек (в соответствии с шириной выборки) на регистр РЧП в соответствии со значением старшей части адреса, находящегося на регистре РАП (два младших разряда регистра РАП обнуляются). Эта подпрограмма реализует микрокоманду обращения к ОП: РЧП := ЧТ(РАП)

ПОДПРОГРАММА IN_OP:

```

IN_OP:
FOR WI:=0 TO 3;
WU:=24-WI*8;
OP[RAP+WI]:=RCHP[WU..(WU+7)];
NEXT;
RETURN;

```

Производит запись в ОП содержимого регистра РЧП (в четыре последовательные ячейки), начиная с адреса, находящегося в регистре РАП (этот адрес должен быть кратным четырем). Эта подпрограмма используется в подпрограмме К_OP.

ПОДПРОГРАММА К_OP:

```

K_OP:
{RAP-Д/Б РАНЕЕ ОПРЕДЕЛЕНО}
GOSUB OUT_OP;
W1:=(NOT (SB-1))*8;
RCHP[W1..31]:=BR[W1..31];
GOSUB IN_OP;
RETURN;

```

Производит запись в ОП блока информации из канала (БР), начиная с адреса, находящегося в регистре РАП (этот адрес должен быть кратным ширине выборки). Число байт информации из БР, которое нужно занести в память, определяется значением счетчика байт SB. Число записываемых в ОП байт меньше 4 при передаче последних байт массива информации, длина которого не кратна 4 (ширине выборки).



Работа подпрограммы: по адресу в РАП на РЧП считывается содержимое соответствующих ячеек ОП (с помощью подпрограммы OUT_OP). Затем на РЧП заменяется требуемое количество байт (равное значению SB; если SB=0, то заменяются все 4 байта) информацией из регистра БР и измененное содержимое РЧП записывается обратно в ОП (с помощью подпрограммы IN_OP).

ПОДПРОГРАММА W_OP:

```
W_OP:
  WRITELN " СОДЕРЖИМОЕ ЯЧЕЕК ОП:";
  FOR WK:=0 TO 1;
  FOR WI:=0 TO 1;
  FOR WJ:=0 TO 3;
  WU:=OP[ADDR+3+WK*8]+WI*4+WJ;
  WRITE $H6 WU," ", $H2 OP[WU]," * ";
  NEXT;
  WRITELN;
  NEXT;NEXT;
RETURN;
```

Распечатывает содержимое ячеек ОП. Распечатываются 16 ячеек - по 8 ячеек, начиная с адресов, указанных в поле адреса каждой канальной команды. В одной строке распечатываются 4 ячейки (в соответствии с шириной выборки).

Например для варианта канальной программы

```
01 000014 6000 0006
02 000038 2000 0003
```

будет получена распечатка вида:

```
СОДЕРЖИМОЕ ЯЧЕЕК ОП:
000014 03 * 000015 46 * 000016 89 * 000017 CC
000018 00 * 000019 00 * 000019 00 * 00001A 00
000038 05 * 000039 FC * 00003A F3 * 00003B C6
00003C 31 * 00003D D8 * 00003E CF * 00003F C6
          /      \
        адрес ячейки  содержимое ячейки
```

ПОДПРОГРАММА Z_RDU:

```
Z_RDU:
  RDU:=PU[WR]; WR:=WR+1;
RETURN;
```

Загружает регистр РДУ очередным значением из PU. Используется при операции "чтение".

ПОДПРОГРАММА ZAGR:

```
ZAGR:
  READ " ВВЕДИТЕ НОМЕР ВАРИАНТА ",WN;
  ADDR:=200+WN*16;
  WR:=0; W7:=WN*7; WF:=0;
  OP[ADDR]:=2; OP[ADDR+8]:=2;
  IF WN[0..0]=1 THEN OP[ADDR]:=1;
  IF WN[0..0]=0 THEN OP[ADDR+8]:=WN[1..1];
```



```
OP[ADDR+7]:=7-W7[0..1]; OP[ADDR+15]:=2+W7[0..1];
IF OP[ADDR+8]=0 THEN OP[ADDR+4]:=160;
IF OP[ADDR+8]<>0 THEN OP[ADDR+4]:=96;
OP[ADDR+12]:=32;
OP[(ADDR+3),2..4]:=W7[0..2]; OP[(ADDR+3),5..6]:=WN[2..3];
OP[ADDR+11]:=OP[ADDR+3]+24+4*WN[0..1];
FOR WI:=0 TO 1;
WJ:=ADDR+8*WI;
IF OP[WJ]=1 THEN WF:=WF+OP[WJ+7];
IF OP[WJ]<>1 THEN GOSUB Z_OP;
NEXT;
WRITELN "ВАРИАНТ НОМЕР ", $D3 WN;
WRITELN "АДРЕС ПЕРВОЙ КАН. КОМАНДЫ ", $H6 ADDR;
WRITELN "КАНАЛЬНАЯ ПРОГРАММА: ";
FOR WI:=0 TO 1;
WRITE " ", $H2 OP[ADDR+WI*8], " ", $H2 OP[ADDR+1+WI*8];
WRITE $H2 OP[ADDR+2+WI*8], $H2 OP[ADDR+3+WI*8], " ";
WRITE $H2 OP[ADDR+4+WI*8], $H2 OP[ADDR+5+WI*8], " ";
WRITELN $H2 OP[ADDR+6+WI*8], $H2 OP[ADDR+7+WI*8];
NEXT;
GOSUB W_OP;
IF WF=0 THEN GOTO WM1;
WRITELN "ЧИСЛА, СЧИТЫВАЕМЫЕ С ВУ:";
FOR WI:=0 TO (WF-1);
PU[WI]:=3+WI*WN*17;
WRITE " ", $H2 PU[WI];
NEXT;
WM1:
WRITELN;
WRITELN "*****";
RETURN;
Z_OP:
FOR WK:=0 TO 7;
OP[OP[WJ+3]+WK]:=5+WK*WN*13+WJ;
NEXT;
RETURN;
```

Формирует исходные данные для моделирования по-задаваемому с клавиатуры номеру варианта WN. Изучение работы данной подпрограммы для студентов не требуется! Подпрограмма формирует канальную программу; адрес первой канальной команды ADDR, начиная с которого загружает канальную программу в ОП; числа, считываемые с ВУ (если есть операция чтения); начальную загрузку данных в ОП (для записи на ВУ) и производит распечатку всех исходных данных в виде:

```
ВАРИАНТ НОМЕР 19
АДРЕС ПЕРВОЙ КАН. КОМАНДЫ 0001F8
КАНАЛЬНАЯ ПРОГРАММА:
01 000014 6000 0006
02 000038 2000 0003
СОДЕРЖИМОЕ ЯЧЕЕК ОП:
000014 00 * 000015 00 * 000016 00 * 000017 00
000018 00 * 000019 00 * 00001A 00 * 00001B 00
```



000038 05 * 000039 FC * 00003A F3 * 00003B EA
00003C E1 * 00003D D8 * 00003E CF * 00003F C6
ЧИСЛА, СЧИТЫВАЕМЫЕ С ВУ:
03 46 89 CC 0F 52

Пример результатов моделирования работы селекторного канала

Рассмотрим результат моделирования селекторного канала в соответствии с заданием для варианта №19. Распечатка результатов моделирования имеет вид:

ВАРИАНТ НОМЕР 19
АДРЕС ПЕРВОЙ КАН. КОМАНДЫ 0001F8
КАНАЛЬНАЯ ПРОГРАММА:
01 000014 6000 0006
02 000038 2000 0003
СОДЕРЖИМОЕ ЯЧЕЕК ОП:
000014 00 * 000015 00 * 000016 00 * 000017 00 *
000018 00 * 000019 00 * 00001A 00 * 00001B 00 *
000038 05 * 000039 FC * 00003A F3 * 00003B EA *
00003C E1 * 00003D D8 * 00003E CF * 00003F C6 *
ЧИСЛА, СЧИТЫВАЕМЫЕ С ВУ:
03 46 89 CC 0F 52

РАКК=0001F8
РКК СЧ.БАЙТ РДК РДУ РАКК
01 000014 6000 0006 0 00 00 00 00 00 000200
РКК СЧ.БАЙТ РДК РДУ РАКК
01 000015 6000 0005 1 03 00 00 00 03 000200
РКК СЧ.БАЙТ РДК РДУ РАКК
01 000016 6000 0004 2 03 46 00 00 46 000200
РКК СЧ.БАЙТ РДК РДУ РАКК
01 000017 6000 0003 3 03 46 89 00 89 000200
РКК СЧ.БАЙТ РДК РДУ РАКК
01 000018 6000 0002 0 03 46 89 CC CC 000200
СОДЕРЖИМОЕ ЯЧЕЕК ОП:
000014 03 * 000015 46 * 000016 89 * 000017 CC *
000018 00 * 000019 00 * 00001A 00 * 00001B 00 *
000038 05 * 000039 FC * 00003A F3 * 00003B EA *
00003C E1 * 00003D D8 * 00003E CF * 00003F C6 *
РКК СЧ.БАЙТ РДК РДУ РАКК
01 000019 6000 0001 1 0F 46 89 CC 0F 000200
РКК СЧ.БАЙТ РДК РДУ РАКК
01 00001A 6000 0000 2 0F 52 89 CC 52 000200
СОДЕРЖИМОЕ ЯЧЕЕК ОП:
000014 03 * 000015 46 * 000016 89 * 000017 CC *
000018 0F * 000019 52 * 00001A 00 * 00001B 00 *
000038 05 * 000039 FC * 00003A F3 * 00003B EA *
00003C E1 * 00003D D8 * 00003E CF * 00003F C6 *
РКК СЧ.БАЙТ РДК РДУ РАКК
02 000038 2000 0003 0 05 FC F3 EA 52 000208
РКК СЧ.БАЙТ РДК РДУ РАКК
02 000039 2000 0002 1 05 FC F3 EA 05 000208
РКК СЧ.БАЙТ РДК РДУ РАКК
02 00003A 2000 0001 2 05 FC F3 EA FC 000208
РКК СЧ.БАЙТ РДК РДУ РАКК
02 00003B 2000 0000 3 05 FC F3 EA F3 000208



Данная канальная программа состоит из 2-х канальных команд. Первая из них хранится в ОП, начиная с адреса 0001F8, а вторая - с 000200 (определяется как 0001F8+8, т.к. длина одной команды равна 8 байт).

В начале в РАКК помещается адрес первой канальной команды, по-которому она из ОП выбирается из ОП и помещается на РКК. Т.к. ширина выборки ОП равна 4, то канальная команда выбирается из ОП за 2 обращения. После каждого обращения в ОП при выборке канальной команды РАКК увеличивается на 4.

После выборки команды из ОП анализируется ее поле кода операции (РКК.КОП). Первая команда оказалась командой чтения, поэтому с ВУ (РДУ) начинается побайтно считываться информация на РДК. При передаче каждого байта на РДК значение СБ и РКК.АДР увеличивается на 1, а РКК.ДЛМ - уменьшается на 1. После передачи 4-х байт информации СБ стал равен 0, что означает, что РДК полностью заполнен. При этом на регистр РАП помещается содержимое поля РКК.АДР и в соответствии со значением его старшей части происходит перепись полученной информации из РДК через БР в ОП. Затем в РДК начинает поступать следующая порция информации. После передачи очередных 2-х байт из ВУ РКК.ДЛМ стало равным 0, что означает, что массив информации из ВУ полностью передан. Поэтому на РАП помещается содержимое поля РКК.АДР и в соответствии со значением его старшей части происходит перепись содержимого РДК в ОП (через БР), при этом значение СБ указывает, сколько байт передается в ОП (сколько байт на РЧП нужно изменить). После записи информации в ОП СБ обнуляется.

Затем анализируется поле признаков РКК.ПРЗ - в данном случае признак "цепочка команд", поэтому выбирается из ОП и помещается на РКК следующая канальная команда и при этом поле РКК.КОП изменяется. Вторая канальная команда оказалась командой записи, поэтому в РАП загружается значение РКК.АДР и из ОП на РДК через БР поступает порция информации, равная ширине выборки ОП. Затем начинается побайтное считывание информации с РДК на ВУ (РДУ). При передаче каждого байта, как и при операции "чтение", увеличивается на 1 значение СБ и РКК.АДР и уменьшается на 1 значение РКК.ДЛМ. После передачи 3-х байт с РДК на РДУ, РКК.ДЛМ стало равно 0. Это означает, что весь массив информации записан на ВУ. При анализе поля признаков РКК.ПРЗ выясняется, что данная канальная команда была последней. На этом моделирование завершается.

Содержание отчета

1. Формулировка задания.
2. Структурная схема селекторного канала.
3. Структурная схема алгоритма моделирования селекторного канала.
4. Программа на языке MCL, моделирующая функционирование селекторного канала.
5. Результаты выполнения программы в **виде распечатки** содержимого поля для вывода текстовой информации **и скриншота** в момент завершения выполнения программы.

Контрольные вопросы

1. Формат канальной команды.
2. Начало работы селекторного канала.
3. Окончание выполнения канальной команды.

Задание к лабораторной работе

Промоделировать работу селекторного канала при выполнении канальной программы в соответствии с вариантом задания. В процессе моделирования распечатать содержимое регистров канала и ячеек памяти в моменты их изменения.



ВАРИАНТ НОМЕР 1

АДРЕС ПЕРВОЙ КК 0000D8
КАНАЛЬНАЯ ПРОГРАММА:
01 00001C 6000 0004
02 000038 2000 0005
СОДЕРЖИМОЕ ЯЧЕЕК ОП:
00001C 00 * 00001D 00 * 00001E 00 * 00001F 00 *
000020 00 * 000021 00 * 000022 00 * 000023 00 *
000038 E5 * 000039 F2 * 00003A FF * 00003B 0C *
00003C 19 * 00003D 26 * 00003E 33 * 00003F 40 *
ЧИСЛА, СЧИТЫВАЕМЫЕ С ВУ:
03 14 25 36

ВАРИАНТ НОМЕР 2

АДРЕС ПЕРВОЙ КК 0000E8
КАНАЛЬНАЯ ПРОГРАММА:
02 000018 6000 0005
01 000038 2000 0004
СОДЕРЖИМОЕ ЯЧЕЕК ОП:
000018 ED * 000019 07 * 00001A 21 * 00001B 3B *
00001C 55 * 00001D 6F * 00001E 89 * 00001F A3 *
000038 00 * 000039 00 * 00003A 00 * 00003B 00 *
00003C 00 * 00003D 00 * 00003E 00 * 00003F 00 *
ЧИСЛА, СЧИТЫВАЕМЫЕ С ВУ:
03 25 47 69

ВАРИАНТ НОМЕР 3

АДРЕС ПЕРВОЙ КК 0000F8
КАНАЛЬНАЯ ПРОГРАММА:
01 000014 6000 0006
02 000038 2000 0003
СОДЕРЖИМОЕ ЯЧЕЕК ОП:
000014 00 * 000015 00 * 000016 00 * 000017 00 *
000018 00 * 000019 00 * 00001A 00 * 00001B 00 *
000038 05 * 000039 2C * 00003A 53 * 00003B 7A *
00003C A1 * 00003D C8 * 00003E EF * 00003F 16 *
ЧИСЛА, СЧИТЫВАЕМЫЕ С ВУ:
03 36 69 9C CF 02

ВАРИАНТ НОМЕР 4

АДРЕС ПЕРВОЙ КК 000108
КАНАЛЬНАЯ ПРОГРАММА:
02 000030 A000 0007
00 000048 2000 0002
СОДЕРЖИМОЕ ЯЧЕЕК ОП:
000030 0D * 000031 41 * 000032 75 * 000033 A9 *
000034 DD * 000035 11 * 000036 45 * 000037 79 *
000048 15 * 000049 49 * 00004A 7D * 00004B B1 *
00004C E5 * 00004D 19 * 00004E 4D * 00004F 81 *

ВАРИАНТ НОМЕР 5

АДРЕС ПЕРВОЙ КК 000118
КАНАЛЬНАЯ ПРОГРАММА:
01 00002C 6000 0004
02 000048 2000 0005
СОДЕРЖИМОЕ ЯЧЕЕК ОП:
00002C 00 * 00002D 00 * 00002E 00 * 00002F 00 *
000030 00 * 000031 00 * 000032 00 * 000033 00 *
000048 25 * 000049 66 * 00004A A7 * 00004B E8 *
00004C 29 * 00004D 6A * 00004E AB * 00004F EC *
ЧИСЛА, СЧИТЫВАЕМЫЕ С ВУ:
03 58 AD 02



ВАРИАНТ НОМЕР 6

АДРЕС ПЕРВОЙ КК 000128
КАНАЛЬНАЯ ПРОГРАММА:
02 000028 6000 0005
01 000048 2000 0004
СОДЕРЖИМОЕ ЯЧЕЕК ОП:
000028 2D * 000029 7B * 00002A C9 * 00002B 17 *
00002C 65 * 00002D B3 * 00002E 01 * 00002F 4F *
000048 00 * 000049 00 * 00004A 00 * 00004B 00 *
00004C 00 * 00004D 00 * 00004E 00 * 00004F 00 *
ЧИСЛА, СЧИТЫВАЕМЫЕ С ВУ:
03 69 CF 35

ВАРИАНТ НОМЕР 7

АДРЕС ПЕРВОЙ КК 000138
КАНАЛЬНАЯ ПРОГРАММА:
01 000024 6000 0006
02 000048 2000 0003
СОДЕРЖИМОЕ ЯЧЕЕК ОП:
000024 00 * 000025 00 * 000026 00 * 000027 00 *
000028 00 * 000029 00 * 00002A 00 * 00002B 00 *
000048 45 * 000049 A0 * 00004A FB * 00004B 56 *
00004C B1 * 00004D 0C * 00004E 67 * 00004F C2 *
ЧИСЛА, СЧИТЫВАЕМЫЕ С ВУ:
03 7A F1 68 DF 56

ВАРИАНТ НОМЕР 8

АДРЕС ПЕРВОЙ КК 000148
КАНАЛЬНАЯ ПРОГРАММА:
02 000040 A000 0007
00 000058 2000 0002
СОДЕРЖИМОЕ ЯЧЕЕК ОП:
000040 4D * 000041 B5 * 000042 1D * 000043 85 *
000044 ED * 000045 55 * 000046 BD * 000047 25 *
000058 55 * 000059 BD * 00005A 25 * 00005B 8D *
00005C F5 * 00005D 5D * 00005E C5 * 00005F 2D *

ВАРИАНТ НОМЕР 9

АДРЕС ПЕРВОЙ КК 000158
КАНАЛЬНАЯ ПРОГРАММА:
01 00005C 6000 0004
02 000078 2000 0005
СОДЕРЖИМОЕ ЯЧЕЕК ОП:
00005C 00 * 00005D 00 * 00005E 00 * 00005F 00 *
000060 00 * 000061 00 * 000062 00 * 000063 00 *
000078 65 * 000079 DA * 00007A 4F * 00007B C4 *
00007C 39 * 00007D AE * 00007E 23 * 00007F 98 *
ЧИСЛА, СЧИТЫВАЕМЫЕ С ВУ:
03 9C 35 CE

ВАРИАНТ НОМЕР 10

АДРЕС ПЕРВОЙ КК 000168
КАНАЛЬНАЯ ПРОГРАММА:
02 000058 6000 0005
01 000078 2000 0004
СОДЕРЖИМОЕ ЯЧЕЕК ОП:
000058 6D * 000059 EF * 00005A 71 * 00005B F3 *
00005C 75 * 00005D F7 * 00005E 79 * 00005F FB *
000078 00 * 000079 00 * 00007A 00 * 00007B 00 *
00007C 00 * 00007D 00 * 00007E 00 * 00007F 00 *
ЧИСЛА, СЧИТЫВАЕМЫЕ С ВУ:
03 AD 57 01



ВАРИАНТ НОМЕР 11

АДРЕС ПЕРВОЙ КК 000178
КАНАЛЬНАЯ ПРОГРАММА:
01 000054 6000 0006
02 000078 2000 0003
СОДЕРЖИМОЕ ЯЧЕЕК ОП:
000054 00 * 000055 00 * 000056 00 * 000057 00 *
000058 00 * 000059 00 * 00005A 00 * 00005B 00 *
000078 85 * 000079 14 * 00007A A3 * 00007B 32 *
00007C C1 * 00007D 50 * 00007E DF * 00007F 6E *
ЧИСЛА, СЧИТЫВАЕМЫЕ С ВУ:
03 BE 79 34 EF AA

ВАРИАНТ НОМЕР 12

АДРЕС ПЕРВОЙ КК 000188
КАНАЛЬНАЯ ПРОГРАММА:
02 000070 A000 0007
00 000088 2000 0002
СОДЕРЖИМОЕ ЯЧЕЕК ОП:
000070 8D * 000071 29 * 000072 C5 * 000073 61 *
000074 FD * 000075 99 * 000076 35 * 000077 D1 *
000088 95 * 000089 31 * 00008A CD * 00008B 69 *
00008C 05 * 00008D A1 * 00008E 3D * 00008F D9 *

ВАРИАНТ НОМЕР 13

АДРЕС ПЕРВОЙ КК 000198
КАНАЛЬНАЯ ПРОГРАММА:
01 00006C 6000 0004
02 000088 2000 0005
СОДЕРЖИМОЕ ЯЧЕЕК ОП:
00006C 00 * 00006D 00 * 00006E 00 * 00006F 00 *
000070 00 * 000071 00 * 000072 00 * 000073 00 *
000088 A5 * 000089 4E * 00008A F7 * 00008B A0 *
00008C 49 * 00008D F2 * 00008E 9B * 00008F 44 *
ЧИСЛА, СЧИТЫВАЕМЫЕ С ВУ:
03 E0 BD 9A

ВАРИАНТ НОМЕР 14

АДРЕС ПЕРВОЙ КК 0001A8
КАНАЛЬНАЯ ПРОГРАММА:
02 000068 6000 0005
01 000088 2000 0004
СОДЕРЖИМОЕ ЯЧЕЕК ОП:
000068 AD * 000069 63 * 00006A 19 * 00006B CF *
00006C 85 * 00006D 3B * 00006E F1 * 00006F A7 *
000088 00 * 000089 00 * 00008A 00 * 00008B 00 *
00008C 00 * 00008D 00 * 00008E 00 * 00008F 00 *
ЧИСЛА, СЧИТЫВАЕМЫЕ С ВУ:
03 F1 DF CD

ВАРИАНТ НОМЕР 15

АДРЕС ПЕРВОЙ КК 0001B8
КАНАЛЬНАЯ ПРОГРАММА:
01 000064 6000 0006
02 000088 2000 0003
СОДЕРЖИМОЕ ЯЧЕЕК ОП:
000064 00 * 000065 00 * 000066 00 * 000067 00 *
000068 00 * 000069 00 * 00006A 00 * 00006B 00 *
000088 C5 * 000089 88 * 00008A 4B * 00008B 0E *
00008C D1 * 00008D 94 * 00008E 57 * 00008F 1A *
ЧИСЛА, СЧИТЫВАЕМЫЕ С ВУ:
03 02 E9 00



Лабораторная работа № 7. Моделирование функционирования мультимплексного канала

Содержание

Цель работы_____	101
Функционирование мультимплексного канала_____	101
Структурная схема мультимплексного канала_____	103
Описание подпрограмм_____	105
Пример результатов моделирования работы мультимплексного канала_____	110
Содержание отчета_____	113
Задание к лабораторной работе_____	113



Цель работы

Изучение функционирования мультиплексного канала при одновременном обслуживании нескольких ВУ.

Функционирование мультиплексного канала

Мультиплексный канал предназначен для одновременного обслуживания нескольких ВУ, каждое из которых работает в соответствии с отдельной канальной программой. Структурная схема мультиплексного канала приведена на рис. 46. Графическое изображение окна программы при выборе лабораторной работы «Мультиплексный канал (процессор ввода-вывода)» приведено на рис. 47.

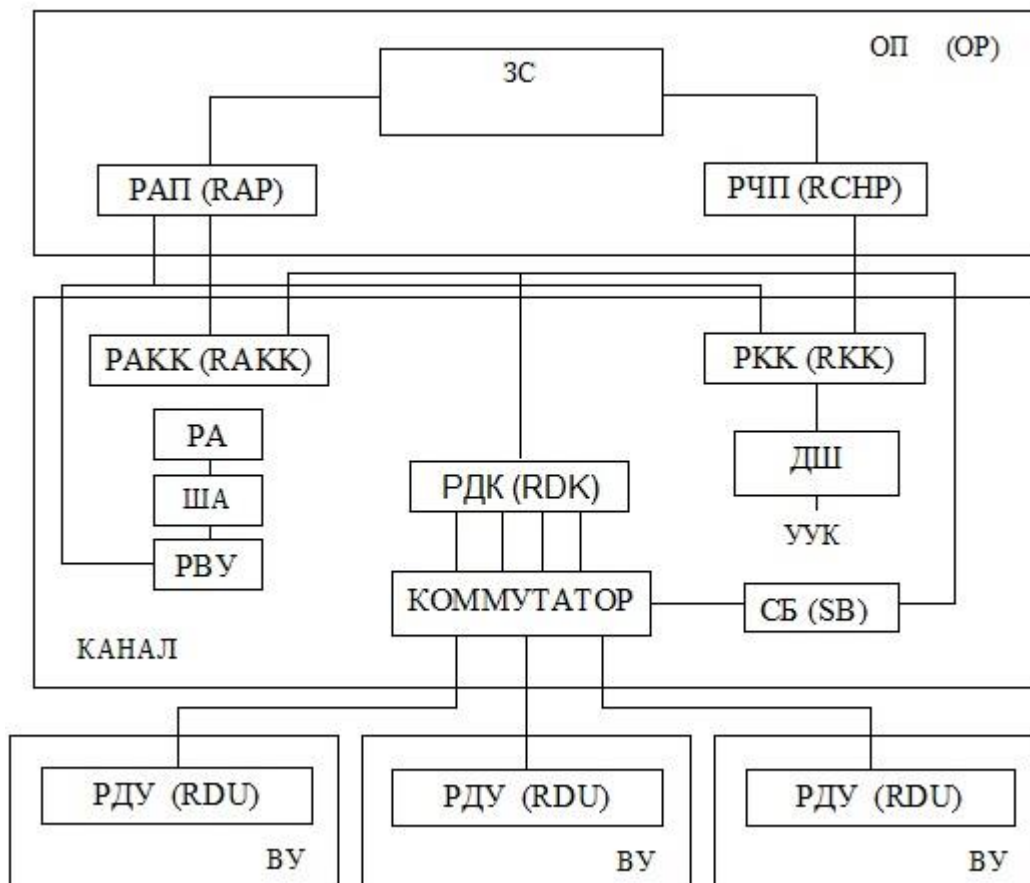


Рис. 46. Структурная схема мультиплексного канала

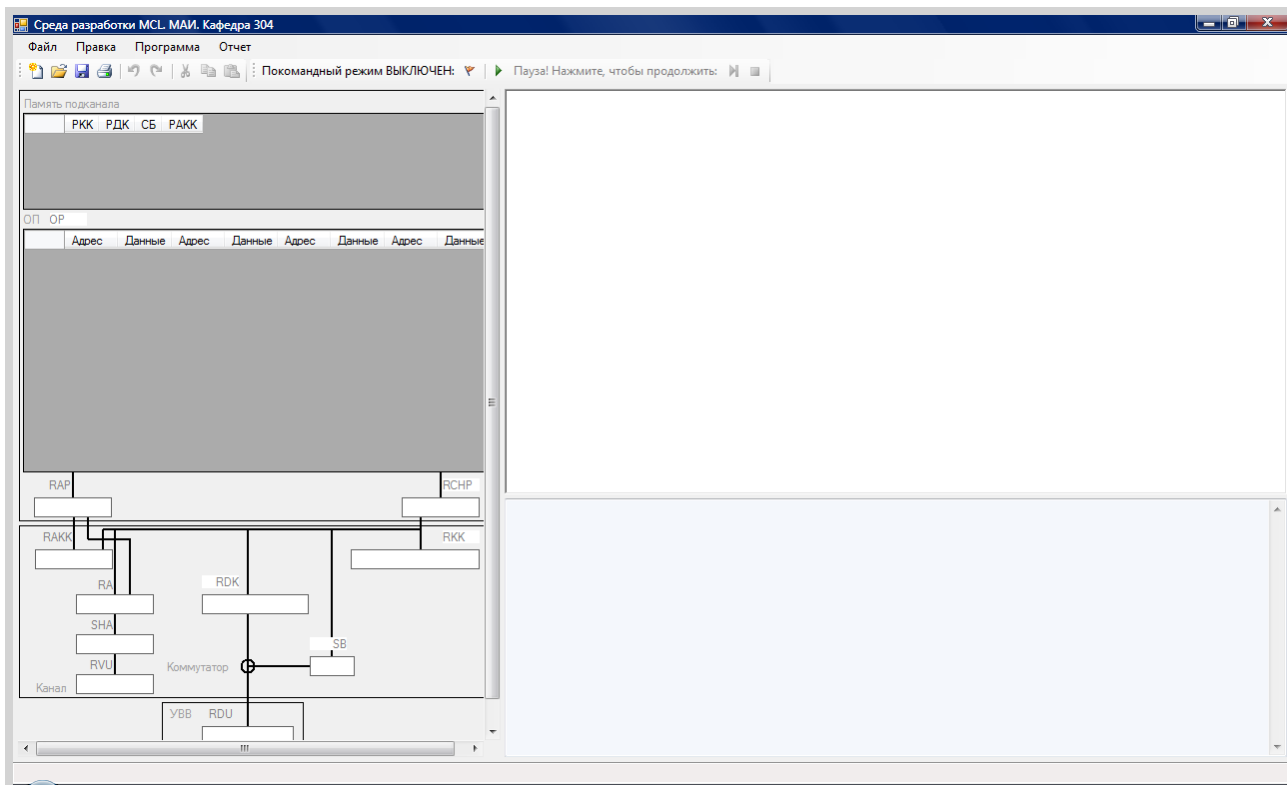


Рис. 47. Окно программы при выборе лабораторной работы «Мультиплексный канал (процессор ввода-вывода)»

Аппаратные средства мультиплексного канала можно разделить на две части: средства, предназначенные для обслуживания отдельных ВУ, и средства, используемые всеми ВУ. Средства, используемые для обслуживания одного ВУ, называются подканалом, который представляет собой запоминающее устройство. Будем считать, что память подканалов реализована как часть ОП. Средства, используемые всеми ВУ - регистры канала РКК, РДК, СБ и РАКК. Они аналогичны соответствующим регистрам селекторного канала (примем те же обозначения регистров и их полей). Как и при моделировании селекторного канала, примем, что ширина выборки ОП - 4 байта, а ширина информационного тракта ВУ - 1 байт. Выполнение операции ввода/вывода в мультиплексном канале включает в себя процедуру начальной выборки и процедуру обслуживания запросов ВУ.

Процедура начальной выборки производится за один сеанс связи, по инициативе ЦП и связана с заданием работы ВУ. ЦП на регистр РАКК передает адрес первой канальной команды, в соответствии с которым на РКК из памяти выбирается канальная команда. Содержимое РАКК увеличивается на 8. В СБ устанавливается номер байта, в который или из которого в РДК будет передан первый байт информации. После этого в память подканала, соответствующего данному ВУ, записывается содержимое РКК, РДК, СБ и РАКК. На этом процедура начальной выборки заканчивается.

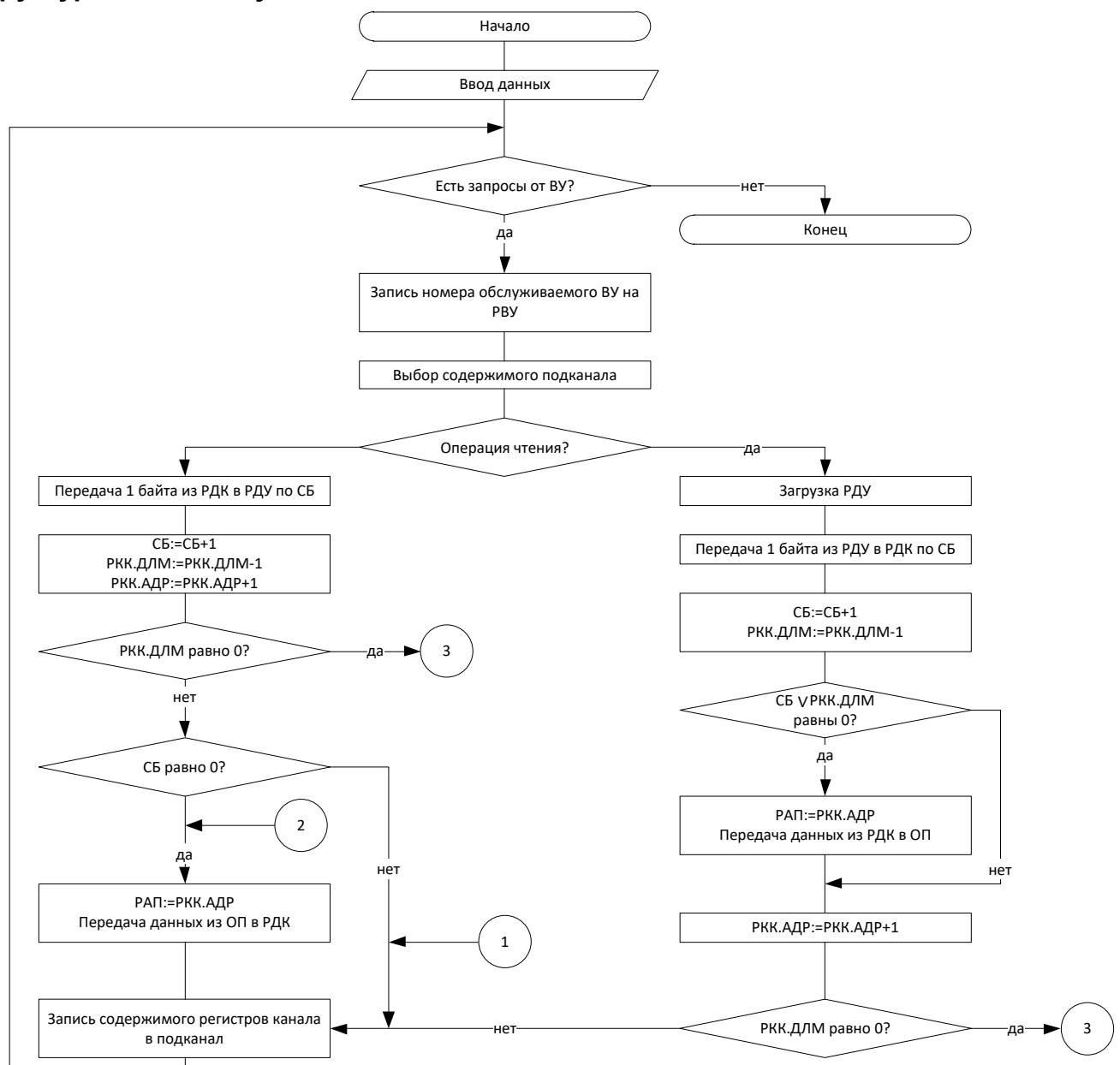
Процедура обслуживания запросов ВУ производится по инициативе ВУ и, как правило, занимает несколько сеансов связи. Запрос от ВУ фиксируется на регистре внешнего устройства РВУ, и из соответствующего подканала выбираются параметры операции и помещаются на регистры канала РКК, РДК, СБ и РАКК. В соответствии с кодом операции РКК.КОП из ВУ или на ВУ пересылается блок информации, равный по размеру ширине информационного тракта ВУ (1 байт). Номер байта на РДК, из которого выбирается или принимается информация, определяется по СБ. После пересылки информации содержимое СБ и РКК.АДР увеличивается на 1, а значение РКК.ДЛМ уменьшается на 1. В том случае, если СБ и РКК.ДЛМ не равны 0, то происходит запись измененных параметров из регистров в память соответствующего подканала



и сеанс связи с ВУ заканчивается. Если РКК.ДЛМ равно 0, то анализируется поле признаков РКК.ПРЗ и в соответствии со значением РАКК из ОП выбирается следующая канальная команда или завершается моделирование, если обнаружен признак "конец канальной команды".

Перед выборкой новой канальной команды в случае операции "чтение" нужно переписать содержимое РДК в ОП. Если вновь выбранная команда окажется командой записи, то в этом же сеансе связи в соответствии с РКК.АДР выбирается блок информации из ОП и помещается на РДК. Если СБ=0 при неравенстве 0 РКК.ДЛМ, то в случае операции "чтение" осуществляется запись информации из РДК в ОП, а в случае операции "запись" - выборка нового блока информации из ОП в соответствии с РКК.АДР, который помещается на РКК. В любом случае сеанс связи с ВУ заканчивается занесением содержимого регистров РКК, РДК, СБ и РАКК в соответствующий подканал. При выполнении лабораторной работы моделируется только процесс обслуживания запросов ВУ. Структурная схема алгоритма моделирования мультиплексного канала приведена на рис. 48.

Структурная схема мультиплексного канала



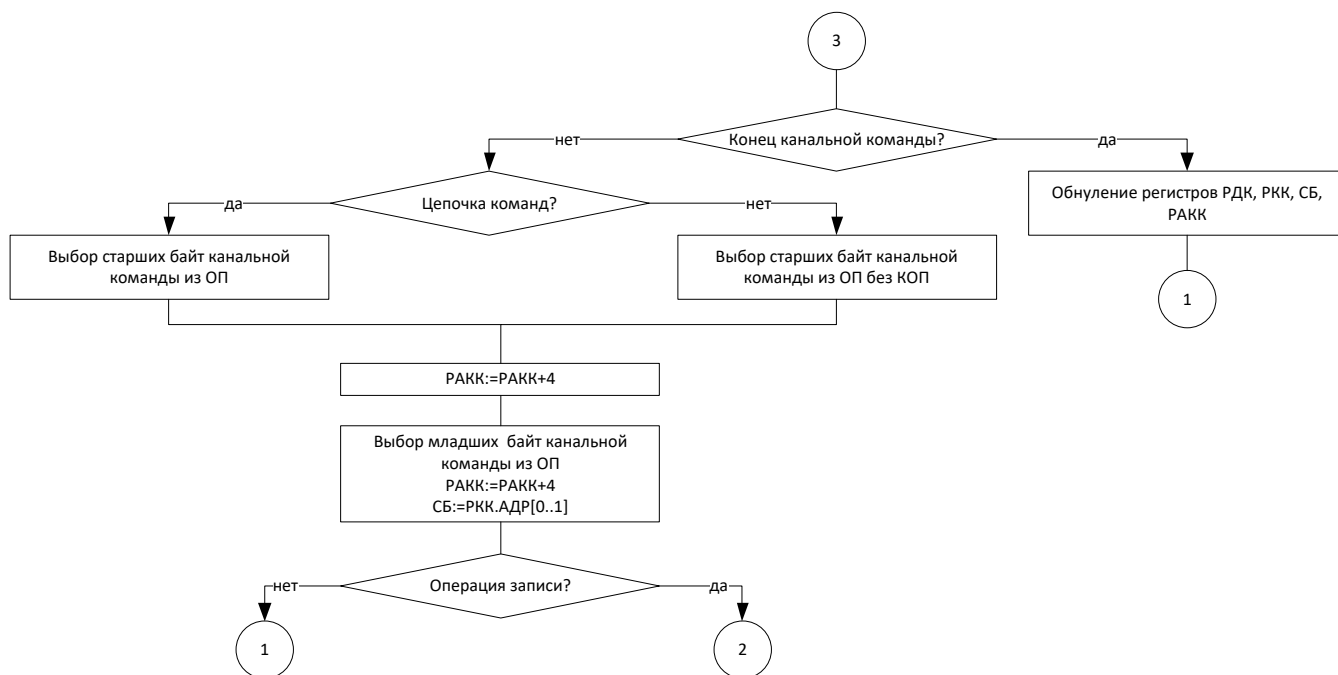


Рис. 48. Структурная схема алгоритма моделирования работы мультиплексного канала

Файл МК.MCL

В файле МК.MCL содержатся подпрограммы работы с памятью, передачи информации из РДК на ВУ (и наоборот), распечатки содержимого подканалов и рабочих ячеек ОП, а так же подпрограмма ввода исходных данных в соответствии с номером варианта. Эти подпрограммы рекомендуется использовать для моделирования работы мультиплексного канала. В них приняты следующие обозначения переменных и регистров (в квадратных скобках размерность регистров):

- WN - номер варианта
- RAKK[24] - регистр адреса канальной команды (РАКК)
- RAP[24] - регистр адреса памяти (РАП)
- RCHP[32] - регистр числа памяти (РЧП)
- BR[32] - буферный регистр данных (БР)
- RDК[32] - регистр данных канала (РДК)
- RDU[8] - регистр данных внешнего устройства (РДУ)
- SB[2] - счетчик байт (СБ)
- RVU[8] - регистр внешнего устройства (РВУ)

Регистр канальной команды (РКК) представлен, как и в случае моделирования селекторного канала, в виде двух структур:

- RKK1[32] с полями KOP[8] и ADR[24] - старшая часть РКК
- RKK2[32] с полями PRZ[16] и DLM[16] - младшая часть РКК

Оперативная память (ОП) описана как MEM OP[1024][8] - линейная память из 1024 ячеек по 1 байту каждая. Подпрограммы работы с памятью представляют, что её ширина выборки равна 4 байта.

MEM PU1[6][8] и MEM PU3[6][8] - память для хранения массива чисел, считываемых с 1-го и 3-го ВУ соответственно (заполняется, когда есть операция "чтение").

MEM ZAPR[9][8] - память для хранения последовательности запросов от ВУ к мультиплексному каналу.

Все числа выводятся в 16-ричной системе счисления. Остальные переменные и регистры, начинающиеся с буквы W, используются как рабочие переменные внутри подпрограмм и физического смысла не имеют, поэтому использовать их в основной программе не рекомендуется.



Описание подпрограмм

ПОДПРОГРАММЫ K_U_K, OUT_OP и IN_OP:

Те же, что и для моделирования селекторного канала (см. описание файла SK.MCL).

ПОДПРОГРАММА K_OP:

```
K_OP:
  GOSUB OUT_OP;
  W1:=(NOT (SB-1))*8;
  RCHP[W1..31]:=RDK[W1..31];
  GOSUB IN_OP;
  RETURN;
```

Производит запись в ОП блока информации из канала (РДК), начиная с адреса, находящегося в регистре РАП (этот адрес должен быть кратным ширине выборки). Число байт информации из РДК, которое нужно занести в память, определяется значением счетчика байт SB. Число записываемых в ОП байт меньше 4 при передаче последних байт массива информации, длина которого не кратна 4 (ширине выборки).

ПОДПРОГРАММА W_OP:

```
W_OP:
  WRITELN " СОДЕРЖИМОЕ ЯЧЕЕК ОП:";
  WX:=0;
  FOR WI:=0 TO 4;
    WX:=WX+WI*20;
    FOR WY:=0 TO 1;
      FOR WJ:=0 TO 3;
        WU:=16+WX+WY*4+WJ;
        WRITE $H6 WU," ", $H2 OP[WU]," * ";
      NEXT;
    WRITELN;
  NEXT;NEXT;
  RETURN;
```

Распечатывает содержимое ячеек ОП. Распечатываются 40 ячеек, по 8 ячеек для каждой канальной команды, начиная с адресов, указанных в поле адреса (т.е. с шестнадцатеричных адресов 10, 24, 4C, 88, D8). В одной строке распечатываются 4 ячейки (в соответствии с шириной выборки). Например:

СОДЕРЖИМОЕ ЯЧЕЕК ОП:

000010	0C	*	000011	98	*	000012	24	*	000013	B0
000014	03	*	000015	46	*	000016	89	*	000017	CC
000024	03	*	000025	7A	*	000026	F1	*	000027	68
000028	00	*	000029	00	*	00002A	00	*	00002B	00
00004C	31	*	00004D	D8	*	00004E	CF	*	00004F	C6
000050	4D	*	000051	D9	*	000052	65	*	000052	F2
000088	2E	*	000089	BA	*	00008A	46	*	00008B	D2
00008C	54	*	00008D	F6	*	00008E	30	*	00008F	14
0000D8	00	*	0000D9	00	*	0000DA	00	*	0000DB	00
0000DC	00	*	0000DD	00	*	0000DE	00	*	0000DF	00

	/	\
адрес	ячейки	содержимое ячейки



ПОДПРОГРАММА ACT:

ACT:

```
RAP:=960+(RVU-1)*16;  
GOSUB OUT_OP;  
RKK1:=RCHP;  
RAP:=RAP+4;  
GOSUB OUT_OP;  
RKK2:=RCHP;  
RAP:=RAP+4;  
GOSUB OUT_OP;  
RDK:=RCHP;  
RAP:=RAP+4;  
GOSUB OUT_OP;  
RAKK:=RCHP[0..23];  
SB:=RCHP[24..25];  
RETURN;
```

Загружает регистры РКК, РДК, СБ и РАКК содержимым из подканала, соответствующего ВУ, запрос от которого обрабатывается.

ПОДПРОГРАММА PAS:

PAS:

```
RAP:=960+(RVU-1)*16;  
RCHP:=RKK1;  
GOSUB IN_OP;  
RAP:=RAP+4;  
RCHP:=RKK2;  
GOSUB IN_OP;  
RAP:=RAP+4;  
RCHP:=RDK;  
GOSUB IN_OP;  
RAP:=RAP+4;  
RCHP[24..31]:=SB; RCHP[0..23]:=RAKK;  
GOSUB IN_OP;  
RETURN;
```

Записывает содержимое регистров РКК, РДК, СБ и РАКК в подканал, соответствующий ВУ, запрос от которого обрабатывался в данный сеанс связи.

ПОДПРОГРАММА W_PK:

W_PK:

```
WRITELN " РДУ=", $H2 RDU;  
WRITELN " СОДЕРЖИМОЕ ПОДКАНАЛОВ: ";  
WRITELN "          РКК          РДК          СБ          РАКК ";  
FOR WI:=0 TO 2;  
FOR WJ:=0 TO 15;  
WRITE $H2 OP[960+16*WI+WJ];  
IF (WJ=0)OR(WJ=3)OR(WJ=5)OR(WJ=8)OR(WJ=9)OR(WJ=10) THEN WRITE " ";  
IF (WJ=7)OR(WJ=11)OR(WJ=12)OR(WJ=15) THEN WRITE " * ";  
NEXT; WRITELN; NEXT;  
WRITELN "*****";  
RETURN;
```



Распечатывает содержимое регистра РДУ и подканалов. Например:

РДУ = АЕ

	РКК	РДК	СБ	РАКК
02	000012 A000 0005 *	0A DC AE 80 *	02 *	000328
02	00004D 6000 0002 *	1B ED BF 91 *	00 *	000338
01	0000D9 2000 0004 *	07 00 00 00 *	01 *	000348

ПОДПРОГРАММА Z_RDU:

```
Z_RDU:
  IF ZAPR[I]=3 THEN GOTO LBL3;
  RDU:=PU1[WK]; WK:=WK+1;
RETURN;
LBL3:
  RDU:=PU3[WL]; WL:=WL+1;
RETURN;
```

Загружает регистр РДУ очередным значением из RU1 или RU3 в зависимости от того, какое ВУ (1-е или 3-е) обслуживается в данном сеансе связи. Используется при операции "чтение".

ПОДПРОГРАММА ZAGR2:

```
ZAGR2:
  READ " ВВЕДИТЕ НОМЕР ВАРИАНТА ",WN;
  WRITELN "*****";
  WRITELN "ИСХОДНЫЕ ДАННЫЕ:";
  WRITELN "ВАРИАНТ НОМЕР ",$D3 WN;
  RAP:=800; RCHP:=0H0200010; GOSUB IN_OP;
  RAP:=RAP+4; RCHP:=0H60000007; GOSUB IN_OP;
  RAP:=RAP+4; RCHP:=0H01000024; GOSUB IN_OP;
  RAP:=RAP+4; RCHP:=0H20000006; GOSUB IN_OP;
  RAP:=RAP+4; RCHP:=0H0200004C; GOSUB IN_OP;
  RAP:=RAP+4; RCHP:=0HA0000002; GOSUB IN_OP;
  RAP:=RAP+4; RCHP:=0H00000088; GOSUB IN_OP;
  RAP:=RAP+4; RCHP:=0H20000004; GOSUB IN_OP;
  RAP:=RAP+4; RCHP:=0H010000D8; GOSUB IN_OP;
  RAP:=RAP+4; RCHP:=0H20000005; GOSUB IN_OP;
  WRITELN "КАНАЛЬНЫЕ ПРОГРАММЫ: ";
  WRITELN "  ДЛЯ 1-го ВУ, В ПАМЯТИ ХРАНИТСЯ С АДРЕСА 000320:";
  WRITELN "    02 000010 6000 0007 ";
  WRITELN "    01 000024 2000 0006 ";
  WRITELN "  ДЛЯ 2-го ВУ, В ПАМЯТИ ХРАНИТСЯ С АДРЕСА 000330:";
  WRITELN "    02 00004C A000 0002 ";
  WRITELN "    00 000088 2000 0004 ";
  WRITELN "  ДЛЯ 3-го ВУ, В ПАМЯТИ ХРАНИТСЯ С АДРЕСА 000340: ";
  WRITELN "    01 0000D8 2000 0005 ";
  { ЗАГРУЗКА ПОДКАНАЛОВ }
  RAP:=960; RCHP:=0H02000012; GOSUB IN_OP;
  RAP:=RAP+4; RCHP:=0H60000005; GOSUB IN_OP;
  RAP:=RAP+8; RCHP:=0H02000328; GOSUB IN_OP;
  RAP:=976; RCHP:=0H0200004C; GOSUB IN_OP;
```



```
RAP:=RAP+4; RCHP:=0HA0000002; GOSUB IN_OP;
RAP:=RAP+8; RCHP:=0H00000338; GOSUB IN_OP;
RAP:=992; RCHP:=0H010000D9; GOSUB IN_OP;
RAP:=RAP+4; RCHP:=0H20000004; GOSUB IN_OP;
RAP:=RAP+8; RCHP:=0H01000348; GOSUB IN_OP;
IF WN<6 THEN GOTO LBL4;
OP[963]:=19; OP[967]:=4; OP[972]:=3; OP[979]:=137;
OP[980]:=32; OP[983]:=3; OP[988]:=1; OP[991]:=64;
OP[995]:=218; OP[999]:=3; OP[1004]:=2;
IF WN>10 THEN GOTO LBL4;
OP[960]:=1; OP[963]:=39; OP[964]:=32; OP[967]:=3;
OP[975]:=48; OP[979]:=77; OP[980]:=160; OP[983]:=1;
OP[991]:=56; OP[995]:=216; OP[999]:=5; OP[1004]:=0;
LBL4:
{ ЗАГРУЗКА ПАМЯТИ ДАННЫМИ }
FOR WI:=0 TO 2;
FOR WJ:=0 TO 7;
OP[16+WI*60+WJ]:=5+WJ*WN*93+WI*17+WJ+WN;
OP[36+WI*90+WJ]:=0;
NEXT; NEXT;
GOSUB W_OP;
{ НАЧАЛЬНЫЕ ЗНАЧЕНИЯ РДК }
RAP:=OP[963]; GOSUB OUT_OP; RAP:=968; GOSUB IN_OP;
RAP:=OP[979]; GOSUB OUT_OP; RAP:=984; GOSUB IN_OP;
{ ФОРМИРОВАНИЕ СПИСКА ЗАПРОСОВ }
RAP:=1008; RCHP:=0H6D9B6D9B;
GOSUB IN_OP;
RAP:=RAP+4; RCHP:=0H799E799E;
GOSUB IN_OP;
RAP:=RAP+4; RCHP:=0H6DE76DE7;
GOSUB IN_OP;
RAP:=RAP+4; RCHP:=0H79B779B7;
GOSUB IN_OP;
RAP:=1008+WN[0..1]*4;
GOSUB OUT_OP;
WJ:=31-WN[2..3]*4;
WRITE "ПОСЛЕДОВАТЕЛЬНОСТЬ ЗАПРОСОВ : ";
FOR WI:=1 TO 8;
ZAPR[WI]:=RCHP[(WJ-1)..WJ];
WJ:=WJ-2;
WRITE $D2 ZAPR[WI];
NEXT;
Writeln;
{ ЧИСЛА ДЛЯ ВВОДА С ВУ }
WK:=0; WL:=0;
FOR WI:=0 TO 5;
PU1[WI]:=3+WI*WN*17;
PU3[WI]:=7+WI*WN*19;
IF (OP[963]-WI)<=36 THEN GOTO LBL1;
OP[968+WK]:=PU1[WI]; WK:=WK+1;
LBL1:
```



```
IF (OP[995]-WI)<=216 THEN GOTO LBL2;
OP[1000+WL]:=PU3[WI]; WL:=WL+1;
LBL2:
NEXT;
WRITE " ЧИСЛА, СЧИТЫВАЕМЫЕ С 1-го ВУ: ";
FOR WI:=WK TO 5;
WRITE $H2 PU1[WI], " ";
NEXT; WRITELN;
WRITE " ЧИСЛА, СЧИТЫВАЕМЫЕ С 3-го ВУ: ";
FOR WI:=WL TO 4;
WRITE $H2 PU3[WI], " ";
NEXT; WRITELN;
GOSUB W_PK;
RETURN;
```

Формирует исходные данные для моделирования по-задаваемому с клавиатуры номеру варианта WN. Изучение работы данной подпрограммы для студентов не требуется!
Подпрограмма формирует канальную программу; содержимое подканалов на момент начала моделирования; последовательность запросов ВУ; числа, считываемые с ВУ (для операции "чтение"); начальную загрузку данных в ОП (для записи на ВУ) и производит распечатку всех исходных данных в виде:

ИСХОДНЫЕ ДАННЫЕ:

ВАРИАНТ НОМЕР 5

КАНАЛЬНЫЕ ПРОГРАММЫ:

ДЛЯ 1-го ВУ, В ПАМЯТИ ХРАНИТСЯ С АДРЕСА 000320:

02 000010 A000 0007

01 000024 2000 0006

ДЛЯ 2-го ВУ, В ПАМЯТИ ХРАНИТСЯ С АДРЕСА 000330:

02 00004C 6000 0002

00 00008820000004

ДЛЯ 3-го ВУ, В ПАМЯТИ ХРАНИТСЯ С АДРЕСА 000340:

01 0000D8 2000 0005

СОДЕРЖИМОЕ ЯЧЕЕК ОП:

000010 0A * 000011 DC * 000012 AE * 000013 80

000014 52 * 000015 24 * 000016 F6 * 000017 C8

000024 00 * 000025 00 * 000026 00 * 000027 00

000028 00 * 000029 00 * 00002A 00 * 00002B 00

00004C 1B * 00004D ED * 00004E BF * 00004F 91

000050 63 * 000051 35 * 000052 07 * 000053 D9

000088 2C * 000089 FE * 00008A D0 * 00008B A2

00008C 74 * 00008D 46 * 00008E 18 * 00008F EA

0000D8 00 * 0000D9 00 * 0000DA 00 * 0000DB 00

0000DC 00 * 0000DD 00 * 0000DE 00 * 0000DF 00

ПОСЛЕДОВАТЕЛЬНОСТЬ ЗАПРОСОВ: 2 1 2 1 3 2 1 3

ЧИСЛА, СЧИТЫВАЕМЫЕ С 1-го ВУ: 03 58 AD 02 57 AC

ЧИСЛА, СЧИТЫВАЕМЫЕ С 3-го ВУ: 66 C5 24 83

РДУ=00

СОДЕРЖИМОЕ ПОДКАНАЛОВ:

РКК РДК СВ РАКК

02 000012 A000 0005 * 0A DC AE 80 * 02 * 000328 *



```
02 00004C 6000 0002 * 1B ED BF 91 * 00 * 000338 *
01 0000D9 2000 0004 * 07 00 00 00 * 01 * 000348 *
*****
```

Пример результатов моделирования работы мультимплексного канала

Рассмотрим результат моделирования мультимплексного канала в соответствии с заданием для варианта №19. Распечатка результатов моделирования имеет вид:

```
*****
ИСХОДНЫЕ ДАННЫЕ:
ВАРИАНТ НОМЕР 19
КАНАЛЬНЫЕ ПРОГРАММЫ:
ДЛЯ 1-го ВУ, В ПАМЯТИ ХРАНИТСЯ С АДРЕСА 000320:
  02 000010 6000 0007
  01 000024 2000 0006
ДЛЯ 2-го ВУ, В ПАМЯТИ ХРАНИТСЯ С АДРЕСА 000330:
  02 00004C A000 0002
  00 000088 2000 0004
ДЛЯ 3-го ВУ, В ПАМЯТИ ХРАНИТСЯ С АДРЕСА 000340:
  01 0000D8 2000 0005
СОДЕРЖИМОЕ ЯЧЕЕК ОП:
000010 18 * 000011 00 * 000012 E8 * 000013 D0 *
000014 B8 * 000015 A0 * 000016 88 * 000017 70 *
000024 00 * 000025 00 * 000026 00 * 000027 00 *
000028 00 * 000029 00 * 00002A 00 * 00002B 00 *
00004C 29 * 00004D 11 * 00004E F9 * 00004F E1 *
000050 C9 * 000051 B1 * 000052 99 * 000053 81 *
000088 3A * 000089 22 * 00008A 0A * 00008B F2 *
00008C DA * 00008D C2 * 00008E AA * 00008F 92 *
0000D8 00 * 0000D9 00 * 0000DA 00 * 0000DB 00 *
0000DC 00 * 0000DD 00 * 0000DE 00 * 0000DF 00 *
ПОСЛЕДОВАТЕЛЬНОСТЬ ЗАПРОСОВ: 1 3 2 1 2 3 1 3
ЧИСЛА, СЧИТЫВАЕМЫЕ С 1-го ВУ: 03 46 89 CC 0F 52
ЧИСЛА, СЧИТЫВАЕМЫЕ С 3-го ВУ: D9 42 AB
РДУ=00
СОДЕРЖИМОЕ ПОДКАНАЛОВ:
РКК РДКСБРАКК
02 000013 6000 0004 * 18 00 E8 D0 * 03 * 000328 *
02 000089 2000 0003 * 3A 22 0A F2 * 01 * 000340 *
01 0000DA 2000 0003 * 07 70 00 00 * 02 * 000348 *
*****
```

```
НОМЕР ОБСЛУЖИВАЕМОГО ЗАПРОСА 1
НОМЕР ОБСЛУЖИВАЕМОГО УСТРОЙСТВА 1
РДУ=D0
СОДЕРЖИМОЕ ПОДКАНАЛОВ:
РКК РДКСБРАКК
02 000014 6000 0003 * B8 A0 88 70 * 00 * 000328 *
02 000089 2000 0003 * 3A 22 0A F2 * 01 * 000340 *
01 0000DA 2000 0003 * 07 70 00 00 * 02 * 000348 *
*****
НОМЕР ОБСЛУЖИВАЕМОГО ЗАПРОСА 2
НОМЕР ОБСЛУЖИВАЕМОГО УСТРОЙСТВА 3
РДУ=D9
СОДЕРЖИМОЕ ПОДКАНАЛОВ:
РКК РДКСБРАКК
02 000014 6000 0003 * B8 A0 88 70 * 00 * 000328 *
02 000089 2000 0003 * 3A 22 0A F2 * 01 * 000340 *
01 0000DB 2000 0002 * 07 70 D9 00 * 03 * 000348 *
*****
НОМЕР ОБСЛУЖИВАЕМОГО ЗАПРОСА 3
```



НОМЕР ОБСЛУЖИВАЕМОГО УСТРОЙСТВА 2

РДУ=22

СОДЕРЖИМОЕ ПОДКАНАЛОВ:

РКК РДКСБРАКК

```
02 000014 6000 0003 * B8 A0 88 70 * 00 * 000328 *
02 00008A 2000 0002 * 3A 22 0A F2 * 02 * 000340 *
01 0000DB 2000 0002 * 07 70 D9 00 * 03 * 000348 *
*****
```

НОМЕР ОБСЛУЖИВАЕМОГО ЗАПРОСА 4

НОМЕР ОБСЛУЖИВАЕМОГО УСТРОЙСТВА 1

РДУ=B8

СОДЕРЖИМОЕ ПОДКАНАЛОВ:

РКК РДКСБРАКК

```
02 000015 6000 0002 * B8 A0 88 70 * 01 * 000328 *
02 00008A 2000 0002 * 3A 22 0A F2 * 02 * 000340 *
01 0000DB 2000 0002 * 07 70 D9 00 * 03 * 000348 *
*****
```

НОМЕР ОБСЛУЖИВАЕМОГО ЗАПРОСА 5

НОМЕР ОБСЛУЖИВАЕМОГО УСТРОЙСТВА 2

РДУ=0A

СОДЕРЖИМОЕ ПОДКАНАЛОВ:

РКК РДКСБРАКК

```
02 000015 6000 0002 * B8 A0 88 70 * 01 * 000328 *
02 00008B 2000 0001 * 3A 22 0A F2 * 03 * 000340 *
01 0000DB 2000 0002 * 07 70 D9 00 * 03 * 000348 *
*****
```

НОМЕР ОБСЛУЖИВАЕМОГО ЗАПРОСА 6

НОМЕР ОБСЛУЖИВАЕМОГО УСТРОЙСТВА 3

СОДЕРЖИМОЕ ЯЧЕЕК ОП:

```
000010 18 * 000011 00 * 000012 E8 * 000013 D0 *
000014 B8 * 000015 A0 * 000016 88 * 000017 70 *
000024 00 * 000025 00 * 000026 00 * 000027 00 *
000028 00 * 000029 00 * 00002A 00 * 00002B 00 *
00004C 29 * 00004D 11 * 00004E F9 * 00004F E1 *
000050 C9 * 000051 B1 * 000052 99 * 000053 81 *
000088 3A * 000089 22 * 00008A 0A * 00008B F2 *
00008C DA * 00008D C2 * 00008E AA * 00008F 92 *
0000D8 07 * 0000D9 70 * 0000DA D9 * 0000DB 42 *
0000DC 00 * 0000DD 00 * 0000DE 00 * 0000DF 00 *
```

РДУ=42

СОДЕРЖИМОЕ ПОДКАНАЛОВ:

РКК РДКСБРАКК

```
02 000015 6000 0002 * B8 A0 88 70 * 01 * 000328 *
02 00008B 2000 0001 * 3A 22 0A F2 * 03 * 000340 *
01 0000DC 2000 0001 * 07 70 D9 42 * 00 * 000348 *
*****
```

НОМЕР ОБСЛУЖИВАЕМОГО ЗАПРОСА 7

НОМЕР ОБСЛУЖИВАЕМОГО УСТРОЙСТВА 1

РДУ=A0

СОДЕРЖИМОЕ ПОДКАНАЛОВ:

РКК РДКСБРАКК

```
02 000016 6000 0001 * B8 A0 88 70 * 02 * 000328 *
02 00008B 2000 0001 * 3A 22 0A F2 * 03 * 000340 *
01 0000DC 2000 0001 * 07 70 D9 42 * 00 * 000348 *
*****
```

НОМЕР ОБСЛУЖИВАЕМОГО ЗАПРОСА 8

НОМЕР ОБСЛУЖИВАЕМОГО УСТРОЙСТВА 3

СОДЕРЖИМОЕ ЯЧЕЕК ОП:

```
000010 18 * 000011 00 * 000012 E8 * 000013 D0 *
000014 B8 * 000015 A0 * 000016 88 * 000017 70 *
000024 00 * 000025 00 * 000026 00 * 000027 00 *
```



```
000028 00 * 000029 00 * 00002A 00 * 00002B 00 *
00004C 29 * 00004D 11 * 00004E F9 * 00004F E1 *
000050 C9 * 000051 B1 * 000052 99 * 000053 81 *
000088 3A * 000089 22 * 00008A 0A * 00008B F2 *
00008C DA * 00008D C2 * 00008E AA * 00008F 92 *
0000D8 07 * 0000D9 70 * 0000DA D9 * 0000DB 42 *
0000DC AB * 0000DD 00 * 0000DE 00 * 0000DF 00 *
РДУ=AB
СОДЕРЖИМОЕ ПОДКАНАЛОВ:
РКК РДКСБРАКК
02 000016 6000 0001 * B8 A0 88 70 * 02 * 000328 *
02 00008B 2000 0001 * 3A 22 0A F2 * 03 * 000340 *
00 000000 0000 0000 * AB 70 D9 42 * 00 * 000000 *
*****
```

Имеются три канальные команды для 3-х ВУ, обслуживаемых одним мультиплексным каналом. На момент начала моделирования уже осуществлена процедура начальной выборки - образованы для всех ВУ соответствующие подканалы и уже обработано каналом несколько запросов от ВУ. В лабораторной работе моделируется обслуживание каналом 8-ми запросов от ВУ (8 сеансов связи) в соответствии с заданной последовательностью запросов.

В 1-ом сеансе связи 1-е ВУ выставило запрос на связь с каналом, поэтому регистры канала РКК, РДК, СБ и РАКК загружаются содержимым подканала, соответствующего 1-му ВУ. Выполняемая операция - операция "запись", СБ=3, поэтому на РДУ передается содержимое 3-го байта РДК. Поле РКК.АДР и СБ увеличиваются на 1, а поле РКК.ДЛМ - уменьшается на 1. При этом СБ становится равным 0 (массив РДУ полностью передан на ВУ), поэтому прежде чем записать содержимое регистров канала в память подканала, в РДУ из ОП пересылается следующий блок передаваемой на ВУ информации, начиная с адреса 000014.

Во 2-ом сеансе связи запрос выставило 3-е ВУ. содержимое подканала, соответствующего 3-му ВУ, помещается на регистры канала. Выполняется операция "чтение" и СБ=2, поэтому информация с РДУ передается на 2-ой байт РДК. Поле РКК.АДР и СБ увеличиваются на 1, а поле РКК.ДЛМ - уменьшается на 1. Т.к. после этого ни СБ, ни поле РКК.ДЛМ не равны 0, то измененные параметры заносятся в память подканала, соответствующему данному ВУ, и сеанс связи завершается.

В 3-ем сеансе связи запрос выставляет 2-е ВУ. содержимое подканала, соответствующего 2-му ВУ, помещается на регистры канала. Выполняется операция "запись" и СБ=1, поэтому информация из 1-го байта РДК передается на РДУ 2-го ВУ. Дальнейшие действия аналогичны действиям в предыдущем сеансе связи.

В 4-ом сеансе связи запрос выставляет 1-е ВУ. Действия аналогичны действиям в 3-ем сеансе связи (содержимое 0-го байта РДК передается на РДУ 1-го ВУ).

В 5-ом сеансе связи запрос выставляет 2-е ВУ. Действия аналогичны действиям в 3-ем сеансе связи (содержимое 2-го байта РДК передается на РДУ 2-го ВУ).

В 6-ом сеансе связи запрос выставляет 3-е ВУ. Выполняется операция "чтение", СБ=3. После передачи информации с РДУ на 3-ий байт РДК СБ становится равным 0 (РДК полностью заполнен), поэтому перед записью измененных параметров в подканал содержимое РДК записывается в ячейки ОП, начиная с адреса 0000D8.

В 7-ом сеансе связи запрос выставляет 1-е ВУ. Действия аналогичны действиям в 3-ем сеансе связи (содержимое 1-го байта РДК передается на РДУ 1-го ВУ).

В 8-ом сеансе связи запрос выставляет 3-е ВУ. Выполняется операция "чтение", СБ=0 - передается информация с РДУ на 0-ой байт РДК. При этом РКК.ДЛМ становится равным 0, т.е. весь массив информации с 3-го ВУ в соответствии с канальной командой передан. При анализе поля признаков РКК.ПРЗ оказывается, что эта канальная команда была последней, поэтому сеанс связи заканчивается обнулением памяти подканала, соответствующего 3-му ВУ. На этом моделирование работы мультиплексного канала завершается.



Содержание отчета

1. Формулировка задания.
2. Структурная схема мультиплексного канала (составить самостоятельно).
3. Структурная схема алгоритма моделирования мультиплексного канала.
4. Программа на языке MCL, моделирующая функционирование мультиплексного канала.
5. Результаты выполнения программы в **виде распечатки** содержимого поля для вывода текстовой информации **и скриншота** в момент завершения выполнения программы.

Контрольные вопросы

1. Основные отличия селекторного канала и мультиплексного канала.
2. Процедура начальной выборки.
3. Завершение процедуры обслуживания запроса.

Задание к лабораторной работе

Промоделировать работу мультиплексного канала при обслуживании запросов от трех ВУ в соответствии с вариантом задания. В процессе моделирования распечатать содержимое подканалов и регистра РДУ после каждого сеанса связи и содержимое рабочих ячеек ОП в моменты их изменения.

ВАРИАНТ НОМЕР 1

```
КАНАЛЬНЫЕ ПРОГРАММЫ:
ДЛЯ 1-го ВУ, В ПАМЯТИ ХРАНИТСЯ С АДРЕСА 000320:
  02 000010 6000 0007
  01 000024 2000 0006
ДЛЯ 2-го ВУ, В ПАМЯТИ ХРАНИТСЯ С АДРЕСА 000330:
  02 00004C A000 0002
  00 000088 2000 0004
ДЛЯ 3-го ВУ, В ПАМЯТИ ХРАНИТСЯ С АДРЕСА 000340:
  01 0000D8 2000 0005
СОДЕРЖИМОЕ ЯЧЕЕК ОП:
000010 06 * 000011 64 * 000012 C2 * 000013 20 *
000014 7E * 000015 DC * 000016 3A * 000017 98 *
000024 00 * 000025 00 * 000026 00 * 000027 00 *
000028 00 * 000029 00 * 00002A 00 * 00002B 00 *
00004C 17 * 00004D 75 * 00004E D3 * 00004F 31 *
000050 8F * 000051 ED * 000052 4B * 000053 A9 *
000088 28 * 000089 86 * 00008A E4 * 00008B 42 *
00008C A0 * 00008D FE * 00008E 5C * 00008F BA *
0000D8 00 * 0000D9 00 * 0000DA 00 * 0000DB 00 *
0000DC 00 * 0000DD 00 * 0000DE 00 * 0000DF 00 *
ПОСЛЕДОВАТЕЛЬНОСТЬ ЗАПРОСОВ: 1 3 2 1 2 1 3 2
ЧИСЛА, СЧИТЫВАЕМЫЕ С 1-го ВУ: 03 14 25 36 47 58
ЧИСЛА, СЧИТЫВАЕМЫЕ С 3-го ВУ: 1A 2D 40 53
СОДЕРЖИМОЕ ПОДКАНАЛОВ:
РКК РДКСВРАКК
02 000012 6000 0005 * 06 64 C2 20 * 02 * 000328 *
02 00004C A000 0002 * 17 75 D3 31 * 00 * 000338 *
01 0000D9 2000 0004 * 07 00 00 00 * 01 * 000348 *
*****
```



ВАРИАНТ НОМЕР 2

КАНАЛЬНЫЕ ПРОГРАММЫ:
ДЛЯ 1-го ВУ, В ПАМЯТИ ХРАНИТСЯ С АДРЕСА 000320:
02 000010 6000 0007
01 000024 2000 0006
ДЛЯ 2-го ВУ, В ПАМЯТИ ХРАНИТСЯ С АДРЕСА 000330:
02 00004C A000 0002
00 000088 2000 0004
ДЛЯ 3-го ВУ, В ПАМЯТИ ХРАНИТСЯ С АДРЕСА 000340:
01 0000D8 2000 0005
СОДЕРЖИМОЕ ЯЧЕЕК ОП:
000010 07 * 000011 C2 * 000012 7D * 000013 38 *
000014 F3 * 000015 AE * 000016 69 * 000017 24 *
000024 00 * 000025 00 * 000026 00 * 000027 00 *
000028 00 * 000029 00 * 00002A 00 * 00002B 00 *
00004C 18 * 00004D D3 * 00004E 8E * 00004F 49 *
000050 04 * 000051 BF * 000052 7A * 000053 35 *
000088 29 * 000089 E4 * 00008A 9F * 00008B 5A *
00008C 15 * 00008D D0 * 00008E 8B * 00008F 46 *
0000D8 00 * 0000D9 00 * 0000DA 00 * 0000DB 00 *
0000DC 00 * 0000DD 00 * 0000DE 00 * 0000DF 00 *
ПОСЛЕДОВАТЕЛЬНОСТЬ ЗАПРОСОВ: 1 2 3 1 3 2 1 3
ЧИСЛА, СЧИТЫВАЕМЫЕ С 1-го ВУ: 03 25 47 69 8B AD
ЧИСЛА, СЧИТЫВАЕМЫЕ С 3-го ВУ: 2D 53 79 9F
СОДЕРЖИМОЕ ПОДКАНАЛОВ:
РКК РДКСБРАКК
02 000012 6000 0005 * 07 C2 7D 38 * 02 * 000328 *
02 00004C A000 0002 * 18 D3 8E 49 * 00 * 000338 *
01 0000D9 2000 0004 * 07 00 00 00 * 01 * 000348 *

ВАРИАНТ НОМЕР 3

КАНАЛЬНЫЕ ПРОГРАММЫ:
ДЛЯ 1-го ВУ, В ПАМЯТИ ХРАНИТСЯ С АДРЕСА 000320:
02 000010 6000 0007
01 000024 2000 0006
ДЛЯ 2-го ВУ, В ПАМЯТИ ХРАНИТСЯ С АДРЕСА 000330:
02 00004C A000 0002
00 000088 2000 0004
ДЛЯ 3-го ВУ, В ПАМЯТИ ХРАНИТСЯ С АДРЕСА 000340:
01 0000D8 2000 0005
СОДЕРЖИМОЕ ЯЧЕЕК ОП:
000010 08 * 000011 20 * 000012 38 * 000013 50 *
000014 68 * 000015 80 * 000016 98 * 000017 B0 *
000024 00 * 000025 00 * 000026 00 * 000027 00 *
000028 00 * 000029 00 * 00002A 00 * 00002B 00 *
00004C 19 * 00004D 31 * 00004E 49 * 00004F 61 *
000050 79 * 000051 91 * 000052 A9 * 000053 C1 *
000088 2A * 000089 42 * 00008A 5A * 00008B 72 *
00008C 8A * 00008D A2 * 00008E BA * 00008F D2 *
0000D8 00 * 0000D9 00 * 0000DA 00 * 0000DB 00 *
0000DC 00 * 0000DD 00 * 0000DE 00 * 0000DF 00 *
ПОСЛЕДОВАТЕЛЬНОСТЬ ЗАПРОСОВ: 1 3 2 1 2 3 1 3
ЧИСЛА, СЧИТЫВАЕМЫЕ С 1-го ВУ: 03 36 69 9C CF 02
ЧИСЛА, СЧИТЫВАЕМЫЕ С 3-го ВУ: 40 79 B2 EB
СОДЕРЖИМОЕ ПОДКАНАЛОВ:
РКК РДКСБРАКК
02 000012 6000 0005 * 08 20 38 50 * 02 * 000328 *
02 00004C A000 0002 * 19 31 49 61 * 00 * 000338 *
01 0000D9 2000 0004 * 07 00 00 00 * 01 * 000348 *



ВАРИАНТ НОМЕР 4

КАНАЛЬНЫЕ ПРОГРАММЫ:
ДЛЯ 1-го ВУ, В ПАМЯТИ ХРАНИТСЯ С АДРЕСА 000320:
02 000010 6000 0007
01 000024 2000 0006
ДЛЯ 2-го ВУ, В ПАМЯТИ ХРАНИТСЯ С АДРЕСА 000330:
02 00004C A000 0002
00 000088 2000 0004
ДЛЯ 3-го ВУ, В ПАМЯТИ ХРАНИТСЯ С АДРЕСА 000340:
01 0000D8 2000 0005
СОДЕРЖИМОЕ ЯЧЕЕК ОП:
000010 09 * 000011 7E * 000012 F3 * 000013 68 *
000014 DD * 000015 52 * 000016 C7 * 000017 3C *
000024 00 * 000025 00 * 000026 00 * 000027 00 *
000028 00 * 000029 00 * 00002A 00 * 00002B 00 *
00004C 1A * 00004D 8F * 00004E 04 * 00004F 79 *
000050 EE * 000051 63 * 000052 D8 * 000053 4D *
000088 2B * 000089 A0 * 00008A 15 * 00008B 8A *
00008C FF * 00008D 74 * 00008E E9 * 00008F 5E *
0000D8 00 * 0000D9 00 * 0000DA 00 * 0000DB 00 *
0000DC 00 * 0000DD 00 * 0000DE 00 * 0000DF 00 *
ПОСЛЕДОВАТЕЛЬНОСТЬ ЗАПРОСОВ: 3 1 2 1 2 3 1 2
ЧИСЛА, СЧИТЫВАЕМЫЕ С 1-го ВУ: 03 47 8B CF 13 57
ЧИСЛА, СЧИТЫВАЕМЫЕ С 3-го ВУ: 53 9F EB 37
СОДЕРЖИМОЕ ПОДКАНАЛОВ:
РКК РДКСВРАКК
02 000012 6000 0005 * 09 7E F3 68 * 02 * 000328 *
02 00004C A000 0002 * 1A 8F 04 79 * 00 * 000338 *
01 0000D9 2000 0004 * 07 00 00 00 * 01 * 000348 *

ВАРИАНТ НОМЕР 5

КАНАЛЬНЫЕ ПРОГРАММЫ:
ДЛЯ 1-го ВУ, В ПАМЯТИ ХРАНИТСЯ С АДРЕСА 000320:
02 000010 6000 0007
01 000024 2000 0006
ДЛЯ 2-го ВУ, В ПАМЯТИ ХРАНИТСЯ С АДРЕСА 000330:
02 00004C A000 0002
00 000088 2000 0004
ДЛЯ 3-го ВУ, В ПАМЯТИ ХРАНИТСЯ С АДРЕСА 000340:
01 0000D8 2000 0005
СОДЕРЖИМОЕ ЯЧЕЕК ОП:
000010 0A * 000011 DC * 000012 AE * 000013 80 *
000014 52 * 000015 24 * 000016 F6 * 000017 C8 *
000024 00 * 000025 00 * 000026 00 * 000027 00 *
000028 00 * 000029 00 * 00002A 00 * 00002B 00 *
00004C 1B * 00004D ED * 00004E BF * 00004F 91 *
000050 63 * 000051 35 * 000052 07 * 000053 D9 *
000088 2C * 000089 FE * 00008A D0 * 00008B A2 *
00008C 74 * 00008D 46 * 00008E 18 * 00008F EA *
0000D8 00 * 0000D9 00 * 0000DA 00 * 0000DB 00 *
0000DC 00 * 0000DD 00 * 0000DE 00 * 0000DF 00 *
ПОСЛЕДОВАТЕЛЬНОСТЬ ЗАПРОСОВ: 2 1 2 1 3 2 1 3
ЧИСЛА, СЧИТЫВАЕМЫЕ С 1-го ВУ: 03 58 AD 02 57 AC
ЧИСЛА, СЧИТЫВАЕМЫЕ С 3-го ВУ: 66 C5 24 83
СОДЕРЖИМОЕ ПОДКАНАЛОВ:
РКК РДКСВРАКК
02 000012 6000 0005 * 0A DC AE 80 * 02 * 000328 *
02 00004C A000 0002 * 1B ED BF 91 * 00 * 000338 *
01 0000D9 2000 0004 * 07 00 00 00 * 01 * 000348 *



ВАРИАНТ НОМЕР 6

КАНАЛЬНЫЕ ПРОГРАММЫ:
ДЛЯ 1-го ВУ, В ПАМЯТИ ХРАНИТСЯ С АДРЕСА 000320:
02 000010 6000 0007
01 000024 2000 0006
ДЛЯ 2-го ВУ, В ПАМЯТИ ХРАНИТСЯ С АДРЕСА 000330:
02 00004C A000 0002
00 000088 2000 0004
ДЛЯ 3-го ВУ, В ПАМЯТИ ХРАНИТСЯ С АДРЕСА 000340:
01 0000D8 2000 0005
СОДЕРЖИМОЕ ЯЧЕЕК ОП:
000010 0B * 000011 3A * 000012 69 * 000013 98 *
000014 C7 * 000015 F6 * 000016 25 * 000017 54 *
000024 00 * 000025 00 * 000026 00 * 000027 00 *
000028 00 * 000029 00 * 00002A 00 * 00002B 00 *
00004C 1C * 00004D 4B * 00004E 7A * 00004F A9 *
000050 D8 * 000051 07 * 000052 36 * 000053 65 *
000088 2D * 000089 5C * 00008A 8B * 00008B BA *
00008C E9 * 00008D 18 * 00008E 47 * 00008F 76 *
0000D8 00 * 0000D9 00 * 0000DA 00 * 0000DB 00 *
0000DC 00 * 0000DD 00 * 0000DE 00 * 0000DF 00 *
ПОСЛЕДОВАТЕЛЬНОСТЬ ЗАПРОСОВ: 3 1 3 2 1 3 1 2
ЧИСЛА, СЧИТЫВАЕМЫЕ С 1-го ВУ: 35 9B 01
ЧИСЛА, СЧИТЫВАЕМЫЕ С 3-го ВУ: 07 79 EB 5D CF
СОДЕРЖИМОЕ ПОДКАНАЛОВ:
РКК РДКСВРАКК
01 000027 2000 0003 * 03 69 CF 00 * 03 * 000330 *
02 00004D A000 0001 * 1C 4B 7A A9 * 01 * 000338 *
01 0000D8 2000 0005 * 07 00 00 00 * 00 * 000348 *

ВАРИАНТ НОМЕР 7

КАНАЛЬНЫЕ ПРОГРАММЫ:
ДЛЯ 1-го ВУ, В ПАМЯТИ ХРАНИТСЯ С АДРЕСА 000320:
02 000010 6000 0007
01 000024 2000 0006
ДЛЯ 2-го ВУ, В ПАМЯТИ ХРАНИТСЯ С АДРЕСА 000330:
02 00004C A000 0002
00 000088 2000 0004
ДЛЯ 3-го ВУ, В ПАМЯТИ ХРАНИТСЯ С АДРЕСА 000340:
01 0000D8 2000 0005
СОДЕРЖИМОЕ ЯЧЕЕК ОП:
000010 0C * 000011 98 * 000012 24 * 000013 B0 *
000014 3C * 000015 C8 * 000016 54 * 000017 E0 *
000024 00 * 000025 00 * 000026 00 * 000027 00 *
000028 00 * 000029 00 * 00002A 00 * 00002B 00 *
00004C 1D * 00004D A9 * 00004E 35 * 00004F C1 *
000050 4D * 000051 D9 * 000052 65 * 000053 F1 *
000088 2E * 000089 BA * 00008A 46 * 00008B D2 *
00008C 5E * 00008D EA * 00008E 76 * 00008F 02 *
0000D8 00 * 0000D9 00 * 0000DA 00 * 0000DB 00 *
0000DC 00 * 0000DD 00 * 0000DE 00 * 0000DF 00 *
ПОСЛЕДОВАТЕЛЬНОСТЬ ЗАПРОСОВ: 2 1 2 3 1 3 1 3
ЧИСЛА, СЧИТЫВАЕМЫЕ С 1-го ВУ: 68 DF 56
ЧИСЛА, СЧИТЫВАЕМЫЕ С 3-го ВУ: 07 8C 11 96 1B
СОДЕРЖИМОЕ ПОДКАНАЛОВ:
РКК РДКСВРАКК
01 000027 2000 0003 * 03 7A F1 00 * 03 * 000330 *
02 00004D A000 0001 * 1D A9 35 C1 * 01 * 000338 *
01 0000D8 2000 0005 * 07 00 00 00 * 00 * 000348 *



ВАРИАНТ НОМЕР 8

КАНАЛЬНЫЕ ПРОГРАММЫ:
ДЛЯ 1-го ВУ, В ПАМЯТИ ХРАНИТСЯ С АДРЕСА 000320:
02 000010 6000 0007
01 000024 2000 0006
ДЛЯ 2-го ВУ, В ПАМЯТИ ХРАНИТСЯ С АДРЕСА 000330:
02 00004C A000 0002
00 000088 2000 0004
ДЛЯ 3-го ВУ, В ПАМЯТИ ХРАНИТСЯ С АДРЕСА 000340:
01 0000D8 2000 0005
СОДЕРЖИМОЕ ЯЧЕЕК ОП:
000010 0D * 000011 F6 * 000012 DF * 000013 C8 *
000014 B1 * 000015 9A * 000016 83 * 000017 6C *
000024 00 * 000025 00 * 000026 00 * 000027 00 *
000028 00 * 000029 00 * 00002A 00 * 00002B 00 *
00004C 1E * 00004D 07 * 00004E F0 * 00004F D9 *
000050 C2 * 000051 AB * 000052 94 * 000053 7D *
000088 2F * 000089 18 * 00008A 01 * 00008B EA *
00008C D3 * 00008D BC * 00008E A5 * 00008F 8E *
0000D8 00 * 0000D9 00 * 0000DA 00 * 0000DB 00 *
0000DC 00 * 0000DD 00 * 0000DE 00 * 0000DF 00 *
ПОСЛЕДОВАТЕЛЬНОСТЬ ЗАПРОСОВ: 2 1 2 3 1 2 3 1
ЧИСЛА, СЧИТЫВАЕМЫЕ С 1-го ВУ: 9B 23 AB
ЧИСЛА, СЧИТЫВАЕМЫЕ С 3-го ВУ: 07 9F 37 CF 67
СОДЕРЖИМОЕ ПОДКАНАЛОВ:
РКК РДКСВРАКК
01 000027 2000 0003 * 03 8B 13 00 * 03 * 000330 *
02 00004D A000 0001 * 1E 07 F0 D9 * 01 * 000338 *
01 0000D8 2000 0005 * 07 00 00 00 * 00 * 000348 *

ВАРИАНТ НОМЕР 9

КАНАЛЬНЫЕ ПРОГРАММЫ:
ДЛЯ 1-го ВУ, В ПАМЯТИ ХРАНИТСЯ С АДРЕСА 000320:
02 000010 6000 0007
01 000024 2000 0006
ДЛЯ 2-го ВУ, В ПАМЯТИ ХРАНИТСЯ С АДРЕСА 000330:
02 00004C A000 0002
00 000088 2000 0004
ДЛЯ 3-го ВУ, В ПАМЯТИ ХРАНИТСЯ С АДРЕСА 000340:
01 0000D8 2000 0005
СОДЕРЖИМОЕ ЯЧЕЕК ОП:
000010 0E * 000011 54 * 000012 9A * 000013 E0 *
000014 26 * 000015 6C * 000016 B2 * 000017 F8 *
000024 00 * 000025 00 * 000026 00 * 000027 00 *
000028 00 * 000029 00 * 00002A 00 * 00002B 00 *
00004C 1F * 00004D 65 * 00004E AB * 00004F F1 *
000050 37 * 000051 7D * 000052 C3 * 000053 09 *
000088 30 * 000089 76 * 00008A BC * 00008B 02 *
00008C 48 * 00008D 8E * 00008E D4 * 00008F 1A *
0000D8 00 * 0000D9 00 * 0000DA 00 * 0000DB 00 *
0000DC 00 * 0000DD 00 * 0000DE 00 * 0000DF 00 *
ПОСЛЕДОВАТЕЛЬНОСТЬ ЗАПРОСОВ: 2 1 3 2 1 3 2 1
ЧИСЛА, СЧИТЫВАЕМЫЕ С 1-го ВУ: CE 67 00
ЧИСЛА, СЧИТЫВАЕМЫЕ С 3-го ВУ: 07 B2 5D 08 B3
СОДЕРЖИМОЕ ПОДКАНАЛОВ:
РКК РДКСВРАКК
01 000027 2000 0003 * 03 9C 35 00 * 03 * 000330 *
02 00004D A000 0001 * 1F 65 AB F1 * 01 * 000338 *
01 0000D8 2000 0005 * 07 00 00 00 * 00 * 000348 *



ВАРИАНТ НОМЕР 10

КАНАЛЬНЫЕ ПРОГРАММЫ:
ДЛЯ 1-го ВУ, В ПАМЯТИ ХРАНИТСЯ С АДРЕСА 000320:
02 000010 6000 0007
01 000024 2000 0006
ДЛЯ 2-го ВУ, В ПАМЯТИ ХРАНИТСЯ С АДРЕСА 000330:
02 00004C A000 0002
00 000088 2000 0004
ДЛЯ 3-го ВУ, В ПАМЯТИ ХРАНИТСЯ С АДРЕСА 000340:
01 0000D8 2000 0005
СОДЕРЖИМОЕ ЯЧЕЕК ОП:
000010 0F * 000011 B2 * 000012 55 * 000013 F8 *
000014 9B * 000015 3E * 000016 E1 * 000017 84 *
000024 00 * 000025 00 * 000026 00 * 000027 00 *
000028 00 * 000029 00 * 00002A 00 * 00002B 00 *
00004C 20 * 00004D C3 * 00004E 66 * 00004F 09 *
000050 AC * 000051 4F * 000052 F2 * 000053 95 *
000088 31 * 000089 D4 * 00008A 77 * 00008B 1A *
00008C BD * 00008D 60 * 00008E 03 * 00008F A6 *
0000D8 00 * 0000D9 00 * 0000DA 00 * 0000DB 00 *
0000DC 00 * 0000DD 00 * 0000DE 00 * 0000DF 00 *
ПОСЛЕДОВАТЕЛЬНОСТЬ ЗАПРОСОВ: 3 2 1 3 1 2 3 1
ЧИСЛА, СЧИТЫВАЕМЫЕ С 1-го ВУ: 01 AB 55
ЧИСЛА, СЧИТЫВАЕМЫЕ С 3-го ВУ: 07 C5 83 41 FF
СОДЕРЖИМОЕ ПОДКАНАЛОВ:
РКК РДКСВРАКК
01 000027 2000 0003 * 03 AD 57 00 * 03 * 000330 *
02 00004D A000 0001 * 20 C3 66 09 * 01 * 000338 *
01 0000D8 2000 0005 * 07 00 00 00 * 00 * 000348 *

ВАРИАНТ НОМЕР 11

КАНАЛЬНЫЕ ПРОГРАММЫ:
ДЛЯ 1-го ВУ, В ПАМЯТИ ХРАНИТСЯ С АДРЕСА 000320:
02 000010 6000 0007
01 000024 2000 0006
ДЛЯ 2-го ВУ, В ПАМЯТИ ХРАНИТСЯ С АДРЕСА 000330:
02 00004C A000 0002
00 000088 2000 0004
ДЛЯ 3-го ВУ, В ПАМЯТИ ХРАНИТСЯ С АДРЕСА 000340:
01 0000D8 2000 0005
СОДЕРЖИМОЕ ЯЧЕЕК ОП:
000010 10 * 000011 10 * 000012 10 * 000013 10 *
000014 10 * 000015 10 * 000016 10 * 000017 10 *
000024 00 * 000025 00 * 000026 00 * 000027 00 *
000028 00 * 000029 00 * 00002A 00 * 00002B 00 *
00004C 21 * 00004D 21 * 00004E 21 * 00004F 21 *
000050 21 * 000051 21 * 000052 21 * 000053 21 *
000088 32 * 000089 32 * 00008A 32 * 00008B 32 *
00008C 32 * 00008D 32 * 00008E 32 * 00008F 32 *
0000D8 00 * 0000D9 00 * 0000DA 00 * 0000DB 00 *
0000DC 00 * 0000DD 00 * 0000DE 00 * 0000DF 00 *
ПОСЛЕДОВАТЕЛЬНОСТЬ ЗАПРОСОВ: 2 3 1 3 1 3 2 1
ЧИСЛА, СЧИТЫВАЕМЫЕ С 1-го ВУ: 03 BE 79 34 EF AA
ЧИСЛА, СЧИТЫВАЕМЫЕ С 3-го ВУ: A9 7A 4B
СОДЕРЖИМОЕ ПОДКАНАЛОВ:
РКК РДКСВРАКК
02 000013 6000 0004 * 10 10 10 10 * 03 * 000328 *
02 000089 2000 0003 * 32 32 32 32 * 01 * 000340 *
01 0000DA 2000 0003 * 07 D8 00 00 * 02 * 000348 *



ВАРИАНТ НОМЕР 12

КАНАЛЬНЫЕ ПРОГРАММЫ:

ДЛЯ 1-го ВУ, В ПАМЯТИ ХРАНИТСЯ С АДРЕСА 000320:

02 000010 6000 0007

01 000024 2000 0006

ДЛЯ 2-го ВУ, В ПАМЯТИ ХРАНИТСЯ С АДРЕСА 000330:

02 00004C A000 0002

00 000088 2000 0004

ДЛЯ 3-го ВУ, В ПАМЯТИ ХРАНИТСЯ С АДРЕСА 000340:

01 0000D8 2000 0005

СОДЕРЖИМОЕ ЯЧЕЕК ОП:

000010 11 * 000011 6E * 000012 CB * 000013 28 *

000014 85 * 000015 E2 * 000016 3F * 000017 9C *

000024 00 * 000025 00 * 000026 00 * 000027 00 *

000028 00 * 000029 00 * 00002A 00 * 00002B 00 *

00004C 22 * 00004D 7F * 00004E DC * 00004F 39 *

000050 96 * 000051 F3 * 000052 50 * 000053 AD *

000088 33 * 000089 90 * 00008A ED * 00008B 4A *

00008C A7 * 00008D 04 * 00008E 61 * 00008F BE *

0000D8 00 * 0000D9 00 * 0000DA 00 * 0000DB 00 *

0000DC 00 * 0000DD 00 * 0000DE 00 * 0000DF 00 *

ПОСЛЕДОВАТЕЛЬНОСТЬ ЗАПРОСОВ: 2 3 1 2 3 1 2 1

ЧИСЛА, СЧИТЫВАЕМЫЕ С 1-го ВУ: 03 CF 9B 67 33 FF

ЧИСЛА, СЧИТЫВАЕМЫЕ С 3-го ВУ: CF B3 97

СОДЕРЖИМОЕ ПОДКАНАЛОВ:

РКК РДКСБРАКК

02 000013 6000 0004 * 11 6E CB 28 * 03 * 000328 *

02 000089 2000 0003 * 33 90 ED 4A * 01 * 000340 *

01 0000DA 2000 0003 * 07 EB 00 00 * 02 * 000348 *



Лабораторная работа № 8. Моделирование секторной буферной памяти

Содержание

Цель работы	121
Функционирование секторного буфера	121
Выполнение работы	123
Описание подпрограмм	125
Пример выполнения	126
Содержание отчета	135
Задание к лабораторной работе	138



Цель работы

Изучение функционирования блока секторного буфера со сквозной записью.

Функционирование секторного буфера

Для повышения производительности ЭВМ между оперативной памятью и ЦП дополнительно используется сверхоперативная буферная память (БП), называемая кэш-памятью. Повышение производительности ЭВМ при использовании кэш-памяти достигается за счет того, что время доступа к БП значительно меньше времени доступа к оперативной памяти.

Уменьшение времени доступа к БП обеспечивается за счет уменьшения ее емкости по сравнению с оперативной памятью. В БП хранятся как программы, так и данные. Однако непосредственно из программы адресоваться к информации, хранимой в БП, нельзя. Функционирование двухуровневой системы памяти, включающей в себя БП и оперативную память, осуществляется следующим образом:

- по заданному адресу в команде определяется, хранится ли требуемая информация в БП.
- в том случае, если информация хранится в БП, то происходит работа с БП.
- в противном случае необходимо информацию перезаписать из оперативной памяти в БП, а затем происходит работа с БП.

Наибольшая производительность достигается в том случае, когда не требуется взаимодействия с оперативной памятью, а работа осуществляется на уровне БП.

В зависимости от способа отображения оперативной памяти на БП выделяют четыре способа построения:

- буферная память прямого соответствия,
- секторный буфер,
- ассоциативный буфер,
- группно-ассоциативный буфер.

Ниже рассматривается функционирование секторного и группно-ассоциативного буфера, являющихся развитием буфера прямого соответствия и ассоциативного буфера соответственно.

Целью моделирования функционирования секторного и группно-ассоциативного буферов является изучение работы кэш-памяти. При этом для уменьшения сложности моделирования рассматривается упрощенный вариант организации оперативной и буферной памяти, включающий ограниченное число блоков и сегментов, начиная с адреса, равного нулю. В дальнейшем для определенности будем считать, что кэш-память используется между ОП и ЦП.

Моделирование функционирования секторного буфера

Для организации функционирования ОП и секторного буфера все поля памяти и буфера разбивается на блоки равной величины, содержащие по L байтов. В свою очередь, блоки объединяются в зоны по N блоков. Зоны в ОП будем называть сегментами, а зоны в БП - секторами. Поскольку в реальных вычислительных системах емкость ОП определяется гигабайтами, а емкость БП определяется мегабайтами, то число сегментов C значительно превышает число секторов S . Любой из C сегментов ОП может быть помещен в любой из S секторов. Однако j – блок ($j=0..N-1$), i -го сегмента ОП ($i=0..C-1$), может быть помещен на место j -го блока k -го сектора БП ($k=0..S-1$). Для указания номеров сегментов, которые присутствуют в БП, в каждом секторе используется специальный регистр номера присутствующего сегмента.

Число регистров номеров присутствующих сегментов равно S . Каждый блок в БП содержит дополнительный бит присутствия блока и бит изменения блока. Бит присутствия блока устанавливается равным 1, если данный блок содержится в БП, в противном случае бит присутствия равен 0. Бит изменения блока равен 1 в том случае, если содержимое в БП изменялось и, таким образом, стало отличным от содержимого, аналогичного блока в ОП.



Номер сегмента, номер блока и номер байта определяют отдельные группы разрядов исполнительного адреса. Номер сегмента определяют старшие $\lceil \log_2(C) \rceil$ разрядов исполнительного адреса, где $\lceil x \rceil$ означает наибольшее целое от x . Следующие $\lceil \log_2(N) \rceil$ разрядов исполнительного адреса определяют номер блока внутри сегмента, а младшие $\lceil \log_2(L) \rceil$ разрядов - номер байта внутри блока.

Основные принципы функционирования секторного буфера.

На основании старших $\lceil \log_2(C) \rceil$ разрядов исполнительного адреса определяется номер сегмента. Этот номер сравнивается с содержимым регистра номера присутствующего сегмента в каждом секторе. Если в некотором секторе произошло совпадение номера сегмента и содержимого регистра номера присутствующего сегмента, то это означает, что сегмент находится в этом секторе. Далее на основании следующих $\lceil \log_2(N) \rceil$ разрядов исполнительного адреса определяется номер блока. В секторе, который содержит адресуемый сегмент, по бит присутствия блока определяется, находится ли блок в БП. Если бит присутствия блока равен 1, то это означает, что адресуемый блок находится в БП и по адресу байта, который определяется младшими $\lceil \log_2(L) \rceil$ разрядами исполнительного адреса, в зависимости от кода операции происходит чтение информации из ЦП в БП, или запись информации из ЦП в БП. Если же бит присутствия блока равен 0, то это означает, что адресуемый блок отсутствует в БП. В связи с этим блок переписывается из ОП в БП (старшие $\lceil \log_2(C) \rceil$ разрядов исполнительного адреса определяют номер сегмента ОП, который содержит требуемый блок, а следующие $\lceil \log_2(N) \rceil$ разрядов определяют номер адресуемого блока как в ОП, так и в БП). Найденный блок в ОП переписывается на соответствующее место в БП в тот сектор, который содержит адресуемый сегмент, а бит присутствия этого блока устанавливается в 1. Параллельно с перезаписью блока в БП, в случае операции чтения, происходит считывание информации в ЦП. В случае операции записи после занесения блока в БП записываются данные из ЦП в БП. Выше рассматривался случай нахождения адресуемого сегмента в БП. В том случае, если ни в одном секторе номер сегмента не совпал с номером регистра присутствующего сегмента, то это означает, что адресуемый сегмент отсутствует в БП. Для перезаписи сегмента в БП нужно один из сегментов, хранящихся в БП, вытеснить. Существуют различные алгоритмы определения номера сегмента, подлежащего вытеснению:

- LRU - вытесняется сегмент наиболее давно использовавшийся;
- FIFO - вытесняется сегмент, раньше других помещенный в буфер;
- LIFO - вытесняется сегмент, позже других поступивший в буфер.

На основании одного из принятых алгоритмов определяется номер сегмента, подлежащий вытеснению. При вытеснении сегмента могут использоваться различные дисциплины. Сначала рассмотрим дисциплину признакового обмена. Для каждого блока вытесняемого сегмента анализируется бит изменения блока. Для тех блоков, у которых бит изменения равен 1, происходит перезапись блоков в ОП. Биты изменения сбрасываются в 0. После перезаписи блоков в ОП в регистре номера присутствующего сегмента устанавливается номер сегмента, который должен быть помещен в БП из ОП, а биты присутствия всех блоков сбрасываются в 0.

При вытеснении сегмента, кроме признакового регистрового обмена, в ряде ЭВМ используется сквозная запись, а также ряд других обменных дисциплин. При использовании сквозной записи запись данных в БП осуществляется одновременно с записью данных в ОП. Поэтому отпадает необходимость использовать биты изменения блоков. При сквозной записи, как и в случае признакового обмена, при вытеснении сегмента необходимо сбросить в 0 все биты присутствия блоков, а в регистр номера присутствующего сегмента поместить номер сегмента, записываемого в БП из ОП.

Буфер прямого соответствия является частным случаем секторного буфера, состоящего из одного сектора. Графическое изображение окна программы при выборе лабораторной работы «Секторный буфер (кэш-память)» приведено на рис. 49.

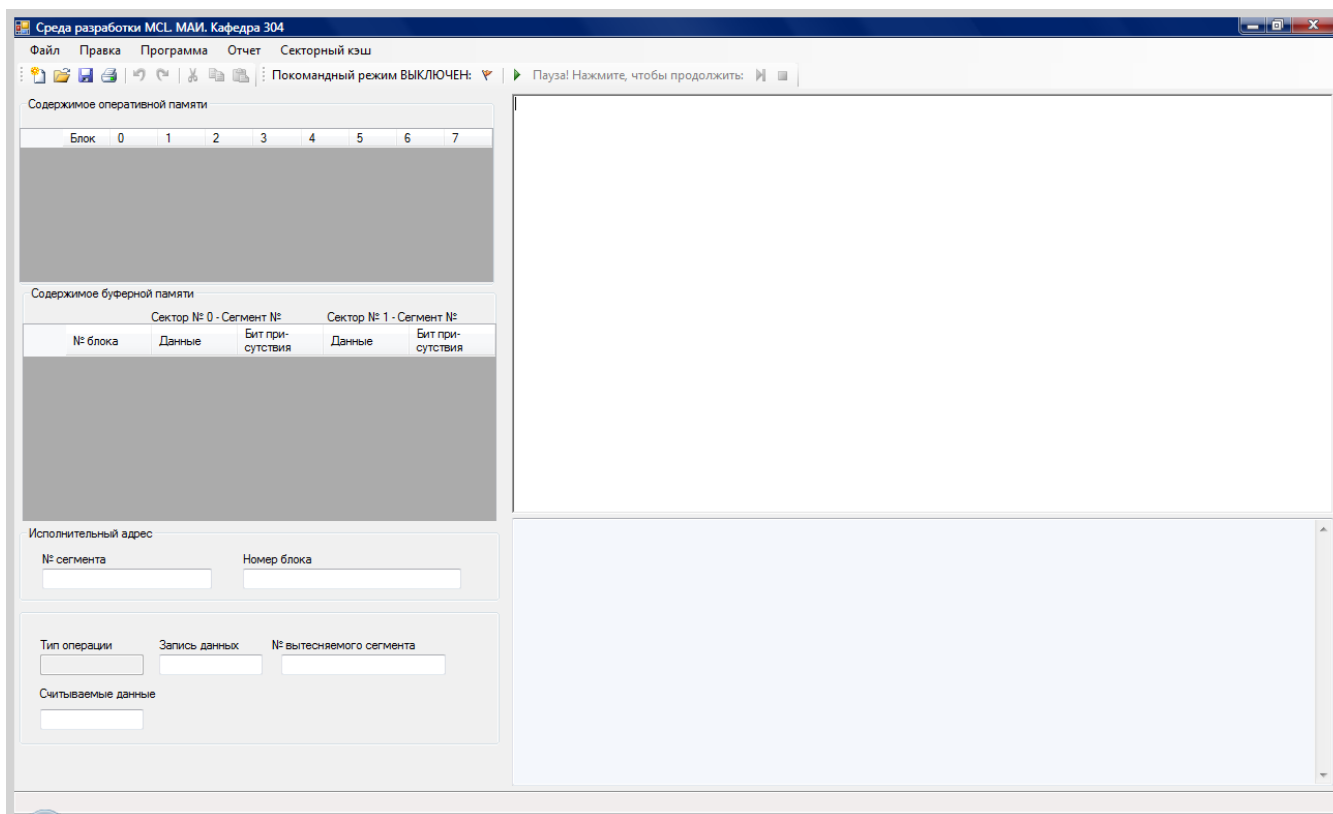


Рис. 49. Окно программы при выборе лабораторной работы «Секторный буфер (кэш-память)»

Выполнение работы

Работы содержит две части: буферная память с признаковым обменом и со сквозной записью. Первая часть представлена в виде примера и содержит код готовой программы. Ознакомившись с ее работой, можно приступить ко второй части, где **необходимо изменить исходный код и показать работу буферной памяти со сквозной записью, при этом программа запускается при выбранном пункте «Сквозная запись», пункт «Признаковый обмен» служит в качестве примера в обучающих целях.**

Работа буфера моделируется на уровне блоков, т.е. адрес ячейки в блоке не используется. Требуется написать программу моделирования буферной памяти со сквозной записью, моделирующую три обращения к памяти. В качестве примера в данном описании приводится текст программы моделирования буферной памяти с признаковым обменом. Соответствующая MCL-программа находится в рабочей директории и имеет имя Секторный КЭШ с признаковым обменом.MCL.

В программе моделирования буферной памяти с признаковым обменом описаны следующие типы (см. программу).

Блок-схема алгоритма функционирования секторного буфера имеет вид:

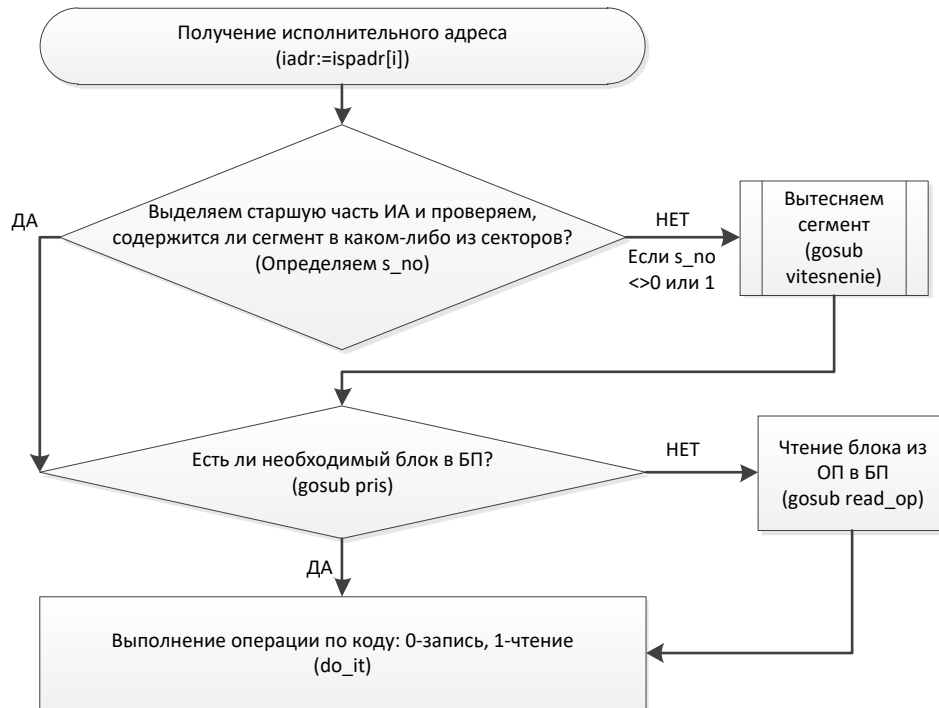


Рис. 50. Блок-схема функционирования секторного буфера

Блок-схема алгоритма вытеснения для секторного буфера при признаковом обмене:



Рис.51. Блок-схема алгоритма вытеснения

Типы, используемые подпрограммами

Внимание! Имена переменных должны быть строго соблюдены, иначе программа может выполняться некорректно.

mem op[32][8] – оперативная память 32 слова по 8 бит



mem bp[16][10] - буферная память 16 слов по 10 бит, при этом 8 бит - данные, 1 бит изменения и 1 бит присутствия. Для работы со словом БП необходимо считать его в структуру BUF_REG[10]
 mem ispadr[3][5] - память исполнительных адресов, содержит три исполнительных адреса
 mem inputs[3][8] - числа для записи, содержит 3 числа записываемых в память
 mem coper[3][1] - память кодов операции, содержит 3 кода операции (1-чтение, 0-запись)
 mem seg_present[2][2] - регистры номеров присутствующих сегментов (два регистра по количеству секторов)
 mem rep_seg[3][2] - вытесняемый сегмент, содержит номера сегментов, вытесняемых из БП при замещении

struct iadr[5] - адрес ячейки ОП 32 слова
 blok[3], - номер блока, всего 8 блоков.
 seg [2]; - номер сегмента, всего 4 сегмента.

В данную структуру считывается исполнительный адрес из памяти, что позволяет различать в нем номера блока и сегмента.

struct bpadr[4] - адрес ячейки БП 16 слов
 blok[3], - номер блока, всего 8 блоков.
 sect[1]; - номер сектора, всего 2 сектора.

Данной структуре присваивается адрес буферной памяти, что позволяет различать в нем номера блока и сегмента.

struct buf_reg[10] - структура слова буферной памяти
 data[8], - данные
 ch_bit[1], - бит изменения
 pres_bit[1]; - бит наличия

В эту структуру считывается слово БП при любой обработке, что позволяет различать в нем биты присутствия, изменения и собственно данные.

var i, j, - вспомогательная переменная
 choice, - номер варианта

Переменные используемые в программе моделирования работы БП:

k, l, - вспомогательная переменная
 s_no; - номер сектора, содержащего необходимый сегмент

Следующие переменные используются для упрощения. В них считываются соответственно значения coper, inputs, и rep_seg для данного обращения к памяти:

op_code, - код операции
 data, - данные
 replace, - номер замещаемого сегмента

Используемые подпрограммы

1. VARIANT - подпрограмма задания исходных данных по варианту и их распечатка. При исполнении подпрограммы на экран выводится сообщение:

Введите номер варианта:



после чего необходимо ввести номер варианта от 1 до 15 включительно. При этом подпрограммой задаются начальные значения следующих описанных выше типов:

op, bp, ispadr, inputs, coper, seg_present, rep_seg.

Исходные данные заданы таким образом, что элементы массивов ispadr, inputs, coper и rep_seg с одинаковым индексом относятся к одному обращению к памяти. То есть если первое обращение к памяти - чтение, а второе запись, то inputs[1] не используется вообще, а во втором обращении используются сразу inputs[2]. Значения исходных данных см. ниже.

2. FIRST, SECOND, THIRD - данные подпрограммы используются исключительно подпрограммой VARIANT.

3. PRINT_BP - подпрограмма печати содержимого буферной памяти.

4. PRINT_BR - подпрограмма печати содержимого буферного регистра, используется исключительно подпрограммой PRINT_BP.

5. PRINT_OP - подпрограмма печати содержимого оперативной памяти.

6. PRINT - подпрограмма печати чисел заданных для записи и номеров вытесняемых сегментов, используется подпрограммами FIRST, SECOND и THIRD.

7. PRIS – подпрограмма для определения присутствия блока в БП, изменяет значение переменной pris на 0 (блок отсутствует) или 1 (блок присутствует).

8. WRITE_DATA – подпрограмма для записи данных в БП.

9. READ_DATA – подпрограмма для чтения данных из БП.

10. READ_OP – подпрограмма для чтения из ОП в БП.

11. VITESNENIE – подпрограмма для вытеснения сегмента из сектора БП.

12. CHANGED – подпрограмма для определения изменялся ли блок в БП, изменяет значение переменной changed на 0 (блок не изменялся) или 1 (блок изменялся).

13. WRITE_OP – подпрограмма для записи из БП в ОП.

14. ZERO_BITS – подпрограмма для обнуления битов присутствия и изменения.

Типы, описанные в данном файле, полностью идентичны типам описанным в программе моделирования БП с признаковым обменом. Отличием является отсутствие поля CH_BIT[1] - бит изменения, структуры BUF_REG. Соответственно иначе описана и память ВР. Так же отсутствуют описания переменных используемых собственно в программе моделирования, которые вводятся по усмотрению студентов. Подпрограммы имеющиеся в файле, имеют те же имена и назначение, что и в описанной выше программе.

Текст программы из файла Секторный КЭШ с признаковым обменом.MCL (признаковый обмен):

```
writeln " *****";
writeln " *          Модель секторной КЭШ памяти с признаковым обменом          *";
writeln " *****";

{
    работа моделируется на уровне блоков
}

{ ***** ОПИСАНИЕ ТИПОВ ***** }

{ типы используемые п/п }

var i,j,k,l,
    choice,
    op_code,
    data,
    replace,
    s_no,
    { вспомогательные переменные }
    { номер варианта }
    { код операции }
    { данные }
    { номер замещаемого сегмента }
    { номер сектора, содержащего необходимый сегмент }
```



```
    pris,                { для п/п проверки присутствия блока в БП }
    changed,             { для п/п проверки изменения блока в БП }
    read_data_flag;

mem op[32][8];           { оперативная память 32 слова по 8 бит }

mem bp[16][10];          { буферная память 16 слов по 10 бит}

mem ispadr[3][5];        { массив для хранения исполнительных адресов}

mem inputs[3][8];        { массив для хранения чисел, которые должны быть записаны в
БП}

mem coper[3][1];         { массив для хранения кодов операций 1-чтение, 0-запись }

mem seg_present[2][2];   { регистры номеров присутствующих сегментов}

mem rep_seg[3][2];       { массив для хранения номеров сегментов подлежащих вытеснению}

struct iadr[5]           { структура для разбиения исполнительного адреса на
старшую и младшую часть }
    blok[3],              { номер блока}
    seg [2];              { номер сегмента }

struct bpadr[4]           { структура для выделения старшей и младшей части в адресе
БП }
    blok[3],              { номер блока }
    sect[1];              { номер сектора }

struct buf_reg[10]        { структура для выделения данных, бита изменения и бита
присутствия}
    data[8],              { данные }
    ch_bit[1],            { бит изменения }
    pres_bit[1];          { бит присутствия }

struct subiadr[5]         { переменная используется в подпрограммах }
    blok[3],
    seg [2];

struct subbpadr[4]        { permennaa ispol'zuetsa v podprogrammah }
    blok[3],
    sect[1];

struct subbuf_reg[10]     { permennaa ispol'zuetsa v podprogrammah }
    data[8],
    ch_bit[1],
    pres_bit[1];

{***** начало программы *****)

gosub variant;           { задание начальных значений всех переменных }
for l:=0 to 2;            { цикл для работы с 3-мя исполнительными адресами }

    read_data_flag:=0;
    {получаем исполнительный адрес}
    iadr:=ispadr[1];
    {получаем код операции}
    op_code:=coper[1];
    {загружаем данные для записи}
    data:=inputs[1];
    {получаем номер вытесняемого сегмента}
```



```
replace:=rep_seg[1];
writeln "Исполнительный адрес - ", $d2 iadr;
pause;

{задаем случайное значение номера сектора не равное 0 или 1}
s_no:=2;

{определяем номер сегмента на основе старших разрядов и номер сектора БП,
содержащего сегмент}
if seg_present[0]=iadr.seg then s_no:=0;
{если сегмент в секторе 0, то переходим к проверке наличия блока}
if s_no=0 then goto get_blok;
{если сегмент в секторе 1, то переходим к проверке наличия блока}
if seg_present[1]=iadr.seg then s_no:=1;
if s_no=1 then goto get_blok;
{если сегмента нет в секторах 0 и 1, то переходим к вытеснению сегмента}
writeln "Сегмент ", $d2 iadr.seg, " отсутствует в буферной памяти";
writeln "Вытеснение сегмента ", $d2 replace;
pause;
gosub vitesnenie;
get_blok:
{проверка есть ли блок в БП}
gosub pris;
{pris=0-блок отсутствует, pris=1-присутствует}
{если блок присутствует, то пропускаем подкачку блока}
if pris=1 then goto est_blok;
{если блок отсутствует, то подгружаем его из ОП}
writeln "Блок ", $d2 iadr.blok, " сегмента ", $d2 iadr.seg, " отсутствует в БП";
writeln "Чтение блока из ОП...";
pause;
gosub read_OP;
pause;
goto do_it;
est_blok:
writeln "Блок ", $d2 iadr.blok, " сегмента ", $d2 iadr.seg, " присутствует в БП";
goto do_it;

do_it: { выполнение операции по коду}
if op_code=1 then gosub read_data; { если 1 то чтение}
{ если 0 то запись}
if op_code=0 then gosub write_data;
pause;
gosub print_bp; { печать содержимого БП }
pause;
gosub print_op; { печать содержимого ОП }
pause;

next;
end;

{***** конец программы *****}

{***** подпрограммы *****}

{ п/п для задания варианта, использует п/п first,second,third }

variant:
s_no:=0;
pause;
{ задание исходных данных по вариантам}
```




```
read " Введите номер варианта: ",choice;

if choice<=5 then gosub first;
if ((choice>5) and (choice<=10)) then gosub second;
if choice>10 then gosub third;

return; { variant }

{ п/п первый вариант }

first:

{ присваивание начальных значений ячейкам ОП }

for i:=0 to 31;
    op[i]:=(choice+2)*2+i-2;
next;

{ присваивание начальных значений ячейкам БП }

for i:=0 to 15;
    buf_reg.data:=(choice+2)*2+i-2;
    buf_reg.ch_bit:=0;
    buf_reg.pres_bit:=1;
    bp[i]:=buf_reg;
next;

for i:=2 to 5;
    buf_reg.data:=choice+i-2;
    buf_reg.ch_bit:=0;
    buf_reg.pres_bit:=0;
    bp[i]:=buf_reg;
next;

{ назначение исполнительных адресов }

ispadr[0]:=4;
ispadr[1]:=29;
ispadr[2]:=15;

{ задание чисел для записи }

for i:=0 to 2;
    inputs[i]:=i+8;
next;

{ задание КОП }

coper[0]:=0;
coper[1]:=1;
coper[2]:=0;

{ задание номеров присутствующих сегментов }

seg_present[0]:=0;
seg_present[1]:=1;

{ задание номеров замещаемых сегментов }

rep_seg[0]:=0;
rep_seg[1]:=0;
```



```
rep_seg[2]:=0;

gosub print_bp;    { печать содержимого БП}

gosub print_op;    { печать содержимого ОП}
gosub print; { печать записываемых чисел и замещаемых сегментов }

return; { first }

{ п/п второй вариант }

second:

{ присваивание начальных значений ячейкам ОП }

for i:=0 to 31;
    op[i]:=(choice+2)*2+i-8;
next;

{ присваивание начальных значений ячейкам БП }

for i:=0 to 15;
    buf_reg.data:=(choice+2)*2+i;
    buf_reg.ch_bit:=0;
    buf_reg.pres_bit:=1;
    bp[i]:=buf_reg;
next;

for i:=0 to 3;
    buf_reg.data:=choice-4;
    buf_reg.ch_bit:=0;
    buf_reg.pres_bit:=0;
    bp[i]:=buf_reg;
next;

{ назначение исполнительных адресов}

ispadr[0]:=10;
ispadr[1]:=30;
ispadr[2]:=19;

{ задание чисел для записи }

for i:=0 to 2;
    inputs[i]:=i+12;
next;

{ задание КОП }

coper[0]:=1;
coper[1]:=0;
coper[2]:=1;

{ задание номеров присутствующих сегментов }

seg_present[0]:=1;
seg_present[1]:=2;

{ задание номеров замещаемых сегментов }
```



```
rep_seg[0]:=0;
rep_seg[1]:=1;
rep_seg[2]:=0;

gosub print_bp; { печать содержимого БП }
gosub print_op; { печать содержимого ОП }
gosub print; { печать записываемых чисел и замещаемых сегментов }

return; { second }

{ п/п третий вариант }

third:

{ присваивание начальных значений ячейкам ОП }

for i:=0 to 31;
    op[i]:=(choice+2)/2+i+5;
next;

{ присваивание начальных значений ячейкам БП }

for i:=0 to 15;
    buf_reg.data:=(choice+2)/2+i+21;
    buf_reg.ch_bit:=0;
    buf_reg.pres_bit:=1;
    bp[i]:=buf_reg;
next;

for i:=4 to 7;
    buf_reg.data:=choice-7;
    buf_reg.ch_bit:=0;
    buf_reg.pres_bit:=0;
    bp[i]:=buf_reg;
next;

{ назначение исполнительных адресов }

ispadr[0]:=21;
ispadr[1]:=8;
ispadr[2]:=24;

{ задание чисел для записи }

for i:=0 to 2;
    inputs[i]:=i+23;
next;

{ задание КОП }

coper[0]:=1;
coper[1]:=1;
coper[2]:=0;

{ задание номеров присутствующих сегментов }

seg_present[0]:=2;
seg_present[1]:=3;

{ задание номеров замещаемых сегментов }
```



```
rep_seg[0]:=0;
rep_seg[1]:=2;
rep_seg[2]:=0;

gosub print_bp;    { печать содержимого БП }

gosub print_op;    { печать содержимого ОП }
gosub print; { печать записываемых чисел и замещаемых сегментов}

return; { third }

{ п/п печати содержимого БП }

print_bp:
  writeln "Содержимое буферной памяти :";
  writeln "      Сектор 0      Сектор 1";
  writeln "Блок   Изм. Прис. Данные   Изм. Прис. Данные";
  for i:=0 to 7;
    write " ", $d2 i;
    subbpadr.sect:=0;
    subbpadr.blok:=i;
    subbuf_reg:=bp[subbpadr];
    gosub print_br;
    subbpadr.sect:=1;
    subbuf_reg:=bp[subbpadr];
    gosub print_br;
    writeln;
  next;
  writeln "Содержимое РНПС:";
  writeln "-----";
  writeln "!      сектор      !      сегмент      !";
  writeln "-----";
  writeln "!      0      !", $d7 seg_present[0], "      !";
  writeln "!      1      !", $d7 seg_present[1], "      !";
  writeln "-----";
return; { print_bp }

{ п/п печати буферного регистра}

print_br:
  write "      ", $b1 subbuf_reg.ch_bit;
  write "      ", $b1 subbuf_reg.pres_bit;
  write "      ", $d4 subbuf_reg.data;
return; { print_br }

{ п/п печати содержимого ОП}

print_op:
  writeln "Содержимое оперативной памяти:";
  writeln "      Блок #      0      1      2      3      4      5      6      7";
  for i:=0 to 3;
    write " Сегмент #", $d1 i, " ";
    subiadr.seg:=i;
    for j:=0 to 7;
      subiadr.blok:=j;
      write $d4 op[subiadr];
    next;
    writeln;
  next;
return; { print_op }
```



```
{ п/п печати записываемых чисел и замещаемых сегментов }

print:
  writeln "-----";
  writeln "! № обращ. ! Числа для ! № вытесн. !";
  writeln "!           ! записи      ! сегмента  !";
  writeln "-----";
  writeln "!      1      !", $d7 inputs[0], "      !", $d6 rep_seg[0], "      !";
  writeln "!      2      !", $d7 inputs[1], "      !", $d6 rep_seg[1], "      !";
  writeln "!      3      !", $d7 inputs[2], "      !", $d6 rep_seg[2], "      !";
  writeln "-----";
return; { print }

{ п/п определения присутствия блока в БП}

pris:
  bpadr.blok:=iadr.blok;
  bpadr.sect:=s_no;
  buf_reg:=bp[bpadr];{ чтение блока из БП в буферный регистр }
  if buf_reg.pres_bit=1 then pris=1; { esli blok prisutstvuet v BP }
  if buf_reg.pres_bit=0 then pris=0; { esli blok otsutstvuet v BP }
return; {pris}

{ п/п записи данных в БП}

write_data:
  buf_reg.data:=data;
  buf_reg.ch_bit:=1;
  buf_reg.pres_bit:=1;

  bp[bpadr]:=buf_reg;
  writeln "Записаны данные ", $d4 data;

return; {write_data}

{ п/п для чтения данных из БП}

read_data:

  data:=buf_reg.data;
  read_data_flag:=1;
  writeln "Прочитаны данные ", $d4 data;
return; {read_data}

{п/п чтения из ОП в БП}

read_OP:
  buf_reg.data:=op[iadr];
  buf_reg.pres_bit:=1;
  buf_reg.ch_bit:=0;
  bp[bpadr]:=buf_reg;
  writeln "Блок прочитан из ОП";
return; {read_OP}

{п/п для вытеснения сегмента из сектора БП}

vitesnenie:
  if seg_present[0]=replace then s_no:=0;      { поиск номера сектора, в котором
находится вытесняемый сегмент}
  if seg_present[1]=replace then s_no:=1;
```



```
for j:=0 to 7; { вытеснение сегмента}
  { проверка, изменялся ли блок}
  gosub changed;
  {changed=0-не изменялся, changed=1-изменялся}
  if changed=0 then goto dont_save; { если блок не изменялся}
  { если блок изменялся, то сохраняем его в ОП}
  gosub write_OP;
  gosub zero_bits; {обнуляем биты присутствия и изменения}
dont_save:
  gosub zero_bits; {обнуляем биты присутствия и изменения}
next;
  seg_present[s_no]:=iadr.seg; {заносим в РНПС номер нового сегментв}
  writeln "Сегмент успешно вытеснен";

return; {vitesnenie}

{п/п определения был ли изменен блок в БП}

changed:
  bpadr.sect:=s_no; { определение адреса в БП}
  bpadr.blok:=j;
  buf_reg:=bp[bpadr];
  if buf_reg.ch_bit=0 then changed=0;
  if buf_reg.ch_bit=1 then changed=1;
  if changed=1 then writeln "Блок ", $d2 j, " сектора ", $d2 s_no, " был изменен";
return; {changed}

{п/п записи из БП в ОП}

write_OP:
  k:=iadr;
  iadr.blok:=bpadr.blok;
  iadr.seg:=seg_present[s_no];
  op[iadr]:=buf_reg.data;
  writeln "Сохранение блока ", $d2 iadr.blok, " сегмента ", $d2 iadr.seg, " в ОП ";
  iadr:=k;
return; {write_OP}

{п/п для обнуления битов присутствия и изменения}

zero_bits:
  buf_reg.ch_bit:=0;
  buf_reg.pres_bit:=0;
  bp[bpadr]:=buf_reg;
return; {zero_bits}
```



Пример выполнения программы с признаковым обменом (файл Секторный КЭШ с признаковым обменом.MCL) для 1-го варианта:

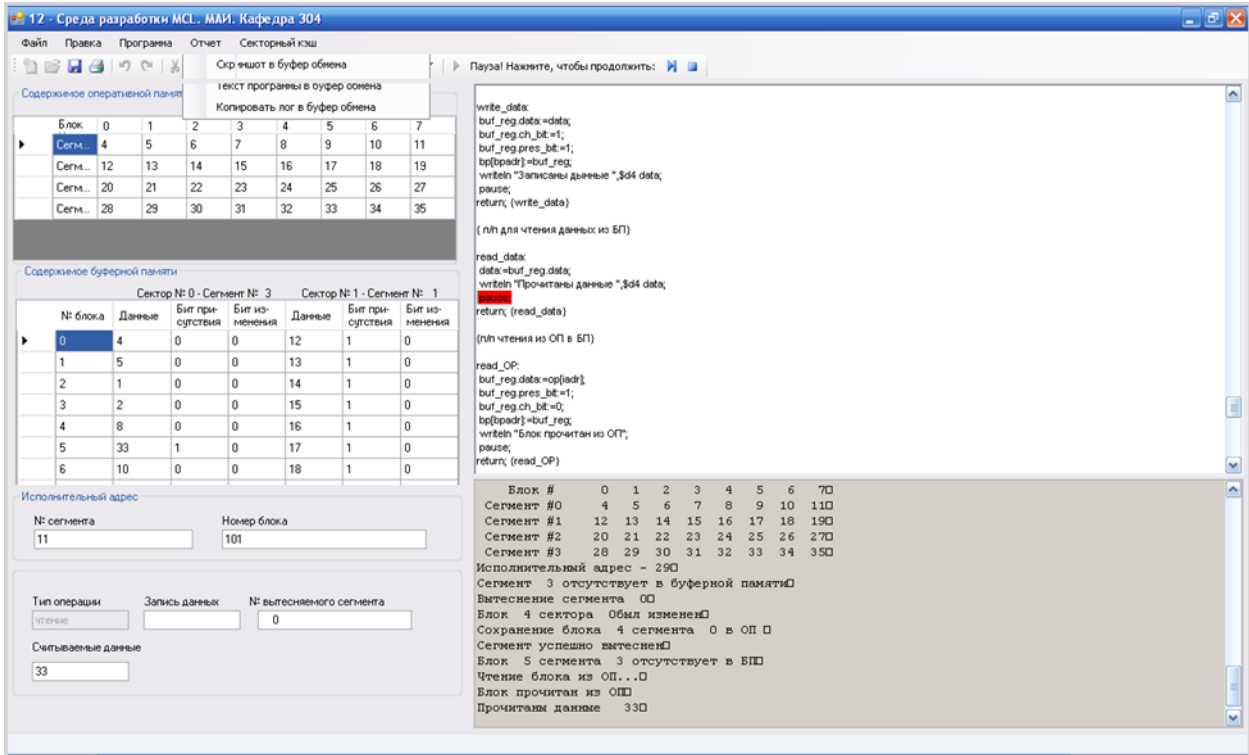


Рис. 52. Результат моделирования секторного буфера с признаковым обменом

```
*****
*          Модель секторной КЭШ памяти с признаковым обменом          *
*****

Введите номер варианта:
1
Содержимое буферной памяти :
      Сектор 0          Сектор 1
Блок  Изм. Прис. Данные  Изм. Прис. Данные
0      0      1      4      0      1      12
1      0      1      5      0      1      13
2      0      0      1      0      1      14
3      0      0      2      0      1      15
4      0      0      3      0      1      16
5      0      0      4      0      1      17
6      0      1     10      0      1      18
7      0      1     11      0      1      19

Содержимое РНПС:
-----
!      сектор      !      сегмент      !
-----
!      0      !      0      !
!      1      !      1      !
-----

Содержимое оперативной памяти:
      Блок #      0      1      2      3      4      5      6      7
Сегмент #0      4      5      6      7      8      9     10     11
```



Сегмент #1	12	13	14	15	16	17	18	19
Сегмент #2	20	21	22	23	24	25	26	27
Сегмент #3	28	29	30	31	32	33	34	35

! № обращ. ! Числа для ! № вытесн. !
! ! записи ! сегмента !

!	1	!	8	!	0	!
!	2	!	9	!	0	!
!	3	!	10	!	0	!

Исполнительный адрес - 4
Блок 4 сегмента 0 отсутствует в БП
Чтение блока из ОП...
Блок прочитан из ОП
Записаны данные 8

Содержимое буферной памяти :

	Сектор 0			Сектор 1		
Блок	Изм.	Прис.	Данные	Изм.	Прис.	Данные
0	0	1	4	0	1	12
1	0	1	5	0	1	13
2	0	0	1	0	1	14
3	0	0	2	0	1	15
4	1	1	8	0	1	16
5	0	0	4	0	1	17
6	0	1	10	0	1	18
7	0	1	11	0	1	19

Содержимое РНПС:

! сектор ! сегмент !

! 0 ! 0 !
! 1 ! 1 !

Содержимое оперативной памяти:

Блок #	0	1	2	3	4	5	6	7
Сегмент #0	4	5	6	7	8	9	10	11
Сегмент #1	12	13	14	15	16	17	18	19
Сегмент #2	20	21	22	23	24	25	26	27
Сегмент #3	28	29	30	31	32	33	34	35

Исполнительный адрес - 29
Сегмент 3 отсутствует в буферной памяти
Вытеснение сегмента 0
Блок 4 сектора 0 был изменен
Сохранение блока 4 сегмента 0 в ОП
Сегмент успешно вытеснен
Блок 5 сегмента 3 отсутствует в БП
Чтение блока из ОП...
Блок прочитан из ОП
Прочитаны данные 33
Содержимое буферной памяти :



Блок	Сектор 0			Сектор 1		
	Изм.	Прис.	Данные	Изм.	Прис.	Данные
0	0	0	4	0	1	12
1	0	0	5	0	1	13
2	0	0	1	0	1	14
3	0	0	2	0	1	15
4	0	0	8	0	1	16
5	0	1	33	0	1	17
6	0	0	10	0	1	18
7	0	0	11	0	1	19

Содержимое РНПС:

!	сектор	!	сегмент	!
!	0	!	3	!
!	1	!	1	!

Содержимое оперативной памяти:

Блок #	0	1	2	3	4	5	6	7
Сегмент #0	4	5	6	7	8	9	10	11
Сегмент #1	12	13	14	15	16	17	18	19
Сегмент #2	20	21	22	23	24	25	26	27
Сегмент #3	28	29	30	31	32	33	34	35

Исполнительный адрес - 15

Блок 7 сегмента 1 присутствует в ВП

Записаны данные 10

Содержимое буферной памяти :

Блок	Сектор 0			Сектор 1		
	Изм.	Прис.	Данные	Изм.	Прис.	Данные
0	0	0	4	0	1	12
1	0	0	5	0	1	13
2	0	0	1	0	1	14
3	0	0	2	0	1	15
4	0	0	8	0	1	16
5	0	1	33	0	1	17
6	0	0	10	0	1	18
7	0	0	11	1	1	10

Содержимое РНПС:

!	сектор	!	сегмент	!
!	0	!	3	!
!	1	!	1	!

Содержимое оперативной памяти:

Блок #	0	1	2	3	4	5	6	7
Сегмент #0	4	5	6	7	8	9	10	11
Сегмент #1	12	13	14	15	16	17	18	19
Сегмент #2	20	21	22	23	24	25	26	27
Сегмент #3	28	29	30	31	32	33	34	35



Содержание отчета

1. Формулировка задания.
2. Исходные данные.
3. Программа на языке MCL, моделирующая буферную память со сквозной записью.
4. Результаты выполнения программы в **виде распечатки** содержимого поля для вывода текстовой информации **и скриншота** (пример на рис. 50) в момент завершения выполнения программы.

Контрольные вопросы

1. Запись и замещение блоков с признаковым обменом.
2. Запись и замещение блоков со сквозным обменом.
3. Запись блоков разных сегментов в один сектор.

Задание к лабораторной работе

Все возможные варианты разбиты на три группы в соответствии с чем задаются исходные данные.

Для вариантов 1-5 задаются следующие данные:

Номер обращения к памяти	Исполнительный адрес			Код операции	
	адрес	N сегмента	N блока		
1	4	0	4	запись	0
2	29	3	5	чтение	1
3	15	1	7	запись	0

Для вариантов 6-10 задаются следующие данные:

Номер обращения к памяти	Исполнительный адрес			Код операции	
	адрес	N сегмента	N блока		
1	10	1	2	чтение	1
2	30	3	6	запись	0
3	19	2	3	чтение	1

Для вариантов 11-15 задаются следующие данные:

Номер обращения к памяти	Исполнительный адрес			Код операции	
	адрес	N сегмента	N блока		
1	21	2	5	чтение	1
2	8	1	0	чтение	1
3	24	3	0	запись	0



Содержимое ОП и БП различно для каждого варианта.

Вариант 1

Содержимое буферной памяти:

Сектор 0				Сектор 1			
Блок	Изм.	Прис.	Данные	Изм.	Прис.	Данные	
0	0	1	4	0	1	12	
1	0	1	5	0	1	13	
2	0	0	1	0	1	14	
3	0	0	2	0	1	15	
4	0	0	3	0	1	16	
5	0	0	4	0	1	17	
6	0	1	10	0	1	18	
7	0	1	11	0	1	19	

Содержимое РНПС:

!	сектор	!	сегмент	!
!	0	!	0	!
!	1	!	1	!

Содержимое оперативной памяти:

Блок #	0	1	2	3	4	5	6	7
Сегмент #0	4	5	6	7	8	9	10	11
Сегмент #1	12	13	14	15	16	17	18	19
Сегмент #2	20	21	22	23	24	25	26	27
Сегмент #3	28	29	30	31	32	33	34	35

!	N обращ.	!	Числа для записи	!	N вытесн. сегмента	!
!	1	!	8	!	0	!
!	2	!	9	!	0	!
!	3	!	10	!	0	!

Вариант 2

Содержимое буферной памяти :

Сектор 0				Сектор 1			
Блок	Изм.	Прис.	Данные	Изм.	Прис.	Данные	
0	0	1	6	0	1	14	
1	0	1	7	0	1	15	
2	0	0	2	0	1	16	
3	0	0	3	0	1	17	
4	0	0	4	0	1	18	
5	0	0	5	0	1	19	
6	0	1	12	0	1	20	
7	0	1	13	0	1	21	

Содержимое РНПС:

!	сектор	!	сегмент	!
!	0	!	0	!
!	1	!	1	!

Содержимое оперативной памяти:

Блок #	0	1	2	3	4	5	6	7
Сегмент #0	6	7	8	9	10	11	12	13
Сегмент #1	14	15	16	17	18	19	20	21
Сегмент #2	22	23	24	25	26	27	28	29
Сегмент #3	30	31	32	33	34	35	36	37

!	N обращ.	!	Числа для записи	!	N вытесн. сегмента	!
!		!		!		!



!	1	!	8	!	0	!
!	2	!	9	!	0	!
!	3	!	10	!	0	!

Вариант 3

Содержимое буферной памяти :

Сектор 0				Сектор 1		
Блок	Изм.	Прис.	Данные	Изм.	Прис.	Данные
0	0	1	8	0	1	16
1	0	1	9	0	1	17
2	0	0	3	0	1	18
3	0	0	4	0	1	19
4	0	0	5	0	1	20
5	0	0	6	0	1	21
6	0	1	14	0	1	22
7	0	1	15	0	1	23

Содержимое РНПС:

!	сектор	!	сегмент	!
!	0	!	0	!
!	1	!	1	!

Содержимое оперативной памяти:

Блок #	0	1	2	3	4	5	6	7
Сегмент #0	8	9	10	11	12	13	14	15
Сегмент #1	16	17	18	19	20	21	22	23
Сегмент #2	24	25	26	27	28	29	30	31
Сегмент #3	32	33	34	35	36	37	38	39

!	N обращ.	!	Числа для	!	N вытесн.	!
!		!	записи	!	сегмента	!
!	1	!	8	!	0	!
!	2	!	9	!	0	!
!	3	!	10	!	0	!

Вариант 4

Содержимое буферной памяти :

Сектор 0				Сектор 1		
Блок	Изм.	Прис.	Данные	Изм.	Прис.	Данные
0	0	1	10	0	1	18
1	0	1	11	0	1	19
2	0	0	4	0	1	20
3	0	0	5	0	1	21
4	0	0	6	0	1	22
5	0	0	7	0	1	23
6	0	1	16	0	1	24
7	0	1	17	0	1	25

Содержимое РНПС:

!	сектор	!	сегмент	!
!	0	!	0	!
!	1	!	1	!

Содержимое оперативной памяти:

Блок #	0	1	2	3	4	5	6	7
Сегмент #0	10	11	12	13	14	15	16	17
Сегмент #1	18	19	20	21	22	23	24	25
Сегмент #2	26	27	28	29	30	31	32	33
Сегмент #3	34	35	36	37	38	39	40	41

!	N обращ.	!	Числа для	!	N вытесн.	!
!		!	записи	!	сегмента	!



!	1	!	8	!	0	!
!	2	!	9	!	0	!
!	3	!	10	!	0	!

Вариант 5

Содержимое буферной памяти :						
Сектор 0				Сектор 1		
Блок	Изм.	Прис.	Данные	Изм.	Прис.	Данные
0	0	1	12	0	1	20
1	0	1	13	0	1	21
2	0	0	5	0	1	22
3	0	0	6	0	1	23
4	0	0	7	0	1	24
5	0	0	8	0	1	25
6	0	1	18	0	1	26
7	0	1	19	0	1	27

Содержимое РНПС:

!	сектор	!	сегмент	!
!	0	!	0	!
!	1	!	1	!

Содержимое оперативной памяти:

Блок #	0	1	2	3	4	5	6	7
Сегмент #0	12	13	14	15	16	17	18	19
Сегмент #1	20	21	22	23	24	25	26	27
Сегмент #2	28	29	30	31	32	33	34	35
Сегмент #3	36	37	38	39	40	41	42	43

! N обращ. ! Числа для ! N вытесн. !
! ! записи ! сегмента !

!	1	!	8	!	0	!
!	2	!	9	!	0	!
!	3	!	10	!	0	!

Вариант 6

Содержимое буферной памяти :						
Сектор 0				Сектор 1		
Блок	Изм.	Прис.	Данные	Изм.	Прис.	Данные
0	0	0	2	0	1	24
1	0	0	2	0	1	25
2	0	0	2	0	1	26
3	0	0	2	0	1	27
4	0	1	20	0	1	28
5	0	1	21	0	1	29
6	0	1	22	0	1	30
7	0	1	23	0	1	31

Содержимое РНПС:

!	сектор	!	сегмент	!
!	0	!	1	!
!	1	!	2	!

Содержимое оперативной памяти:

Блок #	0	1	2	3	4	5	6	7
Сегмент #0	8	9	10	11	12	13	14	15
Сегмент #1	16	17	18	19	20	21	22	23
Сегмент #2	24	25	26	27	28	29	30	31
Сегмент #3	32	33	34	35	36	37	38	39

! N обращ. ! Числа для ! N вытесн. !



!		!	записи	!	сегмента	!
!	1	!	12	!	0	!
!	2	!	13	!	1	!
!	3	!	14	!	0	!

Вариант 7

Содержимое буферной памяти :

Сектор 0				Сектор 1		
Блок	Изм.	Прис.	Данные	Изм.	Прис.	Данные
0	0	0	3	0	1	26
1	0	0	3	0	1	27
2	0	0	3	0	1	28
3	0	0	3	0	1	29
4	0	1	22	0	1	30
5	0	1	23	0	1	31
6	0	1	24	0	1	32
7	0	1	25	0	1	33

Содержимое РНПС:

!	сектор	!	сегмент	!
!	0	!	1	!
!	1	!	2	!

Содержимое оперативной памяти:

Блок #	0	1	2	3	4	5	6	7
Сегмент #0	10	11	12	13	14	15	16	17
Сегмент #1	18	19	20	21	22	23	24	25
Сегмент #2	26	27	28	29	30	31	32	33
Сегмент #3	34	35	36	37	38	39	40	41

!	N обращ.	!	Числа для	!	N вытесн.	!
!		!	записи	!	сегмента	!
!	1	!	12	!	0	!
!	2	!	13	!	1	!
!	3	!	14	!	0	!

Вариант 8

Содержимое буферной памяти :

Сектор 0				Сектор 1		
Блок	Изм.	Прис.	Данные	Изм.	Прис.	Данные
0	0	0	4	0	1	28
1	0	0	4	0	1	29
2	0	0	4	0	1	30
3	0	0	4	0	1	31
4	0	1	24	0	1	32
5	0	1	25	0	1	33
6	0	1	26	0	1	34
7	0	1	27	0	1	35

Содержимое РНПС:

!	сектор	!	сегмент	!
!	0	!	1	!
!	1	!	2	!

Содержимое оперативной памяти:

Блок #	0	1	2	3	4	5	6	7
Сегмент #0	12	13	14	15	16	17	18	19
Сегмент #1	20	21	22	23	24	25	26	27
Сегмент #2	28	29	30	31	32	33	34	35
Сегмент #3	36	37	38	39	40	41	42	43



! N обращ.	! Числа для	! N вытесн.
! ! записи	! сегмента	!

! 1 ! 12 ! 0 !		
! 2 ! 13 ! 1 !		
! 3 ! 14 ! 0 !		

Вариант 9

Содержимое буферной памяти :

Сектор 0				Сектор 1			
Блок	Изм.	Прис.	Данные	Изм.	Прис.	Данные	
0	0	0	5	0	1	30	
1	0	0	5	0	1	31	
2	0	0	5	0	1	32	
3	0	0	5	0	1	33	
4	0	1	26	0	1	34	
5	0	1	27	0	1	35	
6	0	1	28	0	1	36	
7	0	1	29	0	1	37	

Содержимое РНПС:

! сектор	! сегмент	!

! 0 ! 1 !		
! 1 ! 2 !		

Содержимое оперативной памяти:

Блок #	0	1	2	3	4	5	6	7
Сегмент #0	14	15	16	17	18	19	20	21
Сегмент #1	22	23	24	25	26	27	28	29
Сегмент #2	30	31	32	33	34	35	36	37
Сегмент #3	38	39	40	41	42	43	44	45

! N обращ.	! Числа для	! N вытесн.
! ! записи	! сегмента	!

! 1 ! 12 ! 0 !		
! 2 ! 13 ! 1 !		
! 3 ! 14 ! 0 !		

Вариант 10

Содержимое буферной памяти :

Сектор 0				Сектор 1			
Блок	Изм.	Прис.	Данные	Изм.	Прис.	Данные	
0	0	0	6	0	1	32	
1	0	0	6	0	1	33	
2	0	0	6	0	1	34	
3	0	0	6	0	1	35	
4	0	1	28	0	1	36	
5	0	1	29	0	1	37	
6	0	1	30	0	1	38	
7	0	1	31	0	1	39	

Содержимое РНПС:

! сектор	! сегмент	!

! 0 ! 1 !		
! 1 ! 2 !		

Содержимое оперативной памяти:

Блок #	0	1	2	3	4	5	6	7
Сегмент #0	16	17	18	19	20	21	22	23
Сегмент #1	24	25	26	27	28	29	30	31
Сегмент #2	32	33	34	35	36	37	38	39
Сегмент #3	40	41	42	43	44	45	46	47



! N обращ.	! Числа для	! N вытесн.	!
! !	! записи	! сегмента	!
! 1	! 12	! 0	!
! 2	! 13	! 1	!
! 3	! 14	! 0	!

Вариант 11

Содержимое буферной памяти :

Сектор 0				Сектор 1			
Блок	Изм.	Прис.	Данные	Изм.	Прис.	Данные	
0	0	1	27	0	1	35	
1	0	1	28	0	1	36	
2	0	1	29	0	1	37	
3	0	1	30	0	1	38	
4	0	0	4	0	1	39	
5	0	0	4	0	1	40	
6	0	0	4	0	1	41	
7	0	0	4	0	1	42	

Содержимое РНПС:

! сектор	!	сегмент	!
! 0	!	2	!
! 1	!	3	!

Содержимое оперативной памяти:

Блок #	0	1	2	3	4	5	6	7
Сегмент #0	11	12	13	14	15	16	17	18
Сегмент #1	19	20	21	22	23	24	25	26
Сегмент #2	27	28	29	30	31	32	33	34
Сегмент #3	35	36	37	38	39	40	41	42

! N обращ.	! Числа для	! N вытесн.	!
! !	! записи	! сегмента	!
! 1	! 23	! 0	!
! 2	! 24	! 2	!
! 3	! 25	! 0	!

Вариант 12

Содержимое буферной памяти :

Сектор 0				Сектор 1			
Блок	Изм.	Прис.	Данные	Изм.	Прис.	Данные	
0	0	1	28	0	1	36	
1	0	1	29	0	1	37	
2	0	1	30	0	1	38	
3	0	1	31	0	1	39	
4	0	0	5	0	1	40	
5	0	0	5	0	1	41	
6	0	0	5	0	1	42	
7	0	0	5	0	1	43	

Содержимое РНПС:

! сектор	!	сегмент	!
! 0	!	2	!
! 1	!	3	!

Содержимое оперативной памяти:

Блок #	0	1	2	3	4	5	6	7
Сегмент #0	12	13	14	15	16	17	18	19
Сегмент #1	20	21	22	23	24	25	26	27
Сегмент #2	28	29	30	31	32	33	34	35



Сектор #3	36	37	38	39	40	41	42	43
! N обращ.	!	Числа для	!	N вытесн.	!			
!		! записи	!	! сегмента	!			
!	1	!	23	!	0	!		
!	2	!	24	!	2	!		
!	3	!	25	!	0	!		

Вариант 13

Содержимое буферной памяти :

Сектор 0				Сектор 1			
Блок	Изм.	Прис.	Данные	Изм.	Прис.	Данные	
0	0	1	28	0	1	36	
1	0	1	29	0	1	37	
2	0	1	30	0	1	38	
3	0	1	31	0	1	39	
4	0	0	6	0	1	40	
5	0	0	6	0	1	41	
6	0	0	6	0	1	42	
7	0	0	6	0	1	43	

Содержимое РНПС:

!	сектор	!	сегмент	!
!	0	!	2	!
!	1	!	3	!

Содержимое оперативной памяти:

Блок #	0	1	2	3	4	5	6	7
Сектор #0	12	13	14	15	16	17	18	19
Сектор #1	20	21	22	23	24	25	26	27
Сектор #2	28	29	30	31	32	33	34	35
Сектор #3	36	37	38	39	40	41	42	43

! N обращ.	!	Числа для	!	N вытесн.	!
!		! записи	!	! сегмента	!

!	1	!	23	!	0
!	2	!	24	!	2
!	3	!	25	!	0

Вариант 14

Содержимое буферной памяти :

Сектор 0				Сектор 1			
Блок	Изм.	Прис.	Данные	Изм.	Прис.	Данные	
0	0	1	29	0	1	37	
1	0	1	30	0	1	38	
2	0	1	31	0	1	39	
3	0	1	32	0	1	40	
4	0	0	7	0	1	41	
5	0	0	7	0	1	42	
6	0	0	7	0	1	43	
7	0	0	7	0	1	44	

Содержимое РНПС:

!	сектор	!	сегмент	!
!	0	!	2	!
!	1	!	3	!

Содержимое оперативной памяти:

Блок #	0	1	2	3	4	5	6	7
Сектор #0	13	14	15	16	17	18	19	20
Сектор #1	21	22	23	24	25	26	27	28



Сегмент #2	29	30	31	32	33	34	35	36
Сегмент #3	37	38	39	40	41	42	43	44

! N обращ.	! Числа для	! N вытесн.	!
!	! записи	! сегмента	!

!	1	!	23	!	0	!
!	2	!	24	!	2	!
!	3	!	25	!	0	!

Вариант 15

Содержимое буферной памяти :

Сектор 0				Сектор 1			
Блок	Изм.	Прис.	Данные	Изм.	Прис.	Данные	
0	0	1	29	0	1	37	
1	0	1	30	0	1	38	
2	0	1	31	0	1	39	
3	0	1	32	0	1	40	
4	0	0	8	0	1	41	
5	0	0	8	0	1	42	
6	0	0	8	0	1	43	
7	0	0	8	0	1	44	

Содержимое РНПС:

!	сектор	!	сегмент	!
---	--------	---	---------	---

!	0	!	2	!
!	1	!	3	!

Содержимое оперативной памяти:

Блок #	0	1	2	3	4	5	6	7
Сегмент #0	13	14	15	16	17	18	19	20
Сегмент #1	21	22	23	24	25	26	27	28
Сегмент #2	29	30	31	32	33	34	35	36
Сегмент #3	37	38	39	40	41	42	43	44

! N обращ.	! Числа для	! N вытесн.	!
!	! записи	! сегмента	!

!	1	!	23	!	0	!
!	2	!	24	!	2	!
!	3	!	25	!	0	!



Лабораторная работа № 9. Моделирование группно-ассоциативной буферной памяти

Содержание

Цель работы _____	148
Функционирование группно-ассоциативного буфера _____	148
Выполнение работы _____	149
Описание подпрограмм _____	150
Пример выполнения программы _____	159
Содержание отчета _____	159
Задание к лабораторной работе _____	161



Цель работы

Изучение функционирования группно-ассоциативного буфера со сквозной записью.

Функционирование группно-ассоциативного буфера

Для организации функционирования группно-ассоциативного буфера также, как и в случае секторного буфера, все поле памяти разбивается на блоки равной величины, содержащие по L байтов. Блоки объединяются в зоны по N блоков, которые называются сегментами и секторами, соответственно, для ОП и БП. В группно-ассоциативном буфере j -й блок ($j=0..N-1$), i -го сегмента ОП ($i=0..C-1$, где C -число сегментов), может быть помещен на место j -го блока k -го сектора БП ($k=0..S-1$, где S -число секторов), однако в одном секторе буфера могут находиться блоки различных сегментов ОП. Поэтому в группно-ассоциативном буфере каждому блоку придается адресная часть, в которой указывается, какому сегменту ОП принадлежит данный блок. Кроме того, при использовании признакового обмена каждый блок в БП содержит дополнительно бит изменения. Бит изменения блока равен 1, если содержимое блока изменялось, в противном случае бит изменения равен 0.

Номер сегмента, номер блока и номер байта внутри блока определяются по исполнительному адресу так же, как для секторного буфера.

Рассмотрим функционирование группно-ассоциативного буфера. При обращении к памяти по исполнительному адресу определяется номер блока. Затем в БП просматриваются адресные части блоков с выделенным номером и анализируются номера сегментов. В том случае, когда номер сегмента, заданный исполнительным адресом, совпадает с номером сегмента, заданным адресной частью блока в БП, то это означает, что адресуемый блок хранится в БП. При наличии адресуемого блока в БП в зависимости от кода операции происходит чтение информации из БП в ЦП или запись информации в БП из ЦП по адресу, который определяется младшими $\lfloor \log_2(L) \rfloor$ разрядами исполнительного адреса.

В том случае, когда адресуемый блок отсутствует в БП, необходимо вытеснить один из блоков с таким же номером, что и адресуемый блок, но принадлежащий другому сегменту. Вытесняемый блок определяется на основании принятого алгоритма вытеснения, аналогичного алгоритму вытеснения сегмента для секторного буфера. При вытеснении блока для группно-ассоциативного буфера так же, как и при вытеснении сегмента для секторного буфера, могут использоваться различные дисциплины.

Рассмотрим дисциплину признакового обмена. Для блока, подлежащего вытеснению, проверяется бит изменения. В том случае, если бит изменения равен 1, т.е. это означает, что содержимое блока было изменено, блок переписывается в ОП, а бит изменения сбрасывается в 0.

После этого на место вытесняемого блока из ОП записывается адресуемый блок, а в адресную часть блока заносится номер адресуемого сегмента. В том случае, если для вытесняемого блока бит изменения был равен 0, то на место этого блока сразу из ОП перезаписывается адресуемый блок, а в адресную часть заносится номер адресуемого сегмента. Дальнейшая работа с буфером осуществляется по схеме, рассмотренной выше.

Так же, как и в секторном буфере, в группно-ассоциативном буфере, кроме признакового обмена, могут использоваться другие обменные дисциплины. При использовании сквозной записи данные в БП и ОП записываются параллельно. В связи с этим в группно-ассоциативном буфере не используются биты изменения. При сквозной записи в случае отсутствия адресуемого блока на место вытесняемого блока сразу же записывается адресуемый блок, а в адресную часть блока заносится номер адресуемого сегмента.

Ассоциативный буфер является частным случаем группно-ассоциативного буфера, в котором j -й блок ($j=0..N-1$), i -го сегмента ОП ($i=0..C-1$), может быть помещен на место n -го блока ($n=0..N-1$), k -го сектора БП ($k=0..S-1$). Таким образом, если разбить весь объем буфера V на N групп $S=V/N$, то в группно-ассоциативном буфере j -й блок ОП может занять одно из S мест в j -й группе БП. При $N=1$ получаем ассоциативный буфер. Графическое изображение окна



программы при выборе лабораторной работы «Группо-ассоциативный буфер (кэш-память)» приведено на рис. 51.

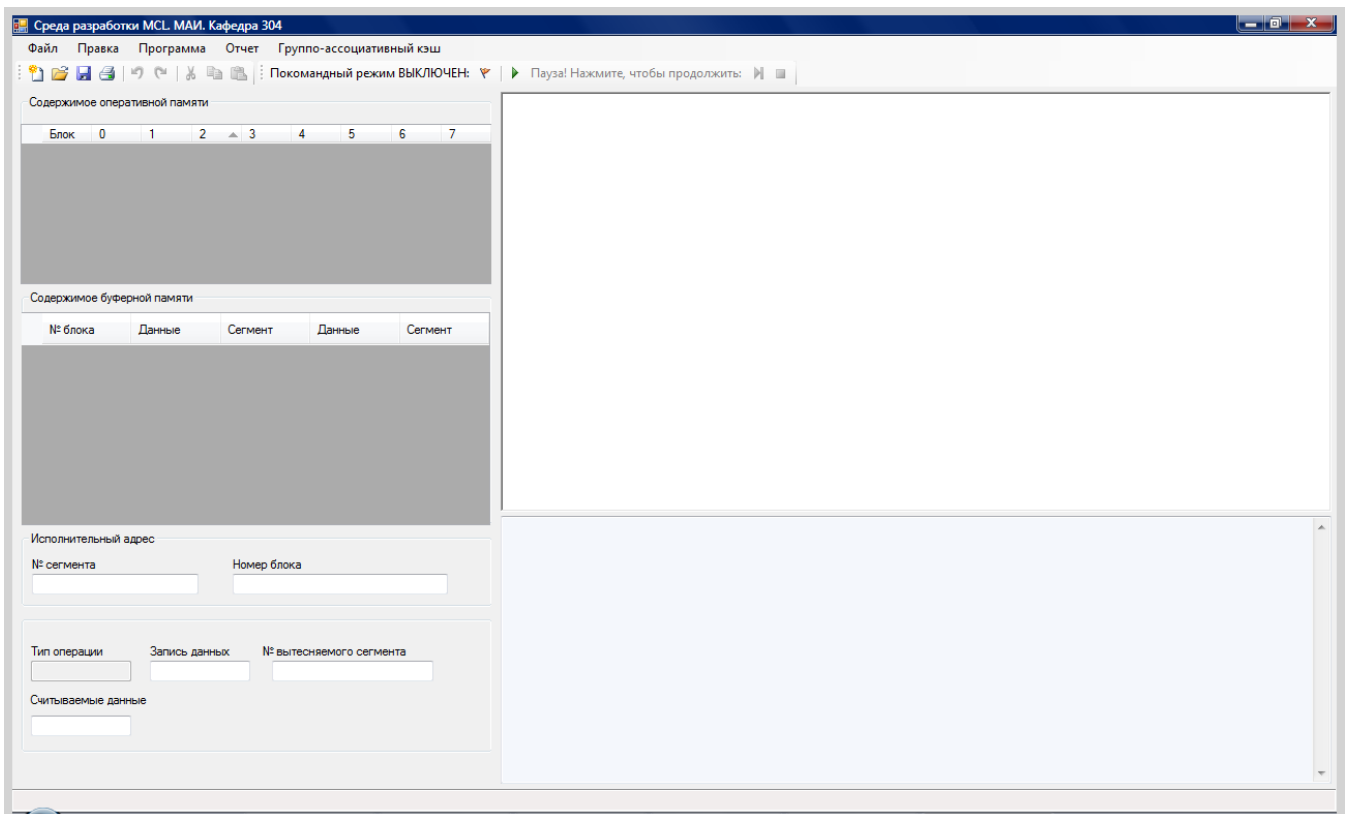


Рис. 53. Окно программы при выборе лабораторной работы «Группо-ассоциативный буфер (кэш-память)»

Выполнение работы

Работы содержит две части: буферная память с признаковым обменом и со сквозной записью. Первая часть представлена в виде примера и содержит код готовой программы. Ознакомившись с ее работой, можно приступить ко второй части, где **необходимо изменить исходный код и показать работу буферной памяти со сквозной записью, при этом программа запускается при выбранном пункте «Сквозная запись», пункт «Признаковый обмен» служит в качестве примера в обучающих целях.**

Работа буфера моделируется на уровне блоков, т.е. адрес ячейки в блоке не используется. Требуется написать программу моделирования буферной памяти со сквозной записью, моделирующую три обращения к памяти. В качестве примера в данном описании приводится текст программы моделирования буферной памяти с признаковым обменом. Соответствующая MCL-программа находится в рабочей директории и имеет имя Группо-ассоциативный КЭШ с признаковым обменом.MCL.

В программе моделирования буферной памяти с признаковым обменом описаны следующие типы (см. программу).

Типы, используемые подпрограммами

Внимание! Имена переменных должны быть строго соблюдены, иначе программа может выполняться некорректно.

mem op[32][8] – оперативная память 32 слова по 8 бит



mem bp[16][11] - буферная память 16 слов по 11 бит, при этом 8 бит - данные, 1 бит изменения и 2 бита - адресная часть указывающая какому сегменту принадлежит данный блок. Для работы со словом БП необходимо считать его в структуру BUF_REG[11]
mem ispadr[3][5] - память исполнительных адресов, содержит три исполнительных адреса
mem inputs[3][8] - числа для записи, содержит 3 числа записываемых в память
mem corer[3][1] - память кодов операции, содержит 3 кода операции (1-чтение, 0-запись)
mem rep_seg[3][2] - вытесняемый сегмент, содержит номера сегментов, вытесняемых из БП при замещении

struct iadr[5] - адрес ячейки ОП 32 слова
blok[3], - номер блока, всего 8 блоков
seg [2]; - номер сегмента, всего 4 сегмента

В данную структуру считывается исполнительный адрес из памяти, что позволяет различать в нем номера блока и сегмента.

struct bpadr[4] - адрес ячейки БП 16 слов
blok[3], - номер блока, всего 8 блоков
sect[1]; - номер сектора, всего 2 сектора

Данной структуре присваивается адрес буферной памяти, что позволяет различать в нем номера блока и сегмента.

struct buf_reg[11] - структура слова буферной памяти
data[8], - данные
ch_bit[1], - бит изменения
address[2]; - адресная часть

В эту структуру считывается слово БП при любой обработке, что позволяет различать в нем биты присутствия, изменения и собственно данные.

var i,j, - вспомогательная переменная
choice, - номер варианта

Переменные используемые в программе моделирования работы БП:

k,l, - вспомогательная переменная
s_no; - номер сектора, содержащего необходимый сегмент

Следующие переменные используются для упрощения. В них считываются соответственно значения corer, inputs, и rep_seg для данного обращения к памяти:

op_code, - код операции
data, - данные
replace, - номер замещаемого сегмента

Описание подпрограмм

1. VARIANT - подпрограмма задания исходных данных по варианту, и их распечатка. При исполнении подпрограммы на экран выводится сообщение:



Введите номер варианта:– после чего необходимо ввести номер варианта от 1 до 15 включительно. При этом подпрограммой задаются начальные значения следующих описанных выше типов: `op`, `bp`, `ispadr`, `inputs`, `coper`, `rep_seg`.

Исходные данные заданы таким образом, что элементы массивов `ispadr`, `inputs`, `coper` и `rep_seg` с одинаковым индексом относятся к одному обращению к памяти. То есть если первое обращение к памяти - чтение, а второе запись, то `inputs[1]` не используется вообще, а во втором обращении используются сразу `inputs[2]`.

2. FIRST, SECOND, THIRD - данные подпрограммы используются исключительно подпрограммой VARIANT.
3. PRINT_BP - подпрограмма печати содержимого буферной памяти.
4. PRINT_BR - подпрограмма печати содержимого буферного регистра, используется исключительно подпрограммой PRINT_BP.
5. PRINT_OP - подпрограмма печати содержимого оперативной памяти.
6. PRINT_DOP - подпрограмма печати чисел заданных для записи и номеров вытесняемых сегментов, используется подпрограммами FIRST, SECOND и THIRD.

Типы, описанные в данном файле, полностью идентичны типам описанным в программе моделирования БП с признаковым обменом. Отличием является отсутствие поля `CH_BIT[1]` - бит изменения, структуры `BUF_REG`. Соответственно иначе описана и память BP. Так же отсутствуют описания переменных используемых собственно в программе моделирования, которые вводятся по усмотрению студентов. Подпрограммы имеющиеся в файле, имеют те же имена и назначение, что и в описанной выше программе.

Блок-схема алгоритма функционирования группно-ассоциативного буфера имеет вид:



Рис.54. Блок-схема функционирования секторного буфера



Блок-схема алгоритма вытеснения для группно-ассоциативного буфера при признаковом обмене:

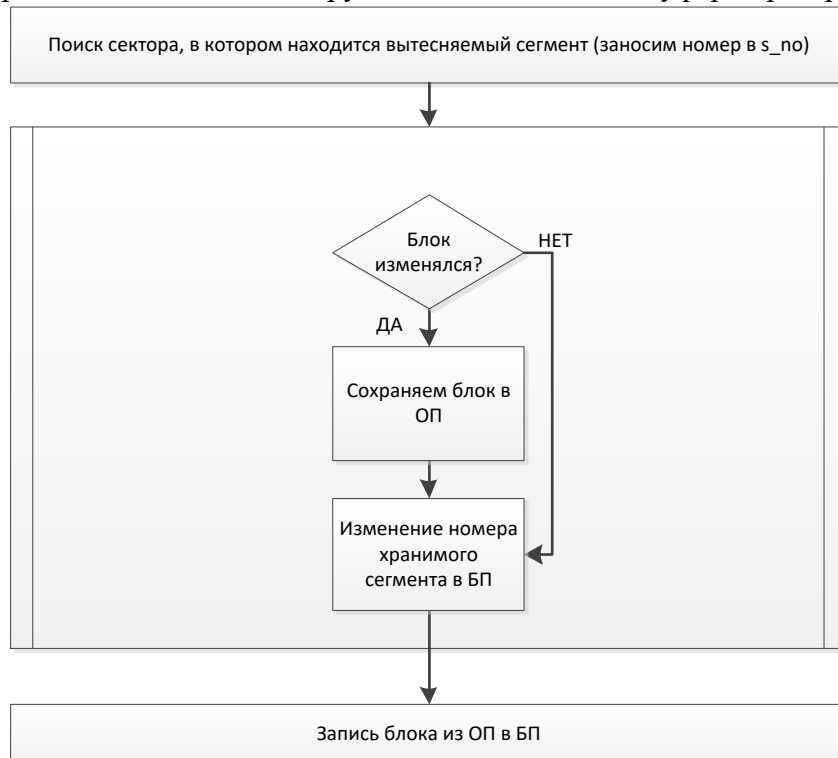


Рис.55. Блок-схема алгоритма вытеснения

Текст программы из файла Группно-ассоциативный КЭШ с признаковым обменом.MCL (признаковый обмен):

```
writeln " *****";
writeln " * Модель группно-ассоциативной КЭШ памяти с признаковым обменом *";
writeln " *****";

{ работа моделируется на уровне блоков }

{***** ОПИСАНИЕ ТИПОВ*****}

{ типы используемые п/п }

var i,j,k,l, { вспомогательные переменные }
    choice, { номер варианта }
    op_code, { код операции }
    data, { данные }
    replace, { номер замещаемого сегмента }
    s_no, { номер сектора, содержащего необходимый сегмент }
    read_data_flag;

mem op[32][8]; { оперативная память 32 слова по 8 бит }

mem bp[16][11]; { буферная память 16 слов по 11 бит}

mem ispadr[3][5]; { массив для хранения исполнительных адресов}

mem inputs[3][8]; { массив для хранения чисел, которые должны быть записаны в
БП}
```




```
mem coper[3][1];      { массив для хранения кодов операций 1-чтение, 0-запись }

mem rep_seg[3][2];    { массив для хранения номеров сегментов подлежащих вытеснению}

struct iadr[5]        { структура для разбиения исполнительного адреса на
старшую и младшую часть }
    blok[3],           { номер блока}
    seg [2];           { номер сегмента }

struct bpadr[4]        { структура для выделения старшей и младшей части в адресе
БП }
    blok[3],           { номер блока }
    sect[1];           { номер сектора }

struct buf_reg[11]    { структура для выделения данных, бита изменения и бита
присутствия}
    data[8],           { данные }
    ch_bit[1],         { бит изменения }
    address[2];        { номер сегмента}

struct subiadr[5]     { переменная используется в подпрограммах }
    blok[3],
    seg [2];

struct subbpadr[4]    { переменная используется в подпрограммах }
    blok[3],
    sect[1];

struct subbuf_reg[11] { переменная используется в подпрограммах }
    data[8],
    ch_bit[1],
    address[2];

{***** начало программы *****)

gosub variant;        { задание начальных значений всех переменных }
for l:=0 to 2;         { цикл для работы с 3-мя исполнительными адресами }
    read_data_flag:=0;
    {получаем исполнительный адрес}
    iadr:=ispadr[l];
    {получаем код операции}
    op_code:=coper[l];
    {загружаем данные для записи}
    data:=inputs[l];
    writeln "Исполнительный адрес - ", $d2 iadr;
    pause;
    bpadr.blok:=iadr.blok; { выделение блоков с нужным номером, номер блока в БП =
номеру блока в исполнительном адресе}
    { Поиск наличия блока в БП }
    gosub poisk_bloka;
    replace:=rep_seg[l];
    if s_no=0 then goto get_blok; { Если блок нужного сегмента присутствует в БП,
то продолжаем работу с ним}
    if s_no=1 then goto get_blok;
    { Если блока нет, то переходим к вытеснению }
    write "Блок ", $d2 iadr.blok;
    writeln " ,сегмента ", $d2 iadr.seg, " отсутствует в БП";
    pause;
```



```
writeln "Вытеснение блока, принадлежащего сегменту",$d2 replace;
{получаем номер вытесняемого сегмента}

pause;
gosub vitesnenie;

get_blok:
writeln "Блок ", $d2 iadr.blok," сегмента ", $d2 iadr.seg, " присутствует в БП";
pause;
{ выполнение операции по коду }
if op_code=1 then gosub read_data; { если 1 то чтение}
if op_code=0 then gosub write_data; { если 0 то запись}
pause;
goto end_job;

end_job:
gosub print_bp; { печать содержимого БП }
gosub print_op; { печать содержимого ОП }
pause;
next;
end;

{***** конец программы *****)}

{***** подпрограммы *****)}

{ п/п для задания варианта, использует п/п first,second,third }

variant:
pause;
s_no:=0;
{ задание исходных данных по вариантам}

read " Введите номер варианта: ",choice;

if choice<=5 then gosub first;
if ((choice>5) and (choice<=10)) then gosub second;
if choice>10 then gosub third;

return; { variant }

{ п/п первый вариант }

first:

{ присваивание начальных значений ячейкам ОП }

for i:=0 to 31;
    op[i]:=(choice+2)*2+i-2;
next;

{ присваивание начальных значений ячейкам БП }

for i:=0 to 7;
    buf_reg.data:=(choice+2)*2+i-2;
    buf_reg.ch_bit:=0;
    buf_reg.address:=0;
    bp[i]:=buf_reg;
next;
```



```
for i:=8 to 15;
  buf_reg.data:=(choice+2)*2+i-2;
  buf_reg.ch_bit:=0;
  buf_reg.address:=1;
  bp[i]:=buf_reg;
next;

{ назначение исполнительных адресов }

ispadr[0]:=4;
ispadr[1]:=28;
ispadr[2]:=22;

{ задание чисел для записи }

for i:=0 to 2;
  inputs[i]:=i+8;
next;

{ задание КОП }

coper[0]:=0;
coper[1]:=1;
coper[2]:=0;

{ задание номеров замещаемых сегментов }

rep_seg[0]:=0;
rep_seg[1]:=0;
rep_seg[2]:=1;

gosub print_bp; { печать содержимого БП}
gosub print_op; { печать содержимого ОП}
gosub print; { печать записываемых чисел и замещаемых сегментов }

return; { first }

{ п/п второй вариант }

second:

{ присваивание начальных значений ячейкам ОП }

for i:=0 to 31;
  op[i]:=(choice+2)*2+i-8;
next;

{ присваивание начальных значений ячейкам БП }

for i:=0 to 7;
  buf_reg.data:=(choice+2)*2+i;
  buf_reg.ch_bit:=0;
  buf_reg.address:=1;
  bp[i]:=buf_reg;
next;

for i:=8 to 15;
  buf_reg.data:=(choice+2)*2+i;
  buf_reg.ch_bit:=0;
  buf_reg.address:=2;
  bp[i]:=buf_reg;
```



```
next;

{ назначение исполнительных адресов }
ispadr[0]:=10;
ispadr[1]:=30;
ispadr[2]:=6;

{ задание чисел для записи }

for i:=0 to 2;
    inputs[i]:=i+12;
next;

{ задание КОП }

coper[0]:=1;
coper[1]:=0;
coper[2]:=1;

{ задание номеров замещаемых сегментов }

rep_seg[0]:=1;
rep_seg[1]:=2;
rep_seg[2]:=3;

gosub print_bp; { печать содержимого БП }
gosub print_op; { печать содержимого ОП }
gosub print; { печать записываемых чисел и замещаемых сегментов }

return; { second }

{ п/п третий вариант }

third:

{ присваивание начальных значений ячейкам ОП }

for i:=0 to 31;
    op[i]:=(choice+2)/2+i+5;
next;

{ присваивание начальных значений ячейкам БП }

for i:=0 to 7;
    buf_reg.data:=(choice+2)/2+i+21;
    buf_reg.ch_bit:=0;
    buf_reg.address:=2;
    bp[i]:=buf_reg;
next;

for i:=8 to 15;
    buf_reg.data:=(choice+2)/2+i+21;
    buf_reg.ch_bit:=0;
    buf_reg.address:=3;
    bp[i]:=buf_reg;
next;

{ назначение исполнительных адресов }

ispadr[0]:=8;
ispadr[1]:=26;
```



```
ispadr[2]:=10;

{ задание чисел для записи }

for i:=0 to 2;
    inputs[i]:=i+23;
next;

{ задание КОП }

coper[0]:=0;
coper[1]:=0;
coper[2]:=1;

{ задание номеров замещаемых сегментов }

rep_seg[0]:=2;
rep_seg[1]:=2;
rep_seg[2]:=3;

gosub print_bp; { печать содержимого БП }
gosub print_op; { печать содержимого ОП }
gosub print; { печать записываемых чисел и замещаемых сегментов }

return; { third }

{ п/п печати содержимого БП }

print_bp:
    writeln "Содержимое буферной памяти :";
    writeln "        Сектор 0                Сектор 1";
    writeln "Блок  Данные                Сег. Изм.  Данные                Сег. Изм.";
    for i:=0 to 7;
        write " ", $d2 i;
        subbpadr.sect:=0;
        subbpadr.blok:=i;
        subbuf_reg:=bp[subbpadr];
        gosub print_br;
        subbpadr.sect:=1;
        subbuf_reg:=bp[subbpadr];
        gosub print_br;
        writeln;
    next;
return; { print_bp }

{ п/п печати буферного регистра }

print_br:
    write "        ", $d4 subbuf_reg.data;
    write "        ", $d4 subbuf_reg.address;
    write "        ", $b1 subbuf_reg.ch_bit;
return; { print_br }

{ п/п печати содержимого ОП }

print_op:
    writeln "Содержимое оперативной памяти:";
    writeln "        Блок #        0    1    2    3    4    5    6    7";
    for i:=0 to 3;
        write " Сегмент #", $d1 i, " ";
        subiadr.seg:=i;
```



```
        for j:=0 to 7;
            subiadr.blok:=j;
            write $d4 op[subiadr];
        next;
        writeln;
    next;
return; { print_op }

{ п/п печати записываемых чисел и замещаемых сегментов }

print:
writeln "-----";
writeln "! № обращ. ! Числа для ! № вытесн. !";
writeln "!           ! записи      ! сегмента  !";
writeln "-----";
writeln "!      1      !", $d7 inputs[0], "      !", $d6 rep_seg[0], "      !";
writeln "!      2      !", $d7 inputs[1], "      !", $d6 rep_seg[1], "      !";
writeln "!      3      !", $d7 inputs[2], "      !", $d6 rep_seg[2], "      !";
writeln "-----";
return; { print }

poisk_bloka:
{задаем случайное значение номера сектора не равное 0 или 1}
s_no:=2;
bpadr.sect:=0;
buf_reg:=bp[bpadr];
if buf_reg.address=iadr.seg then s_no:=0;
bpadr.sect:=1;
buf_reg:=bp[bpadr];
if buf_reg.address=iadr.seg then s_no:=1;
return; { poisk_bloka }

vitesnenie:
{ Поиск вытесняемого блока}
bpadr.sect:=0;
buf_reg:=bp[bpadr];
if buf_reg.address=replace then s_no:=0;
bpadr.sect:=1;
buf_reg:=bp[bpadr];
if buf_reg.address=replace then s_no:=1;
{ Вытеснение сегмента }
bpadr.sect:=s_no; { нахождение сектора}
buf_reg:=bp[bpadr]; { считывание в буферный регистр}
if buf_reg.ch_bit=0 then goto dont_save; { если блок не изменялся}
{ если блок изменялся, то сохраняем его в ОП }
k:=iadr;
iadr.seg:=buf_reg.address;
op[iadr]:=buf_reg.data;
write "Сохранение блока ", $d2 iadr.blok;
writeln " сегмента ", $d2 iadr.seg, " в ОП ";
iadr:=k;
dont_save:
buf_reg.ch_bit:=0; { обнуление бита изменения }
buf_reg.address:=iadr.seg; { занесение номера сегмента в адресную часть}
buf_reg.data:=op[iadr]; { запись адресуемого блока из ОП}
bp[bpadr]:=buf_reg;
return; { vitesnenie }

write_data:
bpadr.sect:=s_no; { нахождение адреса }
buf_reg:=bp[bpadr]; { чтение блока из БП в буферный регистр }
```



```
buf_reg.data:=data;
buf_reg.ch_bit:=1;
bp[bpadr]:=buf_reg; { запись из регистра в буферную память }
writeln "Записаны данные ", $d4 data;
return; {write data}

read_data:
bpadr.sect:=s_no; { нахождение адреса }
buf_reg:=bp[bpadr]; { чтение блока из БП в буферный регистр }
data:=buf_reg.data;
writeln "Прочитаны данные ", $d4 data;
read_data_flag:=1;
return; {read_data}
```

Пример выполнения программы

Данный пример есть выполнение программы с признаковым обменом (файл Группо-ассоциативный КЭШ с признаковым обменом.MCL) для 1-го варианта

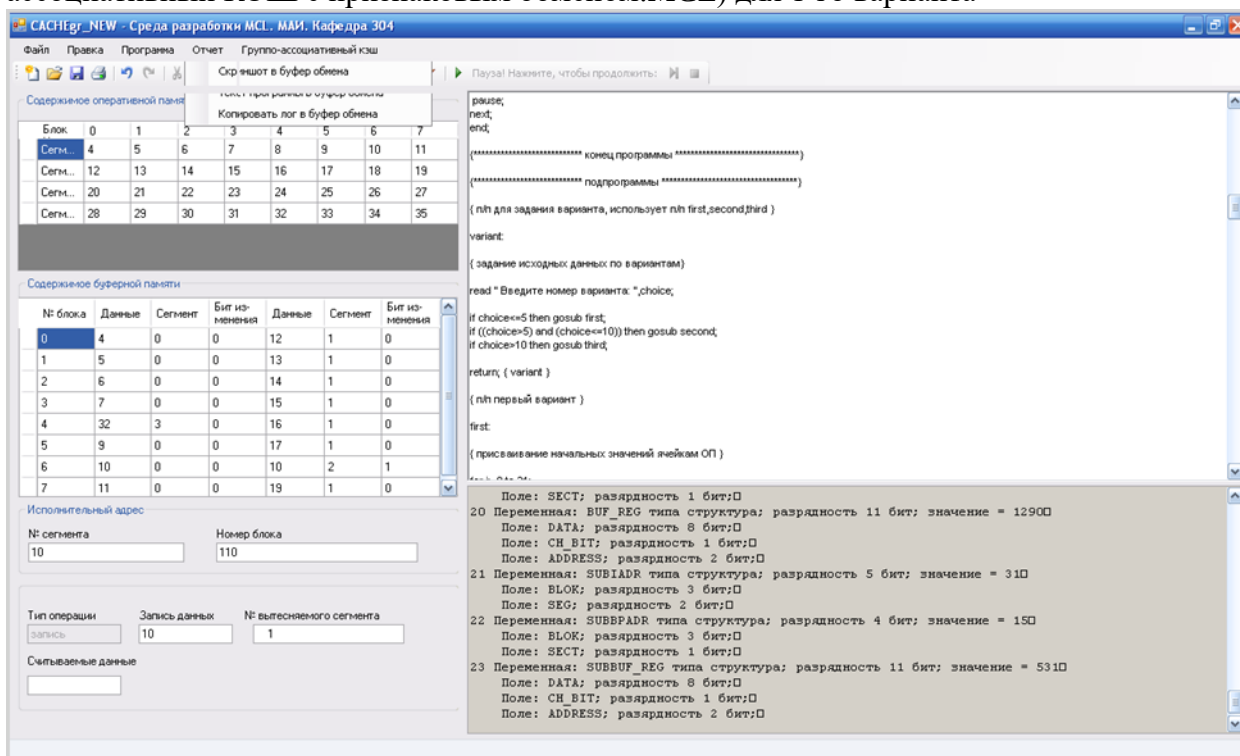


Рис. 52. Результат моделирования группо-ассоциативного буфера с признаковым обменом

* Модель группо-ассоциативной КЭШ памяти с признаковым обменом *

Введите номер варианта:

1

Содержимое буферной памяти :

Сектор 0				Сектор 1		
Блок	Сег.	Изм.	Данные	Сег.	Изм.	Данные
0	0	0	4	1	0	12
1	0	0	5	1	0	13
2	0	0	6	1	0	14
3	0	0	7	1	0	15
4	0	0	8	1	0	16



5	0	0	9	1	0	17
6	0	0	10	1	0	18
7	0	0	11	1	0	19

Содержимое оперативной памяти:

Блок #	0	1	2	3	4	5	6	7
Сегмент #0	4	5	6	7	8	9	10	11
Сегмент #1	12	13	14	15	16	17	18	19
Сегмент #2	20	21	22	23	24	25	26	27
Сегмент #3	28	29	30	31	32	33	34	35

! № обращ. ! Числа для ! № вытесн. !
! ! записи ! сегмента !

! 1 ! 8 ! 0 !
! 2 ! 9 ! 0 !
! 3 ! 10 ! 1 !

Исполнительный адрес - 4

Блок 4 сегмента 0 присутствует в БП

Записаны данные 8

Содержимое буферной памяти :

Сектор 0				Сектор 1		
Блок	Сег.	Изм.	Данные	Сег.	Изм.	Данные
0	0	0	4	1	0	12
1	0	0	5	1	0	13
2	0	0	6	1	0	14
3	0	0	7	1	0	15
4	0	1	8	1	0	16
5	0	0	9	1	0	17
6	0	0	10	1	0	18
7	0	0	11	1	0	19

Содержимое оперативной памяти:

Блок #	0	1	2	3	4	5	6	7
Сегмент #0	4	5	6	7	8	9	10	11
Сегмент #1	12	13	14	15	16	17	18	19
Сегмент #2	20	21	22	23	24	25	26	27
Сегмент #3	28	29	30	31	32	33	34	35

Исполнительный адрес - 28

Блок 4 ,сегмента 3 отсутствует в БП

Вытеснение блока, принадлежащего сегменту 0

Сохранение блока 4 сегмента 0 в ОП

Блок 4 сегмента 3 присутствует в БП

Прочитаны данные 32

Содержимое буферной памяти :

Сектор 0				Сектор 1		
Блок	Сег.	Изм.	Данные	Сег.	Изм.	Данные
0	0	0	4	1	0	12
1	0	0	5	1	0	13
2	0	0	6	1	0	14
3	0	0	7	1	0	15
4	1	0	32	1	0	16



5	0	0	9	1	0	17
6	0	0	10	1	0	18
7	0	0	11	1	0	19

Содержимое оперативной памяти:

Блок #	0	1	2	3	4	5	6	7
Сегмент #0	4	5	6	7	8	9	10	11
Сегмент #1	12	13	14	15	16	17	18	19
Сегмент #2	20	21	22	23	24	25	26	27
Сегмент #3	28	29	30	31	32	33	34	35

Исполнительный адрес – 22

Блок 6 ,сегмента 2 отсутствует в БП

Вытеснение блока, принадлежащего сегменту 1

Блок 6 сегмента 2 присутствует в БП

Записаны данные 10

Содержимое буферной памяти :

Сектор 0				Сектор 1		
Блок	Сег.	Изм.	Данные	Сег.	Изм.	Данные
0	0	0	4	1	0	12
1	0	0	5	1	0	13
2	0	0	6	1	0	14
3	0	0	7	1	0	15
4	1	0	32	1	0	16
5	0	0	9	1	0	17
6	0	0	10	0	1	10
7	0	0	11	1	0	19

Содержимое оперативной памяти:

Блок #	0	1	2	3	4	5	6	7
Сегмент #0	4	5	6	7	8	9	10	11
Сегмент #1	12	13	14	15	16	17	18	19
Сегмент #2	20	21	22	23	24	25	26	27
Сегмент #3	28	29	30	31	32	33	34	35

Содержание отчета

1. Формулировка задания.
2. Исходные данные.
3. Программа на языке MCL, моделирующая буферную память со сквозной записью.
4. Результаты выполнения программы в **виде распечатки** содержимого поля для вывода текстовой информации и **скриншота** (пример на рис. 52) в момент завершения выполнения программы.

Контрольные вопросы

1. Отличие секторной и группно-ассоциативной организации буферной памяти.
2. Запись и замещение блоков с признаковым обменом.
3. Запись и замещение блоков со сквозным обменом.
4. Запись блоков разных сегментов в один сектор.

Задание к лабораторной работе

Все возможные варианты разбиты на три группы в соответствии с чем задаются исходные данные:



Для вариантов 1-5 задаются следующие данные:

Номер обращения к памяти	Исполнительный адрес			Код операции	
	адрес	N сегмента	N блока		
1	4	0	4	запись	0
2	28	3	4	чтение	1
3	22	2	6	запись	0

Для вариантов 6-10 задаются следующие данные:

Номер обращения к памяти	Исполнительный адрес			Код операции	
	адрес	N сегмента	N блока		
1	10	1	2	чтение	1
2	30	3	6	запись	0
3	6	0	6	чтение	1

Для вариантов 11-15 задаются следующие данные:

Номер обращения к памяти	Исполнительный адрес			Код операции	
	адрес	N сегмента	N блока		
1	8	1	0	запись	0
2	26	3	2	запись	0
3	10	1	2	чтение	1

Содержимое ОП и БП различно для каждого варианта:



Вариант1

Содержимое буферной памяти :

Сектор 0				Сектор 1		
Блок	Сег.	Изм.	Данные	Сег.	Изм.	Данные
0	0	0	4	1	0	12
1	0	0	5	1	0	13
2	0	0	6	1	0	14
3	0	0	7	1	0	15
4	0	0	8	1	0	16
5	0	0	9	1	0	17
6	0	0	10	1	0	18
7	0	0	11	1	0	19

Содержимое оперативной памяти:

Блок #	0	1	2	3	4	5	6	7
Сегмент #0	4	5	6	7	8	9	10	11
Сегмент #1	12	13	14	15	16	17	18	19
Сегмент #2	20	21	22	23	24	25	26	27
Сегмент #3	28	29	30	31	32	33	34	35

! N обращ. ! Числа для ! N вытесн. !
! ! записи ! сегмента !

! 1 ! 8 ! 0 !
! 2 ! 9 ! 0 !
! 3 ! 10 ! 1 !

Вариант2

Содержимое буферной памяти :

Сектор 0				Сектор 1		
Блок	Сег.	Изм.	Данные	Сег.	Изм.	Данные
0	0	0	6	1	0	14
1	0	0	7	1	0	15
2	0	0	8	1	0	16
3	0	0	9	1	0	17
4	0	0	10	1	0	18
5	0	0	11	1	0	19
6	0	0	12	1	0	20
7	0	0	13	1	0	21

Содержимое оперативной памяти:

Блок #	0	1	2	3	4	5	6	7
Сегмент #0	6	7	8	9	10	11	12	13
Сегмент #1	14	15	16	17	18	19	20	21
Сегмент #2	22	23	24	25	26	27	28	29
Сегмент #3	30	31	32	33	34	35	36	37

! N обращ. ! Числа для ! N вытесн. !
! ! записи ! сегмента !

! 1 ! 8 ! 0 !
! 2 ! 9 ! 0 !
! 3 ! 10 ! 1 !

Вариант3



Содержимое буферной памяти :

Сектор 0				Сектор 1		
Блок	Сег.	Изм.	Данные	Сег.	Изм.	Данные
0	0	0	8	1	0	16
1	0	0	9	1	0	17
2	0	0	10	1	0	18
3	0	0	11	1	0	19
4	0	0	12	1	0	20
5	0	0	13	1	0	21
6	0	0	14	1	0	22
7	0	0	15	1	0	23

Содержимое оперативной памяти:

Блок #	0	1	2	3	4	5	6	7
Сегмент #0	8	9	10	11	12	13	14	15
Сегмент #1	16	17	18	19	20	21	22	23
Сегмент #2	24	25	26	27	28	29	30	31
Сегмент #3	32	33	34	35	36	37	38	39

! N обращ. ! Числа для ! N вытесн. !
! ! записи ! сегмента !

!	1	!	8	!	0	!
!	2	!	9	!	0	!
!	3	!	10	!	1	!

Вариант4

Содержимое буферной памяти :

Сектор 0				Сектор 1		
Блок	Сег.	Изм.	Данные	Сег.	Изм.	Данные
0	0	0	10	1	0	18
1	0	0	11	1	0	19
2	0	0	12	1	0	20
3	0	0	13	1	0	21
4	0	0	14	1	0	22
5	0	0	15	1	0	23
6	0	0	16	1	0	24
7	0	0	17	1	0	25

Содержимое оперативной памяти:

Блок #	0	1	2	3	4	5	6	7
Сегмент #0	10	11	12	13	14	15	16	17
Сегмент #1	18	19	20	21	22	23	24	25
Сегмент #2	26	27	28	29	30	31	32	33
Сегмент #3	34	35	36	37	38	39	40	41

! N обращ. ! Числа для ! N вытесн. !
! ! записи ! сегмента !

!	1	!	8	!	0	!
!	2	!	9	!	0	!
!	3	!	10	!	1	!

Вариант5



Содержимое буферной памяти :

Сектор 0				Сектор 1		
Блок	Сег.	Изм.	Данные	Сег.	Изм.	Данные
0	0	0	12	1	0	20
1	0	0	13	1	0	21
2	0	0	14	1	0	22
3	0	0	15	1	0	23
4	0	0	16	1	0	24
5	0	0	17	1	0	25
6	0	0	18	1	0	26
7	0	0	19	1	0	27

Содержимое оперативной памяти:

Блок #	0	1	2	3	4	5	6	7
Сегмент #0	12	13	14	15	16	17	18	19
Сегмент #1	20	21	22	23	24	25	26	27
Сегмент #2	28	29	30	31	32	33	34	35
Сегмент #3	36	37	38	39	40	41	42	43

! N обращ. ! Числа для ! N вытесн. !
! ! записи ! сегмента !

!	1	!	8	!	0	!
!	2	!	9	!	0	!
!	3	!	10	!	1	!

Вариант6

Содержимое буферной памяти :

Сектор 0				Сектор 1		
Блок	Сег.	Изм.	Данные	Сег.	Изм.	Данные
0	1	0	16	2	0	24
1	1	0	17	2	0	25
2	1	0	18	2	0	26
3	1	0	19	2	0	27
4	1	0	20	2	0	28
5	1	0	21	2	0	29
6	1	0	22	2	0	30
7	1	0	23	2	0	31

Содержимое оперативной памяти:

Блок #	0	1	2	3	4	5	6	7
Сегмент #0	8	9	10	11	12	13	14	15
Сегмент #1	16	17	18	19	20	21	22	23
Сегмент #2	24	25	26	27	28	29	30	31
Сегмент #3	32	33	34	35	36	37	38	39

! N обращ. ! Числа для ! N вытесн. !
! ! записи ! сегмента !

!	1	!	12	!	1	!
!	2	!	13	!	2	!
!	3	!	14	!	3	!

Вариант7



Содержимое буферной памяти :

Сектор 0				Сектор 1		
Блок	Сег.	Изм.	Данные	Сег.	Изм.	Данные
0	1	0	18	2	0	26
1	1	0	19	2	0	27
2	1	0	20	2	0	28
3	1	0	21	2	0	29
4	1	0	22	2	0	30
5	1	0	23	2	0	31
6	1	0	24	2	0	32
7	1	0	25	2	0	33

Содержимое оперативной памяти:

Блок #	0	1	2	3	4	5	6	7
Сегмент #0	10	11	12	13	14	15	16	17
Сегмент #1	18	19	20	21	22	23	24	25
Сегмент #2	26	27	28	29	30	31	32	33
Сегмент #3	34	35	36	37	38	39	40	41

! N обращ. ! Числа для ! N вытесн. !
! ! записи ! сегмента !

! 1 ! 12 ! 1 !
! 2 ! 13 ! 2 !
! 3 ! 14 ! 3 !

Вариант8

Содержимое буферной памяти :

Сектор 0				Сектор 1		
Блок	Сег.	Изм.	Данные	Сег.	Изм.	Данные
0	1	0	20	2	0	28
1	1	0	21	2	0	29
2	1	0	22	2	0	30
3	1	0	23	2	0	31
4	1	0	24	2	0	32
5	1	0	25	2	0	33
6	1	0	26	2	0	34
7	1	0	27	2	0	35

Содержимое оперативной памяти:

Блок #	0	1	2	3	4	5	6	7
Сегмент #0	12	13	14	15	16	17	18	19
Сегмент #1	20	21	22	23	24	25	26	27
Сегмент #2	28	29	30	31	32	33	34	35
Сегмент #3	36	37	38	39	40	41	42	43

! N обращ. ! Числа для ! N вытесн. !
! ! записи ! сегмента !

! 1 ! 12 ! 1 !
! 2 ! 13 ! 2 !
! 3 ! 14 ! 3 !

Вариант9



Содержимое буферной памяти :

Сектор 0				Сектор 1		
Блок	Сег.	Изм.	Данные	Сег.	Изм.	Данные
0	1	0	22	2	0	30
1	1	0	23	2	0	31
2	1	0	24	2	0	32
3	1	0	25	2	0	33
4	1	0	26	2	0	34
5	1	0	27	2	0	35
6	1	0	28	2	0	36
7	1	0	29	2	0	37

Содержимое оперативной памяти:

Блок #	0	1	2	3	4	5	6	7
Сегмент #0	14	15	16	17	18	19	20	21
Сегмент #1	22	23	24	25	26	27	28	29
Сегмент #2	30	31	32	33	34	35	36	37
Сегмент #3	38	39	40	41	42	43	44	45

! N обращ. ! Числа для ! N вытесн. !
! ! записи ! сегмента !

!	1	!	12	!	1	!
!	2	!	13	!	2	!
!	3	!	14	!	3	!

Вариант10

Содержимое буферной памяти :

Сектор 0				Сектор 1		
Блок	Сег.	Изм.	Данные	Сег.	Изм.	Данные
0	1	0	24	2	0	32
1	1	0	25	2	0	33
2	1	0	26	2	0	34
3	1	0	27	2	0	35
4	1	0	28	2	0	36
5	1	0	29	2	0	37
6	1	0	30	2	0	38
7	1	0	31	2	0	39

Содержимое оперативной памяти:

Блок #	0	1	2	3	4	5	6	7
Сегмент #0	16	17	18	19	20	21	22	23
Сегмент #1	24	25	26	27	28	29	30	31
Сегмент #2	32	33	34	35	36	37	38	39
Сегмент #3	40	41	42	43	44	45	46	47

! N обращ. ! Числа для ! N вытесн. !
! ! записи ! сегмента !

!	1	!	12	!	1	!
!	2	!	13	!	2	!
!	3	!	14	!	3	!

Вариант11



Содержимое буферной памяти :

Сектор 0				Сектор 1		
Блок	Сег.	Изм.	Данные	Сег.	Изм.	Данные
0	2	0	28	3	0	36
1	2	0	29	3	0	37
2	2	0	30	3	0	38
3	2	0	31	3	0	39
4	2	0	32	3	0	40
5	2	0	33	3	0	41
6	2	0	34	3	0	42
7	2	0	35	3	0	43

Содержимое оперативной памяти:

Блок #	0	1	2	3	4	5	6	7
Сегмент #0	12	13	14	15	16	17	18	19
Сегмент #1	20	21	22	23	24	25	26	27
Сегмент #2	28	29	30	31	32	33	34	35
Сегмент #3	36	37	38	39	40	41	42	43

! N обращ. ! Числа для ! N вытесн. !
! ! записи ! сегмента !

! 1 ! 23 ! 2 !
! 2 ! 24 ! 2 !
! 3 ! 25 ! 3 !

Вариант12

Содержимое буферной памяти :

Сектор 0				Сектор 1		
Блок	Сег.	Изм.	Данные	Сег.	Изм.	Данные
0	2	0	28	3	0	36
1	2	0	29	3	0	37
2	2	0	30	3	0	38
3	2	0	31	3	0	39
4	2	0	32	3	0	40
5	2	0	33	3	0	41
6	2	0	34	3	0	42
7	2	0	35	3	0	43

Содержимое оперативной памяти:

Блок #	0	1	2	3	4	5	6	7
Сегмент #0	12	13	14	15	16	17	18	19
Сегмент #1	20	21	22	23	24	25	26	27
Сегмент #2	28	29	30	31	32	33	34	35
Сегмент #3	36	37	38	39	40	41	42	43

! N обращ. ! Числа для ! N вытесн. !
! ! записи ! сегмента !

! 1 ! 23 ! 2 !
! 2 ! 24 ! 2 !
! 3 ! 25 ! 3 !

Вариант13



Содержимое буферной памяти :

Сектор 0				Сектор 1		
Блок	Сег.	Изм.	Данные	Сег.	Изм.	Данные
0	2	0	28	3	0	36
1	2	0	29	3	0	37
2	2	0	30	3	0	38
3	2	0	31	3	0	39
4	2	0	32	3	0	40
5	2	0	33	3	0	41
6	2	0	34	3	0	42
7	2	0	35	3	0	43

Содержимое оперативной памяти:

Блок #	0	1	2	3	4	5	6	7
Сегмент #0	12	13	14	15	16	17	18	19
Сегмент #1	20	21	22	23	24	25	26	27
Сегмент #2	28	29	30	31	32	33	34	35
Сегмент #3	36	37	38	39	40	41	42	43

! N обращ. ! Числа для ! N вытесн. !
! ! записи ! сегмента !

! 1 ! 23 ! 2 !
! 2 ! 24 ! 2 !
! 3 ! 25 ! 3 !

Вариант14

Содержимое буферной памяти :

Сектор 0				Сектор 1		
Блок	Сег.	Изм.	Данные	Сег.	Изм.	Данные
0	2	0	29	3	0	37
1	2	0	30	3	0	38
2	2	0	31	3	0	39
3	2	0	32	3	0	40
4	2	0	33	3	0	41
5	2	0	34	3	0	42
6	2	0	35	3	0	43
7	2	0	36	3	0	44

Содержимое оперативной памяти:

Блок #	0	1	2	3	4	5	6	7
Сегмент #0	13	14	15	16	17	18	19	20
Сегмент #1	21	22	23	24	25	26	27	28
Сегмент #2	29	30	31	32	33	34	35	36
Сегмент #3	37	38	39	40	41	42	43	44

! N обращ. ! Числа для ! N вытесн. !
! ! записи ! сегмента !

! 1 ! 23 ! 2 !
! 2 ! 24 ! 2 !
! 3 ! 25 ! 3 !

Вариант15



Содержимое буферной памяти :

Сектор 0				Сектор 1			
Блок	Сег.	Изм.	Данные	Сег.	Изм.	Данные	
0	2	0	29	3	0	37	
1	2	0	30	3	0	38	
2	2	0	31	3	0	39	
3	2	0	32	3	0	40	
4	2	0	33	3	0	41	
5	2	0	34	3	0	42	
6	2	0	35	3	0	43	
7	2	0	36	3	0	44	

Приостановлено... Нажмите "ENTER" для продолжения

Содержимое оперативной памяти:

Блок #	0	1	2	3	4	5	6	7
Сегмент #0	13	14	15	16	17	18	19	20
Сегмент #1	21	22	23	24	25	26	27	28
Сегмент #2	29	30	31	32	33	34	35	36
Сегмент #3	37	38	39	40	41	42	43	44

! N обращ. ! Числа для ! N вытесн. !
! ! записи ! сегмента !

! 1 ! 23 ! 2 !
! 2 ! 24 ! 2 !
! 3 ! 25 ! 3 !
